

API-based Products

API Introduction | Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

What are APIs?

- API is an interface that software uses to access whatever resource it needs:
 - data,
 - server software,
 - or other applications
- API stands for application programming interface
 - are the little pieces of code that make it possible for digital devices, software applications, and data servers to talk with each other
 - are the essential backbone of so many services we now rely on!



A Blog Post by Ayushi Rowat

https://miro.medium.com/v2/resize:fit:1400/format:webp/1*2GMSybCim32TTec9afVGxw.png

Learning Outcomes

Explain how APIs serve as intermediaries between users and backend systems, providing functionality by communicating with both



Work Integrated Learning Programmes

Application

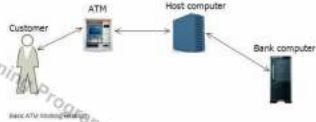
- App provides a function, but it's not doing this all by itself
 - needs to communicate both with the user, and with the backend it's accessing
- App deals in both inputs and outputs
- May be a customer-facing app like a travel booking site, or a back-end app like server software that funnels requests to a database
- Think of an application like an ATM - provides you services by interacting with Bank



<https://www.idfcfirstbank.com/content/dam/idfcfirstbank/images/blog/finance/what-is-atm-717x404.jpg>

Programming

- **Engineering part of the app's software that translates input into output**
 - Accepts, validates and processes user request
- For example, in ATM case
 - translates request for cash to the bank's database
 - verifies there's enough cash in account to withdraw the requested amount
 - the bank grants permission
 - then the ATM communicates back to the bank how much money to withdraw
 - bank can update account balance



<https://www.researchgate.net/publication/220307805/figure/fig1/AS:671513497841676@1537112702690/Basic-ATM-Working-Relation.ppm>

BITS WILP | Generated: 12-May-2026 16:47:54

Interface

- **Interfaces are how we communicate with a machine**
 - With APIs, it's much the same, only we're replacing users with software
- In the case of the ATM,
 - it's the screen, keypad, and cash slot
 - where the input and output occurs



<https://i.pinimg.com/originals/c2/7a/b4/c27ab475b4ed47e061e736357ab276ed.jpg>

BITS WILP | Generated: 12-May-2026 16:47:54

API Working

The customer, a waiter, and a restaurant kitchen!

- Sometimes the easiest way to think about APIs is to think about a metaphor, and a common scenario
 - A customer talks to the waiter and tells the waiter what she wants
 - The waiter takes down the order and communicates it to the kitchen
 - The kitchen does their work, creating the food
 - Then the waiter delivers the order back to the customer



https://www.google.com/imgres?imgurl=https://miro.medium.com/max/951/1*3h95bN2_xe-eltWh_Yavw.png&imgrefurl=https://medium.com/@ama.thanu/what-is-an-api-how-does-it-work-f4ea562d741f&tbid=EEUVQm5EVNB-nM&vet=12ahUKEw_IKqy_BbvAHUS6iGhdkYC1cQMvAqAegUIARChAQ_1&docid=SS-nrZKJLw0tzMw=951&h=588&q=How%20do%20APIs%20work%26ved=2ahUKEw_IKqy_BbvAHUS6iGhdkYC1cQMvAqAegUIARChAQ

BITS WILP | Generated: 12-May-2026 16:47:54

Analogy extended!

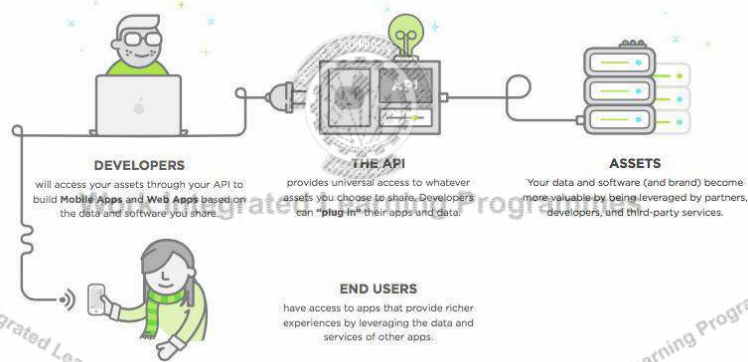
- **The customer, a waiter, and a restaurant kitchen!**
 - In this metaphor, a customer is like a user. who tells the waiter what she wants
- The waiter is like an API,
 - receiving the customer's order
 - translating the order into easy-to-follow instructions that the kitchen then uses to fulfill that order—often following a specific set of codes
 - or input, that the kitchen easily recognizes
- The kitchen is like a server that
 - does the work of creating the order in the manner the customer wants it, hopefully!
- When the food is ready, the waiter picks up the order and delivers it to the customer.



https://www.google.com/imgres?imgres=https://miro.medium.com/max/951/1*3h95bN2_xe-eltWh_Yavw.png&imgrefurl=https://medium.com/@ama.thanu/what-is-an-api-how-does-it-work-f4ea562d741f&tbid=EEUVQm5EVNB-nM&vet=12ahUKEw_IKqy_BbvAHUS6iGhdkYC1cQMvAqAegUIARChAQ_1&docid=SS-nrZKJLw0tzMw=951&h=588&q=How%20do%20APIs%20work%26ved=2ahUKEw_IKqy_BbvAHUS6iGhdkYC1cQMvAqAegUIARChAQ

BITS WILP | Generated: 12-May-2026 16:47:54

Parties involved



BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- API which stands for application programming interface
 - are the little pieces of code that make it possible for digital devices, software applications, and data servers to talk with each other
 - are the essential backbone of so many services we now rely on!

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

- What is the primary function of an API in software applications?
- A) To enhance the visual appearance of a website
- B) To facilitate communication between digital devices, software applications, and data servers
- C) To provide a graphical user interface for users
- D) To store data in a database

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ2

- In the context of an ATM, what role does the software play when a user requests to withdraw cash?
- A) It only provides a user interface
- B) It directly disburses cash to the user
- C) It validates the user's request, communicates with the bank's database, and updates the account balance
- D) It processes transactions without interacting with any external systems

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ3

- Which of the following best describes an API's function when compared to a restaurant analogy?
- A) The API is like the customer, placing an order with the restaurant
- B) The API is like the restaurant kitchen, preparing the food
- C) The API is like the waiter, taking the customer's order and delivering the prepared dish
- D) The API is like the dining area, where the customers enjoy their meal
- Answer: C) The API is like the waiter, taking the customer's order and delivering the prepared dish

API-based Products

API Evolution

Suresh Gopal

Learning Outcomes

- Recognize the **chronological development of APIs** from early, rudimentary forms to sophisticated, feature-rich interfaces

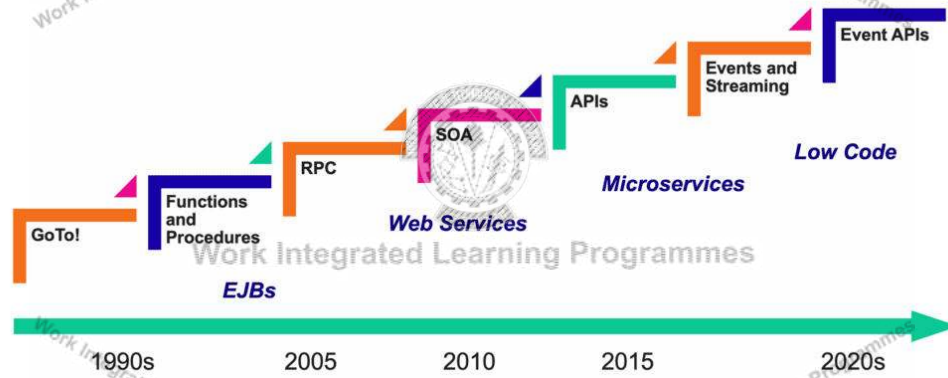


[zenstack](#)

History of APIs

- The history of APIs is a journey marked by significant milestones, reflecting
 - the **evolution of technology**
 - the **changing needs of software development**
- **A brief overview:**
 - Early APIs (1960s-1970s)
 - Graphical User Interfaces (1980s)
 - Internet Era and Web APIs (1990s)
 - REST and the Rise of Web Services (2000s)
 - API Economy and Cloud Computing (2010s)
 - Specialized APIs and Microservices (2010s-2020s)
 - APIs in the Era of IoT and AI (2020s and beyond)

The Evolution of APIs



<https://solace.com/blog/evolution-of-apis-restful-event-driven-apis/>

BITS WILP | Generated: 12-May-2026 16:47:54

Early APIs (1960s-1970s)

- In the early days of computing, APIs were primarily used within operating systems to facilitate interactions between software components
- Examples include
 - system calls in Unix-like operating systems
 - libraries like the Standard I/O library in C

UNIX®
An Open Group Standard



wikipedia

Wikimedia commons

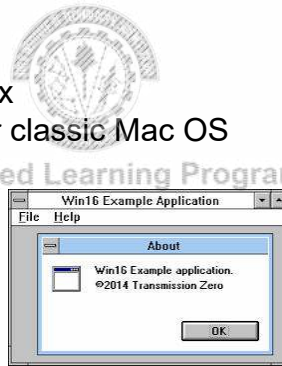
BITS WILP | Generated: 12-May-2026 16:47:54

Graphical User Interfaces (1980s)

With the advent of graphical user interfaces (GUIs), APIs evolved to support the creation of desktop applications

Early examples of GUI APIs

- The Win16 API for Windows 3.x
- The Macintosh Toolbox API for classic Mac OS



Transmission zero

BITS WILP | Generated: 12-May-2026 16:47:54

Internet Era and Web APIs (1990s)

- The rise of the internet led to the development of web APIs, which allowed software systems to communicate over the web
- Common Gateway Interface (CGI)
 - one of earliest methods for creating web APIs,
 - enabling dynamic generation of web content
- SOAP (Simple Object Access Protocol)
 - emerged as a standardized protocol for building distributed systems,
 - although it was complex and heavyweight



SSOURCE

BITS WILP | Generated: 12-May-2026 16:47:54

REST and the Rise of Web Services (2000s)

- Representational State Transfer (REST) gained popularity as a **simpler alternative to SOAP**, emphasizing stateless communication over HTTP
- RESTful APIs became prevalent,
 - enabling developers to **build lightweight and scalable web services**
- Social media platforms like **Twitter and Facebook** popularized the use of RESTful APIs for accessing their data and functionality



BITS WILP | | Generated: 12-May-2026 16:47:54

API Economy and Cloud Computing (2010s)

- The proliferation of cloud computing services and the rise of the **API economy** fueled the growth of APIs across industries
- API management platforms emerged, providing tools
 - for designing, deploying, and managing APIs at scale
- Companies like **Amazon, Google, and Microsoft** offered a wide range of cloud-based APIs for services such as **storage, computation, and machine learning**

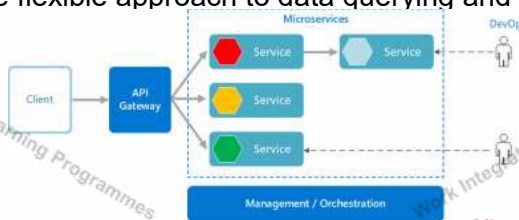


medium

BITS WILP | | Generated: 12-May-2026 16:47:54

Specialized APIs and Microservices (2010s-2020s)

- APIs became **increasingly specialized**, offering access to specific functionalities such as geolocation, payment processing, and AI services
- Microservices architecture gained momentum, **leading to the development of APIs for inter-service communication within distributed systems**
- **GraphQL** emerged as an alternative to REST,
 - offering a more flexible approach to data querying and retrieval



Microsoft

BITS WILP | | Generated: 12-May-2026 16:47:54

APIs in the Era of IoT and AI (2020s and beyond)

- With the **proliferation of Internet of Things (IoT) devices and advances in artificial intelligence (AI)**, APIs continue to evolve to support new use cases and technologies
- **APIs for IoT enable devices to communicate with each other and with cloud-based services**,
 - facilitating the development of smart and interconnected systems
- AI APIs provide **access to pre-trained models and algorithms** for tasks such as
 - natural language processing,
 - computer vision,
 - and recommendation systems



thebrainybits.com

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- APIs like **SOAP and early REST protocols** which provided basic communication between applications, but were cumbersome and lacked flexibility
- **Technological advancements such as the emergence of GraphQL**, microservices architecture, and serverless computing reshaped API design, emphasizing simplicity, scalability, and efficiency
- **Advent of AI and machine learning APIs** expanded the scope of possibilities, enabling developers to leverage advanced capabilities without extensive domain expertise

MCQ1

Web APIs can be implemented with _____.

- A) Common Gateway Interface (CGI)
- B) REST
- C) GraphQL
- D) None of above

MCQ2

Which of the following is dominant way of creating APIs?

- A) REST
- B) SOAP
- C) CGI

MCQ3

Which of the following accurately describes the historical progression of APIs, reflecting the evolution of technology and the changing needs of software development?

- a) Microservices and Specialized APIs (2000s)
- b) Internet Era and Web APIs (1990s)
- c) REST and the Rise of Web Services (1960s-1970s)
- d) Graphical User Interfaces (2020s)

API-based Products

API Paradigms
Exploring Modern Approaches

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

WHEN TO
USE
WHAT:

{ REST }

GraphQL

webhooks

gRPC

NORDICAPIS.COM

Learning Outcomes

- Understand the fundamental principles and importance of API paradigms in modern software development
- Identify and differentiate between various API paradigms, including
 - RESTful API,
 - GraphQL,
 - gRPC,
 - event-driven architecture

REST, GraphQL, gRPC
choose the right protocol
for API Design

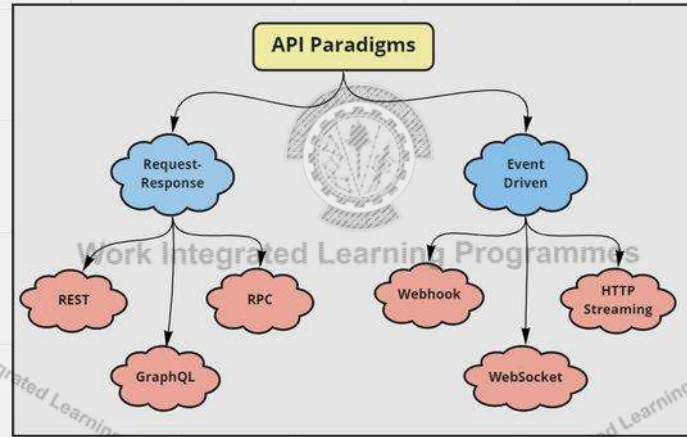


scalac

API paradigms

- API paradigms refer to
 - fundamental principles,
 - architectural styles,
 - and design approaches
- governing the creation and interaction of Application Programming Interfaces (APIs) in software development
- These paradigms shape how APIs are
 - structured,
 - accessed,
 - and utilized within software systems,
- influencing factors such as
 - interoperability,
 - scalability,
 - performance,
 - and developer experience

API paradigms

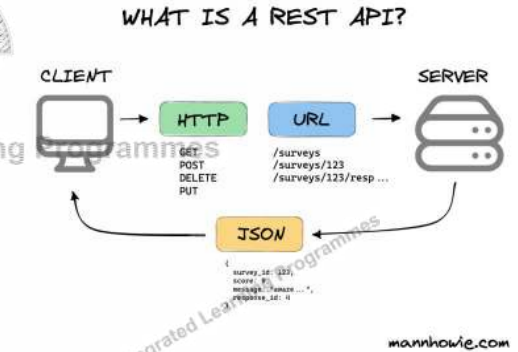


systemdesigntopics.quora.com

BITS WILP | Generated: 12-May-2026 16:47:54

RESTful API paradigm

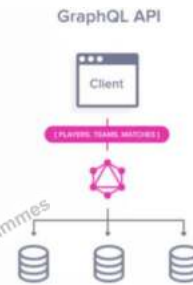
- Representational State Transfer (REST) is an **architectural style** for **designing networked applications**
- RESTful APIs
 - utilize **standard HTTP methods**
 - GET, POST, PUT, DELETE
 - to perform **CRUD**
 - Create, Read, Update, Delete operations
- Emphasizes
 - statelessness,
 - scalability,
 - and uniform resource identification



BITS WILP | Generated: 12-May-2026 16:47:54

GraphQL paradigm

- GraphQL is a **query language and runtime** for APIs developed by Facebook
- Provides a **flexible data query mechanism**, allowing clients to request **specific data structures**
- Enables clients to retrieve multiple resources in a single request,
 - reducing over-fetching and under-fetching of data

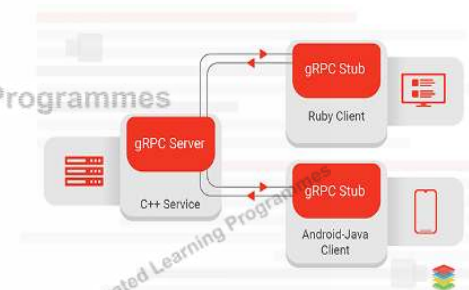


scriptworks.io

BITS WILP | Generated: 12-May-2026 16:47:54

gRPC paradigm

- gRPC is a **high-performance, open-source RPC (Remote Procedure Call) framework** developed by Google
- Utilizes Protocol Buffers (protobuf) for efficient serialization and deserialization of data
- Supports
 - bidirectional streaming,
 - authentication,
 - and load balancing out of the box



medium

BITS WILP | Generated: 12-May-2026 16:47:54

Event-driven architecture paradigm

Event-driven architecture focuses on

- Production,
- Detection,
- Consumption,
- and reaction to events

Utilizes event-driven messaging systems

- Apache Kafka,
- RabbitMQ,
- or AWS SNS/SQS

Enables

- Asynchronous communication,
- Scalability,
- and loose coupling between components



Summary

- **Realm of API paradigms,**
 - which are foundational principles shaping modern software development
- Four key paradigms:
 - **RESTful API,**
 - **GraphQL,**
 - **gRPC,**
 - **and Event-driven architecture**



[Treble blog](#)

MCQ1

Which of the following best describes API paradigms?

- a) Development methodologies
- b) Coding languages
- c) Fundamental principles and design approaches
- d) Software testing techniques

Correct Answer: c) Fundamental principles and design approaches

MCQ2

Which of the following is example of Request – Response API paradigm?

- A) streaming
- B) GraphQL
- C) REST
- D) gRPC

Answer: B, C, D

MCQ3

Which of the following is example of Event driven API paradigm?

- A) streaming
- B) GraphQL
- C) REST
- D) gRPC

Answer: A



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Define the **concept of API-as-a-Product**
- Identify **key characteristics** that differentiate API products
- Understand the **benefits** of adopting an API-as-a-Product mindset
- Grasp **best practices** for developing and managing API products



Work Integrated Learning Programmes



[openapihub](https://openapihub.com)

BITS WILP | Generated: 12-May-2026 16:47:54

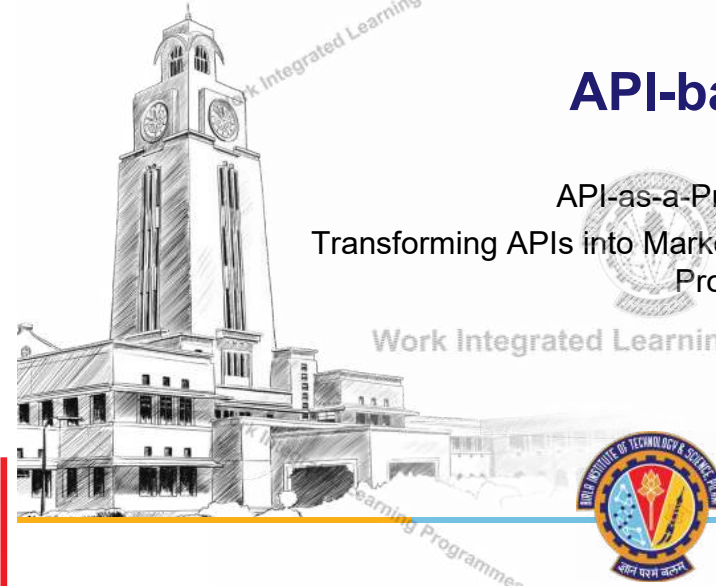


Work Integrated Learning Programmes

API-based Products

API-as-a-Product
Transforming APIs into Marketable Products

Suresh Gopal



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

API-as-a-Product

API-as-a-Product refers to

- Treating APIs not just as technical interfaces.
- But as **fully-fledged products** designed to **meet the needs of developers and end-users**.

API products enable businesses to

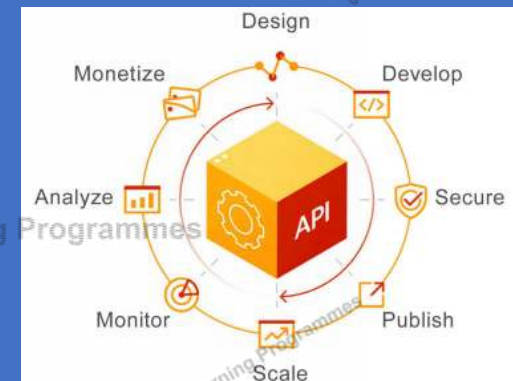
- Leverage their data and services in new ways.
- Fostering innovation, and expanding market reach.



Work Integrated Learning Programmes

Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Source: <https://cloud.google.com/files/apigee/apigee-api-product-mindset-ebook.pdf>



Work Integrated Learning Programmes

Key Characteristics

Developer-Centric

Prioritize developer experience with clear documentation, SDKs, and support

Scalability

APIs should be designed to scale with the growing needs of users and applications

Monetization

Offer flexible pricing models such as freemium, usage-based, or subscription

Analytics and Insights

Provide robust analytics to understand API usage and optimize performance

Security

Implement strong security measures to protect both data and users



Image source: <https://blog.openapihub.com/en-us/why-you-need-to-think-api-as-a-product-now/>

BITS WILP | Generated: 12-May-2026 16:47:54

Benefits of API-as-a-Product

Increased Revenue Streams

Monetizing APIs opens up new revenue channels beyond core products

Enhanced Partner Ecosystem

APIs attract developers and foster partnerships, expanding the ecosystem

Innovation and Agility

Encourages experimentation and innovation by enabling rapid integration of new services

Improved Customer Experience

Empowers developers to create richer, more personalized experiences for end-users

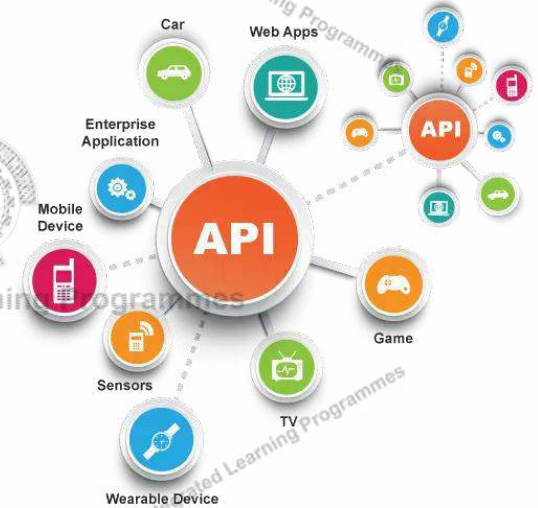


Image source: <https://blog.restcase.com/content/images/2016/11/api-collaboration.png>

[quantumsofttech](https://blog.restcase.com/content/images/2016/11/api-collaboration.png)

BITS WILP | Generated: 12-May-2026 16:47:54

Case Studies

Airbnb

Opened its API to developers, allowing them to integrate Airbnb listings into their apps, expanding its reach and user base

Twilio

Provides a cloud communications platform through APIs, enabling developers to add messaging, voice, and video to their applications

Stripe

Offers a suite of APIs for online payments, enabling businesses to accept payments seamlessly



BITS WILP | Generated: 12-May-2026 16:47:54

Best Practices

User-Centered Design

Prioritize the needs and experience of developers and end-users

Continuous Improvement

Regularly update and iterate APIs based on feedback and changing market demands

Collaboration

Foster collaboration between API teams, developers, and business stakeholders

Documentation and Support

Provide comprehensive documentation and responsive support to assist developers in integrating APIs

Best Practices for API Management



Image source: <https://appinventiv.com/wp-content/uploads/2018/05/Best-Practices-for-Building-the-Right-API.png>

[appinventiv](https://appinventiv.com/wp-content/uploads/2018/05/Best-Practices-for-Building-the-Right-API.png)

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Concept of treating **APIs as marketable products** rather than just technical interfaces
- **Key characteristics of API products**, including
 - Developer-centric design
 - Scalability
 - Monetization options
 - Analytics
 - and Security measures
- **Benefits of adopting an API-as-a-Product mindset**, such as
 - generating additional revenue streams
 - improving customer experience
- **Best practices** for developing and managing API products



[brunofuga](#)

MCQ1

What aspect of API design prioritizes the experience of developers by providing clear documentation, SDKs, and support?

- a) Scalability
- b) Monetization
- c) Analytics and Insights
- d) Developer-Centric

Correct Answer: d) Developer-Centric

MCQ2

Which of the following is a result of monetizing APIs, which opens up additional revenue channels beyond core products?

- a) Expanded Partner Ecosystem
- b) Enhanced User Experience
- c) Encouraged Innovation and Agility
- d) Boosted Revenue Streams

Correct Answer: d) Boosted Revenue Streams

MCQ3

Which principle emphasizes prioritizing the needs and experiences of both developers and end-users in API design?

- a) Continuous Improvement
- b) Collaboration
- c) Documentation and Support
- d) User-Centered Design

Correct Answer: d) User-Centered Design

API-based Products

API-First Thinking
A Paradigm Shift in Software Development

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

API-First Thinking

API-First Thinking is a development approach where APIs are designed and prioritized before any other aspect of the application

Ensures

- Interoperability
- Scalability
- and flexibility of applications

Means developers prioritize designing the API that will serve as the foundation for all interactions between the frontend, backend, and other services



Learning Outcomes

- Grasp the **concept of API-First Thinking**
- Gain knowledge of **Key Principles**
- Recognize the **benefits**
- Develop familiarity with **Implementation Strategies**



[medium](#)

BITS WILP | | Generated: 12-May-2026 16:47:54

Key Principles of API-First Thinking

Design-Driven Development

APIs are designed based on use cases and user needs, promoting a user-centric approach.

Scalability and Flexibility

APIs are built to scale with the evolving needs of applications and adapt to changing requirements.

Interoperability

APIs adhere to standards and best practices to ensure seamless integration with other systems and services.



Image source: <https://www.netsolutions.com/assets/images/API-first-App-Development-1.webp>

BITS WILP | | Generated: 12-May-2026 16:47:54

Benefits of API-First Thinking

Faster Time-to-Market

APIs enable parallel development by decoupling frontend and backend development, reducing time-to-market

Improved Developer Experience

Developers can work independently on frontend and backend components, promoting agility and innovation

Enhanced Collaboration

Clear API contracts facilitate collaboration between development teams, reducing misunderstandings and errors

Benefits of an API-First Approach



Image source: <https://www.netsolutions.com/assets/images/Benefits-of-API-first-approach.webp>

Implementation Strategies

API Documentation

Comprehensive documentation ensures clarity and ease of integration for developers

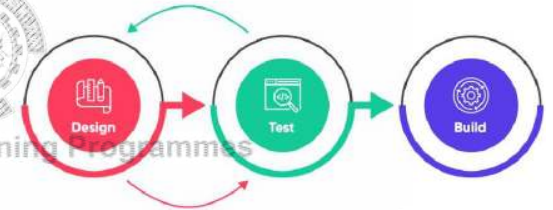
Mocking and Prototyping

Use mock APIs to enable frontend development before backend implementation, speeding up development cycles

Continuous Integration and Testing

Implement automated testing and continuous integration pipelines to ensure the stability and reliability of APIs

Implementing API-first



Case Studies

GitHub

Employs API-First Thinking by providing robust APIs for version control, enabling seamless integration with third-party tools and services



Slack

Prioritizes API development, allowing developers to build custom integrations and automate workflows within the platform



Stripe

Offers a suite of APIs for online payments, providing developers with the tools to accept payments seamlessly



MCQ1

Which approach prioritizes the design of APIs before any other aspect of application development?

- a) Backend-First Thinking
- b) Middleware-First Thinking
- c) API-First Thinking
- d) Frontend-First Thinking

Correct Answer: c) API-First Thinking

MCQ2

Which development approach emphasizes designing APIs based on use cases and user needs, promoting a user-centric approach?

- a) User-Driven Development
- b) Design-Driven Development
- c) Function-First Development
- d) Efficiency-Driven Development

Correct Answer: b) Design-Driven Development

MCQ3

Which practice ensures clarity and ease of integration for developers by providing comprehensive documentation?

- a) Mocking and Prototyping
- b) Continuous Integration and Testing
- c) API Documentation
- d) Backend Implementation

Correct Answer: c) API Documentation

API-based Products

API Product Lifecycle | Suresh Gopal

The five stages of an API lifecycle

While it's crucial to plan for the development of an API, it's just as important to prepare for its retirement. Software teams typically divide the API lifecycle into five distinct phases:

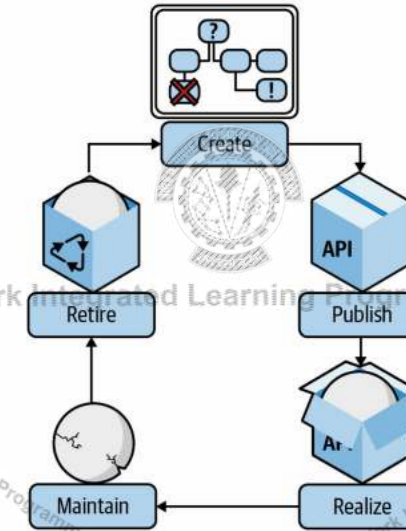


Introduction

- Just like any product, **APIs have a lifecycle with distinct phases**
 - Understanding these phases is crucial for effective API management
- Let's delve into the five stages of API lifecycle and explore how they contribute to continuous API management
 - Create Phase
 - Publish Phase
 - Realize Phase
 - Maintain Phase
 - Retire Phase

BITS WILP | Generated: 12-May-2026 16:47:54

API Product Lifecycle



Source: Adapted from Continuous API Management

Create Phase

Design, prototype, and test phase

- APIs may be complete but not released for wide use
- Are not considered available for wide use
- Interface is likely to change
- Exhibit errors
- Experience periods of non-responsiveness

Characteristics

- A new API or a replacement of API that no longer exists
- Has not been deployed in a production environment
- Has not been made available for reliable use

BITS WILP | Generated: 12-May-2026 16:47:54

Publish Phase

Wide adoption of the API can hinge on whether the publishing effort was adequate for the target community

Characteristics

- API instance has been deployed to a production environment
- Made available to one or more developer communities
- Strategic value of API is not yet being realized

BITS WILP | Generated: 12-May-2026 16:47:54

Publish Phase II

The API is placed into production and made available to one or more developer communities for use.

- Considered reliable, the interface is stable
- Proper security and scalability elements are in place

Making API public likely means investing in

- Documentation
- Training courses
- Sales materials
- Even staffing up support forums and online communities

Realize Phase

Phase focused on realizing the initial reason that product was created.

- Might change the interface to better meet goals,
- Tweak performance, availability, scalability
- Changes are driven by the desire to realize the initial purpose of the product

Good APIs solve a problem

- A good API program keeps track of just how well those APIs are doing at solving their stated problems

Realize Phase II

The work of realizing an API is

- Actually the **work of “proving” that the API product is doing a good job**
- Solving the problem it was designed to address

Characteristics

- Published API instance exists and is available
- Being used in a way that realizes its objective
- Realized value is generally trending upward

Maintain Phase

Assuming API meets our intended goal

- It is realizing it's planned value
- Can eventually place it into “maintenance mode”

Should focus on getting the most value from the product without investing a great deal in changes to it

- May do some minor optimizations,
- Look for ways to reduce operational costs, etc.
- But it is important to **resist making any major changes or costly investments to the product.**

Maintain Phase II

Important to identify when the API reaches the maintain phase

- Placing APIs into maintenance mode means you can
 - Dial back on adding new features
 - Focus on improving performance,
 - Reducing operational costs,
 - and maximizing return.

Characteristics

- Being actively used by one or more consuming applications
- Realized value is stagnant or downward trending
- No longer actively being improved

Retire Phase

Finally, as the realized value starts to wane, as the operational costs outweigh the revenue/saving benefits.

- Time to place the product into retirement

May begin a program to migrate remaining users of this product to a new, more valuable, alternative

- or if the service is no longer needed, simply work out an end-of-life plan for the API
- Begin the “wind-down” process.

Retire Phase II

Retiring an API frees up resources (infrastructure, staff, support, etc.)

Characteristics

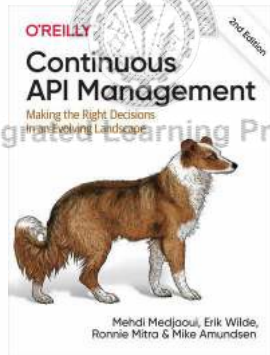
- Published API instance exists and is available
- Realization value is no longer enough to justify continued maintenance
- End of life decision has been made

Summary

- Create phase
 - emphasizes the importance of active design processes and clear use cases to lay a solid foundation for API development
- Publish phase
 - highlighted the significance of proper documentation, training, and support to facilitate wide adoption and success
- Realize phase
 - focuses on tracking metrics and monitoring API performance to ensure that they effectively meet their intended purposes
- Maintain phase
 - APIs are optimized to maximize value while minimizing changes and investments
- Retirement phase,
 - stresses the importance of actively managing the end-of-life process to avoid unnecessary costs and maintain a healthy API ecosystem

Reference

- Continuous API Management, 2nd Edition
 - By Mehdi Medjaoui, Erik Wilde, Ronnie Mitra, Mike Amundsen



BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

Which of the following is a key aspect of effective API management, involving understanding the distinct phases of an API lifecycle?

- a) Product development
- b) Market analysis
- c) Continuous integration
- d) API lifecycle management

Correct Answer: d) API lifecycle management

MCQ2

Which of the following represents a stage in the lifecycle of an API that involves making the API available to users for integration and consumption?

- a) Create Phase
- b) Publish Phase
- c) Realize Phase
- d) Maintain Phase
- e) Retire Phase

Correct Answer: b) Publish Phase

MCQ3

Which stage of the API lifecycle involves the initial development and design of the API, focusing on defining its functionalities and specifications?

- a) Publish Phase
- b) Realize Phase
- c) Maintain Phase
- d) Create Phase
- e) Retire Phase

Correct Answer: d) Create Phase

API-based Products

Trends in API development | Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

API Trends in 2023

- | | |
|--|---|
| 01 API analytics and monitoring platforms | 06 API analytics and monitoring platforms |
| 02 API-as-a-Product | 07 GraphQL Adoptability will Accelerate |
| 03 The proliferation of low code and no code API development | 08 APIs will become more ubiquitous and hyperconnected |
| 04 Machine learning APIs | 09 The Adoption of microservices will promote the development of APIs |
| 05 Improvement in API security standards | 10 Improvement in API security standards |

Source: <https://apitoolkit.io/blog/api-trends/top-10-api-trends.png>

Learning Outcomes

- Develop understanding of Emerging Trends in API development
- Knowledge of API Marketplaces



[geeksforgeeks](https://www.geeksforgeeks.org/)

BITS WILP | Generated: 12-May-2026 16:47:54

API Trends in 2026

- API-as-a-Product (AaaS) is Going Mainstream
- REST is Great, but GraphQL & gRPC Are Growing
- Async APIs Are on the Rise
- API Security as a Top Attack Surface
- AI-Assisted API Design, Testing & Security
- APIs Consumed by AI and Agents (Not Just Humans)
- API Governance, Not Just Management
- The Rise of API Observability & Analytics
- API-First Development
- Multi-Cloud / Hybrid API Governance & Consistent Security
- Regulatory & Domain-Specific APIs
- Observability, Analytics & AI-Assisted Ops
- Policy-as-Code & Automated API Compliance
- API Marketplaces are Exploding

Source: [Top API Trends to Watch in 2026 - Security, AI & Governance](#) [Top 10 API Trends for 2026: Shaping the Future of Development](#)

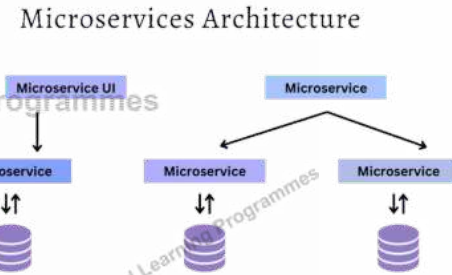
Rise of Microservices Architecture

Microservices architecture is an approach to software development that structures an application as a collection of loosely coupled services

Each service is responsible for a specific business function.

Services are independently deployable and scalable.

Communication between services often occurs via lightweight protocols like HTTP or messaging queues.



Source: <https://www.atatus.com/blog/content/images/size/w960/2022/12/microservice-architecture.png> | Generated: 12-May-2026 16:47:54

Benefits of Microservices in API Development

Enhanced Scalability:

Microservices allow for scaling individual components independently, enabling better resource utilization.

Improved Agility:

Decoupled services facilitate faster development cycles and easier maintenance, promoting agility in API development.

Flexibility:

Microservices enable the use of different technologies and frameworks for each service, optimizing for specific requirements.

Fault Isolation:

Failure in one service does not necessarily impact the entire system, enhancing fault tolerance and reliability.

CHARACTERISTICS OF MICROSERVICE ARCHITECTURE



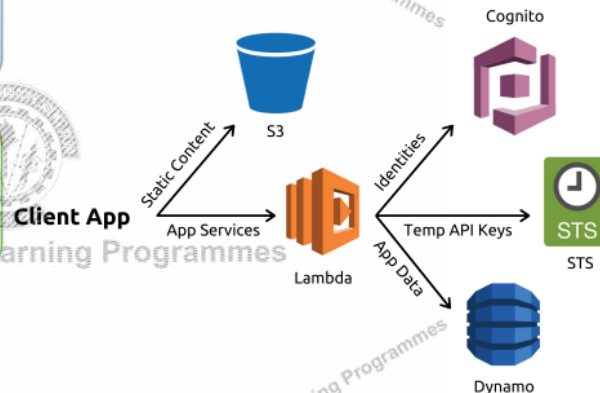
Image Source: https://images.spiceworks.com/wp-content/uploads/2022/04/05115956/7_1.png

Serverless Computing

Serverless computing is a cloud computing model where cloud providers dynamically manage the allocation of machine resources, allowing developers to focus on writing code without worrying about server management.

Characteristics:

- No server provisioning or maintenance required.
- Pay-per-use pricing model based on actual resource consumption.
- Event-driven architecture with automatic scaling.



Source: <https://profjustaftermidnight247.com/wp-content/uploads/2021/01/2.png> | Generated: 12-May-2026 16:47:54

How Serverless Architecture Impacts API Development?

Stateless Functions:

Serverless platforms execute functions in stateless containers, making them ideal for building API endpoints that can scale automatically based on demand.

Focus on Business Logic:

Developers can focus solely on writing application logic without concerning themselves with server provisioning, scaling, or infrastructure management.

Rapid Development and Deployment:

Serverless architectures enable rapid development cycles, allowing developers to deploy updates quickly and iteratively.

Cost Efficiency:

With serverless computing, developers only pay for the resources consumed during function execution, resulting in cost savings compared to traditional server-based architectures.



Image source: <https://www.fortinet.com/resources/cyberglossary/serverless-computing>

AI-Powered APIs

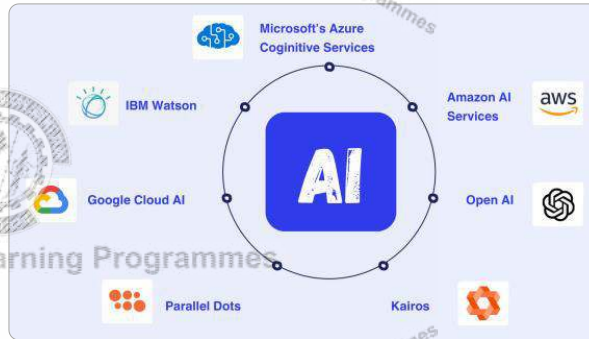
AI-powered APIs leverage artificial intelligence (AI) and machine learning (ML) algorithms to provide advanced functionalities and insights through an API interface

Integration:

AI capabilities are encapsulated within APIs, allowing developers to access and utilize AI functionality without needing to build or train ML models from scratch.

Accessibility:

AI-powered APIs democratize access to AI technologies, enabling developers with varying levels of expertise to incorporate AI capabilities into their applications.



Use Cases of AI in API Development

Natural Language Processing (NLP):

AI-powered APIs for NLP enable tasks such as sentiment analysis, language translation, entity recognition, and text summarization, enhancing the understanding and processing of human language

Image Recognition:

AI-powered APIs for image recognition facilitate tasks such as object detection, image classification, facial recognition, and content moderation, enabling applications to interpret and analyze visual data

Predictive Analytics:

AI-powered APIs for predictive analytics utilize ML algorithms to analyze historical data and make predictions or recommendations, empowering applications with insights for decision-making and optimization



API Marketplaces

API marketplaces are platforms that facilitate the discovery, procurement, and integration of APIs from various providers.

- Acts as market hubs for API consumption and distribution.

Functionality:

API marketplaces offer features such as API search and discovery, documentation access, subscription management, billing, and analytics.

Diverse Offerings:

API marketplaces host a wide range of APIs spanning different categories, industries, and functionalities, catering to diverse developer needs and use cases.

Examples:

- Amazon API Gateway
- RapidAPI
- Google Cloud API Marketplace
- Microsoft Azure Marketplace



Summary

- The evolution of API development, highlighting the transition from monolithic architectures to modern approaches like microservices and serverless computing
- The role of API marketplaces in facilitating API discovery, consumption, and monetization, offering benefits for both API providers and consumers

MCQ1

Which of the following best describes a characteristic of microservices architecture?

- a) Tight coupling between services
- b) Centralized deployment of services
- c) Dependency on heavyweight protocols like CORBA
- d) Independence and scalability of services

Correct Answer: d) Independence and scalability of services

MCQ2

Which of the following characteristics best describes serverless computing?

- a) Manual server provisioning and maintenance
- b) Fixed pricing model regardless of resource consumption
- c) Event-driven architecture with manual scaling
- d) Dynamic management of machine resources by cloud providers

Correct Answer: d) Dynamic management of machine resources by cloud providers

MCQ3

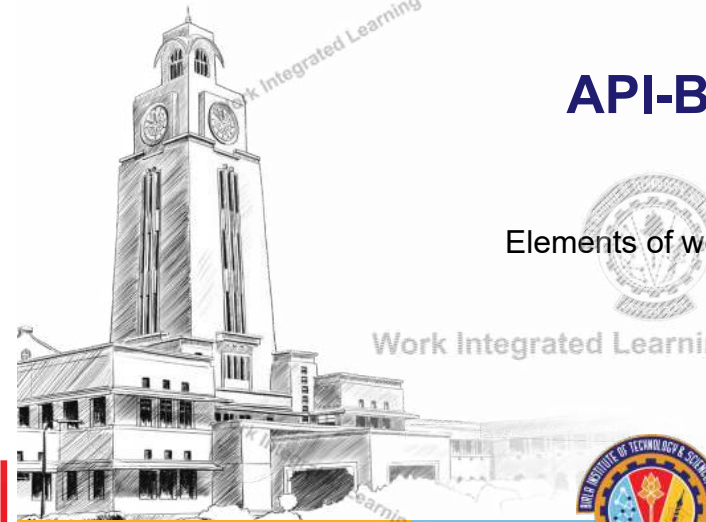
How does serverless architecture impact API development?

- a) Increases server management tasks
- b) Reduces scalability options
- c) Simplifies deployment and maintenance
- d) Adds complexity to code writing

Correct Answer: c) Simplifies deployment and maintenance

API-Based Products

Elements of web API | Suresh Gopal



Learning Outcomes

- Understand the Concept of Web APIs
- Identify Key Elements of Web APIs



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



What are Web APIs?

Web APIs (Application Programming Interfaces) are interfaces that allow different software applications to communicate and interact with each other over the web.

Purpose:

- Enable developers to access functionality and data from other applications or services.

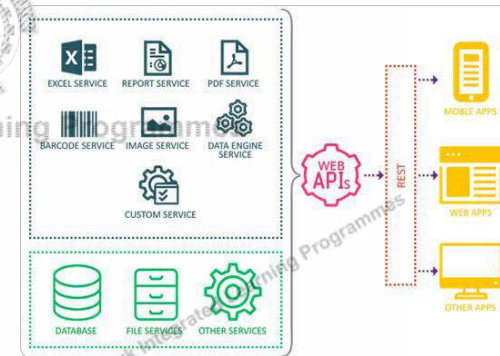


Image source: <https://helpcentral.componentone.com/nethelp/c1webapi/webframe.html#C1WebAPI.html>

BITS WILP | Generated: 12-May-2026 16:47:54

Key Elements of Web APIs

Endpoints:

URLs that represent resources or functionality exposed by the API

HTTP Methods:

Actions that can be performed on resources (GET, POST, PUT, DELETE)

Parameters:

Additional data passed in API requests to specify actions or filter results

Responses:

Data returned by the API in response to client requests

Error Handling:

Mechanisms for communicating errors and status information to clients

BITS WILP | Generated: 12-May-2026 16:47:54



Elements of Web APIs

REST API IN ACTION



Image source: <https://www.altexsoft.com/blog/rest-api-design/>

HTTP Methods and Status Codes

HTTP Methods:

- GET: Retrieve data from the server
- POST: Create new resources on the server
- PUT: Update existing resources on the server
- DELETE: Remove resources from the server

HTTP Status Codes:

- 2xx: Success (e.g., 200 OK)
- 4xx: Client Error (e.g., 404 Not Found)
- 5xx: Server Error (e.g., 500 Internal Server Error)

Status	Method	File
404	HEAD	recipe
200	GET	bootstrap.min.css
200	GET	bootstrap.min.css
404	HEAD	recipe
404	HEAD	recipe
412	POST	app
200	GET	app

Image source: https://metamug.com/article/status-codes-for-rest-api.html#google_vignette

BITS WILP | Generated: 12-May-2026 16:47:54

Authentication and Authorization

Authentication:

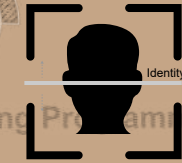
Process of verifying the identity of clients accessing the API

Authorization:

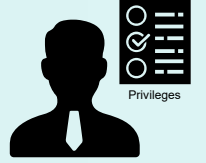
Process of determining whether authenticated clients have permission to access specific resources or perform certain actions

Common Authentication Methods:

- API keys
- OAuth,
- JWT (JSON Web Tokens)



Authentication
are you who you say you are?



Authorization
are you allowed to do this action?

Image source: https://drtatherbewriting.com/learnapidoc/docapis_more_about_authorization.html

Data Formats: JSON and XML

JSON (JavaScript Object Notation):

- Lightweight, human-readable data interchange format.

XML (eXtensible Markup Language):

- Versatile markup language for representing structured data

Considerations:

- Choose the appropriate format based on factors such as compatibility, readability, and performance



Image source: <https://andrewlock.net/formatting-response-data-as-xml-or-json-based-on-the-url-in-asp-net-core/>

API Documentation

Importance:

- Provides developers with essential information on how to use the API

Key Components:

- Endpoint URLs and methods
- Request parameters and payloads
- Response formats and status codes
- Authentication requirements
- Examples and usage scenarios

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Key elements which includes
 - endpoints,
 - HTTP methods,
 - authentication,
 - data formats,
 - and documentation

MCQ1

What are HTTP Methods used for in an API?

- a) Representing resources or functionality
- b) Specifying additional data in requests
- c) Communicating errors and status information
- d) Performing actions on resources

Correct Answer: d) Performing actions on resources

MCQ2

What is the primary purpose of the HTTP method PUT in API communication?

- a) Retrieve data from the server
- b) Create new resources on the server
- c) Update existing resources on the server
- d) Remove resources from the server

Correct Answer: c) Update existing resources on the server

MCQ3

What does the process of authentication involve in API communication?

- a) Determining whether authenticated clients have permission to access specific resources
- b) Verifying the identity of clients accessing the API
- c) Generating JSON Web Tokens (JWT)
- d) Authorizing clients to perform specific actions

Correct Answer: b) Verifying the identity of clients accessing the API

API-Based Products

Why API design matters? Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

Enhancing Developer Experience

Developer experience encompasses all aspects of interaction between developers and APIs which includes

- Ease of use,
- Clarity of documentation,
- Consistency in design,
- and efficiency in implementation.

Clear and intuitive API design significantly enhances developer experience.

- Well-designed APIs provide developers with straightforward ways to achieve their goals.
- Intuitive design reduces the learning curve, enabling developers to quickly grasp the API's functionalities and capabilities.

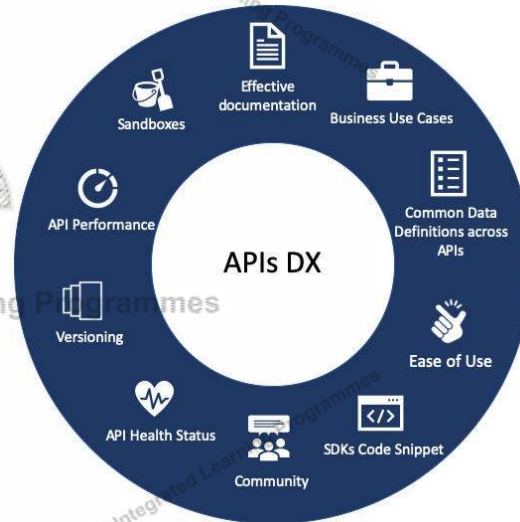


Image source: <https://neosalph.com/api-developer-experience-dx/>
BITS WILP | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Explain how well-designed APIs enhance developer experience
- Analyze the role of APIs in driving business growth, expanding market reach, and increasing revenue



Work Integrated Learning Programmes



BITS WILP | Generated: 12-May-2026 16:47:54

Facilitating Integration

APIs serve as bridges that enable communication and data exchange between disparate applications and platforms.

- Provide a standardized interface for seamless interaction, regardless of the underlying technologies.

Seamless integration is crucial for efficient workflow and data flow across systems.

- Well-designed APIs facilitate integration by offering clear endpoints, standardized data formats, and consistent communication protocols.
- Seamless integration reduces development time, minimizes errors, and enhances overall system interoperability.

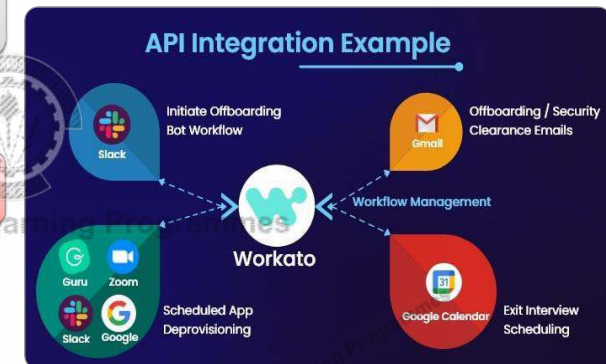


Image source: <https://www.excellentwebworld.com/api-development-knows-what-why-how-guide/>
BITS WILP | Generated: 12-May-2026 16:47:54

Enabling Innovation

APIs play a pivotal role in **unlocking innovation by democratizing access to data, services, and functionalities.**

- Empower developers to leverage existing resources and build upon them to create new and innovative solutions.

Flexible and extensible API design is essential for **accommodating diverse use cases and fostering innovation.**

- APIs designed with flexibility allow developers to adapt and extend functionalities to suit evolving needs **without requiring significant changes to the underlying infrastructure.**

EVERYDAY EXAMPLES OF API



Image source: <https://www.turing.com/kb/7-examples-of-apis>

Driving Business Growth

APIs serve as powerful **enablers of business growth** by facilitating connectivity and collaboration.

- Allow businesses to **leverage their core capabilities and data assets** to create new **revenue streams and business opportunities.**

APIs play a crucial role in **expanding market reach** by enabling businesses to **extend their products and services to new platforms, devices, and ecosystems.**

- Through APIs, businesses can reach **broader audiences, penetrate new markets, and attract new customers.**
- APIs contribute to **revenue generation through various monetization strategies.**

A study of companies running an API platform by Google Cloud shows the breakdown of API usage by industry:

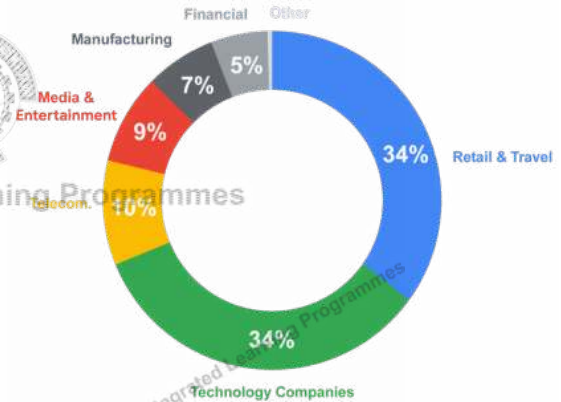


Image source: Google Cloud's State of APIs 2020

Impact of Well-Designed APIs on Business Scalability

Well-designed APIs are essential for **business scalability**, allowing organizations to adapt and grow in response to changing market demands

- Flexible and extensible API design **enables**
 - **Seamless integration with new technologies, systems, and partners.**
- Facilitating business expansion and diversification

Businesses with well-designed APIs gain a **competitive edge** by offering

- Superior customer experiences,
- Faster time-to-market for new products and services,
- Greater agility in responding to market trends and customer needs

Summary

- The pivotal role APIs plays in today's digital landscape, driving
 - innovation,
 - expanding market reach,
 - and enabling business growth

MCQ1

How does clear and intuitive API design impact developer experience?

- a) Increases the learning curve
- b) Decreases efficiency in implementation
- c) Reduces clarity of documentation
- d) Enhances ease of use and efficiency

Correct Answer: d) Enhances ease of use and efficiency

MCQ2

How do well-designed APIs contribute to efficient workflow and data flow across systems?

- a) By increasing development time
- b) By maximizing errors
- c) By offering clear endpoints, standardized data formats, and consistent communication protocols
- d) By complicating system interoperability

Correct Answer: c) By offering clear endpoints, standardized data formats, and consistent communication protocols

MCQ3

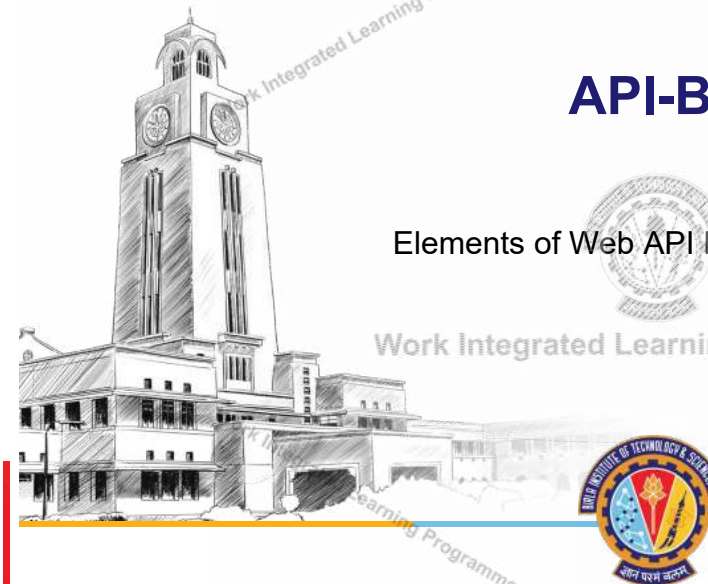
How do APIs contribute to business growth and revenue generation?

- a) By limiting market reach
- b) By increasing development costs
- c) By enabling businesses to extend products and services to new platforms and ecosystems
- d) By reducing collaboration and connectivity

Correct Answer: c) By enabling businesses to extend products and services to new platforms and ecosystems

API-Based Products

Elements of Web API Design | Suresh Gopal



Learning Outcomes

- Develop understanding about
 - Importance of Business Capabilities in API Design
 - Application of Product Thinking to API Design
 - Significance of Developer Experience (DX) in API Development



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



The Elements of Effective Web API Design

The design quality of APIs reflects how organizations value developers

Effective Web API design incorporates three key elements:

- Business capabilities,
- Product thinking,
- and developer experience



Image source: <https://blog.openapihub.com/en-us/why-you-need-to-think-api-as-a-product-now/>

Business Capabilities

Business capabilities describe what an organization brings to the market

For example

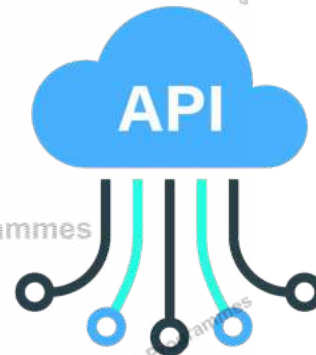
- Unique product design,
- Customer service,
- Sales pipeline management, etc.

Using delivery methods

- Directly by the organization,
- Outsourced,
- or a combination of both

Importance

- APIs digitize business capabilities
- Shaping API design around them ensures alignment with organizational goals



BITS WILP | Generated: 12-May-2026 16:47:54



Product Thinking

Transition from custom integrations to a focus on product thinking

Product thinking shifts focus to scalable, cost-effective API designs that meet market needs.

Benefits

- Enables self-service integration,
- Reduces customization efforts,
- Fosters reusable APIs

Approach

Gather feedback from multiple parties early in the design process to shape APIs for increased opportunities for reuse.

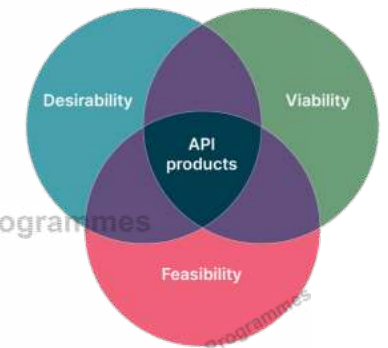


Image source: <https://www.thoughtworks.com/insights/blog/apis/design-thinking-api>



Developer Experience (DX)

DX focuses on engagement with developers, similar to UX for products/services

Importance:

• Great DX

- Drives successful API adoption,
- Improves market traction,
- Empowers developers to deliver value efficiently

Considerations

- First impressions
- Day-to-day usage
- Support
- and documentation



Image source: <https://neosalpha.com/api-developer-experience-dx>

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

The potential impact of prioritizing

- **business capabilities,**
 - **product thinking,**
 - **and developer experience**
- in API design

MCQ1

Effective Web API design incorporates three key elements

- A) business capabilities
- B) digital capabilities
- C) product thinking
- D) developer experience

Answer: A, C , D

Explanation: Digital capabilities represents the APIs itself.

MCQ2

Which of the following delivery methods involves the organization directly managing and executing processes related to its business capabilities?

- a) Outsourced delivery
- b) Combination delivery
- c) Direct delivery
- d) Partnered delivery

Correct Answer: c) Direct delivery

MCQ3

Which approach emphasizes transitioning from custom integrations to a focus on product thinking for API design?

- a) Direct integration
- b) Feedback-driven design
- c) Customization-centric approach
- d) Market-driven integration

Correct Answer: b) Feedback-driven design

API-Based Products

Resource based API design | Suresh Gopal

Learning Outcomes

- Comprehend Resource-Based API Design
- Identify Key Principles of API design with a analysis of Case Study

What is Resource-Based API Design?

An approach to API design that prioritizes

- The identification,
- Representation,
- and management of **resources**

Focus on **resources** rather than **actions**:

- Instead of focusing solely on the actions that can be performed within an application, this approach **emphasizes identifying the core entities or data objects within the system and treating them as resources.**

What is Resource-Based API Design?

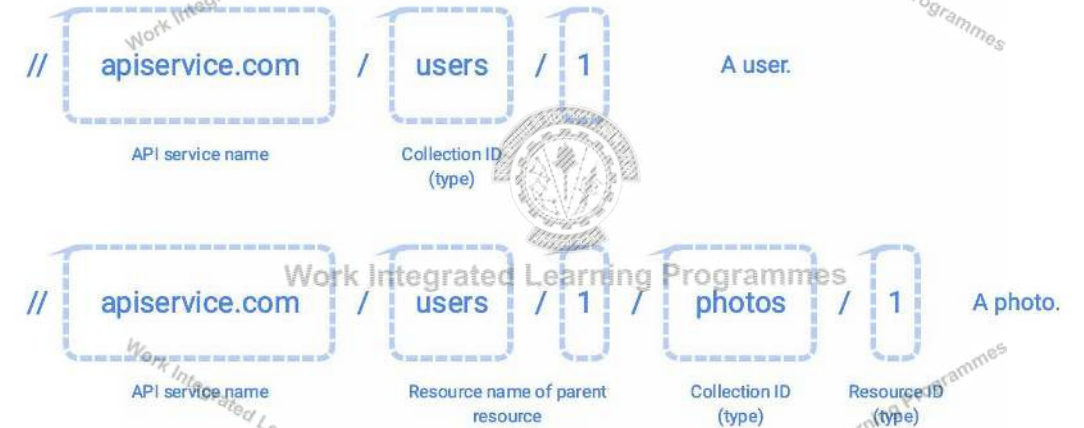
Emphasizes representing entities as resources:

- Entities within the system, such as users, products, or orders, are represented as resources,
- each with a unique identifier (URI) and a set of state-changing operations

Aligns with REST principles:

- Resource-Based API Design **closely aligns with the principles of REST**, which advocate for a stateless client-server communication model, uniform interfaces, and scalability through a layered architecture

Resource-Based API Design



Source: <https://medium.com/@ratrosy/designing-apis-4eed43409f93>

Key Principles of Resource-Based API Design

Resource Identification:

Every resource should have a unique identifier (URL) for access

Resource Representation:

Use standardized formats (JSON, XML) to represent resources

Resource State Management:

API should manage the state of resources explicitly

Key Principles of Resource-Based API Design

Uniform Interface:

Design a uniform interface for interacting with resources (HTTP methods)

Statelessness:

Each request should contain all necessary information; the server should not rely on the client's state

Layered System:

APIs should be designed in a layered architecture to promote scalability and flexibility

Benefits of Resource-Based API Design

Improved Scalability:

By focusing on resources and their interactions, Resource-Based APIs are inherently scalable

Enhanced Interoperability:

Promote interoperability by using standardized formats and protocols

Simplified Client-Server Communication:

With a clear emphasis on resources and a uniform interface, Resource-Based APIs simplify client-server communication

Better Performance Through Caching:

Leverage caching mechanisms to improve performance. By caching responses at various levels (client, server, or intermediary), redundant requests can be avoided, reducing latency and improving overall responsiveness

Case Study: Twitter API

Twitter's Approach implements Resource-Based API Design.

Twitter's API is built around the concept of **resources**, representing various entities such as **tweets**, **users**, **hashtags**, and more

follows **RESTful principles**, providing developers with a standardized and intuitive interface for interacting with Twitter's data and functionalities



Case Study: Twitter API

Key Resources

- Tweets:** Individual messages posted by users. Each tweet has a unique identifier, text content, timestamp, and metadata
- Users:** Twitter users, each with a unique identifier (user ID), username, profile information, and follower/following relationships.
- Hashtags:** Tags used to categorize and organize tweets. Hashtags start with the '#' symbol and allow users to discover related content.



Image source: <https://www.dominzylod.com/2018/12/twitter-hashtags-what-do-they-mean-and.html>

Case Study: Twitter API

HTTP Methods Interaction

- GET:**
 - Used to retrieve resource information
 - Example: `GET /tweets/{tweet_id}` retrieves details of a specific tweet
- POST:**
 - Creates a new resource or performs an action
 - Example: `POST /tweets` creates a new tweet
- PUT:**
 - Updates an existing resource
 - Example: `PUT /users/{user_id}` updates user profile information
- DELETE:**
 - Removes a resource
 - Example: `DELETE /tweets/{tweet_id}` deletes a tweet

Summary

- the fundamental concepts, principles, and best practices of designing APIs with a focus on resources
- the key principles of resource-based API design, including
 - resource identification,
 - representation,
 - state management,
 - uniform interface design,
 - statelessness,
 - layered system architecture,
 - and cacheability

MCQ1

What approach to API design emphasizes the identification, representation, and management of resources?

- a) Function-oriented design
- b) Resource-oriented design
- c) Action-driven design
- d) Protocol-oriented design

Correct Answer: b) Resource-oriented design

MCQ2

What aspect of API design emphasizes the usage of standardized formats such as JSON or XML to represent resources?

- a) Resource Identification
- b) Resource Representation
- c) Resource State Management
- d) Uniform Interface
- e) Statelessness
- f) Layered System

Correct Answer: b) Resource Representation

MCQ3

Which benefit is associated with Resource-Based APIs?

- a) Reduced latency
- b) Increased complexity
- c) Enhanced security
- d) Improved scalability

Correct Answer: d) Improved scalability

API-Based Products

API Design-first vs Code-first approach

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Overview of API Design Methodologies

API design methodologies are **frameworks or approaches used to design and develop APIs systematically.**

- Help streamline the API development process
- Ensure the creation of effective, user-friendly interfaces.

Common API Design Methodologies

- Design-First Approach
- Code-First Approach



Image source: <https://www.gravitee.io/blog/update-gravitee-api-designer> Generated: 12-May-2026 16:47:54

Learning Outcomes

- Identify and describe the two primary API design methodologies
- Evaluate the characteristics, advantages, and disadvantages of both Design-First and Code-First approaches



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

API Design-First Approach

API design-first approach is a methodology that **prioritizes the creation of API specifications before any coding begins**

Focuses on defining the API contract or specification before writing any code

- Emphasizes clear documentation and collaboration between stakeholders

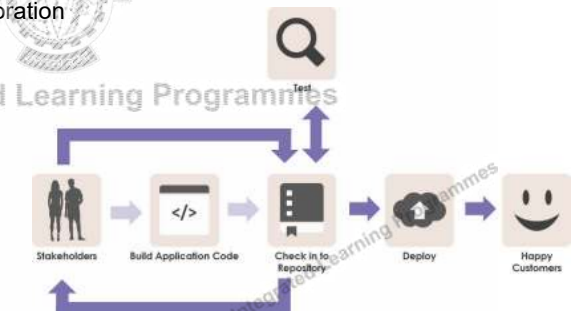


Image source: <https://www.visual-paradigm.com/guide/development/code-first-vs-design-first/> Generated: 12-May-2026 16:47:54

Design-First Approach

Key characteristics

- Focus on defining API contract/specification first
- Design-driven development process
- Utilizes tools like OpenAPI, RAML, or API Blueprint

Advantages of Design-First approach

- Promotes clear API contract definition
- Facilitates collaboration between stakeholders
- Enables early feedback and validation

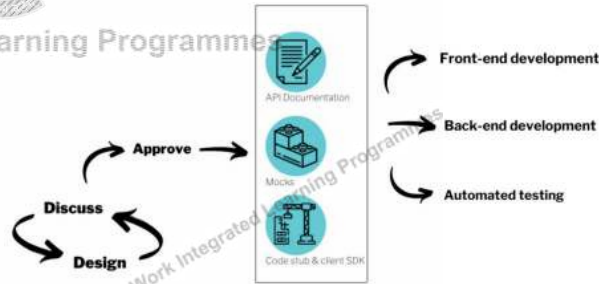


Image source: <https://www.gravitee.io/blog/update-gravitee-api-designer>

BITS WILP | Generated: 12-May-2026 16:47:54

Code-First Approach

Involves writing code to implement the API functionality, with the API contract derived from the implemented code.

- Offers **flexibility** and allows for quick prototyping.
- But may **lack in upfront clarity and documentation**.

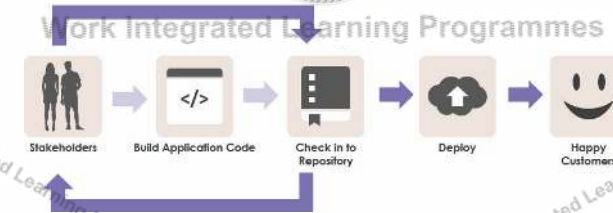


Image source: <https://www.visual-paradigm.com/guide/development/code-first-vs-design-first/>

BITS WILP | Generated: 12-May-2026 16:47:54

Code-First Approach

Key characteristics

- Development starts with writing code to implement the API functionality
- API contract is derived from the implemented code
- Often used in agile or iterative development environments

Advantages of Code-First approach

- Quick prototyping and development
- Allows for flexibility and experimentation
- Easier integration with existing codebases

API-First, Design-First and Code-First

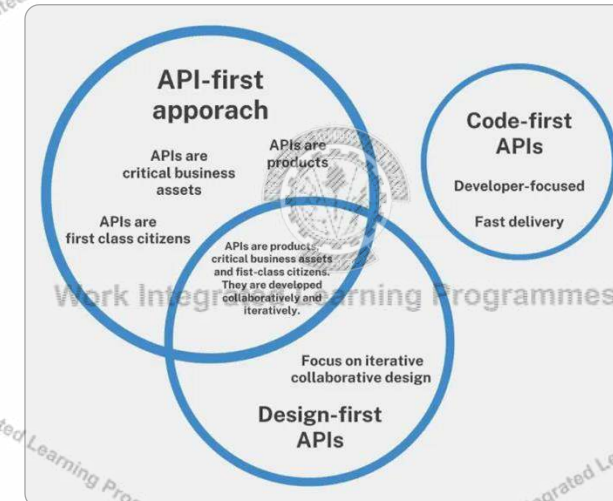


Image source: <https://blog.trafficparrot.com/2023/04/what-is-difference-between-api-first.html#gsc.tab=0>

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Design-First approach emphasizes defining API contract/specification first
- Code-First approach involves writing code to implement API functionality



Work Integrated Learning Programmes



MCQ1

What is the primary purpose of API design methodologies?

- a) To complicate the API development process
- b) To streamline the API development process
- c) To increase user frustration with interfaces
- d) To discourage systematic development

Correct Answer: b) To streamline the API development process



Work Integrated Learning Programmes



MCQ2

What is the primary focus of the API design-first approach?

- a) Writing code
- b) Testing functionality
- c) Defining API specifications
- d) Debugging errors

Correct Answer: c) Defining API specifications



Work Integrated Learning Programmes

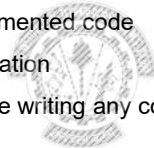


MCQ3

What is the characteristic of the code-first approach to API development?

- a) Prioritizes documentation over code implementation
- b) Derives API specifications from the implemented code
- c) Emphasizes upfront clarity and documentation
- d) Involves creating API specifications before writing any code

Correct Answer: b) Derives API specifications from the implemented code



Work Integrated Learning Programmes



API-Based Products

Importance of Collaboration in
Creating Effective APIs

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Learn the importance of Collaboration in Creating Effective APIs
- Understand the diverse roles involved in API design and development

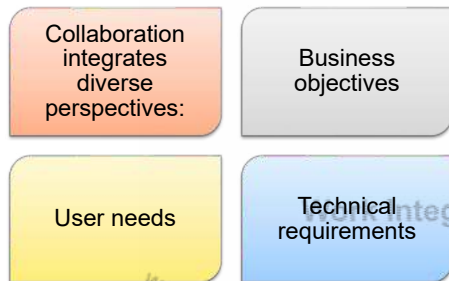


Work Integrated Learning Programmes

Importance of Collaboration in Creating Effective APIs

Effective API development requires collaboration among diverse stakeholders.

• Cross-Functional Expertise



BITS WILP | | Generated: 12-May-2026 16:47:54

Cross-Functional Expertise

Alignment with Business Goals

Collaboration ensures API design aligns with:

Strategic objectives

Revenue generation targets

Market positioning

Enhanced User Experience

Collaboration between developers, designers, and product managers:

Improves usability

Enhances developer experience (DX)

Fosters user satisfaction

BITS WILP | | Generated: 12-May-2026 16:47:54

Cross-Functional Expertise

Optimized Technical Implementation

Collaboration among architects, developers, and QA engineers:

Ensures scalable, reliable, and secure architecture

Facilitates efficient implementation

Reduces technical debt and potential issues

Importance of Collaboration

Iterative Improvement

Collaboration supports continuous iteration:

Incorporating user feedback

Addressing evolving business needs

Adapting to technological advancements

Mitigation of Risks

Collaboration identifies and addresses potential risks early:

Security vulnerabilities

Performance bottlenecks

Compatibility issues



Roles in API design

Various roles

API Product Manager

Developer Experience (DX) Designer

API Evangelist

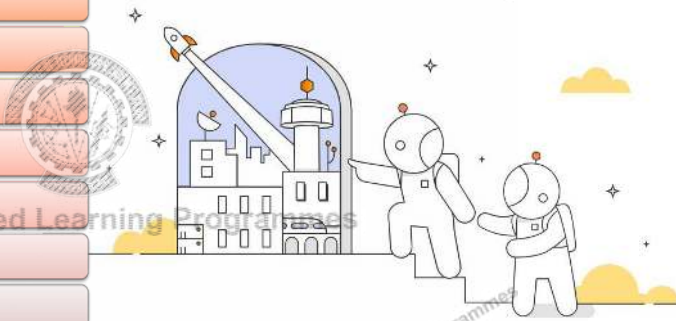
API Architect

Backend Developer

Frontend Developer

Quality Assurance (QA) Engineer

Technical Writer



Summary

- Importance of collaboration and communication in API design
- Roles involved in API design



MCQ1

What aspect of collaboration among stakeholders is crucial for effective API development?

- a) Specialized expertise
- b) Singular focus
- c) Cross-functional expertise
- d) Limited perspectives

Correct Answer: c) Cross-functional expertise

MCQ2

What is the primary benefit of collaboration among architects, developers, and QA engineers in API development?

- a) Increased technical debt
- b) Enhanced user experience
- c) Alignment with business goals
- d) Reduced collaboration

Correct Answer: c) Alignment with business goals

MCQ3

What is a key advantage of collaboration in API development?

- a) Decreased iteration
- b) Increased risks
- c) Iterative improvement
- d) Ignored feedback

Correct Answer: c) Iterative improvement

API-Based Products

Roles involved in API design –
Non-Technical

Suresh Gopal



Learning Outcome

- Understand the diverse non-technical roles involved in API design and development



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Non-Technical Roles in API design

Various roles

API Product Manager

Developer Experience (DX) Designer

API Evangelist

API Architect

Backend Developer

Frontend Developer

Quality Assurance (QA) Engineer

Technical Writer



Image source: <https://blog.openapihub.com/en-us/why-you-need-to-think-api-as-a-product-now/>,
<https://blog.postman.com/collaborate-with-postman-workspaces-and-collections/>

API Product Manager

The API Product Manager oversees the **entire lifecycle of the API product**

- act as a bridge between business objectives and technical implementation

Setting the Strategic Direction for the API

- Define the overarching **vision and goals for the API**
- Identify opportunities **for growth and innovation in the API ecosystem**

Aligning API Design with Business Goals and User Needs

- Ensure that API features and functionalities align with business objectives
- Conduct user research to understand and prioritize **user needs**

API Product Manager



Optimize developer experience

Manage the API roadmap

Prepare for API monetization

Articulate the business value of APIs

Image source: <https://www.linkedin.com/pulse/defining-api-product-manager-role-mark-o-neill>



API Product Manager

Example Tasks

Defining Feature Priorities

- Determine which features are essential for meeting user requirements and business objectives
- Prioritize feature development based on impact and feasibility**

Market Analysis

- Conduct market research to identify competitors and market trends
- Identify opportunities and gaps in the market to inform API development**

Roadmap Planning

- Develop a roadmap outlining the timeline and milestones for API development
- Align roadmap with business goals, user needs, and technical constraints**



BITS WILP | Generated: 12-May-2026 16:47:54

API Product Manager

Key Responsibilities

- Strategic planning
- User research
- Roadmap development
- Stakeholder management

Example Outcome

- A well-defined API roadmap that prioritizes features and aligns with both business goals and user needs, resulting in the successful delivery of a valuable API product



Image source: <https://cloud.google.com/files/apigee/apigee-api-monetization-ebook.pdf>

Developer Experience (DX) Designer

The Developer Experience (DX) Designer focuses on optimizing the experience developers have when using the API

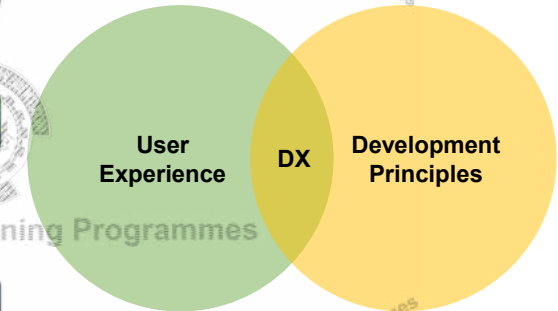
- to ensure that developers can easily understand, integrate, and work with the API

Focusing on the Usability and Developer Experience of the API

- Prioritize usability to make the API intuitive and easy to use
- Enhance developer satisfaction by streamlining integration processes and reducing friction points

Designing Intuitive Interfaces and Documentation

- Design user interfaces (UI) and application programming interfaces (API) to be intuitive and user-friendly
- Create clear and comprehensive documentation to guide developers through API usage and integration



BITS WILP | Generated: 12-May-2026 16:47:54

Developer Experience (DX) Designer

Example Tasks

Designing API Endpoints

- Define API endpoints that are intuitive and aligned with developer expectations
- Ensure consistency and coherence across API endpoints

Creating SDKs (Software Development Kits)

- Develop SDKs to simplify API integration for developers using different programming languages
- Provide code samples and libraries to facilitate rapid development

Writing Clear Documentation

- Document API usage, endpoints, parameters, and responses in a clear and concise manner
- Include tutorials, guides, and examples to help developers understand and implement the API



Image source: <https://www.thoughtworks.com/insights/blog/experience-design/approaches-for-a-better-developer-experience>

BITS WILP | Generated: 12-May-2026 16:47:54

Developer Experience (DX) Designer

Key Responsibilities

- Usability optimization
- Interface design
- Documentation creation
- SDK development

Example Outcome

- Well-designed API interfaces and documentation that enhance the developer experience, leading to increased developer adoption and satisfaction

BITS WILP | Generated: 12-May-2026 16:47:54

API Evangelist

An API Evangelist serves as a passionate advocate for the API both internally and externally

Play a pivotal role in

Promoting the API,

Building community engagement,

and driving adoption



Image source: <https://nordicapis.com/day-in-the-life-of-an-api-developer-evangelist/>

API Evangelist

Example Tasks

Community Engagement

- Actively participate in developer forums, communities, and events to raise awareness about the API
- Organize workshops, webinars, and meetups to educate developers and foster collaboration

Content Creation

- Produce compelling content, such as blog posts, articles, and videos, to showcase the value and capabilities of the API
- Share success stories, case studies, and use cases to demonstrate real-world applications of the API



Image source: <https://apievangelist.com/2014/01/31/10-api-commandments-for-providers/>

API Evangelist

Example Tasks

Developer Advocacy

- Advocate for developers' needs and feedback within the organization, influencing API design and roadmap decisions
- Provide technical support and guidance to developers, helping them overcome challenges and maximize the API's potential

Partnership Development

- Collaborate with external partners, including third-party developers, startups, and enterprises, to explore integration opportunities
- Establish strategic partnerships to expand the API ecosystem and drive innovation

API Evangelist

Example Outcome

Increased awareness and adoption of the API within the developer community

Growing engagement and collaboration through developer events, content, and partnerships

A vibrant ecosystem of third-party applications and integrations, driving business growth and innovation

Summary

- Roles involved in API design
 - API Product Manager
 - Developer Experience (DX) Designer
 - API Evangelist
- Responsibilities associated with each role



Work Integrated Learning Programmes



MCQ1

Which role is primarily responsible for ensuring that the API aligns with the organization's strategic objectives and meets the needs of its target users?

- a) API Product Manager
- b) Developer Experience (DX) Designer
- c) API Evangelist
- d) API Architect
- e) Backend Developer
- f) Frontend Developer
- g) Quality Assurance (QA) Engineer
- h) Technical Writer

Correct Answer: a) API Product Manager



Work Integrated Learning Programmes



MCQ2

Which role is primarily responsible for designing and creating the user interface that interacts with the API?

- a) API Product Manager
- b) Developer Experience (DX) Designer
- c) API Evangelist
- d) API Architect
- e) Backend Developer
- f) Frontend Developer
- g) Quality Assurance (QA) Engineer
- h) Technical Writer

Correct Answer: f) Frontend Developer



Work Integrated Learning Programmes



MCQ3

What is the primary responsibility of an API Evangelist?

- a) Writing API documentation
- b) Developing API architecture
- c) Promoting the API and driving adoption
- d) Designing the API user interface

Correct Answer: c) Promoting the API and driving adoption



Work Integrated Learning Programmes



API-Based Products

API Design-First Approach | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

API Design-First Approach

API design-first approach is a methodology that **prioritizes the creation of API specifications before any coding begins**

Shifts the focus from coding to designing the API contract, ensuring

- Clarity,
- Consistency,
- and usability from the outset

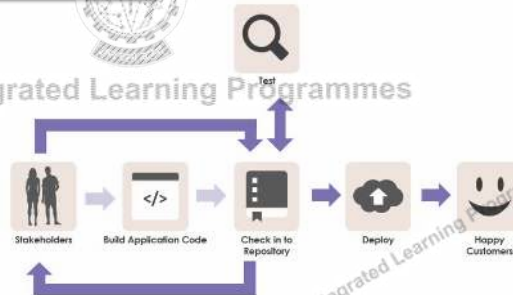


Image Source: <https://www.visual-paradigm.com/guide/development/code-first-vs-design-first/>

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Explain the importance of adopting an API design-first approach for creating scalable and discoverable APIs
- Outline the iterative phases of the API design-first approach



BITS WILP | | Generated: 12-May-2026 16:47:54

Key Elements

Focus on Defining the API Contract

- Establishes a clear understanding of what the API does, its endpoints, methods, and data formats
- Defines the expected behavior and communication protocols

Prioritize User Experience and Usability

- Designs APIs with the end-user in mind, ensuring intuitive interfaces and easy integration
- Improves developer experience by providing well-documented and predictable APIs

Enables Collaboration Between Stakeholders

- Facilitates communication and collaboration among developers, designers, product managers, and other stakeholders
- Encourages feedback loops and iterative improvements, resulting in better-designed APIs

BITS WILP | | Generated: 12-May-2026 16:47:54

Key Steps in API Design-First Approach

Define API Requirements

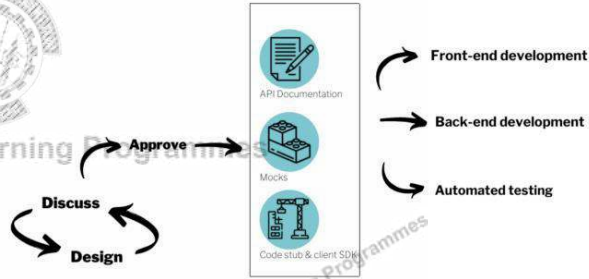
- Understand user needs and business objectives
- Define endpoints, data models, and authentication methods

Create API Specifications

- Use tools like OpenAPI Specification (OAS) or RAML
- Document endpoints, request/response payloads, and error handling

Review and Iterate

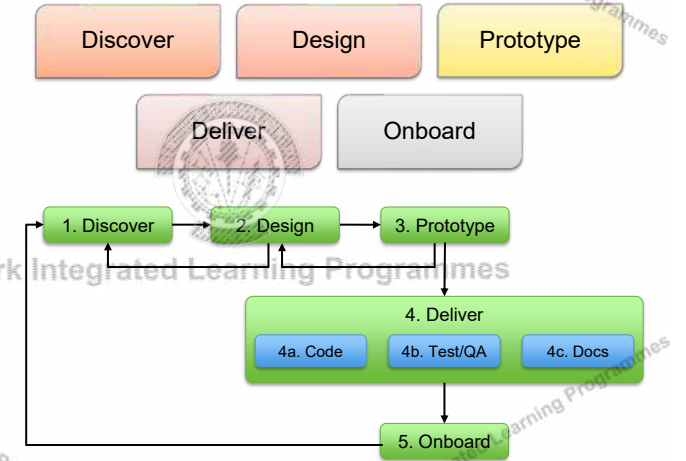
- Gather feedback from stakeholders
- Refine API specifications based on feedback



Source Image: <https://www.gravitee.io/blog/why-design-first-when-building-apis>

API Design-First Approach in Practice

The five iterative phases



The five phases of API design-first

Source: Principles of Web API design by James Higginbotham | Generated: 12-May-2026 16:47:54

The Iterative Design Process

Discover

- Identify the digital functionalities that the API must provide, while also exploring existing APIs that could fulfill these requirements.

Design

- Create an initial API design or enhance an existing API design to cater to the needed digital capabilities that are currently unavailable

Prototype

- Develop a prototype or mock API to elicit feedback from stakeholders on the current design
- Review and adjust previous steps based on the received feedback

BITS WILP | Generated: 12-May-2026 16:47:54

The Iterative Design Process

Deliver

- The API is rolled out through a collaborative effort involving developers, quality assurance, operations, and documentation teams, working in parallel
- API capabilities are released incrementally, following an iterative approach rather than a single release, guided by the mutually agreed-upon API design

Onboard

- Ensure that customers, partners, and internal developers successfully adopt the API and seamlessly integrate it into their solutions
- Providing support is essential during this phase to assist teams in addressing any complex integration requirements they may encounter

BITS WILP | Generated: 12-May-2026 16:47:54

Principle : APIs Should Never be Designed in Isolation

Effective API design requires collaboration among various stakeholders, encompassing both technical and non-technical participants throughout the design process

Restricting the API design responsibility solely to development teams significantly diminishes the likelihood of fully realizing the API's potential

Tools for API Design-First Approach

OpenAPI Specification (OAS)

- Standardized format for API specifications
- Supported by a wide range of tools and libraries

API Blueprint

- Markdown-based format for API documentation
- Focuses on simplicity and readability

RAML (RESTful API Modeling Language)

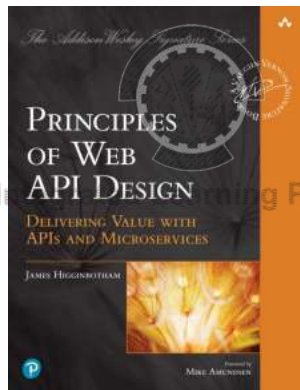
- YAML-based language for describing APIs
- Supports reusable components and libraries



Reference

Principals of Web API Design

- By James Higginbotham



Summary

- API design-first approach that prioritizes API specifications before implementation
- Key stages and tools involved in API design-first approach
- Principles and benefits of collaborative API design

MCQ1

Question: What is the primary focus of the API design-first approach methodology?

- A. Writing code to implement the API functionalities
- B. Creating API specifications before coding begins
- C. Refactoring existing code to optimize API performance
- D. Conducting user testing to refine API usability

Correct Answer: B. Creating API specifications before coding begins

MCQ2

Question: What are the key benefits of focusing on defining the API contract?

- A. Enhancing developer experience through well-documented APIs
- B. Establishing a clear understanding of API functionality, endpoints, and data formats
- C. Prioritizing user experience and usability
- D. Facilitating collaboration among stakeholders

Correct Answer: B. Establishing a clear understanding of API functionality, endpoints, and data formats

MCQ3

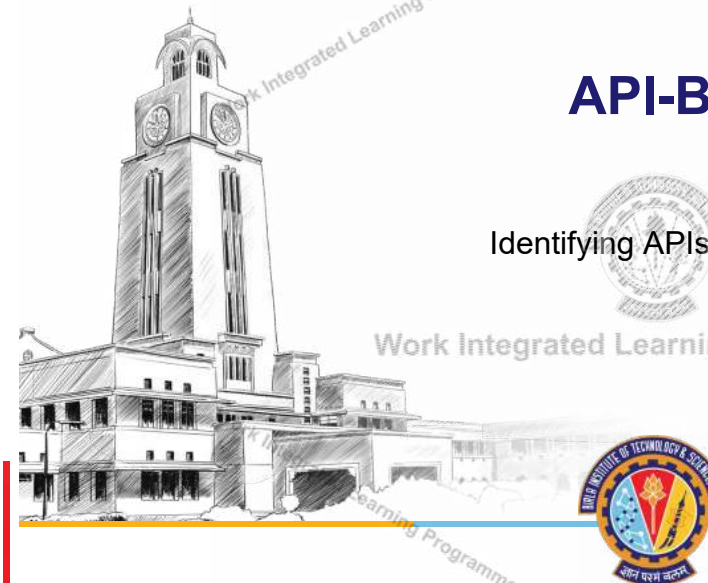
Question: What are the primary steps involved in defining API requirements?

- A. Creating API specifications and gathering feedback
- B. Understanding user needs and business objectives
- C. Reviewing and iterating on API design
- D. Implementing authentication methods and data models

Correct Answer: B. Understanding user needs and business objectives

API-Based Products

Identifying APIs Goals Suresh Gopal



Learning Outcomes

- Understand and apply the steps involved in identifying the API goals
- Introduce the concept of API Goals Canvas



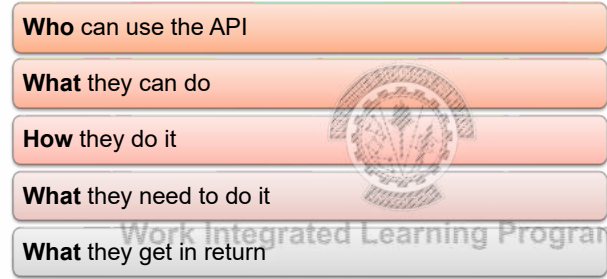
Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Designing the APIs

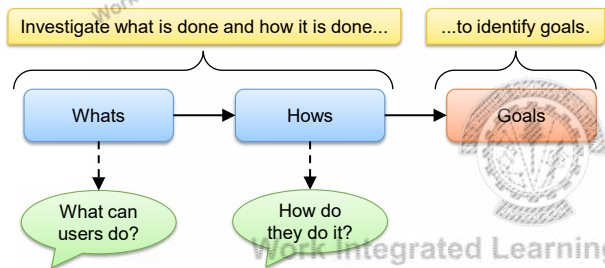
Fundamental in designing an API to have a deep, accurate, and precise knowledge of



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

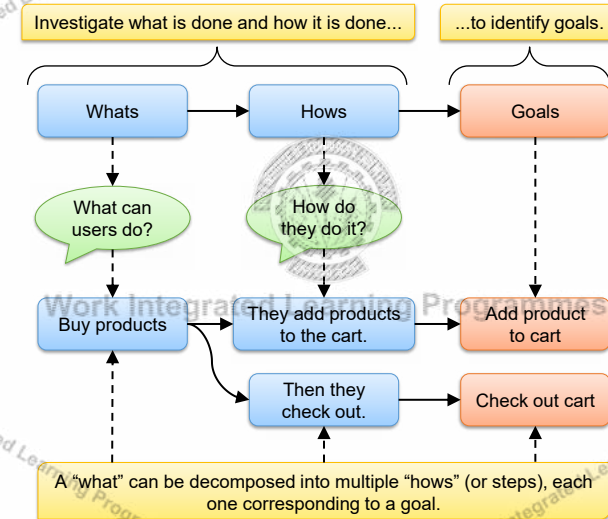
Identifying the What's and the How's



Adapted from : Design of Web APIs



Identifying the What's and the How's



A "what" can be decomposed into multiple "hows" (or steps), each one corresponding to a goal.

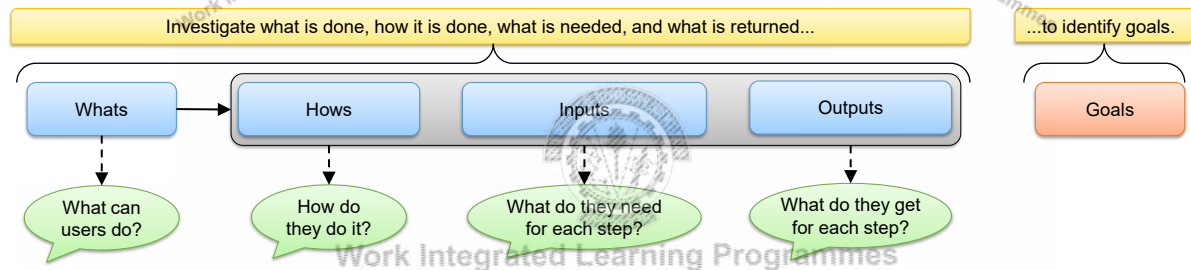
The Shopping API whats and hows

BITS WILP | Generated: 12-May-2026 16:47:54

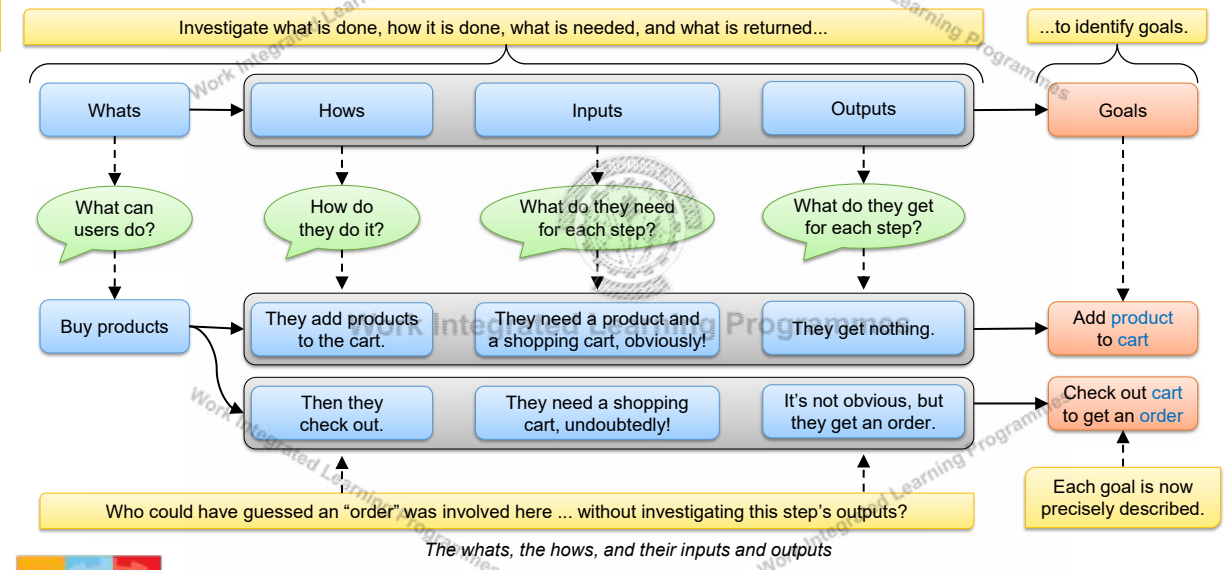
Adapted from : Design of Web APIs



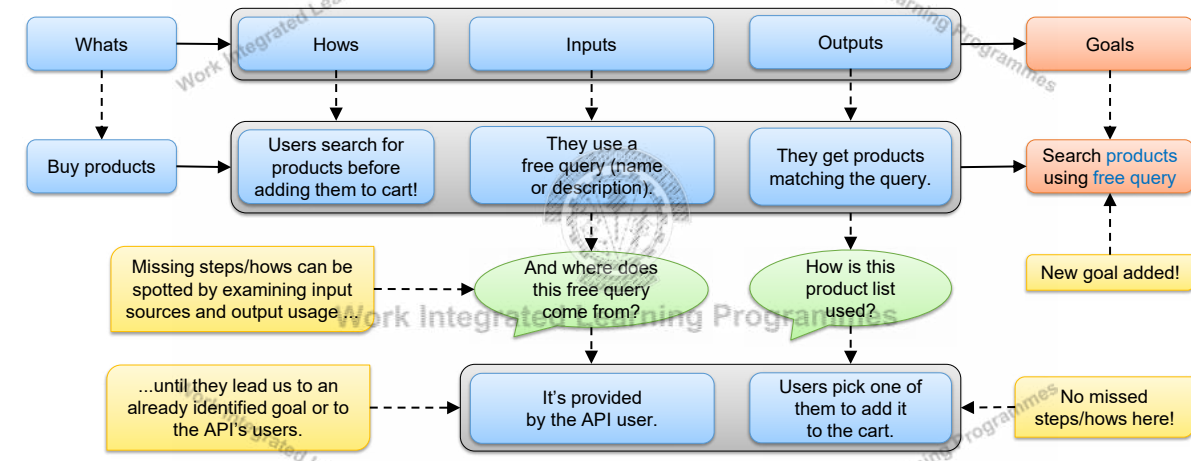
Identifying Inputs and Outputs



Identifying Inputs and Outputs

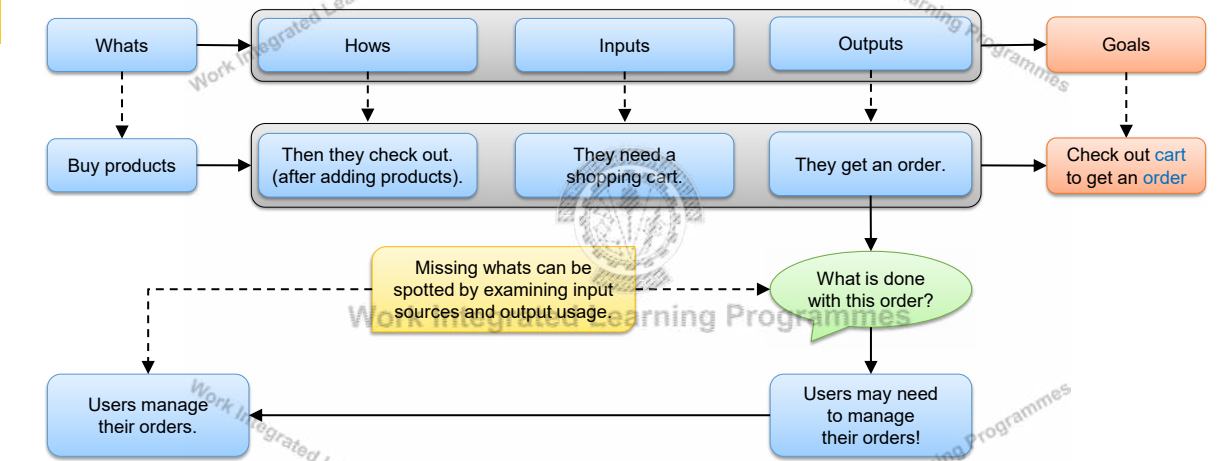


Identifying Missing Goals



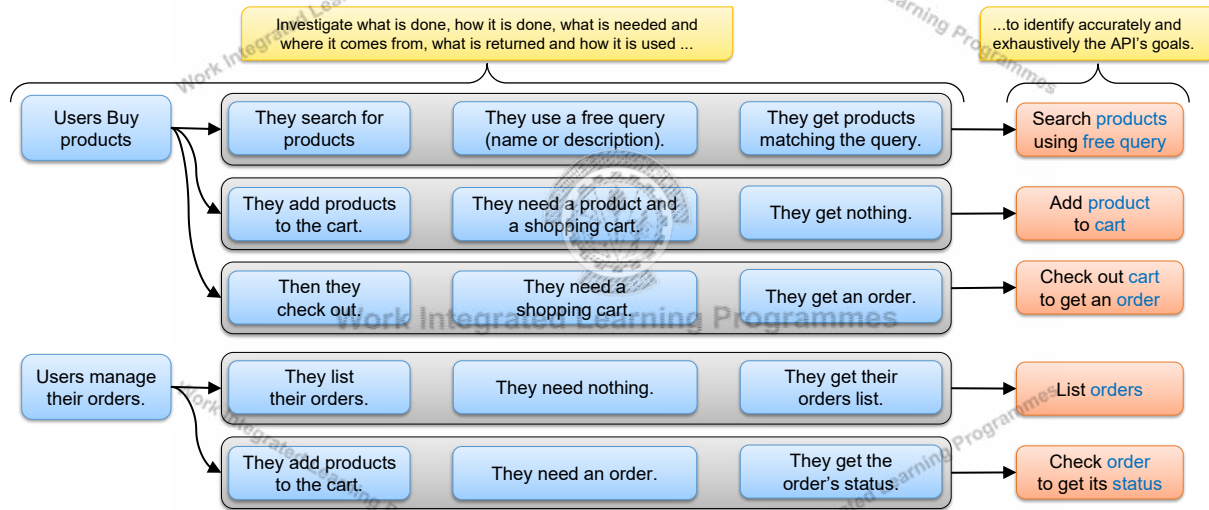
Adding a missing how

Identifying Missing Goals



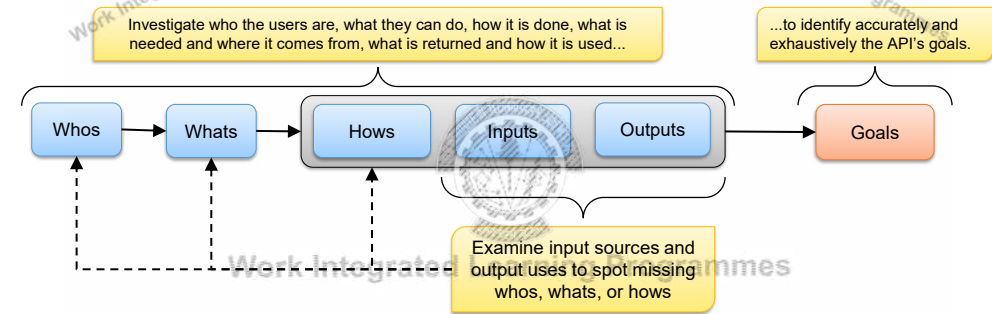
Adding a missing what

Identifying Missing Goals



Shopping API goals list with the new user's manage their orders what

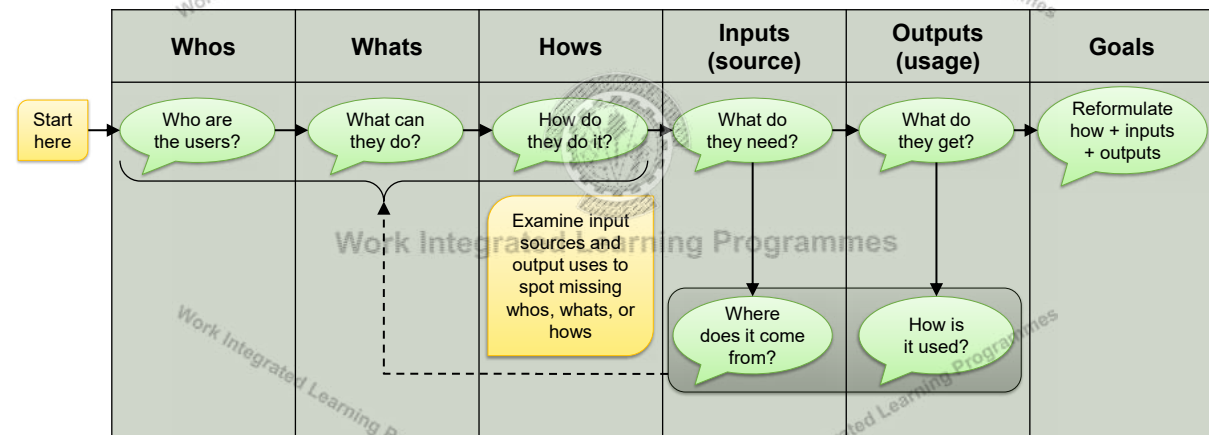
Identifying All Users



Investigating the whos, whats, hows, inputs, and outputs to identify goals

Using the API Goals Canvas

Draw this table on a whiteboard, flipchart, sheet of paper, or spreadsheet and start the questioning.



The API goals canvas

The Shopping API Goals Canvas (Partial View)

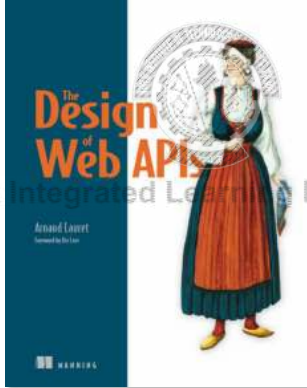
Identifying an API's goals

Whos	Whats	Hows	Inputs (source)	Outputs (usage)	Goals
Customers	Buy products	Search for products	Catalog (manage catalog), free query (provided by user)	Products (add product to cart)	Search for products in catalog using free query
		Add product to cart	Product (search for products), cart (owned by user)		Add product to cart
Admin	Manage catalog	Add product to catalog	Catalog (owned by user), product (provided by user)		Add product to catalog

Reference

The Design of Web APIs

- By Arnaud Laurent



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- The API goals canvas and how to design it



Work Integrated Learning Programmes

MCQ1

Question: What fundamental aspects should be considered when designing an API for an online shopping website or mobile application?

- a) Knowing only who can use the API
- b) Understanding what users can do with the API
- c) Having a deep, accurate, and precise knowledge of who can use the API, what they can do, how they do it, what they need to do it, and what they get in return
- d) Focusing solely on the technical implementation details

Correct Answer: c) Having a deep, accurate, and precise knowledge of who can use the API, what they can do, how they do it, what they need to do it, and what they get in return

MCQ2

Question: Which component of the API goals canvas involves decomposing each action into steps?

- a) Who's
- b) What's
- c) How's
- d) Inputs (source)
- e) Output (usage)
- f) Goals

Correct Answer: c) How's

MCQ3

Question: What are the components of the API goals canvas?

- a) Who's, What's, How's, Inputs, and Outputs
- b) Who's, What's, How's, Inputs (source), Output (usage), and Goals
- c) Users, Actions, Steps, Inputs, and Outputs
- d) Users, Objectives, Procedures, Requirements, and Results

Correct Answer: b) Who's, What's, How's, Inputs (source), Output (usage), and Goals

API-Based Products

API Design Anti-Patterns | Suresh Gopal

Learning Outcomes

- Develop Understanding about API Design Antipatterns
 - Leaky Abstraction
 - Next Release Design Fix
 - Unused API

Adopting an Effective API Design Process

Streamlines Development

Facilitates a structured approach from conception to implementation, enhancing efficiency and reducing development time

Ensures Consistency

Promotes uniformity in API design across projects, simplifying maintenance and enhancing interoperability

Enhances User Experience

Focuses on user-centric design, resulting in APIs that are intuitive, easy to use, and meet consumer needs effectively

Consequences of Failing to Adopt Effective API Design Process

Increased Development Costs

Lack of a structured process leads to inefficiencies, rework, and increased development costs

Poor Developer Experience

Inconsistent design and lack of clarity make APIs difficult to understand and utilize, leading to frustration among developers

Compatibility Issues

Inadequate design may result in compatibility issues with existing systems or future updates, hindering integration and scalability

BITS WILP | Generated: 12-May-2026 16:47:54

Leaky Abstraction Antipattern

Definition

Refers to a situation where the internal complexities of a system "leak" into its external interface, violating the principle of abstraction

Consequences

Results in consumers needing to understand and deal with the internal workings of the system, which should ideally be abstracted away

Work Integrated Learning Programmes

Leaky Abstraction Antipattern

Occurrence

Starting with Code, Not Design

Without a formal API design process, developers may begin by implementing code before fully designing the API interface

Incorporating Internal Technology Decisions

Lack of a structured process may lead to API designs that reflect internal technology decisions, such as database choices or cloud vendor specifics

Example

A public API for a recommendation engine may require users to understand the intricacies of Apache Lucene, a specific technology used internally

Requirement stems from a lack of abstraction in the API design process

BITS WILP | Generated: 12-May-2026 16:47:54

Next Release Design Fix Antipattern

Definition

Refers to the practice of deferring necessary design improvements to future releases rather than addressing them in the current development cycle

Context

Often arises due to the complexity of API design decisions and challenges in accommodating changes within the current release timeline

BITS WILP | Generated: 12-May-2026 16:47:54

Next Release Design Fix Antipattern

Inadequate API Design Decisions

Design decisions made without sufficient consideration of future implications or scalability may necessitate fixes in subsequent releases

Complexity and Technical Debt

The complexity of underlying code changes or technical debt accumulation may lead to challenges in implementing design improvements within the current release

Impact on Development Cycle

Results in pushing an inadequate API design into production, leading to increased maintenance efforts and the risk of compatibility issues

BITS WILP | | Generated: 12-May-2026 16:47:54

Unused API Antipattern

Definition

Refers to the situation where APIs are released but remain unused or underutilized by the intended audience

Context

Often arises when API designs fail to align with the goals, needs, and desires of the target audience, resulting in low adoption rates and limited integration

BITS WILP | | Generated: 12-May-2026 16:47:54

Unused API Antipattern

Common Scenario of Underutilization

Limited Integrations

Despite being released to great fanfare, APIs may struggle to attract integrations due to their failure to address critical use cases or provide significant value

Encounter Bugs

When integrations are attempted, encountered bugs due to design brittleness or implementation flaws further deter adoption

Resulting Consequences

APIs languish with few, if any, integrations, undermining the project's success and potentially damaging the organization's reputation

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- The importance of avoiding API design antipatterns
- The benefits of implementing an effective API design process

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: What are the primary benefits of following a structured approach in API design?

- a) Increased development time and complexity
- b) Reduced efficiency and inconsistency in design
- c) Streamlined development, enhanced efficiency, and reduced development time
- d) Inconsistency in design and decreased interoperability

Correct Answer: c) Streamlined development, enhanced efficiency, and reduced development time

MCQ2

Question: What does the term "leaky abstraction pattern" refer to in software engineering?

- a) It describes a situation where a system's interface is impenetrable, preventing any external interactions
- b) It refers to the seamless integration of external systems with minimal interference
- c) It indicates a situation where the internal complexities of a system are exposed through its external interface, violating the principle of abstraction
- d) It represents a scenario where the internal workings of a system are completely hidden from external users

Correct Answer: c) It indicates a situation where the internal complexities of a system are exposed through its external interface, violating the principle of abstraction

MCQ3

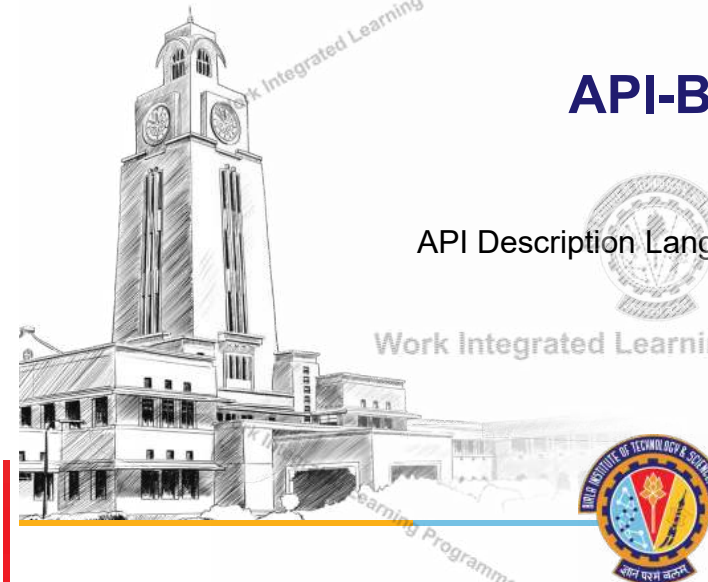
Question: What does the term "unused API pattern" refer to in software development?

- a) It describes an API that is used excessively by its intended audience
- b) It refers to the situation where APIs are released but remain unused or underutilized by the intended audience
- c) It indicates an API that is not implemented properly and leads to system failures
- d) It represents an API that is used in a way that deviates from its intended purpose

Correct Answer: b) It refers to the situation where APIs are released but remain unused or underutilized by the intended audience

API-Based Products

API Description Languages | Suresh Gopal



Learning Outcomes

- Develop familiarity of API Description Languages
- Differentiate between API Description Languages and API Development Languages



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54



API Description Languages

APIs need to be **described** in some form

- Especially when it needs to be **communicated** among various stakeholders
- should be as **easy** as possible to describe APIs

API description languages

- **Specialized languages** can support the crafting of useful API descriptions by providing appropriate abstractions and language concepts

Original purpose was a language for creating API documentation

- Similar as JavaDoc provides a language for documenting Java programs

Description Languages - Today

Today, API description languages can be used for **many additional purposes** during the design and development process of APIs

- Are **machine readable specifications** of the API
- Can be used for **automating tasks** in API development
- Can also be used for the **partially automated generation of code for the API**



Swagger



apibuildprint

RAML

BITS WILP | | Generated: 12-May-2026 16:47:54



What are API Description Languages?

Domain specific languages, which are especially suited for describing APIs

- Both **human readable** and **machine readable languages**, much like programming languages

Are intuitive languages that **can be easily written, read and understood by API developers and API designers** alike

- Are also precise, leave little room for ambiguity and are very expressive and powerful
- Have a well-defined syntax, which makes it possible to process them automatically by software



BITS WILP | | Generated: 12-May-2026 16:47:54

What are API Description Languages?

Compared to programming languages or API implementation languages

- Use a higher level of abstraction and a declarative paradigm
- Can be used to express the "what" instead of the "how"

For example,

- Define the data structure of the possible responses (the "what")
- Instead of describing how the response is computed (the "how")
- Very well suited for expressing the architecture of each API proxy and the design of the API portfolio as a whole

BITS WILP | | Generated: 12-May-2026 16:47:54

API Description Language vs. API Development Language

An API platform should provide two types of languages

First language that should be provided is a higher-level language - API description language

- Can be used for designing APIs and for expressing the "what"

Second language that should be provided by an API platform is a lower-level language - API development language

- Used for implementing APIs and for expressing the "how".

BITS WILP | | Generated: 12-May-2026 16:47:54

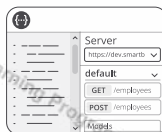
API Description Language vs. API Development Language

Relation between API description languages and API development languages

First, APIs should be designed using an API description language

- Some API platforms have support for API description languages built in
- Means that the platform supports the parsers, and code generators for a specific API description language

The generated implementations should be expressed in the API development language



Design



Build



Document

Image source: <https://swagger.io/tools/open-source/>

BITS WILP | | Generated: 12-May-2026 16:47:54

Language Choices

The most popular and widespread API description languages

- Swagger, RAML, Blueprint, Mashery I/O Docs, WADL and WSDL

WADL and WSDL

- Have been around for a long time, their use is widespread, they can be used for API design,
- However, they have not been created specifically for API design and thus are lacking important features

Swagger (OpenAPI), RAML, API Blueprint and Mashery I/O Docs

- Have been created specifically for RESTful API design

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- API Description Languages and its necessity
- Difference between description language and development language
- Choices available for API description languages



Work Integrated Learning Programmes



MCQ1

What is the purpose of an API description language?

- a) To write code for implementing APIs
- b) To document and define the structure, functionalities, and endpoints of an API
- c) To test the performance of APIs
- d) To secure APIs from unauthorized access



Answer: b) To document and define the structure, functionalities, and endpoints of an API

MCQ2

Which of the following is not an API description language?

- a) YAML
- b) JSON
- c) XML
- d) SQL



Work Integrated Learning Programmes



MCQ3

Which API description language is often used for describing RESTful APIs?

- a) WSDL
- b) Swagger (OpenAPI)
- c) SOAP
- d) GraphQL



Answer: b) Swagger (OpenAPI)

API-Based Products

The Align-Define-Design-Refine
Process

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Requirements to Deliverable

Converting varied business requirements captured in

- Use cases,
- Spreadsheets,
- Wireframes

Into an API design poses a major challenge for API design teams

One critical challenge is ensuring alignment of scope and deliverables among both technical and non-technical team members



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Develop understanding about API-design first approach named A-D-D-R process



Work Integrated Learning Programmes

A-D-D-R Process

Align-Define-Design-Refine (ADDR) process guides teams through an API design-first approach

Groups step-by-step into four distinct phases:

Align: Ensures a shared understanding and agreement on objectives across business, product, and technology domains to achieve desired outcomes

Define: Translates business and customer needs into digital functionalities, serving as the foundation for one or more APIs to accomplish desired outcomes

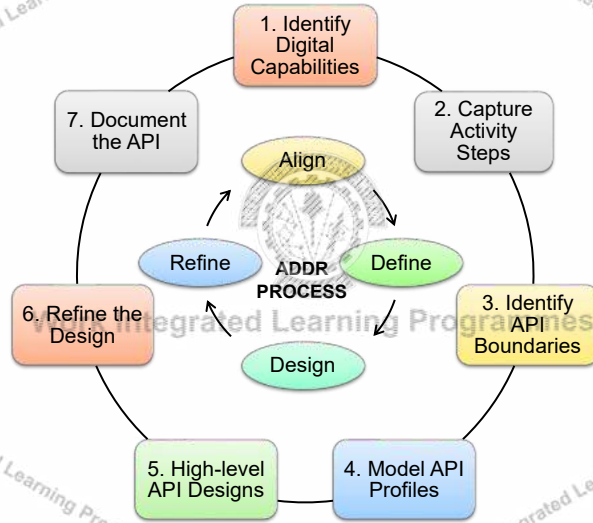
Design: Utilizes tailored design methodologies for each API, incorporating various API styles to fulfill the desired objectives effectively

Refine: Improves the API design iteratively by incorporating feedback from developers, alongside documentation, prototyping, and testing endeavors



BITS WILP | | Generated: 12-May-2026 16:47:54

Process Steps



Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Seven Steps Across the Phases

Identify digital capabilities:

Identify the customer needs and desired outcomes, including the corresponding digital capabilities that are required

Capture activity steps:

Expand the digital capabilities to include a unified understanding and clarity through collaborative API design sessions

Identify API boundaries:

Group the digital capabilities into API boundaries and determine whether the APIs already exist or new APIs are required

Seven Steps Across the Phases

Model API profiles:

Use a collaborative API modeling session to define high-level API design, including resources and operations into an API profile.

High-level API designs:

Select one or more API styles that each API profile will offer and document the high-level design elements.

Refine the design:

Incorporate design feedback from API consumers using techniques that encourage improvement in the developer experience.

Document the API:

Complete the API documentation, including reference documentation and getting started guides, to accelerate integration.

Process Objectives

Produce an API design that addresses and resolves customer issues using terminology they comprehend.

Minimize the frequent redesigning often associated with informal design approaches.

Streamline the entire organizational structure, ensuring that API design and delivery are optimized across all departments, not solely focused on developers.

Streamline the process by eliminating unnecessary steps wherever feasible to expedite delivery.

Process Objectives

Establish a replicable methodology that facilitates the creation of API designs involving a blend of technical and non-technical roles, some of whom may not possess a deep understanding of API intricacies but can contribute valuable insights.

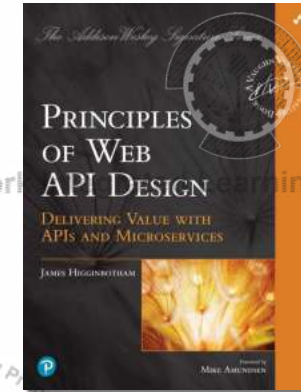
Generate artifacts that can be utilized for reference within the team and shared throughout the organization, rather than relying on rudimentary whiteboard sketches that fail to convey the rationale and intent behind the resultant API design.

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- A-D-D-R Process
 - i.e. Align – Define – Design – Refine process
- Seven steps in A-D-D-R Process
 - Identify digital capabilities
 - Capture activity steps
 - Identify API boundaries
 - Model API profiles
 - High-level API designs
 - Refine the design
 - Document the API
- Objectives associated with process

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: In the ADDR process, which phase translates business and customer needs into digital functionalities?

- a) Align
- b) Define
- c) Design
- d) Refine

Correct Answer: b) Define

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Question: During which phase of the ADDR process is tailored design methodologies utilized for each API?

- a) Align
- b) Define
- c) Design
- d) Refine

Correct Answer: c) Design

MCQ3

Question: What activity is primarily undertaken in the Refine phase of the ADDR process?

- a) Translating business needs into digital functionalities
- b) Utilizing tailored design methodologies for API creation
- c) Improving the API design iteratively based on developer feedback
- d) Ensuring a shared understanding and agreement on objectives

Correct Answer: c) Improving the API design iteratively based on developer feedback

API-Based Products

Identify Digital Capabilities | Suresh Gopal

Learning Outcomes

- Introduces the concepts of digital capabilities
 - Explains how they relate to APIs
- Outlines an approachable method for mapping requirements into a format that identifies necessary digital capabilities

Aligning API outcomes

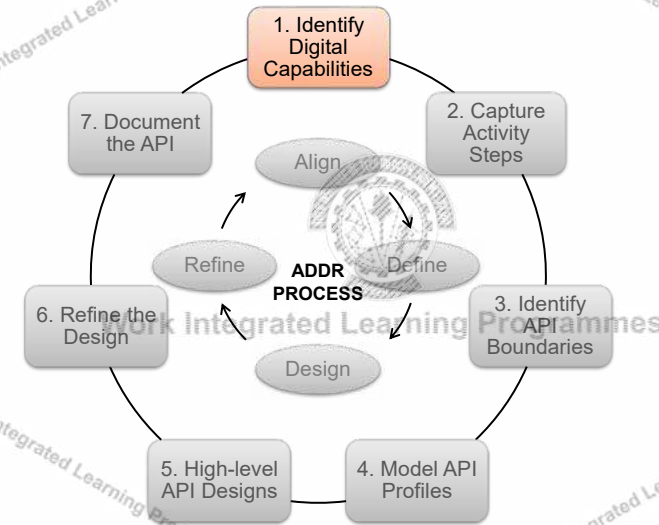
In designing an API, teams often struggle to bridge the gap between business requirements and the actual API design

- Aim to ensure that the API they create fulfills stakeholder needs and that both business and technical teams are on the same page to avoid last-minute revisions

Align phase of the ADDR process addresses these challenges

- Consists of
 - Identify Digital Capabilities
 - Capture Activities and Steps

Step 1 - Identify Digital Capabilities



The Align phase begins with identifying digital capabilities.

Adapted From: Principles of Web API Design

Digital Capabilities

Business capabilities:

Enablers an organization brings to market

Examples:

- Consumer product design,
- Product manufacturing,
- Customer support

Digital Capabilities

Digital capabilities:

Assets that actualize desired outcomes through automation

Allow interaction with the organization digitally

Examples:

- REST APIs,
- Webhook-based asynchronous APIs,
- SOAP services,
- message streams

Relationship

Business capabilities defined by business architects, associated with KPIs or OKRs

Digital capabilities focus on producing outcomes and delivering business capabilities

Business capabilities define the "what," while digital capabilities define the "how."

Example

Digital Capabilities Realized by a REST-Based Project Management API

Digital Capability	Example REST-Based API Design
Manage project from start to finish	POST /projects
Add collaborators to a project	POST /projects/{projectId}/collaborators
Subdivide a project into issues	POST /issues
Mark issue complete	POST /issues/{issueId}/completed
View incomplete issues	GET /issues?status=incomplete
View active projects	GET /projects?status=active

Jobs To Be Done (JTBD)

Identified needs fulfilled by a product or service

- Includes customer problem, task, and desired outcome

Introduced by Clayton Christensen to focus on customer viewpoint in product design

Ensures product addresses specific needs for better market adoption

Begins with identifying customer needs and defining how product fills those needs

JTBD applies to APIs and all aspects of product and software design

Job Stories

Capture jobs to be done for any product, including customer motivations, events, and expectations

Frame design problems from the customer's viewpoint

- Identify customer problems and desired outcomes

APIs provide digital capabilities to fulfill these jobs and achieve desired outcomes

- Focus is on outcomes rather than technical intricacies

Job stories help capture business requirements in a customer-centric manner

The Components of a Job Story

Consist of three elements structured in the "When, I want to, so I can" format:

When: establishes causality by pinpointing the triggering event or situation prompting the customer's desire for a specific outcome

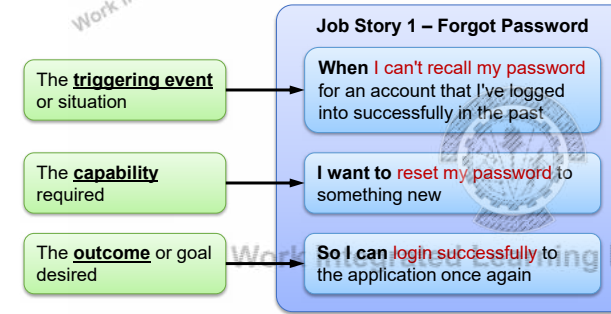
I want to: represents the action identified by the customer as necessary to achieve their goal

- Defines the essential role of the API in delivering the desired outcome and aids in dissecting the operations it will perform

So I can: represents the desired end state resulting from applying the capability when the triggering event occurs

- Outcome sets the criteria for accepting the API design

Example Job Story

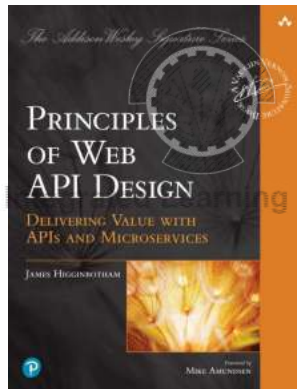


A job story and its three components.

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- Importance of aligning requirements to the deliverables
- Difference between business capabilities and digital capabilities
- How Align phase focus identifying digital capabilities
- How job stories helps in discovering and capturing the digital capabilities

MCQ1

Question: What do business capabilities and digital capabilities represent in an organization?

- a) Business capabilities focus on technological assets, while digital capabilities encompass market enablers
- b) Business capabilities are assets that actualize desired outcomes through automation, while digital capabilities represent enablers an organization brings to market
- c) Business capabilities are assets for customer support, while digital capabilities focus on product manufacturing
- d) Business capabilities encompass consumer product design, while digital capabilities involve message streams

Correct Answer: b) Business capabilities are assets that actualize desired outcomes through automation, while digital capabilities represent enablers an organization brings to market

MCQ2

Question: What is the primary focus when capturing jobs to be done for a product and framing design problems?

- a) Technical intricacies
- b) Customer motivations and expectations
- c) Identifying software bugs
- d) Developing complex algorithms

Correct Answer: b) Customer motivations and expectations

MCQ3

Question: What are the three elements structured in the "When, I want to, so I can" format used in API design?

- a) Cause, Action, Result
- b) Trigger, Action, Outcome
- c) Event, Task, Purpose
- d) When, I want to, So I can

Correct Answer: d) When, I want to, So I can

API-Based Products

Job Stories

Suresh Gopal

Learning Outcomes

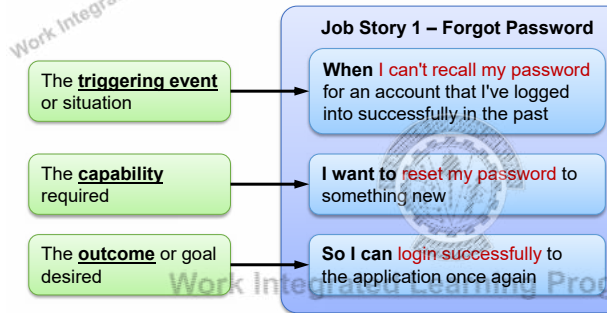
- Outline an approachable method for mapping requirements into a format that identifies necessary digital capabilities



Work Integrated Learning Programmes



Example Job Story



A job story and its three components.

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Writing Job Stories for APIs

Method 1: When the Problem Is Known

Method 2: When the Desired Outcome Is Known

Method 3: When the Digital Capability Has Been Identified



Method 1: When the Problem Is Known

Common approach when customers can identify problems needing resolution

Questions to explore:

- What is the desired outcome to solve the problem?
- What job is required to achieve this outcome?
- Does the original problem description best fit as the triggering situation, or is there a better way to express it?



BITS WILP | Generated: 12-May-2026 16:47:54

BITS WILP | Generated: 12-May-2026 16:47:54

Method 2: When the Desired Outcome Is Known



Occurs when desired outcomes are clear but triggering situations are not

Guiding questions:

- What problem, as described by the customer, leads to the desired outcome?
- What job is necessary to achieve this outcome? Condense multiple tasks into one job.
- Should the desired outcome be rewritten to better express the need?

Method 3: When the Digital Capability Has Been Identified



Happens when customers pinpoint desired digital capabilities

Validation questions:

- What desired outcome does the customer wish to experience?
- What problem or triggering situation demands this outcome, as described by the customer or stakeholder?
- Does the identified digital capability contribute to the desired outcome? If not, how can it be reworded or better suited to solve the problem?

Techniques for Capturing Job Stories



Spreadsheets:

Universal tool for capturing job stories

One job story per row, with columns for components: "When," "I want to," "so I can."

Include a column for notes

Supports collaborative editing, facilitating review and contribution by multiple team members

Techniques for Capturing Job Stories



Documents:

Useful for mimicking index card style

Start with a heading indicating job story identifier

Separate each component ("When," "I want to," "so I can") on separate lines for readability

Leave space for additional insights or details as bulleted items

Separate each job story with a blank space or assign one per page

Techniques for Capturing Job Stories

Markdown files:

Text files with approachable syntax for capturing job stories

Can be used to export job stories into HTML, PDF, and other formats

Combine with version control system (e.g., git) for viewing changes history

Suited for teams with deeper technical expertise

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

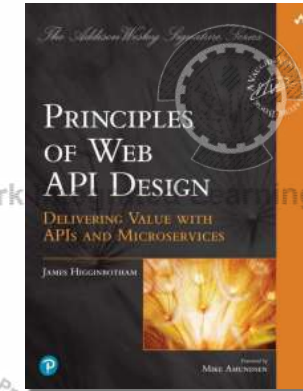
- How job stories helps in discovering and capturing the digital capabilities
- Writing the job stories
- Techniques used to capture job stories

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: What is a common approach when customers can identify problems needing resolution?

- a) Ignoring the problem until it escalates
- b) Asking customers to solve the problem themselves
- c) Proactively exploring questions to understand the desired outcome and required actions
- d) Implementing solutions without customer input

Correct Answer: c) Proactively exploring questions to understand the desired outcome and required actions

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Question: What does it mean when desired outcomes are clear but triggering situations are not?

- a) The customer is not interested in resolving the problem
- b) The problem description is insufficient to understand the desired outcome
- c) The desired outcome is well-defined, but it's unclear what events or situations prompt the need for it
- d) There are no clear solutions available to achieve the desired outcome

Correct Answer: c) The desired outcome is well-defined, but it's unclear what events or situations prompt the need for it

MCQ3

Question: What typically happens when customers pinpoint desired digital capabilities?

- a) The capabilities are ignored by the development team
- b) The capabilities are implemented immediately without validation
- c) Validation questions are asked to understand the desired outcome and its alignment with the problem or triggering situation
- d) The development team proceeds with designing the solution without further investigation

Correct Answer: c) Validation questions are asked to understand the desired outcome and its alignment with the problem or triggering situation

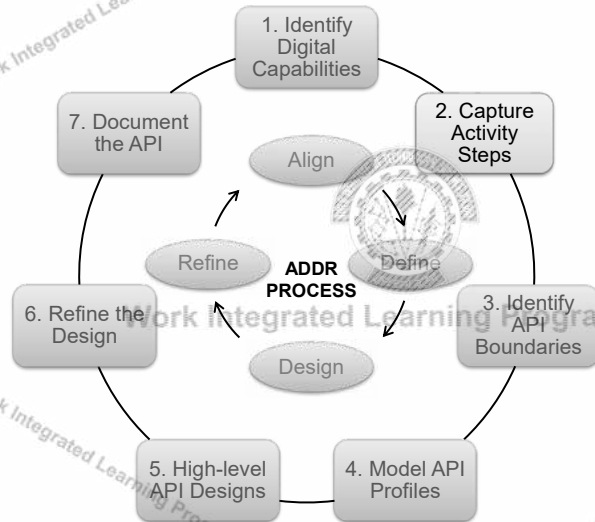
API-Based Products

Capture Activities and Steps | Suresh Gopal

Learning Outcomes

- Learn about the next step in Align phase of A-D-D-R process
- Understand about activity and activity steps
- Learn steps to decompose activity into activity steps

Align: Capture activities and steps



The next step in the Align phase is to capture activity steps.

Adapted From: Principles of Web API Design | BITS WILP | Generated: 12-May-2026 16:47:54

What Next?

First step i.e. "Identify Digital Capabilities" helped in understanding desired outcomes in API design process

- Next step: Gathering details from stakeholders, development teams, and business domain experts

Capturing domain details and expected behavior using activity-based structure

Result:

- Deeper understanding of domain
- Alignment among team members
- Foundation for defining and designing APIs to deliver necessary digital capabilities

Extending Job Stories into Activities and Steps

Creating **job stories** helps identify the desired **outcomes** along with the digital capabilities necessary to produce the outcomes

- Next step is to **detail the digital capabilities as the activities and activity steps** required to achieve these outcomes

Activity

- **Work that contributes toward a desired outcome**
- May be performed by only one participant or by a combination of multiple participants

Activity steps

- **Decompose activities into individual tasks that need to be performed to complete the activity**
- Once all necessary activities are completed, the job story outcome will be met

Two Step Process

Two quick steps to capture details:

- **Identify the activities for each job story,**
- **Decompose each activity into individual steps**

Results are used to identify API boundaries

All team members gain deeper understanding and alignment on the solution

- **If requirements are vague or uncertainty remains,** the team may choose to proceed into a collaborative **EventStorming** session to explore the solution further.

Identify the Activities for Each Job Story

Goal: Find bigger units of work contributing to the outcome

- Start by identifying activities for each job story that produce the desired outcome

Activities are high level and may require one or more steps to accomplish

Example Activities for JSON's Bookstore Place an Order Job Story

Digital Capability	Activity	Participants	Description
Place an Order	Browse for Books	Customer	Browse or search for books
Place an Order	Shop for Books	Customer, Call Center	A customer adds books to a cart
Place an Order	Create an Order	Customer, Call Center	A customer places the order using the contents of the shopping cart

BITS WILP | Generated: 12-May-2026 16:47:54

Decompose Each Activity into Steps

Activities consist of steps, each captured with a level of detail ensuring execution by one participant at a time

- If a step necessitates simultaneous execution by multiple participants, further decompose it into smaller, independent steps for each participant

Some activities may have only a single step, whereas others may have multiple steps

- As some activities are more complex than others

Decompose Each Activity into Steps

Deconstructing activities into individual steps requires a deep understanding of how the API addresses real-world issues, often requiring insights from domain or subject matter experts (SMEs)

Involve SMEs in the process of defining activities and steps

In case an SME is not accessible, dedicate time to interview both SMEs and customers for better comprehension of needs

Allow ample time for research to address all inquiries, eliminating assumptions about the problem domain

When feasible, entrust the product manager with conducting interviews

BITS WILP | Generated: 12-May-2026 16:47:54

Example

Example Activity Steps for JSON's Bookstore

Digital Capability	Activity	Activity Step	Participants	Description
Place an Order	Browse for Books	List Books	Customer, Call Center	List books by category or release date
Place an Order	Browse for Books	Search for Books	Customer, Call Center	Search for books by author, title
Place an Order	Browse for Books	View Book Details	Customer, Call Center	View the details of a book
Place an Order	Shop for Books	Add Books to Cart	Customer, Call Center	Add a book to the customer's cart
Place an Order	Shop for Books	Remove Books from Cart	Customer, Call Center	Remove a book from the customer's cart
Place an Order	Shop for Books	Clear Cart	Customer, Call Center	Remove all books from the customer's cart
Place an Order	Shop for Books	View Cart	Customer, Call Center	View the current cart and total
Place an Order	Create an Order	Checkout	Customer, Call Center	Create an order from the contents of the cart
Place an Order	Create an Order	Pay for Order	Customer, Call Center	Accept and process payment for the order

BITS WILP | Generated: 12-May-2026 16:47:54

What If Requirements Aren't Clear?

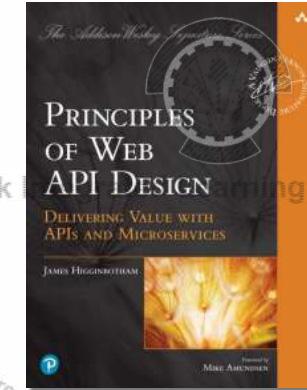
In cases where the team lacks familiarity with certain domains, it might be essential to delve deeper into the problem space before activities and steps become evident

- EventStorming stands out as the preferred method for comprehensively grasping and harmonizing requirements through collaborative efforts

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- Activities which are composed of steps, with each step executed by one participant
- Decomposing activities into steps requires a deep understanding of real-world problems
- Involvement of domain experts or SMEs is crucial in capturing activities and steps

MCQ1

Question: What is the purpose of creating job stories in the context of identifying desired outcomes and digital capabilities?

- a) To detail the technical specifications of the digital capabilities
- b) To outline the roles and responsibilities of team members
- c) To identify the desired outcomes along with the digital capabilities necessary to produce the outcomes
- d) To describe the history and background of the project

Correct Answer: c) To identify the desired outcomes along with the digital capabilities necessary to produce the outcomes

MCQ2

Question: What are the two quick steps involved in capturing details when using job stories?

- a) Identifying the desired outcomes and outlining the technical specifications
- b) Decomposing each activity into individual steps and identifying the activities for each job story
- c) Conducting a collaborative EventStorming session and outlining the roles of team members
- d) Defining the scope of the project and assigning tasks to team members

Correct Answer: b) Decomposing each activity into individual steps and identifying the activities for each job story

MCQ3

Question: In scenarios where a team lacks familiarity with certain domains, what method stands out as the preferred approach for comprehensively grasping and harmonizing requirements through collaborative efforts?

- a) Agile methodology
- b) EventStorming
- c) Waterfall methodology
- d) Scrum framework

Correct Answer: b) EventStorming

API-Based Products

EventStorming

Suresh Gopal

Learning Outcomes

- Develop familiarity with EventStorming process for software or API design
- Learn how EventStorming is used for Collaborative Understanding of domain

Using EventStorming for Collaborative Understanding

EventStorming, a collaborative process, aids in visualizing

- Business processes,
- Requirements,
- and domain events

Developed by Alberto Brandolini,

- Adapted globally to suit various organizational needs
- Aims to establish a shared understanding of the domain

Outputs and insights from EventStorming sessions serve as inputs for software design, including API design

Conduction

Typically conducted in-person,

- Can be conducted remotely if necessary

Facilitators play a crucial role

- Guides the group through the process and maintaining session focus

Active participation from all attendees is expected, involving

- Offering insights,
- Posing clarifying questions,
- and identifying missing information for further research

Conduction

People

- Business owners
- SMEs
- Technical leads
- Security experts
- Individual software developers and contributors

Benefits

Shared understanding of requirements and scope of the problem being modeled

Shared understanding of the workflow processes, business rules, and constraints

Establishment of a shared domain vocabulary, replacing multiple terms with a ubiquitous language

Identification of unknowns that require follow-up and clarification prior to software design and development

Identification of boundaries within the solution, useful for scoping team efforts and division of labor to minimize cross-team dependency coordination

Steps

Step 1: Identify Business Domain Events

- (30–60 minutes)

Step 2: Create an Event Narrative

- (90–120 minutes)

Step 3: Review the Narrative and Identify Gaps

- (60–90 minutes)

Step 4: Expand Domain Understanding

- (30–60 minutes)

Step 5: Review the Final Narrative

- (30 minutes)

Step 1: Identify Business Domain Events

Time Frame: 30–60 minutes

Participants begin by **identifying business domain events for a job story or a set of job stories**

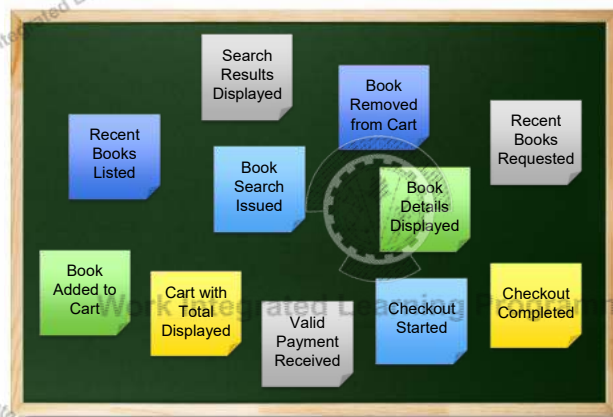
- Events are **recorded on stickies** of the same color (usually orange) and placed on the canvas

Domain events **are expressed in past tense** to signify completed actions, aiding in clarity and consistency

- Typically involves **two passes of 15 to 30 minutes** each, with additional passes for sessions covering broader scopes

Attendees are **encouraged to identify missing events and causation events between passes**, ensuring comprehensive coverage

Step 1: Identify Business Domain Events



An example of domain event sticky notes for the Bookstore online store. The events have now been captured and will be organized in the next step.

Step 2: Create an Event Narrative

Duration: 90–120 minutes

Organize domain event stickies into a coherent narrative sequence

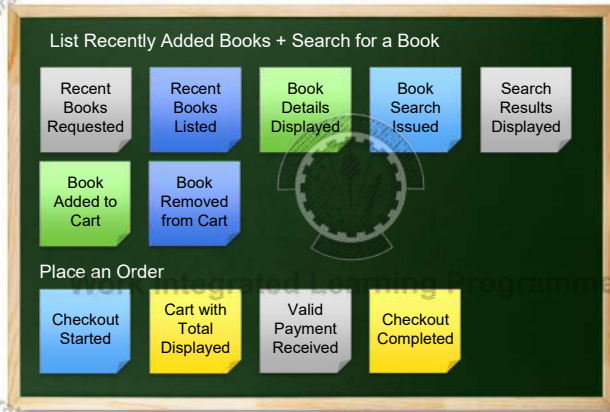
- Remove duplicates and make necessary clarifications to ensure clarity and coherence

The facilitator guides the process by **asking clarifying questions** and ensuring proper composition of the narrative

- **Address any potential roadblocks**, such as branching or parallel narratives, by focusing on a single narrative and ordering events accordingly

If needed, **mark certain domain events for revisiting later** or denote them with hotspot (typically hot pink) stickies for follow-up

Step 2: Create an Event Narrative



Business domain event sticky notes arranged into a linear narrative for Bookstore.

BITS WILP | Generated: 12-May-2026 16:47:54

Adapted From: Principles of Web API Design

Step 3: Review the Narrative and Identify Gaps

Duration: 60–90 minutes

Check the narrative flow from left to right to ensure completeness and clarity

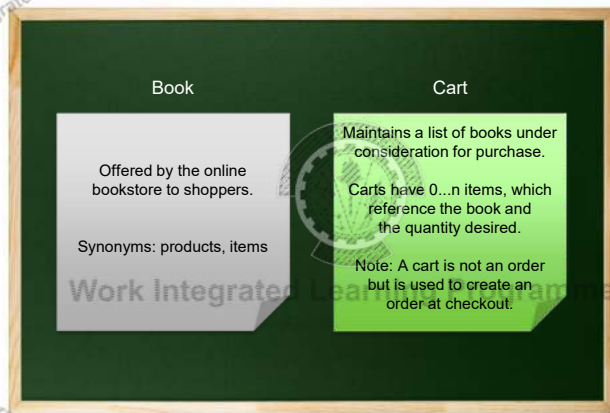
- Adjust or add events as needed to fill any gaps or clarify ambiguities

Establish a unified vocabulary for all domain concepts, forming the foundation for consistent communication

- Introduce **common vocabulary cards** if necessary, and update existing domain events to align with the new terminology
- **Anticipate the group's quick adoption of the new terminology** as discussions progress

BITS WILP | Generated: 12-May-2026 16:47:54

Step 3: Review the Narrative and Identify Gaps



Two examples of cards used to capture vocabulary clarifications during a session. These cards will become part of the ubiquitous language for a bounded context identified in the next step of the process.

Adapted From: Principles of Web API Design

Step 4: Expand Domain Understanding

Duration 30–60 minutes

After the events have been ordered, additional sticky note colors are used to expand domain understanding

Business event (orange): The result of an action or policy that indicates forward progression through a workflow or process

Hotspot (bright pink): Unknown or missing data that requires research and follow-up after the session

Command (dark blue): An action taken by a user or system

BITS WILP | Generated: 12-May-2026 16:47:54

Step 4: Expand Domain Understanding

Aggregate (red): Behavior or logic that executes as a result of a command and behavior and often results in one or more events

Policy (lilac): The triggering event or motivation for why a new command is executed and is always required

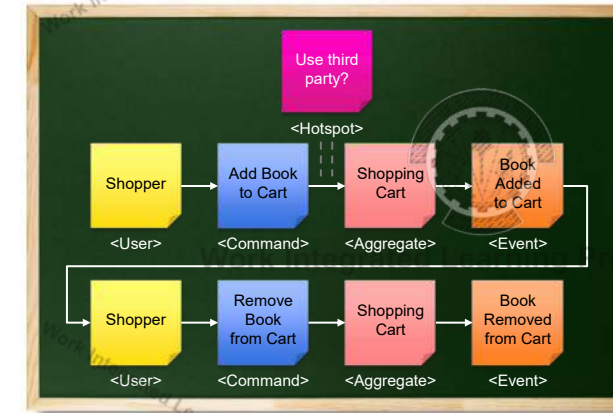
External system (pale pink): Systems outside of the solution

User interface (grey): A user interface that will offer one or more roles the capability to execute a command against an aggregate

User (yellow): A specific role that is interacting with the system, typically via a UI

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 4: Expand Domain Understanding



Adapted From: Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 5: Review the Final Narrative

Duration: 30 minutes

Review the narrative both forwards and backwards to ensure all elements are captured

- Clearly mark important events and triggers to highlight key transitions between steps

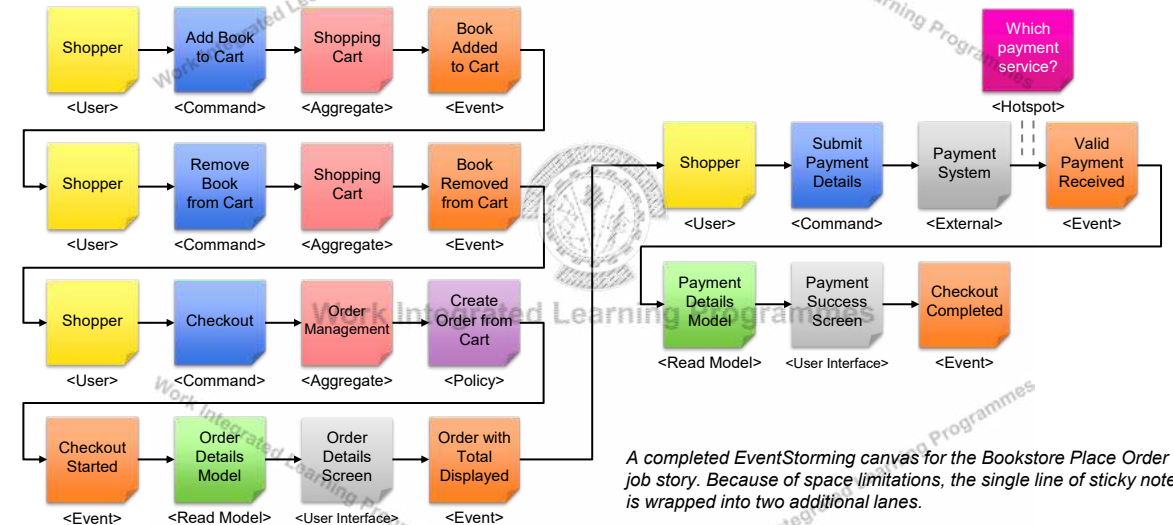
Save the EventStorming canvas for future reference, as it informs subsequent steps in the API design process

- Consider **sharing photos** of the canvas with the team for collaboration and reference
- If using a digital tool, **export the work as a PDF or image** for inclusion in project assets.

Document identified activities and activity steps in the outlined format to conclude the session

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 5: Review the Final Narrative



A completed EventStorming canvas for the Bookstore Place Order job story. Because of space limitations, the single line of sticky notes is wrapped into two additional lanes.

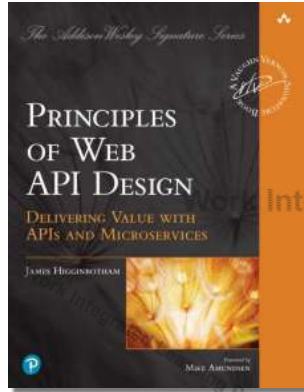
Adapted From: Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- EventStorming as tool for collaboration understanding
- How EventStorming can be used to capture the activities and steps in Align phase of A-D-D-R process

MCQ1

Question: What does EventStorming primarily aid in visualizing?

- a) Technical architecture
- b) Business processes, requirements, and domain events
- c) Software testing strategies
- d) Project management workflows

Correct Answer: b) Business processes, requirements, and domain events

MCQ2

Question: Who are the typical participants in an EventStorming session?

- a) Customers only
- b) Business owners, project managers, and marketing executives
- c) Business owners, subject matter experts, technical leads, security experts, and software developers
- d) Designers and developers only

Correct Answer: c) Business owners, subject matter experts, technical leads, security experts, and software developers

MCQ3

Question: What is one of the primary outcomes of conducting EventStorming sessions?

- a) Generating software code
- b) Establishing team hierarchies
- c) Creating project timelines
- d) Establishing a shared understanding of requirements and scope of the problem being modeled

Correct Answer: d) Establishing a shared understanding of requirements and scope of the problem being modeled

Learning Outcomes

- Identify candidate APIs
- Understand API Boundary Antipatterns
- Learn methods for API Boundary Identification
- Enhance API Design through Boundary Identification

By This Time...

Initial Digital Capabilities Identification

Used **job stories** to identify digital capabilities required

Captured **activities** necessary to achieve outcomes from insights gained in EventStorming sessions

Identify candidate APIs based on the boundaries established in **EventStorming sessions** or from the **list of activities**

Ensuring alignment with customer needs and business objectives by centering on outcomes and activities

Keep teams aligned with the overarching goals throughout the API design process

API-Based Products

Identifying API Boundaries

Suresh Gopal

Defining Candidate APIs

Emergence of API Profiles:

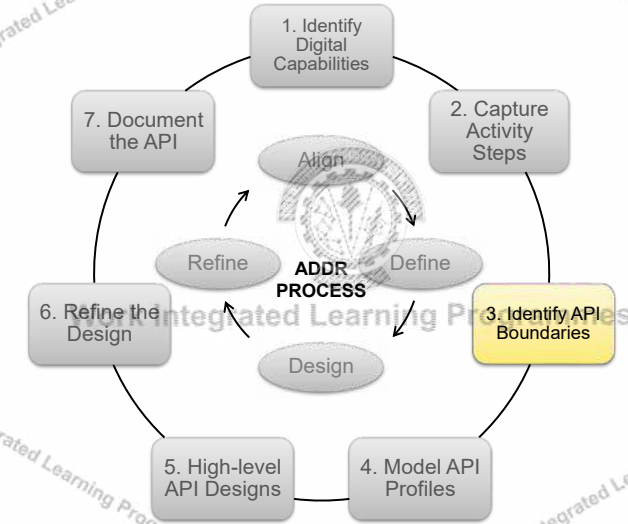
- As boundaries are identified, API profiles start to become clearer
- API profiles provide detailed clarity about resources and operations required for the API design

Clarity in API Definition:

- Need to define API resources and operations through API profiles
- Ensure the eventual API design is focused on delivering desired outcomes for customers, partners, and the workforce

BITS WILP | Generated: 12-May-2026 16:47:54

Identifying API Boundaries



The first step in the Define phase is to identify API boundaries.

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Why Identifying API Boundaries?

APIs provide a **mental model for developers** to integrate and achieve desired outcomes

- Defining the **scope and responsibilities of an API** aids in shaping this mental model

Responsibilities of each API are clearly outlined **based on artifacts from the Align phase**

- Output from activity step identification aids in finding boundaries and candidate APIs

Mistakes in defining API boundaries can lead to a poor developer experience!

BITS WILP | Generated: 12-May-2026 16:47:54

Avoiding API Boundary Antipatterns

Mega All-in-One API Antipattern:

- Designing a **single, large API product** can lead to difficulties for developers in **finding necessary functionalities quickly**
- Conversely, creating **many small API products** may result in **fragmentation and frustration**
- Clear API boundaries mitigate confusion stemming from excessively large or fragmented APIs

Overloaded API Antipattern:

- Attempting to design a **single, comprehensive API for multiple products** often results in an **overloaded API** that struggles to perform any task effectively
- Well-intentioned goals can lead to APIs that try to encompass everything but end up lacking in functionality and usability

BITS WILP | Generated: 12-May-2026 16:47:54

Finding API Boundaries Using EventStorming

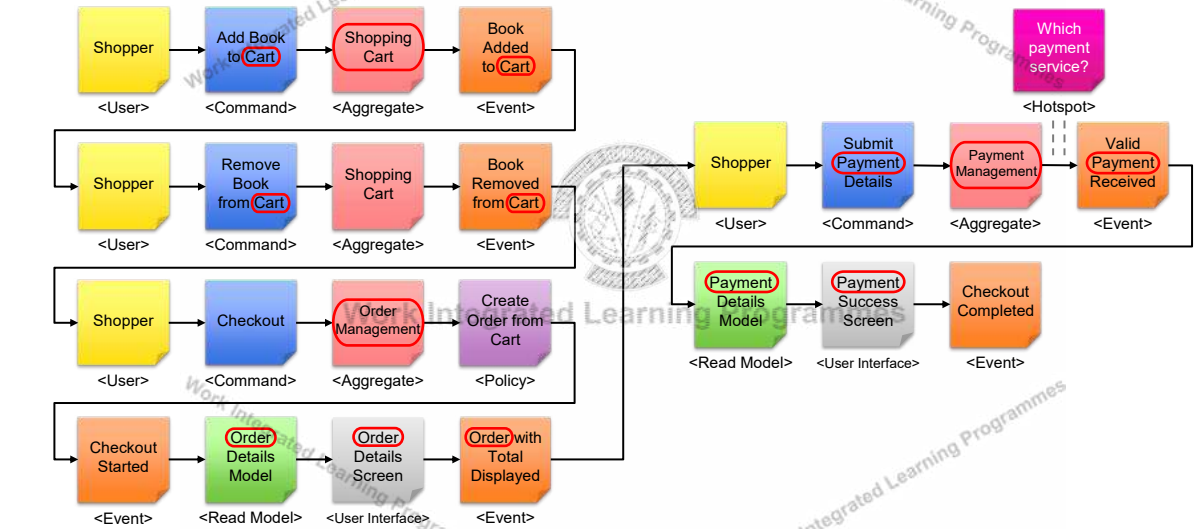
EventStorming for Unified Understanding:

- EventStorming aligns teams with shared terminology and understanding of processes, policies, and system interactions
- **Language shifts in the EventStorming canvas hint at possible API boundaries** emerging as terms and focus change

Identification and Naming of Boundaries:

- **Analyze language patterns in EventStorming to identify and name API boundaries**
- **Aggregates identified in EventStorming can provide hints for API scope**, though not always definitive

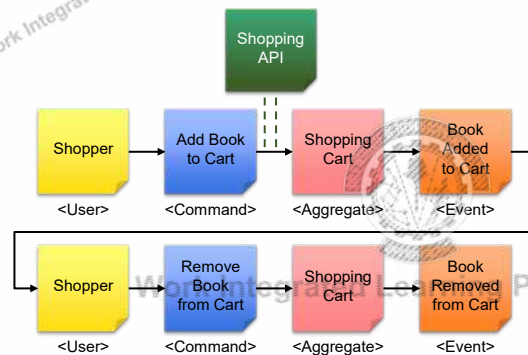
EventStorming Canvas Used to Find Shifts in Language



Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

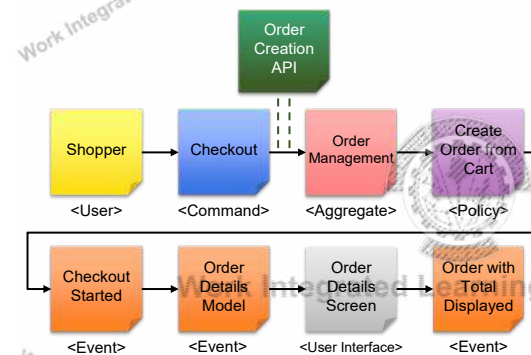
API Boundaries Identified...



Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

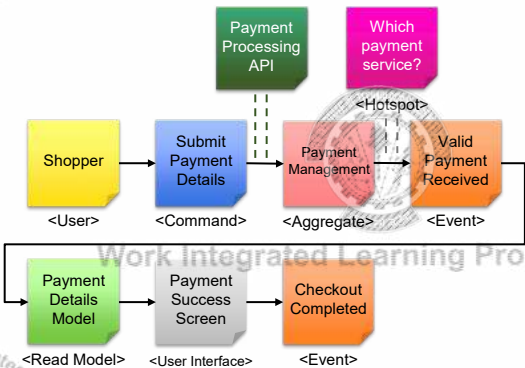
API Boundaries Identified...



Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

API Boundaries Identified...



Adapted From: Principles of Web API Design

Activity Steps - Separators Indicate Shifts in Vocabulary

Digital Capability	Activity	Activity Step	Participants	Description
Place an Order	Browse for Books	List Books	Customer, Call Center	List books by category or release date
Place an Order	Browse for Books	Search for Books	Customer, Call Center	Search for books by author, title
Place an Order	Browse for Books	View Book Details	Customer, Call Center	View the details of a book
Place an Order	Shop for Books	Add Books to Cart	Customer, Call Center	Add a book to the customer's cart
Place an Order	Shop for Books	Remove Books from Cart	Customer, Call Center	Remove a book from the customer's cart
Place an Order	Shop for Books	Clear Cart	Customer, Call Center	Remove all books from the customer's cart
Place an Order	Shop for Books	View Cart	Customer, Call Center	View the current cart and total
Place an Order	Create an Order	Checkout	Customer, Call Center	Create an order from the contents of the cart
Place an Order	Create an Order	Pay for Order	Customer, Call Center	Accept and process payment for the order

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Finding API Boundaries through Activities

Activity Review for Boundary Identification:

Utilize activities and activity steps produced by subject matter experts to identify API boundaries

- Effective when considerable effort has already been invested in capturing requirements

Language Shifts as Boundary Indicators:

Look for shifts in language within activity step names and descriptions

Note changes in nouns, as they may indicate potential boundaries

- When a new set of nouns is introduced, mark the location as the starting point for a new boundary

BITS WILP | Generated: 12-May-2026 16:47:54

API Boundaries Defined...

Shopping API, Discovered through Boundary Identification, with Corresponding Activity Steps from Bookstore

Digital Capability	Activity	Activity Step	Participants	Description
Place an Order	Browse for Books	List Books	Customer, Call Center	List books by category or release date
Place an Order	Browse for Books	Search for Books	Customer, Call Center	Search for books by author, title
Place an Order	Browse for Books	View Book Details	Customer, Call Center	View the details of a book
Place an Order	Shop for Books	Add Books to Cart	Customer, Call Center	Add a book to the customer's cart
Place an Order	Shop for Books	Remove Books from Cart	Customer, Call Center	Remove a book from the customer's cart
Place an Order	Shop for Books	Clear Cart	Customer, Call Center	Remove all books from the customer's cart
Place an Order	Shop for Books	View Cart	Customer, Call Center	View the current cart and total
Place an Order	Create an Order	Checkout	Customer, Call Center	Create an order from the contents of the cart
Place an Order	Create an Order	Pay for Order	Customer, Call Center	Accept and process payment for the order

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Order Creation API, Discovered through Boundary Identification, with Corresponding Activity Steps from Bookstore

Digital Capability	Activity	Activity Step	Participants	Description
Place an Order	Create an Order	Checkout	Customer, Call Center	Create an order from the contents of the cart

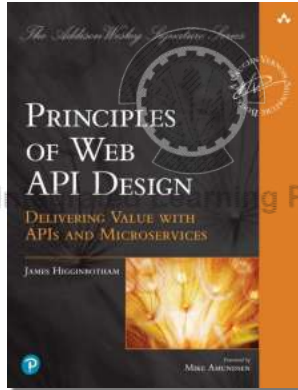
Payment Processing API, Discovered through Boundary Identification, with Corresponding Activity Steps from Bookstore

Digital Capability	Activity	Activity Step	Participants	Description
Place an Order	Create an Order	Pay for Order	Customer, Call Center	Accept and process payment for the order

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Awareness about API Boundary Antipattern
- Approaches to API Boundary Identification
 - EventStorming
 - Activity Steps

MCQ1

Question: What role do API profiles play in the API design process?

- a) They define the boundaries of the API and outline its scope
- b) They provide detailed clarity about resources and operations required for the API design
- c) They establish the naming conventions for API endpoints and methods
- d) They determine the authentication methods and security protocols to be implemented

Correct Answer: b) They provide detailed clarity about resources and operations required for the API design

MCQ2

Question: What are the potential drawbacks of the Mega All-in-One API and Overloaded API Antipatterns?

- a) Difficulty in finding necessary functionalities quickly and fragmentation of functionalities
- b) Confusion stemming from clear API boundaries and frustration for developers
- c) Fragmentation of APIs and effective task performance
- d) Lack of functionality and usability due to well-intentioned goals

Correct Answer: a) Difficulty in finding necessary functionalities quickly and fragmentation of functionalities

MCQ3

Question: What role does EventStorming play in API design?

- a) It primarily focuses on identifying system interactions and policies
- b) It aligns teams with shared terminology and understanding, facilitating the identification and naming of API boundaries
- c) It defines clear API boundaries based on predetermined language patterns
- d) It provides definitive hints for API scope through aggregates identified during the process

Correct Answer: b) It aligns teams with shared terminology and understanding, facilitating the identification and naming of API boundaries

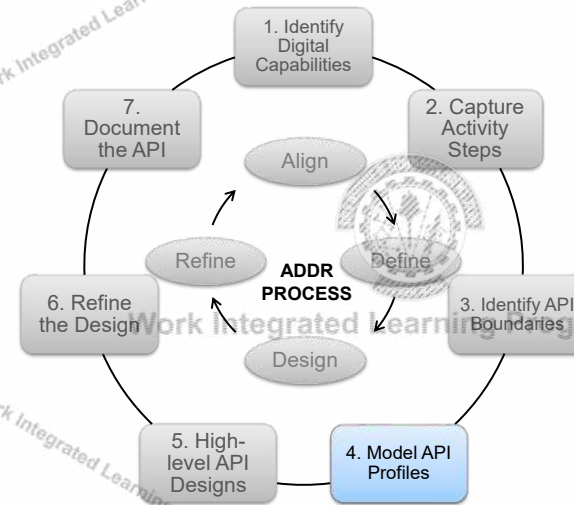
Learning Outcomes

- Develop understanding about
 - API modeling concepts
 - API modeling process

API-Based Products

Model API Profiles

Define: Model API Profiles



The final step of the Define phase is to create API profiles in preparation for transitioning to the Design phase.

Adapted From: Principles of Web API Design

API Modeling

API modeling is a **tracer bullet** for API design

- Technique for exploring the necessary elements of an API prior to the design and delivery process

API modeling helps to **bring together the insights and artifacts from the previous steps**

- Creates an API profile that describes the scope and intent of the APIs needed to deliver the desired outcomes of end users

BITS WILP | | Generated: 12-May-2026 16:47:54

What Is API Modeling?

Just as a beautiful Web design begins from a wireframe, **a great API design starts with an API model**, which defines its scope and responsibilities

- The objective of API modeling is to **thoroughly comprehend and validate the requirements of developers and end users**

API modeling efficiently identifies issues early on, facilitating resolution before coding begins

- **Job stories, activities, and activity steps serve as inputs for API modeling**, resulting in cohesive **API profiles**
- An API profile captures **key characteristics of the API**, including its name, scope, operations, and emitted events essential for achieving desired outcomes.

BITS WILP | | Generated: 12-May-2026 16:47:54

API Profile Structure

API profiles capture all necessary information about an API,

The independent of the API style or styles that it will expose

Used to drive the design of an API, but **also provides the beginnings of the API documentation**

BITS WILP | | Generated: 12-May-2026 16:47:54

API Profile Structure

An API profile captures the following details about each API:

The **name** and a **short description** of the API

The **scope** of the API (internal, public, partner, etc.)

API **operations** with **input** and **output** message details

Participants allowed to perform each operation, in preparation for securing the API

Events generated by each API operation, to drive extensibility beyond the API's original intent

(Optional) **architectural requirements** identified, such as a service-level agreement (SLA)

BITS WILP | | Generated: 12-May-2026 16:47:54

API Modeling Process

Goal of the API modeling process is to produce one or more API profiles, one for each API identified during modeling

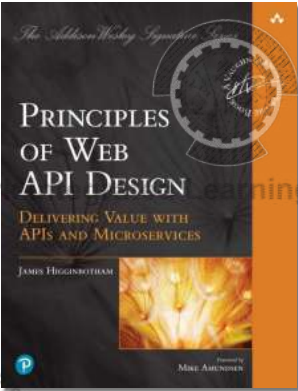
Steps:

- Divided into five quick steps
- Each step adds additional detail to the API profile until a blueprint of the API emerges
- Step 1: Capture API Profile Summary
- Step 2: Identify the Resources
- Step 3: Define the Resource Taxonomy
- Step 4: Add Operation Events
- Step 5: Expand Operation Details

Reference

Principles of Web API Design

- By James Higginbotham



Template

My API—Description goes here

API scope (internal, public, partner, etc.)
Architectural requirements (service-level agreements, standards compliance, etc.)

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listThingies()	List/search for thingies	Customer, Shopper	Thingy	Thingies Listed	Request Parameters: vendorId, ... Returns: Thingy[]
...	...	...	...	...	...

Summary

- API modeling
- API profile structure
- API Modeling process
- Template to capture API details

MCQ1

Question: What is the primary purpose of API modeling in the API design process?

- a) To finalize the technical implementation details of the API
- b) To explore and define the necessary elements of an API before the design and delivery process
- c) To create a detailed blueprint of the API
- d) To document the API endpoints and methods

Correct Answer: b) To explore and define the necessary elements of an API before the design and delivery process

MCQ2

Question: What is the primary objective of API modeling in the API design process?

- a) To define the technical implementation details of the API
- b) To create a wireframe for the API design
- c) To thoroughly comprehend and validate the requirements of developers and end users
- d) To finalize the API endpoints and methods

Correct Answer: c) To thoroughly comprehend and validate the requirements of developers and end users

MCQ3

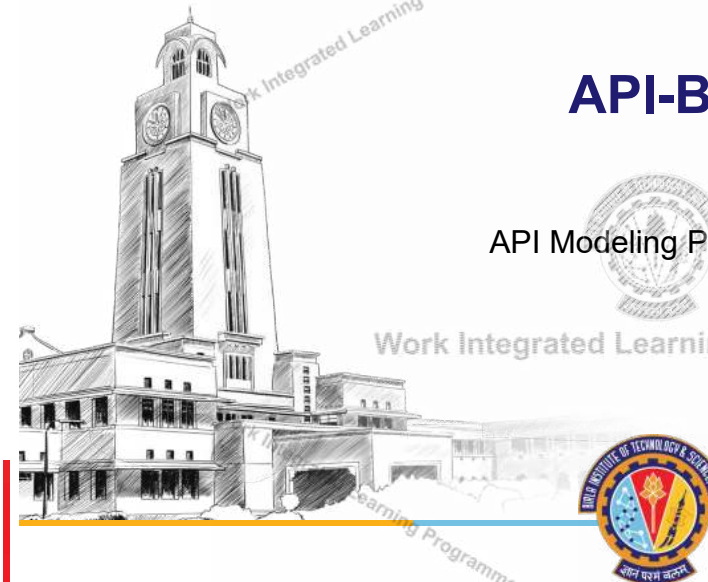
Question: What information does an API profile typically capture?

- a) Only the technical implementation details of the API
- b) The scope of the API and its architectural design
- c) The participants allowed to perform each operation and the events generated by each API operation
- d) The API name, description, scope, operations with input and output message details, participants, events, and optional architectural requirements

Correct Answer: d) The API name, description, scope, operations with input and output message details, participants, events, and optional architectural requirements

API-Based Products

API Modeling Process | Suresh Gopal



Learning Outcomes

- Understand how API modeling process is performed



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

API Modeling Process

Goal of the API modeling process is to produce one or more API profiles, one for each API identified during modeling

Steps:

Divided into five quick steps

Each step adds additional detail to the API profile until a blueprint of the API emerges

Step 1: Capture API Profile Summary

Step 2: Identify the Resources

Step 3: Define the Resource Taxonomy

Step 4: Add Operation Events

Step 5: Expand Operation Details

BITS WILP | Generated: 12-May-2026 16:47:54

Step 1: Capture API Profile Summary

Capture API Profile Summary

- Begin by filling out essential details of the API profile, including the API name, a brief description, and its scope
- Scope - internal, public, and partner realms supported by the organization
- Details are flexible and can be adjusted as the team gains deeper insights into each API's purpose and responsibilities

Further Actions:

- Identify API operation names and participants based on previously captured activities and activity steps
- Convert each activity step into an operation name following a consistent naming convention, such as lowerCamelCase

BITS WILP | Generated: 12-May-2026 16:47:54

Step 1: Capture API Profile Summary

Shopping API - Supports the book browsing experience and cart management
Public

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listBooks()	List books by category or release date	Customer, Call Center			
searchBooks()	Search for books by author, title	Customer, Call Center			
viewBook()	View book details	Customer, Call Center			
addBookToCart()	Add a book to the customer's cart	Customer, Call Center			
removeBookFromCart()	Remove a book from the customer's cart	Customer, Call Center			
clearCart()	Remove all books from the customer's cart	Customer, Call Center			
viewCart()	View the current cart and total	Customer, Call Center			

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Identify the Resources

Utilize the API profile to identify resources for the API, typically domain entities operated upon by the API

- Resource identification can be **challenging** initially but is **informed by the Align and Define phases** of the process

Resources:

- Create a **table** for each candidate resource, detailing its name, known properties, and a description for alignment
- Focus on capturing only essential properties for each resource to expedite the modeling process and maintain focus on the API profile

Step 2: Identify the Resources

Book Resource		Cart Resource	
Property Name	Description	Property Name	Description
title	The book title	books	The books currently in the cart for purchase
isbn	The unique ISBN of the book	subtotal	The total cost of all books in the cart
authors	List of Book Author resources	salesTax	The sales tax to be applied
Book Author Resource		vatTax	Any value-added tax to be applied
Property Name	Description	cartTotal	The total cost of the cart
fullName	The full name of the author		

Capturing each resource for the Shopping API, along with some basic details about each one as a starting point.

Step 2: Identify the Resources

Shopping API - Supports the book browsing experience and cart management
Public

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listBooks()	List books by category or release date	Customer, Call Center	Book		
searchBooks()	Search for books by author, title	Customer, Call Center	Book, Book Author		
viewBook()	View book details	Customer, Call Center	Book		
addBookToCart()	Add a book to the customer's cart	Customer, Call Center	Cart		
removeBookFromCart()	Remove a book from the customer's cart	Customer, Call Center	Cart		
clearCart()	Remove all books from the customer's cart	Customer, Call Center	Cart		
viewCart()	View the current cart and total	Customer, Call Center	Cart		

Identifying the Book and Cart resources for Bookstore's Shopping API profile.

Step 3: Define the Resource Taxonomy

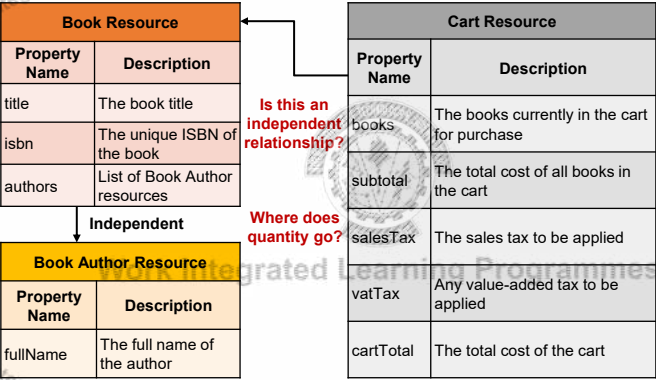
Following resource identification, **delineate the relationships among resources** to formulate the API taxonomy

- API taxonomy** entails classifying concepts and their arrangement within the API, encapsulating the set of offered resources and their interrelations

Three relationship types among resources:

- Independent:** Resources stand alone and don't rely on other resources for existence, though they may reference each other
- Dependent:** One resource necessitates the existence of a parent resource
- Associative:** Resources may exist independently, but their relationship requires additional properties, resulting in a third resource representing their association

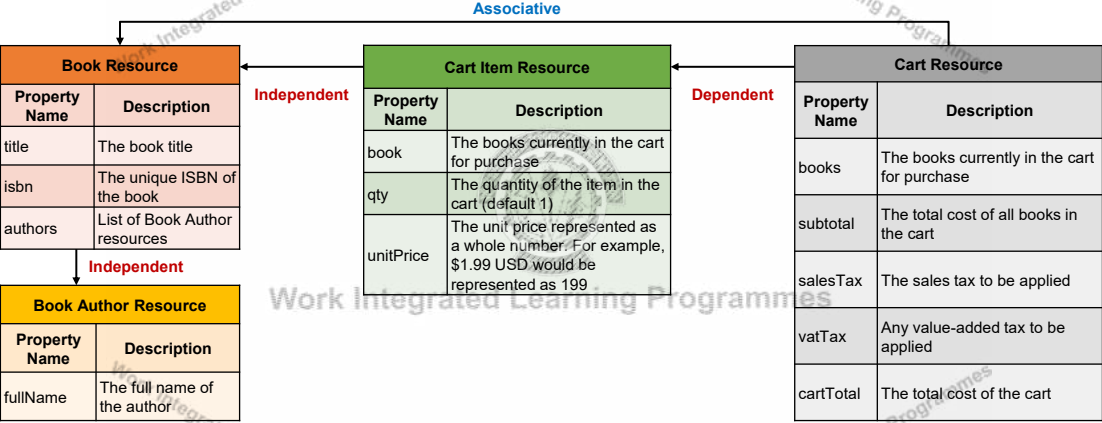
Step 3: Define the Resource Taxonomy



Reviewing the resource relationships introduces a challenge: where does the quantity of books belong?

Adapted From: Principles of Web API Design

Step 3: Define the Resource Taxonomy



A new resource, Cart Item, is added because of the associative relationship between Book and Cart.

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 3: Define the Resource Taxonomy

Shopping API - Supports the book browsing experience and cart management
Public

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listBooks()	List books by category or release date	Customer, Call Center	Book, Book Author		
searchBooks()	Search for books by author, title	Customer, Call Center	Book, Book Author		
viewBook()	View book details	Customer, Call Center	Book		
addItemToCart()	Add a book to the customer's cart	Customer, Call Center	Cart Item, Cart		
removeItemFromCart()	Remove a book from the customer's cart	Customer, Call Center	Cart Item, Cart		
clearCart()	Remove all books from the customer's cart	Customer, Call Center	Cart		
viewCart()	View the current cart and total	Customer, Call Center	Cart		

The Shopping API profile, revised to reflect the introduction of the Cart Item resource.

Adapted From: Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 4: Add Operation Events

After finalizing the API taxonomy, enhance each API operation by specifying the significant events it will emit

- Serve various purposes, including data analytics or triggering reactions in other systems

Utilize EventStorming canvas from the Align phase to identify important business events

Ensure event names adhere to organizational standards and practices, presented in past tense

- API profile now encompasses the events emitted by identified operations, with some operations emitting multiple events while others may emit none at all

BITS WILP | Generated: 12-May-2026 16:47:54

Step 4: Add Operation Events

Shopping API - Supports the book browsing experience and cart management
Public

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listBooks()	List books by category or release date	Customer, Call Center	Book, Book Author	Books.Listed	
searchBooks()	Search for books by author, title	Customer, Call Center	Book, Book Author	Books.Searched	
viewBook()	View book details	Customer, Call Center	Book	Book.Viewed	
addItemToCart()	Add a book to the customer's cart	Customer, Call Center	Cart Item, Cart	Cart.ItemAdded	
removeItemFromCart()	Remove a book from the customer's cart	Customer, Call Center	Cart Item, Cart	Cart.ItemRemoved	
clearCart()	Remove all books from the customer's cart	Customer, Call Center	Cart	Cart.Cleared	
viewCart()	View the current cart and total	Customer, Call Center	Cart	Cart.Viewed	

JSON's Bookstore's Shopping API, now with emitted events added for each operation in the API profile.

Adapted From: Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 5: Expand Operation Details

Finalize each operation by expanding its details to include vital input and output specifications

- Focus on capturing essential resources and parameters to facilitate team understanding, avoiding excessive detail
- Schema definition and property typing are unnecessary at this stage; reserve detailed design for later phases
- Use a "parking lot" to note items requiring further attention as the design progresses

Consider the synchronous or asynchronous nature of operations, noting the distinction for future design considerations

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 5: Expand Operation Details

Pay attention to safety classifications for HTTP operations, ensuring alignment with appropriate HTTP method specifications

- Three safety classifications:

Safe: Read-based operations (e.g., GET) that don't alter resource states.

Idempotent: Operations (e.g., PUT, DELETE) that produce the same result when executed with the same input.

Unsafe: Operations (e.g., POST, PATCH) that may yield different results when called multiple times with the same input.

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 5: Expand Operation Details

Shopping API - Supports the book browsing experience and cart management
Public

Operation Name	Description	Participants	Resource(s)	Emitted Events	Operation Details
listBooks()	List books by category or release date	Customer, Call Center	Book, Book Author	Books.Listed	Request Parameters: categoryId, releaseDate Returns: Book[] safe / synchronous
searchBooks()	Search for books by author, title	Customer, Call Center	Book, Book Author	Books.Searched	Request Parameters: searchQuery Returns: Book[] safe / synchronous
viewBook()	View book details	Customer, Call Center	Book	Book.Viewed	Request Parameters: bookId Returns: Book safe / synchronous
addItemToCart()	Add a book to the customer's cart	Customer, Call Center	Cart Item, Cart	Cart.ItemAdded	Request Parameters: cartId, bookId, quantity Returns: Cart unsafe / synchronous
removeItemFromCart()	Remove a book from the customer's cart	Customer, Call Center	Cart Item, Cart	Cart.ItemRemoved	Request Parameters: cartItemId Returns: Cart idempotent / synchronous
clearCart()	Remove all books from the customer's cart	Customer, Call Center	Cart	Cart.Cleared	Request Parameters: cartId Returns: Cart safe / synchronous
viewCart()	View the current cart and total	Customer, Call Center	Cart	Cart.Viewed	Request Parameters: cartId Returns: Cart safe / synchronous

JSON's Bookstore Shopping API with operation details added to the API profile.

Adapted From: Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Validating the API Model with Sequence Diagrams



Final actions include **validating the API model to identify any gaps** and **gathering feedback from stakeholders**

Validate API profiles against **original job stories and activities** to ensure all requirements are addressed

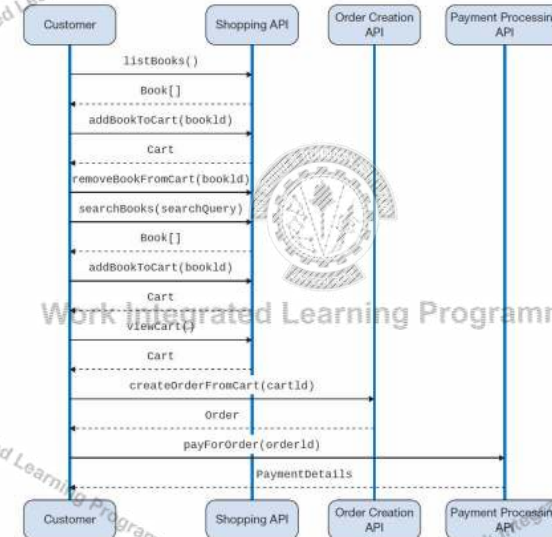
- **Use sequence diagrams** to illustrate typical usage scenarios, drawing scenarios from EventStorming canvas, job stories, and other sources

Share the validated model with stakeholders and solicit feedback

- Incorporate feedback obtained during the process if API modeling involved the entire team, including business and product stakeholders

BITS WILP | | Generated: 12-May-2026 16:47:54

Validating the API Model with Sequence Diagrams



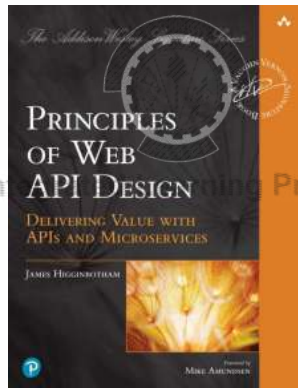
BITS WILP | | Generated: 12-May-2026 16:47:54

Image Source: Principles of Web API Design

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Steps involved in API modeling process
 - Step 1: Capture API Profile Summary
 - Step 2: Identify the Resources
 - Step 3: Define the Resource Taxonomy
 - Step 4: Add Operation Events
 - Step 5: Expand Operation Details

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: What is the goal of the API modeling process, divided into five quick steps?

- a) To establish API documentation for each identified API
- b) To create a detailed blueprint of the API
- c) To outline the high-level functionalities of the API
- d) To generate API profiles for stakeholder review

Correct Answer: b) To create a detailed blueprint of the API

MCQ2

Question: What is the primary purpose of capturing the API Profile Summary?

- a) To outline the technical implementation details of the API
- b) To define the naming convention for API operations
- c) To provide essential information about the API, including its name, description, and scope
- d) To identify participants involved in API operations

Correct Answer: c) To provide essential information about the API, including its name, description, and scope

MCQ3

Question: What is the purpose of delineating relationships among resources to formulate the API taxonomy?

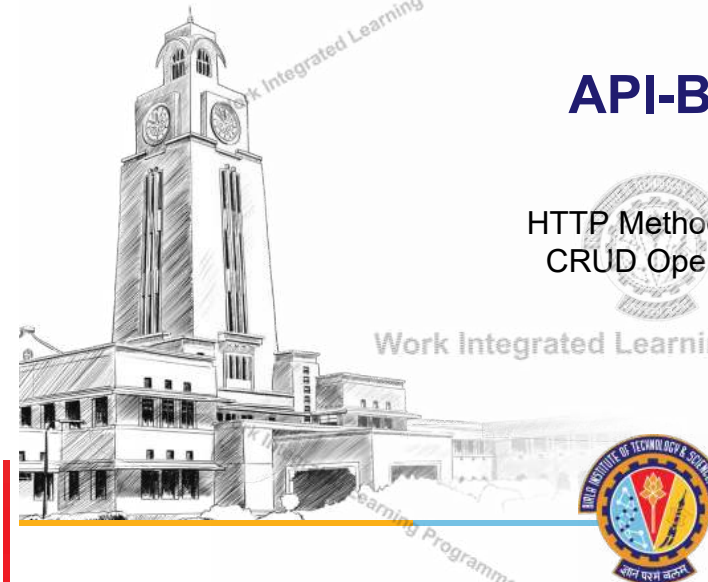
- a) To define the naming conventions for resources within the API
- b) To classify concepts and arrange resources within the API
- c) To identify the authentication methods required for each resource
- d) To determine the HTTP methods supported by each resource

Correct Answer: b) To classify concepts and arrange resources within the API

API-Based Products

HTTP Methods and
CRUD Operations

Suresh Gopal



Learning Outcomes

- Understand importance of HTTP methods and CRUD operations in building RESTful APIs

HTTP and REST

REST, short for Representational State Transfer, is an architectural style for designing networked applications

- Emphasizes a stateless client-server communication model where interactions are based on standard HTTP methods and resource representations

RESTful systems are characterized by

- Resources (identified by URIs),
- Standard HTTP methods (GET, POST, PUT, PATCH, DELETE),
- and hypermedia as the engine of application state (HATEOAS),

HTTP and REST

Identification of resources prompts questions regarding consumer interactions and interests

- Guide the selection of appropriate HTTP verbs for each resource

HTTP verbs are integral to RESTful API design, defining actions on resources

- Commonly used HTTP verbs include POST, GET, PUT, and DELETE, corresponding to CRUD operations
- Additional HTTP verbs like HEAD, OPTIONS, TRACE, and CONNECT exist in the HTTP 1.1 spec.

Overview of Each HTTP Method's

HTTP Method	Description
GET	Retrieves data from the server without modifying resources.
POST	Submits data to the server to create a new resource.
PUT	Updates a resource on the server or creates it if not exist.
PATCH	Applies partial modifications to a resource.
DELETE	Removes a resource from the server.

CRUD Operations

CRUD Operation	HTTP Method	Description
Create	POST	Used to create a new resource on the server
Read	GET	Used to retrieve a resource from the server
Update	PUT/PATCH	PUT: Typically used to update an existing resource by replacing it entirely PATCH: Used to apply partial modifications to an existing resource
Delete	DELETE	Used to remove a resource from the server

Examples of CRUD Operations

How CRUD operations are performed using HTTP methods in REST APIs

Creating a new user:

POST /users

Retrieving user data:

GET /users/{id}

Updating user information:

PUT /users/{id}
or PATCH /users/{id}

Deleting a user account:

DELETE /users/{id}

Idempotent and Safe Methods

Idempotent methods maintain consistent results regardless of how many times they are called

- Idempotency pertains to the method's outcome, not the resource itself
- For instance, a **GET** method on a resource yields the same result unless the resource changes

Safe methods do not alter the resource's state when called

- GET and HEAD methods, for example, are considered safe as they do not modify resources

Idempotent and/or Safe HTTP Methods

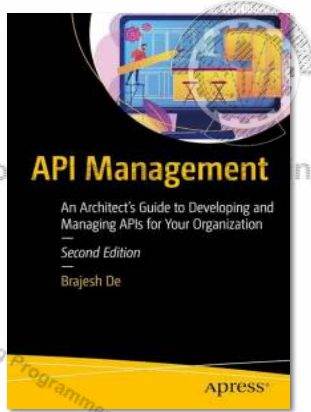
HTTP Method	Idempotent?	Safe?	Description
GET	Yes	Yes	Retrieves data from a server without altering its state
HEAD	Yes	Yes	Retrieves headers from a server without fetching the resource
PUT	Yes	No	Replaces an existing resource or creates a new one
DELETE	Yes	No	Removes a resource from the server
OPTIONS	Yes	Yes	Retrieves the HTTP methods that a server supports for a URL
TRACE	Yes	Yes	Echoes received request back to the client for diagnostic purposes
CONNECT	No	No	Establishes a tunnel to the server for HTTPS connections
POST	No	No	Submits data to be processed by a specified resource

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

How in RESTful APIs,

- CRUD operations are mapped to HTTP methods
- Idempotent and/or Safe HTTP Methods



Work Integrated Learning Programmes

MCQ1

Which HTTP method is commonly used to retrieve data from a server without causing any side effects?

- a) POST
- b) GET
- c) PUT
- d) DELETE

Answer: b) GET



Work Integrated Learning Programmes

MCQ2

Which RESTful CRUD operation corresponds to the HTTP method PATCH?

- a) Create
- b) Read
- c) Update
- d) Delete

Answer: c) Update



Work Integrated Learning Programmes

MCQ3

In a RESTful API, which HTTP method is typically used to delete a resource from the server?

- a) GET
- b) PUT
- c) DELETE
- d) POST

Answer: c) DELETE



API-Based Products

REST Request and Response
Formats

Suresh Gopal



Learning Outcomes

- Develop a comprehensive overview of the key concepts and practices in building and consuming RESTful API



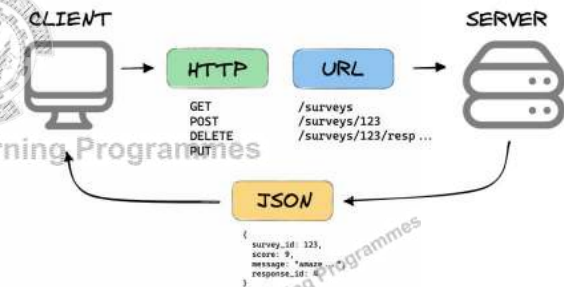
REST Request and Response Formats

REST (Representational State Transfer) is an architectural style for designing networked applications

RESTful APIs use standard formats for requests and responses to enable interoperability and scalability

- Emphasizes a **stateless client-server communication** where interactions are based on standard HTTP methods
- Use **resource-based URLs** to represent data entities and utilize **various HTTP methods for CRUD** (Create, Read, Update, Delete) operations

WHAT IS A REST API?



REST API Request Format

Requests in REST APIs are typically made using HTTP methods like GET, POST, PUT, and DELETE

- The request format includes:

HTTP Method:

Specifies the action to be performed on the resource (GET, POST, PUT, DELETE)

URL:

Represents the resource being accessed or manipulated

Headers:

Provide additional information such as authentication tokens, content type, etc.

Body (for POST and PUT requests):

Contains data to be sent to the server, usually in JSON or XML format

Example of REST API Request

HTTP Method:

- GET

URL:

- https://api.example.com/users/123

Headers:

- Authorization: Bearer <token>
- Accept: application/json

Body:

- N/A

HTTP method

Endpoint

Headers

Body

REST API Response Format

Responses in REST APIs include:

Status Code:

- Indicates the outcome of the request
- 200 for success,
- 404 for not found,
- 500 for server error

Headers:

Provide additional metadata such as content type, caching directives, etc.

Body:

Contains the requested data or an error message, usually in JSON or XML format

```
HTTP/1.1 200 OK
Content-Type: application/vnd.acme.account+json
Content-Length: ...

{
  "account": {
    "account_number": 12345,
    "balance": {
      "currency": "usd",
      "value": 100.00
    },
    "links": {
      "deposit": "/accounts/12345/deposit",
      "withdraw": "/accounts/12345/withdraw",
      "transfer": "/accounts/12345/transfer",
      "close": "/accounts/12345/close"
    }
  }
}
```

Example of REST API Response

Successful Response (200 OK):

Status Code:

- 200 OK

Headers:

- Content-Type: application/json

Body:

```
{
  "id": 123,
  "name": "John Doe",
  "email": "john@example.com"
}
```

Common Data Formats for REST

JSON (JavaScript Object Notation):

- Lightweight
- Human-readable data interchange format

XML (eXtensible Markup Language):

- Markup language for encoding data
- both human-readable and machine-readable

Other Formats:

- YAML,
- Protocol Buffers,
- MessagePack, etc.

{JSON}

<xml />

YAML

protobuf
Protocol Buffers

BITS WILP | | Generated: 12-May-2026 16:47:54

Best Practices for Request and Response Formats

Use descriptive URLs

- To represent resources

Follow consistent naming conventions

- For JSON or XML payloads in request and response bodies

Minimize the size of payloads to improve performance and efficiency

Choose appropriate HTTP methods

- For different operations (GET for retrieval, POST for creation, PUT for update, DELETE for deletion)

Provide descriptive error messages and status codes

- For easy troubleshooting

BITS WILP | | Generated: 12-May-2026 16:47:54

Error Handling in REST APIs

REST APIs use **status codes** to convey error conditions to clients

Common HTTP status codes for errors include

- 4xx series for client errors
 - e.g., 404 Not Found
- 5xx series for server errors
 - e.g., 500 Internal Server Error

Error responses typically include an error message

- Optionally, additional details about the error

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
Content-Language: en

{
  "type": "https://example.net/validation-error",
  "title": "Your request parameters didn't validate.",
  "invalid-params": [
    {
      "name": "age",
      "reason": "must be a positive integer"
    },
    {
      "name": "color",
      "reason": "must be 'green', 'red' or 'blue'"
    }
  ]
}
```

HTTP Status

Error Code / Type

Source: <https://joshimisko.com/posts/rest-api-error-handling>

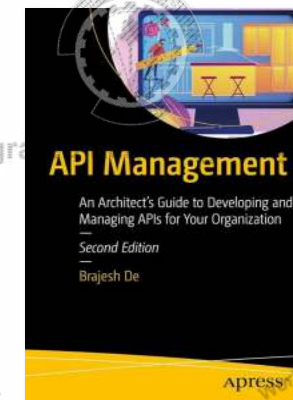
BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- The essential components of REST API communication, including
 - Request methods (GET, POST, PUT, DELETE),
 - Resource-based URLs,
 - Headers, and
 - Response format and status codes



Work Integrated Learning Programmes



MCQ1

What HTTP method is commonly used to retrieve data from a server in a RESTful API?

- a) GET
- b) POST
- c) PUT
- d) DELETE

Answer: a) GET



Work Integrated Learning Programmes



MCQ2

Which component of a REST API request typically contains data to be sent to the server for processing?

- a) URL
- b) Headers
- c) HTTP Method
- d) Body

Answer: d) Body



Work Integrated Learning Programmes



MCQ3

What is the purpose of HTTP status codes in REST API responses?

- a) To specify the content type of the response
- b) To indicate the outcome of the request
- c) To define the structure of the response body
- d) To authenticate the client

Answer: b) To indicate the outcome of the request



Work Integrated Learning Programmes



API-Based Products

HTTP Headers

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

HTTP Headers

Information sent between a client (e.g., browser) and a server in HTTP requests and responses

- Every HTTP transaction includes headers
- Consist of key-value pairs

Structure of HTTP Headers

- General format: HeaderName: Value
- Separated by a colon (:)
- May contain multiple headers in a single request or response

Request URI: <http://www.example.com>

```
HTTP/1.1 200 OK
Content-Encoding: gzip
Age: 521648
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Fri, 06 Mar 2020 17:36:11 GMT
Etag: "3147526947+gzip"
Expires: Fri, 13 Mar 2020 17:36:11 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECS (dcb/7EC9)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 648
```

Learning Outcomes

- Understand the role and importance of HTTP headers in web communications

Importance of HTTP Headers

- **Control and Optimization:**
 - Enable control over
 - Caching,
 - Content negotiation,
 - and redirection
- **Security:**
 - Allow for secure communication through headers like
 - Content-Security-Policy
 - Strict-Transport-Security
- **Customization:**
 - Support custom headers for specific application needs

Types of HTTP Headers

Request Headers:

- Sent by the client to the server
- Examples: User-Agent, Accept, Cookie

Response Headers:

- Sent by the server to the client
- Examples: Content-Type, Set-Cookie, Server

Entity Headers:

- Associated with the body of the request or response
- Examples: Content-Length, Content-Encoding

Common HTTP Headers

User-Agent:

Identifies the client application making request

Accept:

Specifies the media types accepted by client

Content-Type:

Indicates the media type of the resource sent in the response

Cache-Control:

Directives for caching mechanisms

Cookie:

Used for sending stored cookies from the client to the server

Example Use Case: Content Negotiation

Content negotiation is the **process by which servers select the most appropriate representation of a resource to send to a client based on client preferences**

The Accept Header

- A crucial component of content negotiation in HTTP requests
- Specifies the media types that the client is willing to accept, typically in order of preference

Example Use Case: Content Negotiation

Client Preferences

- Clients can express their preferences through the "Accept" header, indicating the desired content types, character sets, languages, and encodings
- Example: "Accept: application/json;q=0.8, text/html;q=0.5, /;q=0.1"

Server Selection

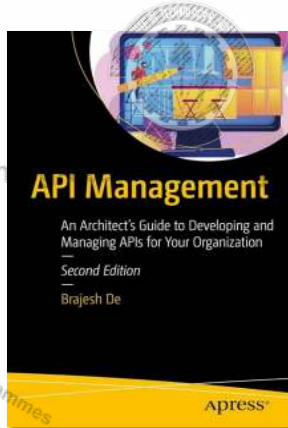
- Upon receiving a request with the "Accept" header, the **server examines the provided preferences and available representations of the requested resource**
- The server then **selects the most appropriate representation based on factors such as content type, language, encoding, and quality parameters**

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



Summary

- Different types of HTTP headers
- Importance of HTTP headers in web communication

MCQ1

What is the purpose of the "Accept" header in HTTP requests?

- A) To specify the desired content type of the requested resource
- B) To provide authentication credentials to the server
- C) To indicate the user agent making the request
- D) To instruct the server on caching directives

Answer: A) To specify the desired content type of the requested resource

MCQ2

Which negotiation approach allows the server to select the representation without considering client preferences?

- A) Agent-driven negotiation
- B) Server-driven negotiation
- C) Content-driven negotiation
- D) Client-driven negotiation

Answer: B) Server-driven negotiation

MCQ3

How do HTTP headers contribute to web security?

- A) By specifying the user agent in requests
- B) By enabling content negotiation between clients and servers
- C) By implementing mechanisms like Cross-Origin Resource Sharing (CORS) and Content Security Policy (CSP)
- D) By controlling caching behavior and reducing latency

Answer: C) By implementing mechanisms like Cross-Origin Resource Sharing (CORS) and Content Security Policy (CSP)

API-Based Products

API Authentication and Authorization | Suresh Gopal

Learning Outcomes

- Explain the significance of authentication and authorization in securing RESTful APIs

Overview of Authentication and Authorization in RESTful APIs:

Authentication and authorization play crucial roles in ensuring the security and integrity of RESTful APIs

Authentication: Verifying the identity of clients

Authorization: Granting access to resources based on permissions



Image source: <https://nordicapis.com/what-is-the-difference-between-authentication-and-authorization/>

Authentication

Authentication is the process of validating the identity of clients or users attempting to access an API

- Ensures that **only authorized entities can perform actions within the system**, protecting against unauthorized access and misuse
- Serves as **the first line of defense against unauthorized access and data breaches**

Authentication Mechanism

Username and Password

API Keys

OAuth (Open Authorization)

JSON Web Tokens (JWT)

BITS WILP | Generated: 12-May-2026 16:47:54

Authentication Mechanisms

Username and Password:

- Basic authentication where clients provide credentials (username and password) with each request
- Not be secure for all scenarios

API Keys:

- Unique tokens generated by the server and provided to clients for authentication
- often included in request headers or query parameters

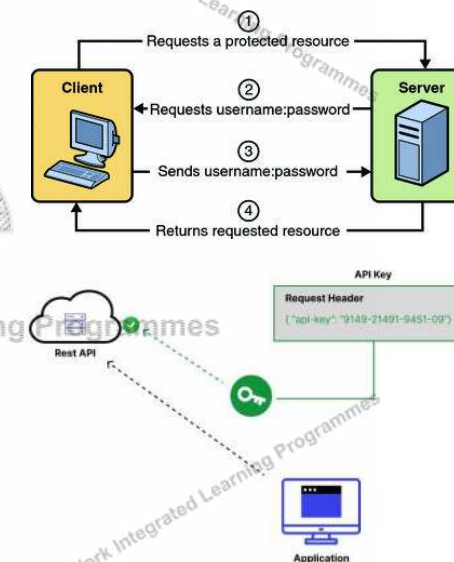


Image source: <https://docs.oracle.com/cd/E19159-01/819-3669/bncbo/index.html>
<https://www.wallarm.com/what/what-is-an-api-authentication>

BITS WILP | Generated: 12-May-2026 16:47:54

Authentication Mechanisms

OAuth (Open Authorization):

- An industry-standard protocol for delegated authorization, allowing clients to access resources on behalf of a user
- Supports scenarios such as user authentication, authorization, and API access delegation

JSON Web Tokens (JWT):

- Compact, URL-safe tokens that contain claims (such as user identity and permissions) and are digitally signed for authentication and data integrity
- Commonly used for stateless authentication in distributed systems

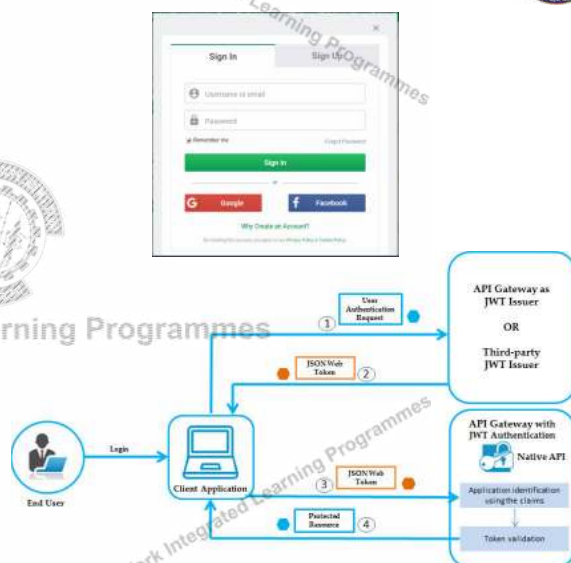


Image Source: <https://www.geeksforgeeks.org/what-is-oauth-open-authorization/>
https://documentation.softwareag.com/webmethods/api_gateway/yai/10-7-10-7-API_Gateway_webhelp/yai-webhelp/images/workflow_jwt.png

BITS WILP | Generated: 12-May-2026 16:47:54

Authentication Mechanisms



Source: https://dev.to/angha_ramdhokar_0b6505c2/api-authentication-jwt-vs-oauth-802

Authorization

Authorization is the process of controlling access to resources or operations within an API based on the authenticated identity of clients.

Ensures that authenticated users or systems only have access to the resources and functionalities they are explicitly permitted to use

Enforces access control policies to protect sensitive data and functionalities from unauthorized access or misuse

BITS WILP | Generated: 12-May-2026 16:47:54

Authorization Mechanisms

Role-Based Access Control (RBAC):

- Assigning roles (e.g., admin, user) to users or entities and defining permissions associated with each role
- Simplifies access control management by grouping users with similar privileges

Attribute-Based Access Control (ABAC):

- Making access decisions based on various attributes (e.g., user attributes, resource properties, environmental factors)
- Provides fine-grained access control based on dynamic conditions

RBAC: Role Based Access Control

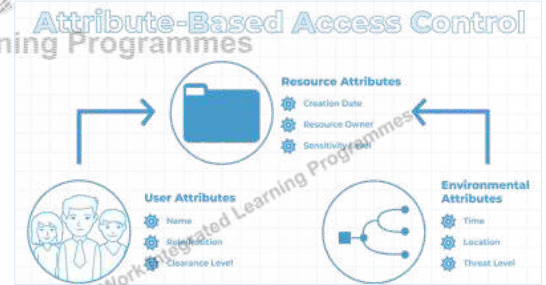
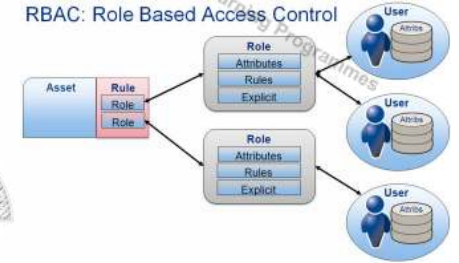


Image Source: <https://identitysander.wordpress.com/2009/11/03/rbac-and-abac-and-roles-oh-my/>
<https://www.okta.com/identity-101/role-based-access-control-vs-attribute-based-access-control/>

Authorization Mechanisms

OAuth Scopes:

- In OAuth, scopes define the specific permissions granted to a client when accessing protected resources
- Allow for granular control over API access, ensuring that clients only have access to the resources they need

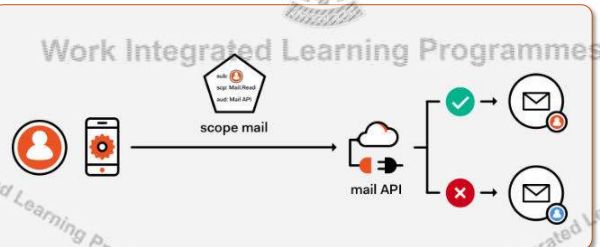


Image source: <https://auth0.com/blog/on-the-nature-of-oauth2-scopes/>

Common Methods

API Keys:

Simple and effective for authenticating clients, but may lack granular access control.

OAuth:

Provides robust authentication and authorization capabilities, suitable for scenarios involving user authentication and third-party access delegation.

JWT:

Offers stateless authentication and enables secure transmission of user claims, suitable for microservices architectures and distributed systems.

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Difference between authentication and authorization
- Various mechanisms used for authentication and authorization



Work Integrated Learning Programmes



MCQ1

Which authentication method involves the use of unique tokens provided to clients for identification?

- A) Basic Authentication
- B) OAuth
- C) API Keys
- D) JWT

Answer: C) API Keys



Work Integrated Learning Programmes



MCQ2

Which authorization method involves assigning roles to users or entities and defining permissions associated with each role?

- A) Role-Based Access Control (RBAC)
- B) Attribute-Based Access Control (ABAC)
- C) OAuth Scopes
- D) Basic Authentication

Answer: A) Role-Based Access Control (RBAC)



Work Integrated Learning Programmes



MCQ3

Which authentication protocol allows users to grant access to their resources without sharing their credentials and supports scenarios like user authentication and third-party access delegation?

- A) Basic Authentication
- B) OAuth
- C) API Keys
- D) JWT

Answer: B) OAuth



Work Integrated Learning Programmes



API-Based Products

Error Handling and Status Codes in REST APIs

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Importance of Error Handling

Error handling is a critical aspect of building robust and reliable APIs

- **Facilitates** clear **communication** between clients and servers
- **Enhances** user experience by providing meaningful **error messages**
- **Simplifies** **debugging** and **troubleshooting** for developers

Foundational element that underpins the reliability, usability, and trustworthiness of RESTful APIs

Learning Outcomes

- Understand the importance of error handling in building robust and reliable APIs



Work Integrated Learning Programmes

Best Practices for Error Handling

Provide **descriptive error messages**:

Clearly explain the encountered issue

Use appropriate **HTTP status codes**:

Select the most suitable status code for each scenario

Maintain **consistency**:

Ensure uniformity across endpoints for error responses

Leverage **standard formats**:

Adopt common error response formats like JSON API or Problem Details



Problem Details for HTTP APIs (RFC7807)

Error Response Structure

Structure of an error response:

Status code: HTTP status code indicating the nature of the error

Error message: A human-readable message describing the error

Error code (optional): Machine-readable code for programmatic error handling

Additional details (optional): Supplementary information, such as error causes or resolution steps

```
Body Cookies Headers (4) Test Results
Pretty Raw Preview Visualize JSON
1 {
2   "type": "https://example.net/validation-error",
3   "title": "Validation Error",
4   "status": 400,
5   "detail": "The request parameters failed to validate.",
6   "invalidParams": [
7     {
8       "name": "name",
9       "reason": "Cannot be blank."
10    },
11    {
12      "name": "age",
13      "reason": "Must be greater than or equal to 18."
14    }
15  ]
16 }
```

Common HTTP Status Codes

Overview of common status code categories:

2xx: Success (e.g., 200 OK)

3xx: Redirection (e.g., 301 Moved Permanently)

4xx: Client Errors (e.g., 400 Bad Request, 404 Not Found)

5xx: Server Errors (e.g., 500 Internal Server Error)

HTTP Status Codes		
Level 200	Level 400	Level 500
200: OK 201: Created 202: Accepted 203: Non-Authoritative Information 204: No content	400: Bad Request 401: Unauthorized 403: Forbidden 404: Not Found 409: Conflict	500: Internal Server Error 501: Not Implemented 502: Bad Gateway 503: Service Unavailable 504: Gateway Timeout 599: Network Timeout

Example Error Scenarios

Various error scenarios and their corresponding status codes:

Invalid Input (400 Bad Request)

Unauthorized Access (401 Unauthorized)

Resource Not Found (404 Not Found)

Method Not Allowed (405 Method Not Allowed)

Request Timeout (408 Request Timeout)

Conflict (409 Conflict)

Rate Limit Exceeded (429 Too Many Requests)

Server Error (500 Internal Server Error)

Service Unavailable (503 Service Unavailable)

Handling Authentication and Authorization Errors

Authentication errors (e.g., 401 Unauthorized)

- Should prompt clients to provide valid credentials or tokens to access protected resources

Authorization errors (e.g., 403 Forbidden)

- Should inform clients that they lack the necessary permissions to perform the requested operation

Clear error messages and appropriate status codes help guide users and developers in resolving authentication and authorization issues effectively

Rate Limiting and Throttling

Rate limiting and throttling are essential mechanisms employed by API providers to manage the rate of incoming requests from clients.

- Serve to prevent abuse, ensure fair usage of resources, and maintain the overall stability and performance of the API infrastructure

Return appropriate status codes (e.g., 429 Too Many Requests) when rate limits are exceeded.

- Include rate limit information in API documentation for client awareness

Internationalization and Localization

Support internationalization by offering **error messages in multiple languages**

Utilize the **Accept-Language header** to determine the preferred language of the client

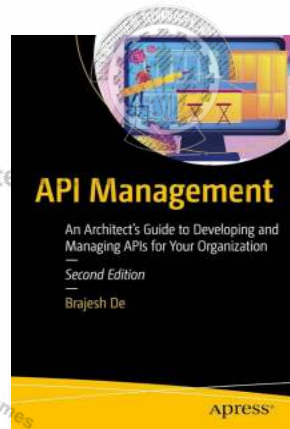
Serve **localized error messages** based on client preferences to enhance user experience

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



Summary

- Common HTTP status codes
 - Their meanings,
 - Distinguishing between success, client error, and server error categories
- Best practices for error handling, including
 - Providing descriptive error messages,
 - Using appropriate status codes,
 - and maintaining consistency across endpoint

MCQ1

What HTTP status code should be returned when a client exceeds the rate limit in an API?

- a) 401 Unauthorized
- b) 403 Forbidden
- c) 429 Too Many Requests
- d) 500 Internal Server Error

Answer: c) 429 Too Many Requests

MCQ2

Which of the following is NOT a recommended practice for implementing rate limiting in REST APIs?

- a) Defining rate limits based on user account or API key
- b) Using a token bucket algorithm for rate limiting
- c) Providing no information about rate limits in API documentation
- d) Specifying rate limits granularity (e.g., per minute, per hour)

Answer: c) Providing no information about rate limits in API documentation

MCQ3

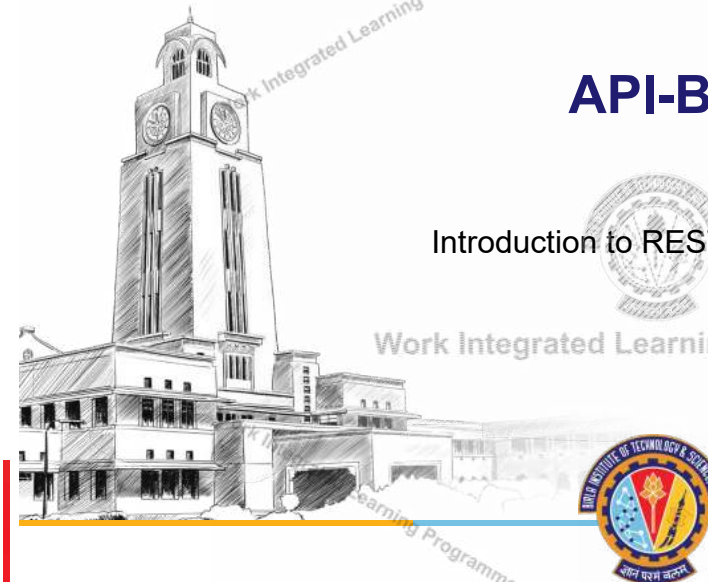
Why is it important to include rate limit information in API documentation?

- a) To confuse API consumers
- b) To provide guidance on handling rate limit responses programmatically
- c) To keep rate limits secret from users
- d) To discourage API usage

Answer: b) To provide guidance on handling rate limit responses programmatically

API-Based Products

Introduction to REST APIs | Suresh Gopal



Learning Outcomes

- Learn the foundational principles and constraints of REST architecture.
 - For designing scalable, reliable, and interoperable networked applications.



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Introduction to REST

What is REST?

Stands for **Representational State Transfer**

An **architectural style** for designing networked applications

Emphasizes

- Scalability,
- Performance,
- Simplicity,
- and reliability

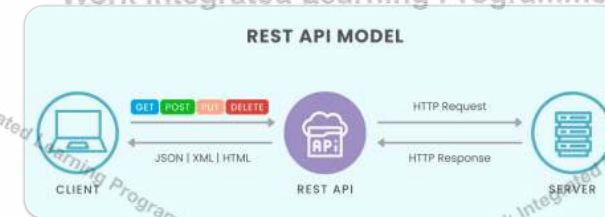


Image source: <https://www.skiplevel.co/blog/part-2-rest-api-components-how-to-read-them>

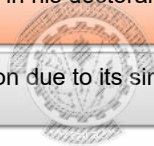
BITS WILP | Generated: 12-May-2026 16:47:54

Introduction to REST

History

Introduced by Roy Fielding in his doctoral dissertation in 2000

Gained widespread adoption due to its simplicity and flexibility



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



REST Constraints

Client-Server:

Separation of concerns between client and server

Statelessness:

Each request from the client to the server must contain all the necessary information to understand the request, and the server cannot store any client state between requests

Cacheability:

Responses from the server must be explicitly labeled as cacheable or non-cacheable

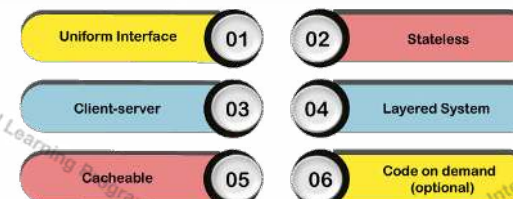


Image source: <https://static.javatpoint.com/blog/images/what-is-rest.png>

BITS WILP | Generated: 12-May-2026 16:47:54



REST Constraints

Uniform Interface:

The interface between client and server must be uniform, which promotes simplicity, scalability, and reusability

Layered System:

A client cannot ordinarily tell whether it is connected directly to the server or to an intermediary along the way, such as a load balancer or proxy

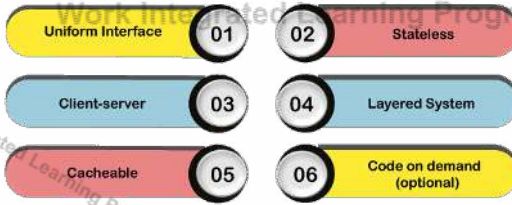


Image source: <https://static.javatpoint.com/blog/images/what-is-rest.png>

Key Principles of REST

Resource Identification

- **Resources:** Everything in the system is a resource, which can be a document, image, or any other representation of data
- **Identification:** Each resource must have a unique identifier, typically expressed as a URI (Uniform Resource Identifier)

Representation

- **Representation:** Resources can have multiple representations (e.g., JSON, XML, HTML)
- **Content Negotiation:** Clients and servers must agree on the representation format during interactions

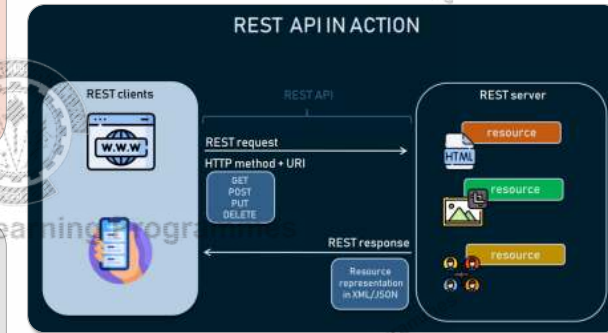


Image source: <https://www.altexsoft.com/blog/rest-api-design/>

BITS WILP | Generated: 12-May-2026 16:47:54

Key Principles of REST

Statelessness

- **Stateless Communication:** Each request from a client to a server must contain all the information necessary to understand the request
- **No Client Context:** The server should not store any client state between requests

Uniform Interface

- **Consistent Interface:** The interface between client and server should be uniform
- **Four Aspects:**
 - Identification of resources,
 - Manipulation of resources through representations,
 - Self-descriptive messages
 - and hypermedia as the engine of application state (HATEOAS).

BITS WILP | Generated: 12-May-2026 16:47:54

Key Principles of REST

Hypermedia as the Engine of Application State (HATEOAS).

- **Dynamic Navigation:** Clients interact with a REST application entirely through hypermedia provided dynamically by the server
- **Decoupling:** Decouples client and server, allowing server-side changes without affecting client behavior



Source: <https://grapeup.com/blog/how-to-build-hypermedia-api-with-spring-hateoas/>

BITS WILP | Generated: 12-May-2026 16:47:54

Benefits of REST

Scalability:

RESTful architectures are inherently scalable due to their stateless nature and caching capabilities

Simplicity:

REST principles promote simplicity in design and implementation, making it easier to develop and maintain systems

Flexibility:

Supports multiple data formats and can adapt to changing requirements

Reliability:

Stateless communication reduces the likelihood of server-side errors due to the absence of client context

Interoperability:

RESTful services can be accessed from any platform or programming language that supports HTTP

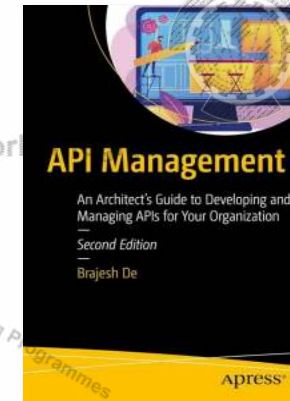
BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- REST principles and constraints which provides a flexible and scalable architectural style for designing networked applications
 - Ensuring simplicity, reliability, and interoperability



Work Integrated Learning Programmes

MCQ1

What does REST stand for in the context of networked applications?

- A) Responsive State Transition
- B) Representational State Transfer
- C) Resourceful Service Technology
- D) Remote Execution System

Answer: B) Representational State Transfer



Work Integrated Learning Programmes

MCQ2

Which of the following is NOT one of the constraints of REST architecture?

- A) Statelessness
- B) Cacheability
- C) Session Persistence
- D) Uniform Interface

Answer: C) Session Persistence

MCQ3

What does HATEOAS stand for in the context of REST architecture?

- A) Hypermedia as the Engine of Application State
- B) Hypertext Application Token Endpoint and Object Storage
- C) Hierarchical Attribute Transmission for Enhanced Object Serialization
- D) Hyperlink and Textual Encoding for Application State

Answer: A) Hypermedia as the Engine of Application State

API-Based Products

Resources

Suresh Gopal

Learning Outcomes

- Understand the key abstraction named "resource" in RESTful API design

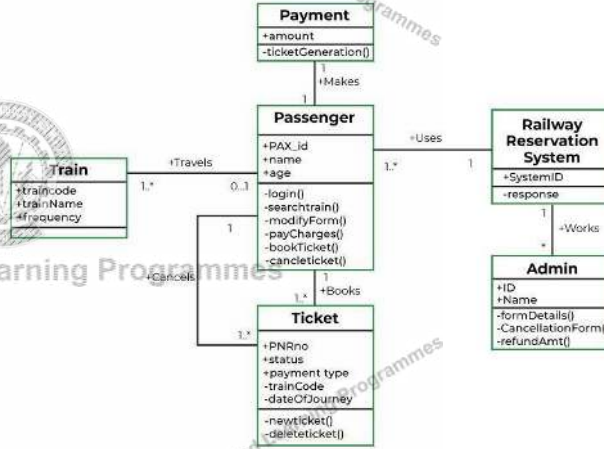
Understanding Resources

Resources are fundamental entities in web-based systems, identified by unique identifiers (URIs) and manipulated via a uniform interface

Examples of resources in a travel portal REST API include

Customers Reservations Tickets

Hotels Flights Buses Cars



<https://www.geeksforgeeks.org/online-railway-ticket-reservation-system/>

BITs WILP | Generated: 12-May-2026 16:47:54

Understanding Resources

Resources can be singular entities or collections of entities

A resource's URI provides its name and network address,

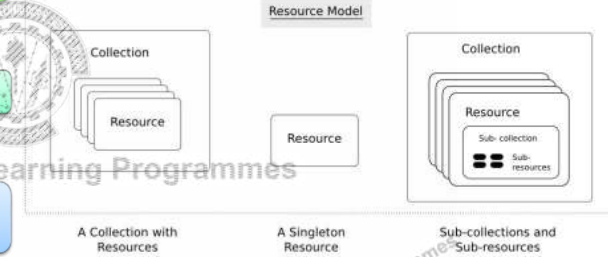
- Enabling clients to connect to servers and access resources using methods like GET

Resources can be related to each other,

- Forming sub-resources within a hierarchical structure

An example URI, such as <http://www.foo.com/v1/customers>,

- Identifies a resource named "customers"



<https://restful-api-design.readthedocs.io/en/latest/resources.html>

BITs WILP | Generated: 12-May-2026 16:47:54

Resource Naming Conventions

Well-designed resources contribute to an intuitive API experience, making it easy for developers to understand and use.

Resource Naming Conventions:

- Each resource should have a **meaningful, noun-based name** to identify itself
- URIs should **reference things rather than actions**, avoiding CRUD function names in resource names or URIs
- For example, instead of `"/getCustomers,"` a resource for customer entities should be named `"/customers."`

Resource Representation

Resources are conceptually **distinct from their representations returned to clients**

When a server responds to a request for a resource, it returns a representation

- (e.g., XML, JSON, HTML) rather than the resource itself

Representations provide a structured view of the resource's data,

- Facilitating communication between clients and servers.

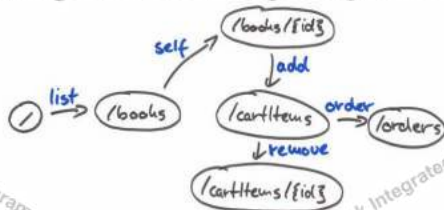
Hypermedia as the Engine of Application State (HATEOAS)

Resource state information often includes links to other resources,

- Guiding clients on next actions and traversal through related resources

For example, a RESTful service providing account information might include links for actions like deposit, withdraw, or transfer

- These links are embedded within the response and provide URLs for subsequent actions
- The presence or absence of links in a resource representation reflects the resource's current state



<https://reflecting.io/rest-hypermedia/>

BITS WILP | Generated: 12-May-2026 16:47:54

Best Practices for Identifying REST API Resources

Avoid overly fine-grained resources

- To prevent excessive communication between the consumer and provider,
- Can degrade app performance

Steer clear of overly coarse-grained resources

- Can result in APIs that are challenging to use and maintain

Design resources to prevent migration of control flow business logic to the API consumer side,

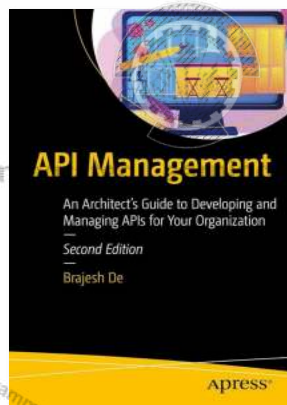
- Ensuring that the server retains responsibility for executing necessary flow logic

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



Summary

- RESTful API design principles, including
 - Resource identification,
 - Resource representation,
 - Resource naming conventions
 - Use of Hypermedia as the Engine of Application State (HATEOAS)

MCQ1

What is a key consideration when designing resources for a RESTful API?

- a) Maximizing fine-grained resources for detailed control
- b) Minimizing coarse-grained resources to simplify API usage
- c) Avoiding self-descriptive messages to reduce message overhead
- d) Promoting tight coupling between API consumer and provider

Answer: b) Minimizing coarse-grained resources to simplify API usage

MCQ2

Which HTTP header is commonly used to specify the format of a message in a self-descriptive RESTful API?

- a) Content-Size
- b) Content-Encoding
- c) Content-Type
- d) Content-Language

Answer: c) Content-Type

MCQ3

How does Hypermedia as the Engine of Application State (HATEOAS) contribute to RESTful API design?

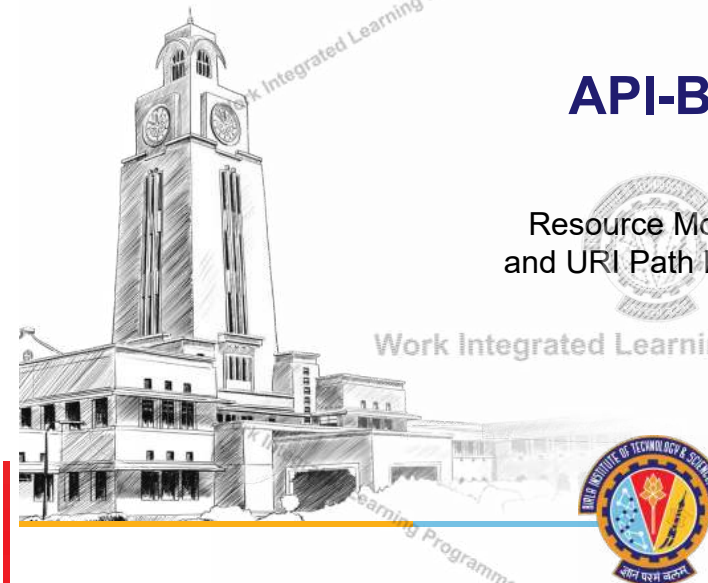
- a) By reducing the need for self-descriptive messages
- b) By embedding links in resource representations for dynamic navigation
- c) By minimizing the use of resource naming conventions
- d) By enforcing tight coupling between API consumer and provider

Answer: b) By embedding links in resource representations for dynamic navigation

API-Based Products

Resource Modeling
and URI Path Design

Suresh Gopal



Learning Outcomes

- Understand the principles and importance of resource modeling and URI path design in RESTful API development



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Resources

Per Roy Fielding's dissertation a resource "is any concept that might be the target of an author's hypertext reference must fit within the definition of a resource"

Resource modeling in REST API design encompasses any concept that can be targeted by a hypertext reference

Resources can represent single objects or collections,

- Including business processes and capabilities



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

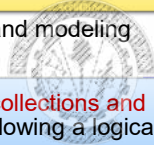


Modelling Resources and Sub Resources

The process of modeling resources starts with analyzing the business domain and identifying relevant objects from the consumer's perspective

- Granularity is crucial in selecting and modeling resources effectively

URI paths are structured to access collections and individual instances of resources, following a logical hierarchy



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



URI Example

A resource can be a collection of customers in an online store or it can be a single customer

- Can identify a collection of 'customers' using `/customers`,
- A single instance of a customer can be identified using `/customers/{customerId}`

Each customer may further have multiple orders

- Refer to the subcollection of 'orders' is modelled as `/customers/{customerId}/orders`
- A single instance of the order may be identified by `/customers/{customerId}/orders/{orderId}`

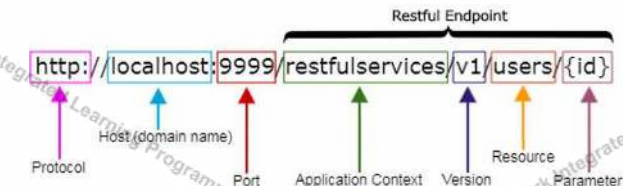
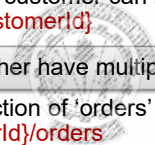


Image source: <https://medium.com/@nadinCodeHat/rest-api-naming-conventions-and-best-practices-1c4e781eb6a5>



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



URI Path Design

Each collection and resource in an API is assigned its own unique URL,

- Typically derived from a combination of collection and resource path segments relative to the API entry point

Advisable to design URLs with an alternative combination of these segments,

While sub-resources can be nested to arbitrary levels, recommended to keep nesting depth limited to two or three levels, where feasible,

- To avoid excessively long URLs that can be cumbersome to manage

Resource Hierarchy

URL	Description
/api	Main entry point /api
/api/{ResourceName}	Accessing top level resource /api/users
/api/{ResourceName}/{ResourceID}	Accessing individual user /api/users/user1
/api/{ResourceName}/{ResourceID}/{SubResourceName}	Accessing resource under top level resource /api/users/user1/accounts
/api/{ResourceName}/{ResourceID}/{SubResourceName}/{SubResourceID}	Accessing particular resource under top level resource /api/users/user1/accounts/acct1

URI and URN

The URI, or Uniform Resource Identifier.

Serves as a concise string of characters for pinpointing an abstract or tangible resource, according to RFC 2396

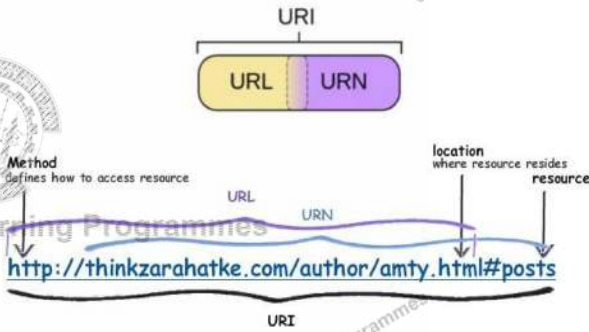
Two principal forms of URIs exist:

Uniform Resource Locators (URLs)

- Indicate the online whereabouts of a particular resource
- http://www.foo.com/users/dhoni

Uniform Resource Names (URNs)

- Serve as persistent, location-independent identifiers
- urn:user:mike

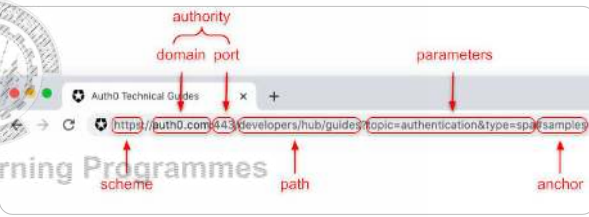


URI Format

The URI syntax comprises various components:

scheme:[//authority] [/]path[?query] [#fragment]

- Scheme name:** Specifies the protocol, like FTP, HTTP, or HTTPS.
- Authority:** Refers to the server's DNS resolution, typically encompassing the hostname or IP address
- Path:** Represents a sequence of segments divided by forward slashes
- Query:** Houses supplementary identification data, separated by a question mark, and typically non-hierarchical
- Fragment:** Guides to a secondary resource within the primary one identified by the authority and path, delimited by a hash symbol



Naming Conventions for URI Paths

Use plural nouns for collection resources,

`http://www.foo.com/api/customers`

Use Singular nouns for individual resources,

`http://www.foo.com/api/customers/customer1`

Use verbs for controller resources,

`http://www.foo.com/api/customers/customer1234/register`

Avoid CRUD operation names in URIs,

`http://www.foo.com/api/getcustomers`

Naming Conventions for URI Paths

Utilization of lowercase letters

Exclusively in URI naming to streamline typing and reading

Preference for hyphens over spaces or underscores

For improved aesthetics and readability, as spaces in URLs are transformed into URL-encoded %20s, complicating readability

Steering clear of characters

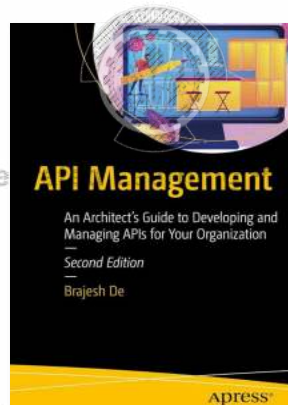
Requiring URL encoding, such as spaces, to maintain simplicity and clarity

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



Summary

- The fundamental concepts of resource modeling and its pivotal role in REST API design
- The principles of URI path design
- The best practices for constructing URI paths
- The recommended naming conventions

MCQ1

What is the recommended naming convention for identifying a collection resource in a RESTful API URI?

- A) Singular noun
- B) Plural noun
- C) Verb
- D) Controller name

Answer: B) Plural noun

MCQ2

According to RFC 2396, which of the following best describes the difference between a Uniform Resource Locator (URL) and a Uniform Resource Name (URN)?

- A) A URL provides the identity of a resource, while a URN provides its location.
- B) A URL functions like a person's name, while a URN functions like a street address.
- C) A URL is a compact string of characters, while a URN is a hierarchical sequence of components.
- D) A URL identifies an online location of a resource, while a URN serves as a persistent, location-independent identifier.

Answer: D) A URL identifies an online location of a resource, while a URN serves as a persistent, location-independent identifier.

MCQ3

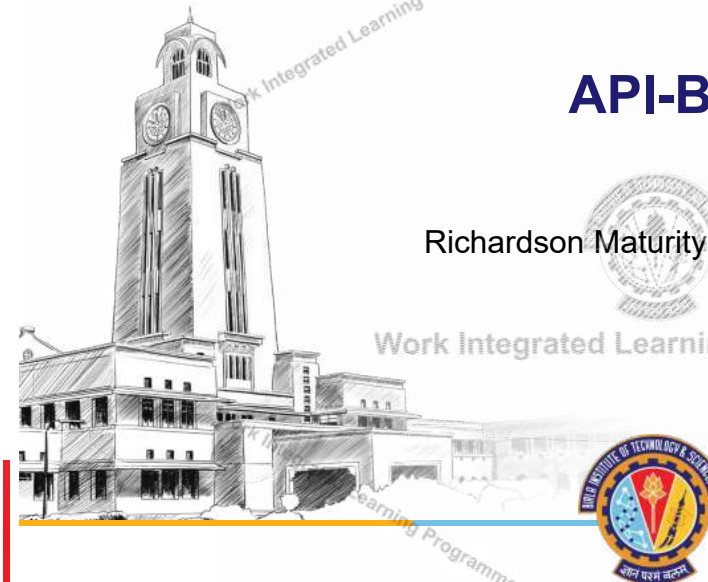
What is the recommended approach for designing URI paths to access resources in a RESTful API?

- A) Use uppercase letters for improved readability.
- B) Avoid hyphens and prefer spaces for aesthetic appeal.
- C) Follow a hierarchical structure with predictable patterns.
- D) Include CRUD operation names in the URI for clarity.

Answer: C) Follow a hierarchical structure with predictable patterns.

API-Based Products

Richardson Maturity Model | Suresh Gopal



Learning Outcomes

- Gain the importance of adopting the Richardson Maturity Model for API design and development



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Richardson Maturity Model

The Richardson Maturity Model is a framework proposed by Leonard Richardson that **categorizes web APIs based on their levels of maturity**

Importance:

- Helps in assessing the quality and readiness of an API

Four Levels

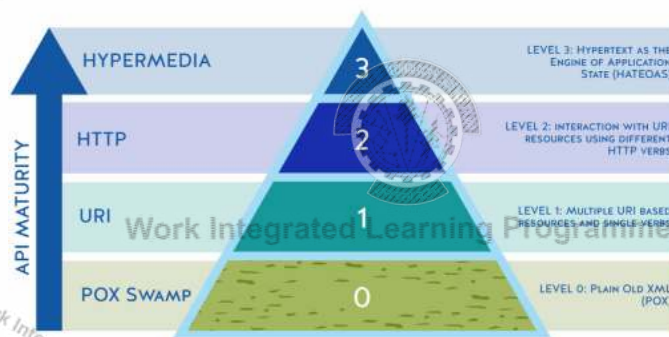
- Level 0: Swamp of POX
- Level 1: Resources
- Level 2: HTTP verbs
- Level 3: Hypermedia controls



BITS WILP | Generated: 12-May-2026 16:47:54

Richardson Maturity Model

RICHARDSON MATURITY MODEL



The Four Levels of the RMM

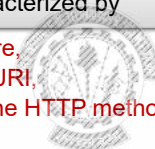
Level 0: The Swamp of POX (Plain Old XML)

APIs at this level are characterized by

- Absence of URI structure,
- Usage of only a single URI,
- and utilization of only one HTTP method (usually POST)

Example:

- SOAP-based web services without RESTful principles



The Four Levels of the RMM

Level 1: Resources

APIs at this level **start introducing separate URIs for resources**,

- But all operations are performed using a **single HTTP method** (mostly POST)

Example:

- Basic CRUD operations on resources without utilizing HTTP methods correctly

The Four Levels of the RMM

Level 2: HTTP Verbs

APIs at this level utilize HTTP methods (GET, POST, PUT, DELETE) correctly to perform operations on resources

Example:

- RESTful APIs where each HTTP method is used for its intended purpose
 - GET for retrieval,
 - POST for creation,
 - PUT for update,
 - DELETE for deletion

The Four Levels of the RMM

Level 3: Hypermedia Controls (HATEOAS)

APIs at this level **include hypermedia controls in responses**,

- Enabling clients to navigate through the API dynamically

Example:

- APIs where responses include links to related resources, allowing clients to discover available actions dynamically

Advantages of Using the RMM

Provides a clear framework for evaluating the maturity of APIs

Helps in setting goals for API design and development

Facilitates communication and understanding among development teams

Encourages adherence to RESTful principles, leading to more scalable and maintainable APIs

Best Practices for Advancing API Maturity



Start with a solid foundation:

Ensure adherence to RESTful principles from the beginning of API development

Incremental Improvements:

Gradually evolve APIs from lower to higher maturity levels, focusing on one aspect at a time

Testing and Validation:

Regularly test APIs against the Richardson Maturity Model to track progress and identify areas for improvement

BITS WILP | | Generated: 12-May-2026 16:47:54

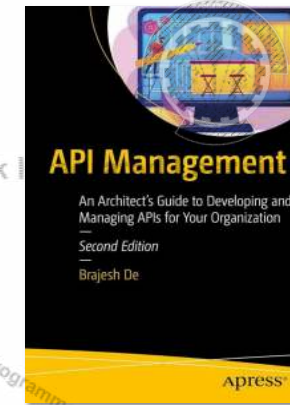
Reference



API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary



- The Richardson Maturity Model which
 - Provides a structured framework for evaluating and advancing the maturity of web APIs,
 - Guiding developers through four levels from basic resource handling to advanced hypermedia controls.



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1



What is the Richardson Maturity Model primarily used for?

- a) Evaluating the performance of web servers
- b) Assessing the maturity of web APIs
- c) Ranking programming languages
- d) Analyzing database structures



Correct answer: b) Assessing the maturity of web APIs

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Which level of the Richardson Maturity Model introduces separate URIs for resources and correct usage of HTTP methods?

- a) Level 0: The Swamp of POX (Plain Old XML)
- b) Level 1: Resources
- c) Level 2: HTTP Verbs
- d) Level 3: Hypermedia Controls (HATEOAS)

Correct answer: c) Level 2: HTTP Verbs

MCQ3

What is the main advantage of advancing to Level 3 in the Richardson Maturity Model?

- a) Increased utilization of Plain Old XML (POX)
- b) Simplification of API structure
- c) Introduction of hypermedia controls enabling dynamic navigation
- d) Reduction in the number of HTTP methods used

Correct answer: c) Introduction of hypermedia controls enabling dynamic navigation

API-Based Products

REST API Design process | Suresh Gopal

Learning Outcomes

- Understand the process of REST API design

REST API Design Process

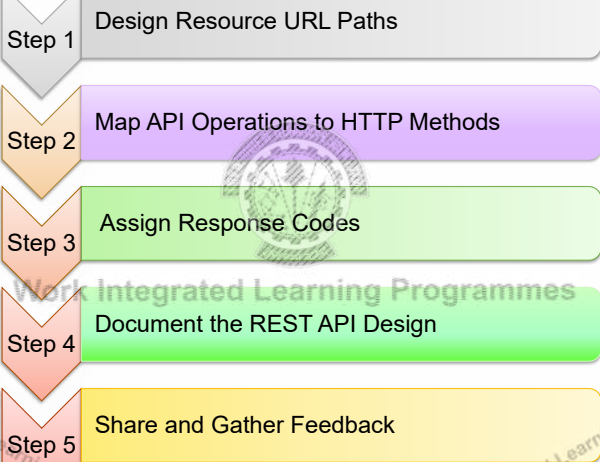
API modeling captures and explores API requirements through profiles

API design maps profiles to HTTP using REST-based principles

Time to transition from API modeling to API design

- With emphasis on modeling effort in API design

Five Quick Steps for High-level API Design



Step 1: Design Resource URL Paths

Utilize the API resources and relationships identified during API modeling

Transfer the list of resources from the API profile into a tabular format

Transition operations, along with their descriptions and request/response details, into a new tabular layout aimed at capturing the high-level design

Step 1: Design Resource URL Paths

Indent dependent resources slightly to indicate relationships, aiding in URL path establishment

Convert each resource name to a URL-friendly format, using lowercase letters and hyphens instead of spaces

- Begin with a leading slash and pluralize the resource name to signify a collection of instances

Nest dependent resources under the parent, requiring the parent identifier in the path for interaction

Resource collections typically employ plural names, although not mandatory, following REST API conventions

Resource Path
Books
Book Authors
Carts
Cart Items

Resource Path
/books
/carts
/carts/{cartId}/items
/authors

Step 1: Design Resource URL Paths

Transfer the list of resources from the API profile into a tabular format

Resource Path	Operation Name	HTTP Method	Description	Request	Response
/books	listBooks()		List books by category or release date	categoryIdreleaseDate	Book[]
/books/search	searchBooks()		Search for books by author, title	searchQuery	Book[]
/books/{bookId}	viewBook()		View book details	bookId	Book
/carts/{cartId}	viewCart()		View the current cart and total	cartId	Cart
/carts/{cartId}	clearCart()		Remove all books from the customer's cart	cartId	Cart
/carts/{cartId}/items	addItemToCart()		Add a book to the customer's cart	cartId	Cart
/carts/{cartId}/items/{cartItemId}	removeItemFromCart()		Remove a book from the customer's cart	cartIdcartItemId	Cart
/authors	getAuthorDetails()		Retrieve the details of an author	authorId	BookAuthor

Migrate the API profile's list of operations, descriptions, and request/response details into a new design-centric tabular format.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Map API Operations to HTTP Methods

Determine suitable HTTP methods for each operation, considering their safety classifications

- Identify common verbs in operation names or descriptions from the API profile, often indicating the most suitable HTTP method
- Combine safety classifications with operation mappings to select appropriate HTTP methods

Include the resource identifier in the path if the operation interacts with a specific resource instance

Common HTTP Methods

Method	Description
GET	Requests a resource. Additional data can be passed to the server through query strings in the URL (e.g. ?param=value).
POST	Sends data to the server. Can handle numerous types of input such as text, PDFs, and binary data. This data is attached to the request body after the headers. This method is commonly used when submitting information (forms/logins) or uploading to websites such as images and documents.
HEAD	Requests the headers that would be returned if a GET request were made to the server. It is usually done to check the size of the response before downloading the resources.
PUT	Creates new resources on the server. Allowing this method without proper controls can lead to the upload of malicious resources.
DELETE	Deletes existing resources on the web server. It can lead to a DoS (Denial of Service) attack if not protected correctly, removing critical files from the server.
OPTIONS	Returns information about the server such as the methods it accepts.
PATCH	Applies partial modifications to the resources at the specified location.

Source: <https://medium.com/@gwenil/orac/http-methods-and-codes-5a10270bde13>

Safety Classifications for Common HTTP Methods

HTTP Method	Method Descriptio	Safety Classification	Safety Description
GET	Returns requested data	Safe	No state changes are made
POST	Used in a variety of scenarios, from calculations to creating new resources	Unsafe	Cannot guarantee the same results for multiple calls with the same input
PUT	Representation from client used to replace resource	Idempotent	Guarantees the same results for multiple calls with the same input as the client is providing to the entire resource representation
PATCH	Performs a partial update of a resource	Unsafe	Cannot guarantee the same results for multiple calls with the same input, as the client provides only a partial representation of the resource
DELETE	Deleted a resource from the server	Idempotent	Multiple calls to delete the same resource will still result in the resource being deleted (even if it doesn't exist)

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Mapping Common Verbs Found in Operation Names/Descriptions to HTTP

Operation Verb	Typical HTTP Method + Resource with Examples
List, Search, Match, View All	GETresource collection GET /books
Show, Retrieve, View	GETresource instance GET /books/12345
Create, Add	POSTresource collection POST /books
Replace	PUTresource instance or collection PUT /cars/123 PUT /cars/123/items
Update	PATCHresource instance PATCH /cars/123
Delete All, Remove All, Clear, Reset	DELETEResource collection DELETE /cars/123/items
Delete	DELETEResource instance DELETE /cars/123/items/456
Search, Secure Search	POSTcustom search action on the resource collection POST /cars/search
Other verbs>	POSTas a custom action on a resource collection or instance POST /books/123/deactivate

Adapted from Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Assign the Appropriate Path and HTTP Method to Each Operation

Resource Path	Operation Name	HTTP Method	Description	Request	Response
/books	listBooks()	GET	List books by category or release date	categoryIdreleaseDate	Book[]
/books/search	searchBooks()	POST	Search for books by author, title	searchQuery	Book[]
/books/{bookId}	viewBook()	GET	View book details	bookId	Book
/cars/{cartId}	viewCart()	GET	View the current cart and total	cartId	Cart
/cars/{cartId}	clearCart()	DELETE	Remove all books from the customer's cart	cartId	Cart
/cars/{cartId}/items	addItemToCart()	POST	Add a book to the customer's cart	cartId	Cart
/cars/{cartId}/items/{cartItemId}	removeItemFromCart()	DELETE	Remove a book from the customer's cart	cartIdcartItemId	Cart
/authors	getAuthorDetails()	GET	Retrieve the details of an author	authorId	BookAuthor

Using the list of paths identified earlier, assign the appropriate path and HTTP method to each operation based on the intended usage.

Adapted from Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

Step 3: Assign Response Codes

HTTP response status codes fall into three primary families: 2xx, 4xx and 5xx.

- **Select appropriate response codes based on their intended usage**, ensuring alignment with the most current RFC for guidance.

While API clients should be prepared for various response codes, it's **unnecessary to capture every possible code**.

- Initially, **identify at least one success response code and any explicit error codes for each operation**.
- **Start with typical error codes that the API may return**, without aiming for a comprehensive list.

BITS WILP | | Generated: 12-May-2026 16:47:54

Common HTTP Response Codes Used in API Design

Type	Description
1XX	Provides information and does not affect the processing of the request
2XX	Returned when a request is successfully completed
3XX	Returned when the server redirects the client
4XX	Improper requests made by the client. For example, requesting a resource that does not exist or was in an incorrect format.
5XX	Returned when there is a problem with the HTTP server (internal problem).

Source: <https://medium.com/@gwenlorac/http-methods-and-codes-5a10270bde13>

BITS WILP | | Generated: 12-May-2026 16:47:54

Common HTTP Response Codes Used in API Design

Code	Description
200 OK	Returned on a successfully completed request.
302 Found	Redirects the client to another URL. For example, redirecting the user to a dashboard after a successful login.
400 Bad Request	Returned when a request is poorly formed.
403 Forbidden	Means that the client does not have access to the resource. It can be returned when the server detects malicious user input.
404 Not Found	Returned when the client requests a resource that does not exist on the server.
500 Internal Server Error	Returned when the server cannot process the request.

Source: <https://medium.com/@gwenlorac/http-methods-and-codes-5a10270bde13>

Expand API Design with Success and Error Response Codes

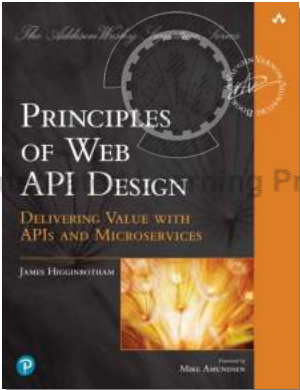
Resource Path	Operation Name	HTTP Method	Description	Request	Response
/books	listBooks()	GET	List books by category or release date	categoryIdreleaseDate	Book[] 200
/books/search	searchBooks()	POST	Search for books by author, title	searchQuery	Book[] 200
/books/{bookId}	viewBook()	GET	View book details	bookid	Book 200, 404
/carts/{cartId}	viewCart()	GET	View the current cart and total	cartid	Cart 200, 404
/carts/{cartId}	clearCart()	DELETE	Remove all books from the customer's cart	cartid	Cart 204,404
/carts/{cartId}/items	addItemToCart()	POST	Add a book to the customer's cart	cartid	Cart 201,400
/carts/{cartId}/items/{cartItemId}	removeItemFromCart()	DELETE	Remove a book from the customer's cart	cartidcartitemid	Cart 204, 404
/authors	getAuthorDetails()	GET	Retrieve the details of an author	authorId	BookAuthor 200, 404

Expanding the Shopping API design with success and error response codes captured in the Response column of the table.

Reference

Principals of Web API Design

- By James Higginbotham



Summary

- The iterative process of API design, encompassing steps from
 - Modeling and resource path definition to HTTP method mapping,
 - Response code assignment,
- Ensuring robust and user-friendly API implementations

MCQ1

What is the primary purpose of API modeling?

- A) To define HTTP methods for API operations
- B) To explore and capture API requirements
- C) To generate mock API implementations
- D) To document response codes for API operations

Answer: B) To explore and capture API requirements

MCQ2

Which step of API design involves converting resource names into URL-friendly formats?

- A) Step 1
- B) Step 2
- C) Step 3
- D) Step 4

Answer: A) Step 1

MCQ3

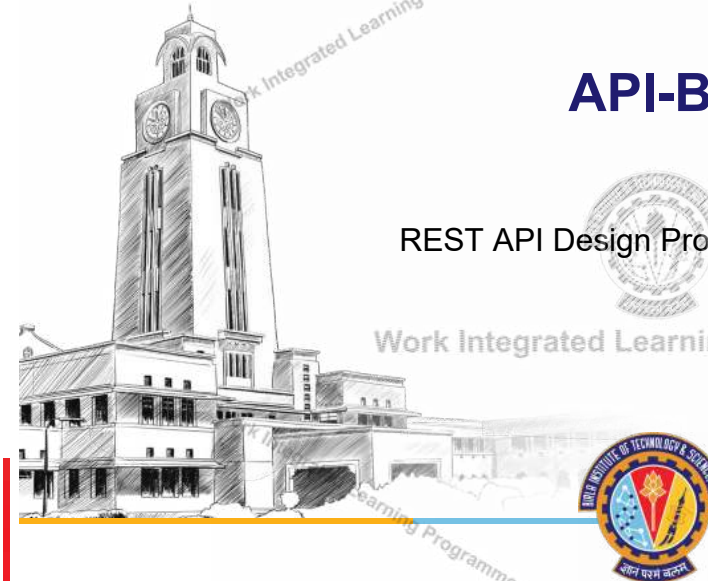
In API design, what do HTTP 400 status codes typically indicate?

- A) Success
- B) A failure in the request that the client may fix and resubmit
- C) A failure on the server not attributable to the client
- D) A retry at a future time, if appropriate

Answer: B) A failure in the request that the client may fix and resubmit

API-Based Products

REST API Design Process II | Suresh Gopal

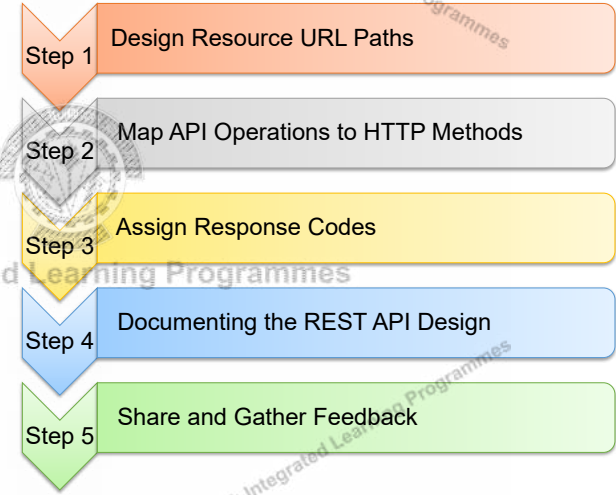


Learning Outcomes

- Understand the process of REST API design

REST API Design Process

Five quick steps for high-level API design



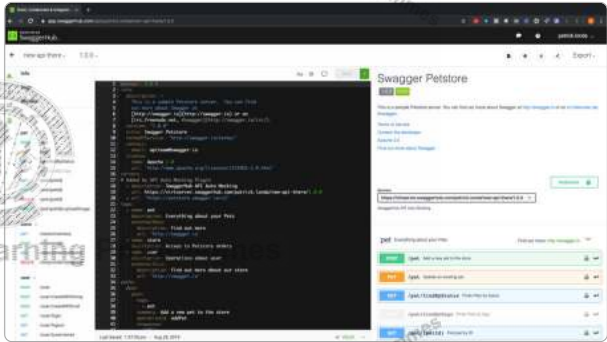
Step 4: Documenting the REST API Design

Completion of step 3 marks the conclusion of high-level API design work, prompting the documentation phase

- Choose a preferred API description format, such as OpenAPI Specification (OAS) or API Blueprint, for sharing the design across teams

Regardless of the chosen format, aim for a machine-readable version of the API design to facilitate

- Review,
- API reference documentation creation,
- and tooling support.



Step 4: Documenting the REST API Design

Commence the documentation process by incorporating API details captured during modeling and design,

- Including the API name, description, and collaboration with other APIs

Summarize the API's purpose and offered operations

- Without delving into internal implementation details, which can be documented separately

Capture operation details, starting with paths and supported HTTP methods

- Include an operation ID to ease mapping back to the API profile

Step 4: Documenting the REST API Design

Ensure all path parameters and query arguments are documented

Validate the API design against

- Job stories,
- EventStorming,
- and API modeling using sequence diagrams,

Showcasing typical interaction patterns

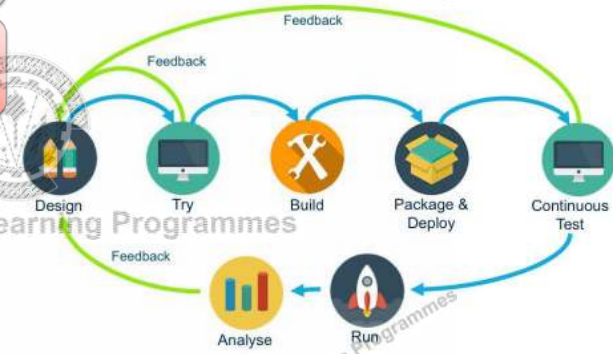
Share the API design documentation and sequence diagrams to solicit feedback, concluding the API design process

Step 5: Share and Gather Feedback

Share the API design with the immediate team, organization's API architects, and internal/external teams planning integration upon API readiness

Once the API is officially released and integrated, its design becomes locked, permitting only nonbreaking changes

- New endpoints or additional fields in resource representations can be added
- Modifying existing endpoints may disrupt existing API consumers, potentially causing customer dissatisfaction or churn



Step 5: Share and Gather Feedback

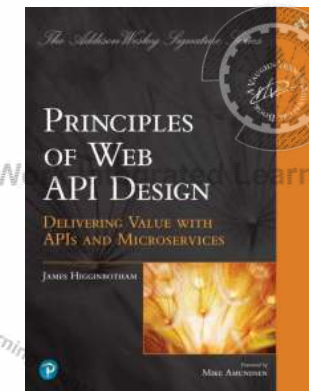
Early sharing of the API design for feedback is crucial to avoid significant design changes post-release, ensuring accuracy from the outset

- Mock implementations of APIs serve as valuable tools for exploring designs, providing developers with a hands-on experience using mock data to understand API functionality
- Emerging tools accept API description formats like OAS to generate mock API implementations, streamlining the process without coding

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- The iterative process of API design, encompassing steps from
 - Documentation,
 - and feedback integration,
- Ensuring robust and user-friendly API implementations



Work Integrated Learning Programmes



MCQ1

What is the purpose of capturing operation details in API documentation?

- A) To list all possible HTTP methods
- B) To specify URL paths for each operation
- C) To document expected response codes
- D) To describe the purpose and details of each API operation

Answer: D) To describe the purpose and details of each API operation



Work Integrated Learning Programmes



MCQ2

Why is early sharing of API designs for feedback important?

- A) To ensure nonbreaking changes can be made post-release
- B) To minimize customer churn by avoiding design changes
- C) To delay integration until the API is officially released
- D) To prevent immediate integration by internal/external teams

Answer: B) To minimize customer churn by avoiding design changes



Work Integrated Learning Programmes



MCQ3

What is the purpose of mock implementations in API design?

- A) To generate API documentation
- B) To simulate API functionality using real data
- C) To automate API testing
- D) To provide a hands-on experience for developers with mock data

Answer: D) To provide a hands-on experience for developers with mock data



Work Integrated Learning Programmes



API-Based Products

Querying, Filtering and Pagination | Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

Querying, Filtering, and Pagination

Enterprises use REST APIs to provide access to their data and services

- Large datasets returned by REST APIs can cause bandwidth and processing issues
- User interfaces typically show limited data, requiring efficient data management

REST APIs should support querying, filtering, and pagination for better data handling

- Consumers can specify parameters and criteria for data retrieval and define response ranges
- can be based on elements, date and time, or offset and limit values



REST API Design:
Filtering, Sorting, and
Pagination

Learning Outcomes

- Design and implement RESTful APIs with efficient filtering and pagination functionalities.



Work Integrated Learning Programmes

Limiting via Query-String Parameters

Filtering and pagination in an API can be effectively managed using query-string parameters

- The API interface can be designed with parameters such as offset and limit
- Offset specifies the starting item number in a collection,
- Limit indicates the maximum number of items to return

For instance,

GET <http://www.foo.com/books?offset=0&limit=25>,

- Meaning retrieve the first 25 books
- Subsequent items require adjusting the offset value

Limiting via Query-String Parameters

The parameters, **offset and limit**, are **not standardized**

- Can vary among API providers
- Other examples of similar parameters include start, count, page, and rpp (records per page)

API designers have **flexibility in naming these parameters** based on business requirements

Filtering

Filtering restricts the results returned in an API response by specifying additional search criteria

- **Complex filtering** may be necessary for APIs supporting intricate sets of search criteria based on **resource attributes**
- **Simple filtering** criteria like **starts-with** or **contains** can help manage complexity

Filtering criteria are typically **specified using a filter query-string with delimiter-separated name/value pairs**

- Example: GET
`http://www.foo.com/users?filter="name::mahesh|city::mumbai"`



Image source: <https://www.atatus.com/blog/rest-api-design-filtering-sorting-and-pagination>

Filter Implementation

Three approaches

Mapping filter criteria to backend database SQL queries to retrieve matching data with minimal messaging

Implementing filter criteria in the service layer, applying them to fetched data, especially for complex criteria or business logic

Implementing filters on the API's intermediary layer, useful when database or service layers remain unchanged, although this may be complex due to limited programming support

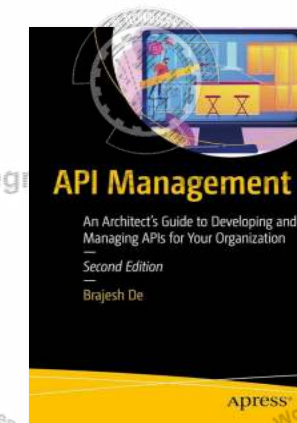
Deciding on the approach should **prioritize implementing filtering as close to the resource data store as possible for optimal efficiency.**

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



Summary

- Effective API design which involves implementing features
 - Like filtering and pagination



Work Integrated Learning Programmes



MCQ1

Which delimiter is commonly used to separate individual filter phrases in a query-string parameter for filtering in RESTful APIs?

- a) Comma (,)
- b) Vertical bar (|)
- c) Colon (:)
- d) Ampersand (&)

Correct answer: b) Vertical bar (|)



Work Integrated Learning Programmes

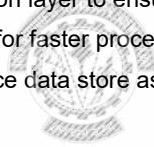


MCQ2

What is the recommended approach when deciding where to implement filtering in an API architecture?

- a) Implement filtering in the API's intermediary layer for simplicity.
- b) Apply filtering in the service implementation layer to ensure flexibility.
- c) Map filter criteria directly to the front-end for faster processing.
- d) Implement filtering as close to the resource data store as possible.

Correct answer: d) Implement filtering as close to the resource data store as possible.

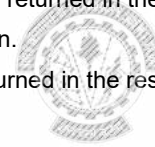


MCQ3

In a REST API call, what does the "limit" query-string parameter signify?

- a) The maximum number of data sources allowed for filtering.
- b) The maximum number of attributes to be returned in the response.
- c) The beginning item number in a collection.
- d) The maximum number of items to be returned in the response.

Correct answer: d) The maximum number of items to be returned in the response.



API-Based Products

Common REST Design Patterns | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Common REST Design Patterns

The fundamental patterns frequently encountered in REST-based API development:

Create-Read-Update-Delete

Extended Resource Lifecycle Support

Singleton Resources

Background (Queued) Jobs

Long-Running Transaction Support in REST

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Delve into different REST API design patterns



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

Create-Read-Update-Delete

Encompass resource collections containing instances, each supporting create, read, update, and delete operations

Offers a consistent approach to managing resource lifecycles

GET

POST

GET

PUT

PATCH

DELETE

Recommended to **avoid fine-grained CRUD APIs** due to transactional complexities

Advise to **design resources around digital capabilities** rather than backend data models

BITS WILP | | Generated: 12-May-2026 16:47:54

Extended Resource Lifecycle Support

Common to encounter state transitions in APIs that extend beyond CRUD interactions

- Necessitating additional operations while adhering to HTTP specifications

For example, in a content management system, managing Articles may require review and approval workflows beyond CRUD

- Involves introducing functional operations like submission, approval, decline, and publishing
- Using this functional operation approach, article resource instances are able to support the workflow necessary

BITS WILP | Generated: 12-May-2026 16:47:54

Singleton Resources

Singleton resources represent individual instances outside of resource collections

Can serve as virtual resources for direct interaction or as singular instances in parent-child relationships within APIs

Typically already exist and do not require client creation

- **Example:**

May support GET, PUT, and/or PATCH methods

GET /me

PUT /customer/1974/profile

BITS WILP | Generated: 12-May-2026 16:47:54

Background (Queued) Jobs

HTTP, being a request/response protocol, necessitates a response for each submitted request

However, for lengthy operations, it's impractical for applications to maintain open connections waiting for completion

- To address this, HTTP offers the 202 Accepted response code
- AKA the fire-and-follow-up pattern

For instance, an API operation for bulk importing user accounts.

Alternatively, if the client doesn't require monitoring, it can disregard the provided URL, following the fire-and-forget pattern

BITS WILP | Generated: 12-May-2026 16:47:54

Long-Running Transaction Support in REST

In certain cases, transactions in REST may require multiple API operations to finalize

- Aka builder pattern

Imagine an API where customers can select individual products to include in their bundle

- However, finalizing the bundle involves multiple steps, such as checking product availability, applying discounts, and confirming shipping details

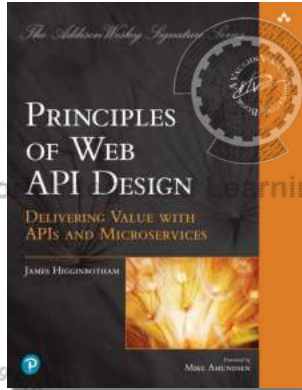
Instead of requiring separate API calls for each product in the bundle, the API could support a transactional approach using a "Cart" or "Bundle" resource

BITS WILP | Generated: 12-May-2026 16:47:54

Reference

Principals of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- A range of REST API design patterns, encompassing
 - CRUD operations,
 - Extended lifecycle support for resources,
 - Singleton resource management,
 - Handling background jobs
 - and managing long-running transactions

MCQ1

What HTTP response code is commonly used to indicate that a request has been accepted but not yet processed?

- A) 200 OK
- B) 202 Accepted
- C) 204 No Content
- D) 400 Bad Request

Answer: B) 202 Accepted

MCQ2

Which HTTP method is typically used to create a new resource in a REST API?

- A) GET
- B) POST
- C) PUT
- D) DELETE

Answer: B) POST

MCQ3

Which design pattern is recommended to handle long-running transactions in REST APIs without resorting to costly transaction managers?

- A) Singleton Resources
- B) Background (Queued) Jobs
- C) Extended Resource Lifecycle Support
- D) Builder Design Pattern

Answer: D) Builder Design Pattern



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Design and implement RESTful APIs with efficient filtering and pagination functionalities.



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54



Work Integrated Learning Programmes



Work Integrated Learning Programmes



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

API-Based Products

Querying, Filtering and Pagination | Suresh Gopal



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

Querying, Filtering, and Pagination

Enterprises use REST APIs to provide access to their data and services

- **Large datasets** returned by REST APIs can cause **bandwidth and processing issues**
- **User interfaces typically show limited data**, requiring efficient data management

REST APIs should **support querying, filtering, and pagination for better data handling**

- Consumers **can specify parameters and criteria for data retrieval and define response ranges**
- can be based on **elements, date and time, or offset and limit values**



REST API Design:
Filtering, Sorting, and
Pagination

Image source: <https://kernicaldev.hashnode.dev/filtering-sorting-and-paging-for-aspnet-core-rest-apis-part-1>

BITS WILP | | Generated: 12-May-2026 16:47:54

Limiting via Query-String Parameters

Filtering and pagination in an API can be effectively managed using query-string parameters

- The API interface can be designed with parameters such as **offset and limit**
- **Offset** specifies the starting item number in a collection,
- **Limit** indicates the maximum number of items to return

For instance,

GET <http://www.foo.com/books?offset=0&limit=25>,

- Meaning retrieve the first 25 books
- Subsequent items require adjusting the offset value

Limiting via Query-String Parameters

The parameters, **offset and limit**, are **not standardized**

- Can vary among API providers
- Other examples of similar parameters include start, count, page, and rpp (records per page)

API designers have **flexibility in naming these parameters** based on business requirements

Filtering

Filtering restricts the results returned in an API response by specifying additional search criteria

- **Complex filtering** may be necessary for APIs supporting intricate sets of search criteria based on **resource attributes**
- **Simple filtering** criteria like **starts-with** or **contains** can help manage complexity

Filtering criteria are typically specified using a filter query-string with delimiter-separated name/value pairs

- Example: GET [http://www.foo.com/users?filter="name::mahesh|city::mumbai"](http://www.foo.com/users?filter=name::mahesh|city::mumbai)



Filter Implementation

Three approaches

Mapping filter criteria to backend database SQL queries to retrieve matching data with minimal messaging

Implementing filter criteria in the service layer, applying them to fetched data, especially for complex criteria or business logic

Implementing filters on the API's intermediary layer, useful when database or service layers remain unchanged, although this may be complex due to limited programming support

Deciding on the approach should prioritize implementing filtering as close to the resource data store as possible for optimal efficiency.

Pagination

Five major pagination strategies

1. Offset & Limit Pagination

Example: GET /products?offset=100&limit=20

- ✓ **Pros**
 - Very simple to implement.
 - Easy for clients to understand.
 - Works well for small datasets.
- ✗ **Cons**
 - **Slow for large offsets** — database must scan/skips many rows.
 - **Inconsistent results** if data changes between requests (insertions/deletions).
 - $O(N)$ cost for large offsets.

Best for:

- Small datasets
- Admin dashboards
- Non-real-time data

Pagination

Five major pagination strategies

2. Page-Based Pagination

Example: GET /products?page=5&pageSize=50

- ✓ **Pros**
 - Human-friendly page numbers.
 - Easy to cache.
 - Simple to implement.
- ✗ **Cons**
 - Suffers from the same issues as offset-based pagination.
 - Page numbers become meaningless if data changes frequently.

Best for:

- Stable datasets
- UI-driven pagination (e.g., e-commerce pages)

Pagination

Five major pagination strategies

3. Cursor-Based (Seek) Pagination

Example: GET /messages?cursor=8756&limit=20

- ✓ **Pros**
 - **Highly efficient** for large datasets.
 - **Consistent** even when data changes during pagination.
 - Avoids large-offset performance issues.
 - Great for infinite scrolling.
- ✗ **Cons**
 - More complex to implement.
 - Requires a **unique, immutable, indexed** column (e.g., id, createdAt).
 - Harder to debug manually.

Best for:

- Social feeds
- Messaging systems
- Real-time data streams
- Very large datasets

Pagination

Five major pagination strategies

4. Time-Based Pagination

Example: GET /logs?start=2025-01-01T00:00:00Z&end=2025-01-02T00:00:00Z

- ✓ **Pros**
 - Ideal for **time-series data** (logs, events).
 - Easy to archive and cache.
 - Natural ordering.
- ✗ **Cons**
 - Records with identical timestamps may cause duplicates or gaps.
 - Requires consistent timestamp precision.

Best for:

- Logs
- Analytics
- Event streams

Pagination

Five major pagination strategies

5. Keyset Pagination

(Technically a form of cursor pagination, but often treated separately.)

- ✓ **Pros**
 - Very fast — uses indexed keys.
 - Stable ordering.
 - No performance degradation with large datasets.
- ✗ **Cons**
 - Requires strict ordering on a unique key.
 - Cannot jump to arbitrary pages (e.g., “page 50”).

Best for:

- Large relational datasets
- High-performance APIs
- Systems where consistency matters more than random access

Pagination

Link Headers

REST best practices recommend including **Link headers** for navigation: rel="next", rel="prev", rel="first", rel="last"

- ✓ **Pros**
 - Standardized navigation.
 - Reduces client-side logic.
 - Helps prevent DoS by enforcing server-side limits.
- ✗ **Cons**
 - Slightly more work for the server.
 - Clients must parse headers.

Pagination

Five major pagination strategies

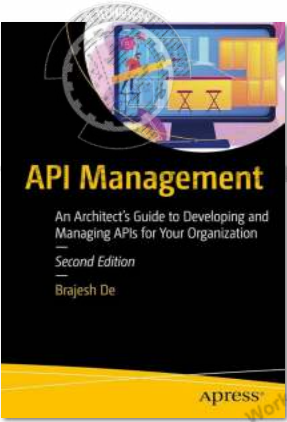
Technique	Performance	Consistency	Complexity	Best Use Case
Offset/Limit	Low for large offsets	Low	Low	Small datasets, admin UIs
Page-Based	Low for large pages	Low	Low	Human-readable pagination
Cursor-Based	High	High	Medium/High	Feeds, infinite scroll
Time-Based	High	Medium	Medium	Logs, time-series
Keyset	Very High	High	Medium	Large datasets, strict ordering

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



Summary

- Effective API design which involves implementing features
 - Like filtering and pagination



Work Integrated Learning Programmes



MCQ1

Which delimiter is commonly used to separate individual filter phrases in a query-string parameter for filtering in RESTful APIs?

- a) Comma (,)
- b) Vertical bar (|)
- c) Colon (:
- d) Ampersand (&)

Correct answer: b) Vertical bar (|)



Work Integrated Learning Programmes

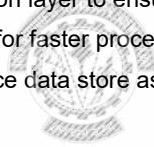


MCQ2

What is the recommended approach when deciding where to implement filtering in an API architecture?

- a) Implement filtering in the API's intermediary layer for simplicity.
- b) Apply filtering in the service implementation layer to ensure flexibility.
- c) Map filter criteria directly to the front-end for faster processing.
- d) Implement filtering as close to the resource data store as possible.

Correct answer: d) Implement filtering as close to the resource data store as possible.

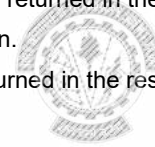


MCQ3

In a REST API call, what does the "limit" query-string parameter signify?

- a) The maximum number of data sources allowed for filtering.
- b) The maximum number of attributes to be returned in the response.
- c) The beginning item number in a collection.
- d) The maximum number of items to be returned in the response.

Correct answer: d) The maximum number of items to be returned in the response.



API-Based Products

Common REST Design Patterns | Suresh Gopal



Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

Common REST Design Patterns

The fundamental patterns frequently encountered in REST-based API development:

Create-Read-Update-Delete

Extended Resource Lifecycle Support

Singleton Resources

Background (Queued) Jobs

Long-Running Transaction Support in REST

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Delve into different REST API design patterns



Work Integrated Learning Programmes

Create-Read-Update-Delete

Encompass resource collections containing instances, each supporting create, read, update, and delete operations

Offers a consistent approach to managing resource lifecycles

GET

POST

GET

PUT

PATCH

DELETE

Recommended to **avoid fine-grained CRUD APIs** due to transactional complexities

Advise to **design resources around digital capabilities** rather than backend data models

BITS WILP | | Generated: 12-May-2026 16:47:54

Extended Resource Lifecycle Support

Common to encounter state transitions in APIs that extend beyond CRUD interactions

- Necessitating additional operations while adhering to HTTP specifications

For example, in a content management system, managing Articles may require review and approval workflows beyond CRUD

- Involves introducing functional operations like submission, approval, decline, and publishing
- Using this functional operation approach, article resource instances are able to support the workflow necessary

BITS WILP | Generated: 12-May-2026 16:47:54

Singleton Resources

Singleton resources represent individual instances outside of resource collections

Can serve as virtual resources for direct interaction or as singular instances in parent-child relationships within APIs

Typically already exist and do not require client creation

- **Example:**

May support GET, PUT, and/or PATCH methods

GET /me

PUT /customer/1974/profile

BITS WILP | Generated: 12-May-2026 16:47:54

Background (Queued) Jobs

HTTP, being a request/response protocol, necessitates a response for each submitted request

However, for lengthy operations, it's impractical for applications to maintain open connections waiting for completion

- To address this, HTTP offers the 202 Accepted response code
- AKA the fire-and-follow-up pattern

For instance, an API operation for bulk importing user accounts.

Alternatively, if the client doesn't require monitoring, it can disregard the provided URL, following the fire-and-forget pattern

BITS WILP | Generated: 12-May-2026 16:47:54

Long-Running Transaction Support in REST

In certain cases, transactions in REST may require multiple API operations to finalize

- Aka builder pattern

Imagine an API where customers can select individual products to include in their bundle

- However, finalizing the bundle involves multiple steps, such as checking product availability, applying discounts, and confirming shipping details

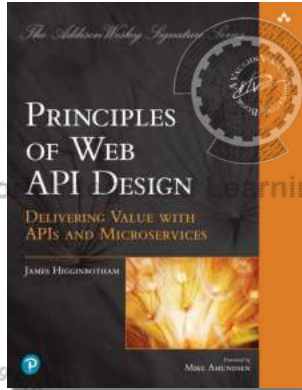
Instead of requiring separate API calls for each product in the bundle, the API could support a transactional approach using a "Cart" or "Bundle" resource

BITS WILP | Generated: 12-May-2026 16:47:54

Reference

Principals of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- A range of REST API design patterns, encompassing
 - CRUD operations,
 - Extended lifecycle support for resources,
 - Singleton resource management,
 - Handling background jobs
 - and managing long-running transactions

MCQ1

What HTTP response code is commonly used to indicate that a request has been accepted but not yet processed?

- A) 200 OK
- B) 202 Accepted
- C) 204 No Content
- D) 400 Bad Request

Answer: B) 202 Accepted

MCQ2

Which HTTP method is typically used to create a new resource in a REST API?

- A) GET
- B) POST
- C) PUT
- D) DELETE

Answer: B) POST

MCQ3

Which design pattern is recommended to handle long-running transactions in REST APIs without resorting to costly transaction managers?

- A) Singleton Resources
- B) Background (Queued) Jobs
- C) Extended Resource Lifecycle Support
- D) Builder Design Pattern

Answer: D) Builder Design Pattern



API-Based Products

Remote Procedure Call | Suresh Gopal



Learning Outcomes

- Define RPC (Remote Procedure Call)
- Explain the Working Principle of RPC-based API
- Identify the Advantages of Using RPC-based API



What is RPC?

RPC stands for Remote Procedure Call

RPC is a protocol that allows a program to execute code on a remote server, as if it were local

- Enables communication between different processes on different systems

Key Components:

- **Client:** Initiates the RPC request
- **Server:** Receives the request, processes it, and returns a response



How RPC-Based API Works

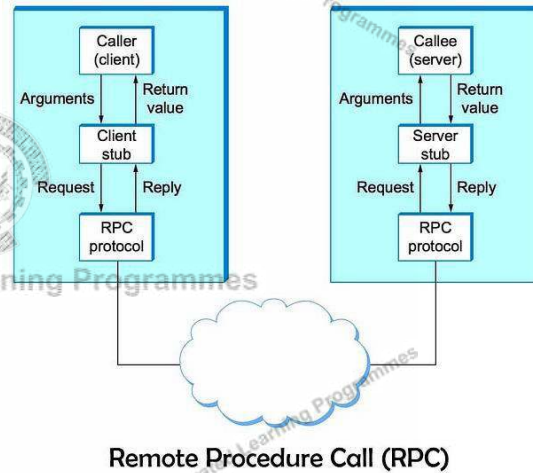
Process:

The client sends a request to the server

The server

- Receives the request,
- Executes the specified procedure,
- and sends back the result

The client receives the response and continues its execution



Remote Procedure Call (RPC)

Image source: <https://faun.pub/remote-procedure-call-rpc-32c10594a79c>

BITS WILP | Generated: 12-May-2026 16:47:54

Advantages of RPC-Based API

Efficiency:

RPC allows for efficient communication between client and server, reducing overhead

Simplified Development:

Developers can focus on application logic without worrying about low-level network communication

Language Agnostic:

RPC can be implemented in various programming languages, allowing flexibility in development

Tight Integration:

RPC enables seamless integration between distributed systems, facilitating modular design.

Use Cases and Examples

Use Cases:

Microservices Architecture:

RPC is commonly used in microservices-based systems for **inter-service communication**

Distributed Computing:

RPC facilitates **communication between nodes** in distributed computing environments

Example:

Google's Protocol Buffers (protobuf) uses RPC for service definition and code generation,

Enabling efficient communication between services in distributed systems



BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- RPC-based APIs which provides
 - Efficient and Language-agnostic communication between client and server, enabling seamless integration in distributed systems



Work Integrated Learning Programmes

Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

Which of the following statements accurately describe Remote Procedure Call (RPC)?

- A) RPC is an abbreviation for "Real-time Process Control."
- B) RPC enables communication between different processes on the same system.
- C) The client initiates the RPC request, while the server receives it, processes it, and returns a response.
- D) RPC stands for "Remote Procedure Call" and allows a program to execute code on a remote server as if it were local.

Correct Answer: D

MCQ2

Which advantages are associated with Remote Procedure Call (RPC)?

- A) Increased overhead due to excessive communication between client and server.
- B) Complex development process, requiring meticulous attention to network communication details.
- C) Language dependency, restricting implementation to a single programming language.
- D) Efficient communication, simplified development, language agnosticism, and seamless integration between distributed systems.

Correct Answer: D

MCQ3

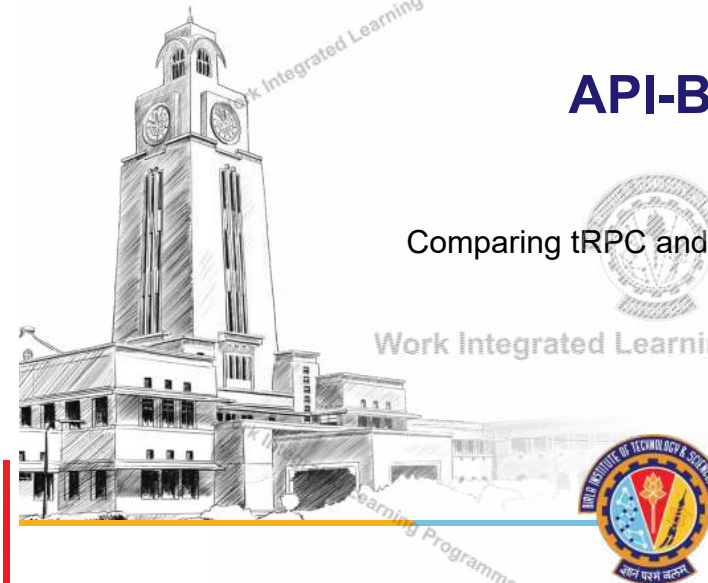
What are the key features and benefits of RPC-based APIs?

- A) Limited communication efficiency and dependency on specific programming languages.
- B) Complex integration process and lack of real-world applications.
- C) Efficient, language-agnostic communication facilitating seamless integration in distributed systems.
- D) RPC fundamentals understanding without practical applications.

Correct Answer: C

API-Based Products

Comparing tRPC and gRPC | Suresh Gopal



Learning Outcomes

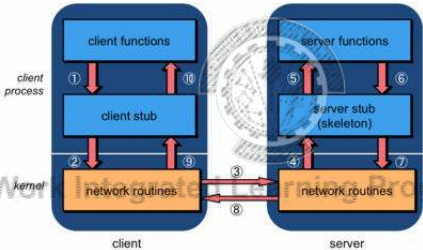
- Identify Key Features of tRPC and gRPC
- Differentiate Between tRPC and gRPC
- Assess Performance Considerations



Work Integrated Learning Programmes

Brief Introduction to RPC

Remote Procedure Call (RPC) is a protocol that enables communication between different processes on different systems



tRPC and gRPC are both RPC frameworks used for building efficient and scalable APIs, but they have differences in their approach and features

gRPC Overview

gRPC is an open-source RPC framework developed by Google,

Widely used for building efficient and scalable APIs

Key Features:

Language Agnostic:

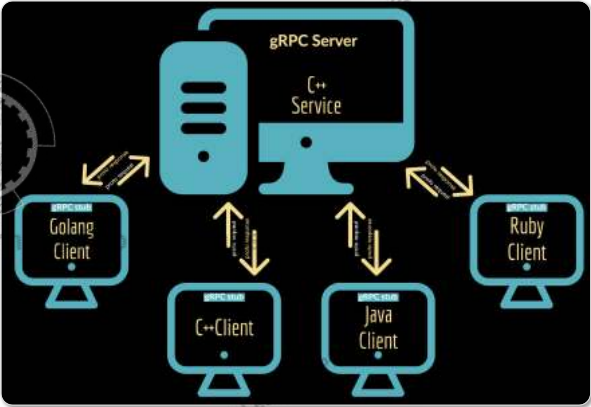
Supports multiple programming languages, including C++, Java, Python, and Go, among others

Performance:

Uses HTTP/2 for transport and Protocol Buffers for serialization, resulting in high-performance communication

Mature Ecosystem:

Has a mature ecosystem with support for various tools and libraries, making it suitable for enterprise-grade applications



tRPC Overview

tRPC (TypeScript RPC) is a modern RPC framework built specifically for TypeScript

Key Features:

Type-Safety:

tRPC provides strong type safety by leveraging TypeScript's static typing system

Simplicity:

Offers a simpler API compared to gRPC, making it easier to use and understand

Expressiveness:

Allows developers to express complex API structures more intuitively using TypeScript decorators

Comparison

Factor	tRPC	gRPC
Language Support	TypeScript	Multiple languages including C++, Java, Python, Go, etc.
Type Safety	Provides strong type safety due to TypeScript integration	Type safety depends on the language used and the Protobuf schema definition
Ecosystem	Till evolving with a growing community and ecosystem	Mature ecosystem with extensive tooling and support
Performance	May vary but generally optimized for TypeScript environments	Optimized for high performance with HTTP/2 and Protocol Buffers

Use Cases and Considerations

Use Cases:

- **tRPC**: Ideal for TypeScript-heavy projects where strong type safety and simplicity are valued
- **gRPC**: Suitable for multi-language environments and performance-critical applications

Considerations:

- **tRPC**: Offers simplicity and type safety but may lack some features and tooling compared to gRPC
- **gRPC**: Provides high performance and language flexibility but may have a steeper learning curve and complexity

Summary

- tRPC and gRPC, both are powerful RPC frameworks with their own strengths and use cases
- How to choose between tRPC and gRPC based on
 - Project requirements,
 - Language preferences,
 - Performance considerations,
 - and ecosystem maturity

MCQ1

What are the key features and benefits of RPC-based APIs?

- A) Limited communication efficiency and dependency on specific programming languages.
- B) Complex integration process and lack of real-world applications.
- C) Efficient, language-agnostic communication facilitating seamless integration in distributed systems.
- D) RPC fundamentals understanding without practical applications.

Correct Answer: C

MCQ2

What are the key features of gRPC, the open-source RPC framework developed by Google?

- A) Limited language support, poor performance, and lack of ecosystem maturity.
- B) Language agnosticism, performance via HTTP/2 and Protocol Buffers, and a mature ecosystem.
- C) Single language support, low-performance communication, and absence of supporting libraries.
- D) Incomplete documentation, restricted use cases, and insufficient community support.

Correct Answer: B

MCQ3

Which statement accurately reflects the use cases and considerations for tRPC and gRPC?

- A) tRPC is recommended for performance-critical applications due to its simplicity and strong type safety, while gRPC is suitable for TypeScript-heavy projects requiring multi-language support.
- B) gRPC is ideal for TypeScript-heavy projects due to its simplicity and strong type safety, while tRPC is recommended for performance-critical applications needing language flexibility.
- C) tRPC is well-suited for TypeScript-heavy projects valuing strong type safety and simplicity, while gRPC is preferred for multi-language environments and performance-critical applications despite its steeper learning curve.
- D) gRPC is preferable for simplicity and type safety in TypeScript-heavy projects, whereas tRPC excels in multi-language environments and performance-critical applications despite lacking some features and tooling.

Correct Answer: C

API-Based Products

RPC-Based API | Suresh Gopal

Learning Outcomes

- Understand RPC-Based API Fundamentals
- Recognize Implications of Tight Coupling
- Consider Factors When Using RPC
- Understand Limitations and Workarounds

Introduction to RPC-Based APIs

RPC-based APIs execute a unit of code, known as a procedure, over the network as if it were being executed locally

Key Components:

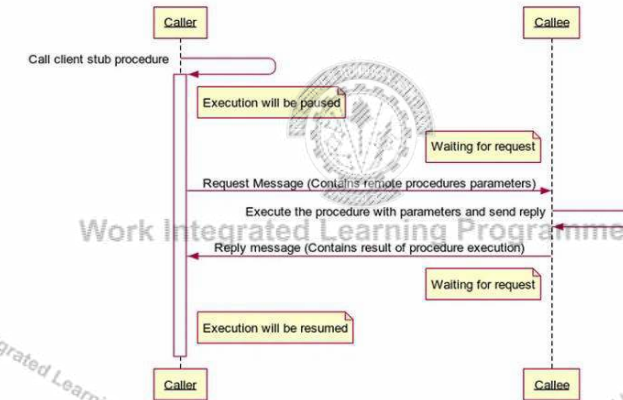
- Client,
- Server,
- Procedures,
- Parameters,
- Response Structure

Importance:

- Facilitates communication between distributed systems, enabling seamless integration of remote services

RPC Working

RPC Model



Source: <https://medium.com/codenv/remoteprocedurecall-rpc-api-explained-3d4a494ff28b>

Tight Coupling Between Client and Server

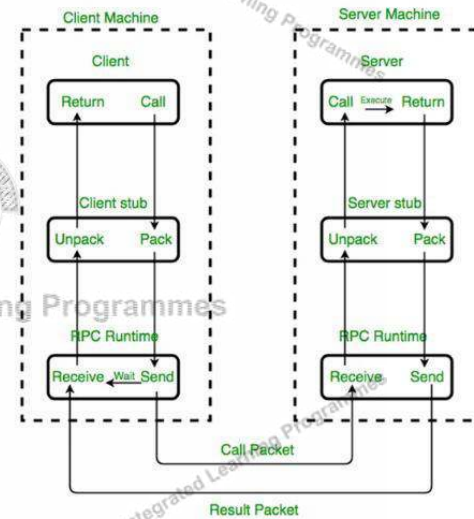
Client tightly coupled to server's procedures

Impact of Modifications:

- Modifications or removal of procedures on the server necessitate adjustments in client code

Trade-off:

- Tight coupling often results in better performance
- But requires synchronization between client and server



Implementation of RPC mechanism

Specification and Standards

Specification Agreement:

RPC-based APIs must agree to a specification for marshalling procedure invocation for target programming languages

Historical Context:

Early RPC standards like

- RMI,
- CORBA,
- XML-RPC,
- SOAP RPC,
- JSON-RPC

Modern standards like

- gRPC

Helper Libraries and Code Generation

Code Generation:

- Uses of helper libraries and code generation tools for generating client and server stubs

Failure Handling:

- Incorporation of failure-handling support in client and server stubs due to the fallacies of distributed computing

Benefits:

- **Faster design and implementation** of RPC-based APIs
- but **less resilience to method renaming and parameter reordering**

Factors to Consider When Using RPC

Performance vs. Tight Coupling:

- Has to consider trade-off between performance and tight coupling

Advantages:

- Speedy development with code generation
- Suitable for organizations owning both client and server

Disadvantages:

- Tight coupling
- Fixed serialization format
- Limitations in customization and browser support (e.g., gRPC)

Limitations and Considerations

Limitations:

- Fixed serialization format
- Lack of HTTP-based content negotiation,
- Custom middleware requirements for browser support and security

Workarounds:

- Projects like grpcweb to transform HTTP/1 requests into gRPC-based procedure calls

RPC-based APIs are suitable when an organization owns both client and server but require diligence in keeping client code updated

Summary

- RPC-based APIs which
 - Enable efficient communication between distributed systems
 - But require careful consideration of tight coupling and limitations

MCQ1

What is a characteristic feature of RPC-based APIs?

- A) They execute multiple units of code simultaneously.
- B) They execute code exclusively on the client side.
- C) They execute a unit of code over the network as if it were local.
- D) They execute code only within the server's environment.

Correct Answer: C

MCQ2

Which statement accurately reflects the role of specification agreements in RPC-based APIs and their historical context?

- A) RPC-based APIs do not require specification agreements as they can execute procedures across any programming language seamlessly.
- B) Specification agreements are essential for RPC-based APIs to ensure proper marshaling of procedure invocations across different programming languages.
- C) Historical RPC standards like RMI, CORBA, and XML-RPC have rendered modern standards like gRPC obsolete due to their robustness in specification agreements.
- D) Specification agreements were only relevant in the early stages of RPC development and are no longer necessary in modern RPC frameworks.

Correct Answer: B

MCQ3

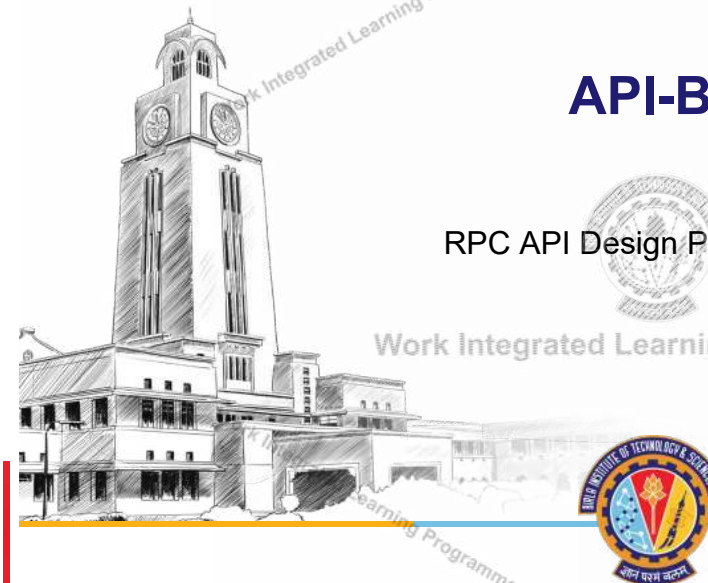
What considerations must be weighed regarding performance and coupling in RPC-based systems?

- A) Performance is always prioritized over coupling, leading to rapid development and flexible serialization formats.
- B) Coupling is essential for performance optimization, allowing for speedy development and enhanced browser support.
- C) A trade-off between performance and tight coupling must be balanced, with advantages including rapid development via code generation and suitability for organizations controlling both client and server.
- D) Tight coupling is preferred to ensure fixed serialization formats and limitations in customization, even if it hampers performance.

Correct Answer: C

API-Based Products

RPC API Design Process | Suresh Gopal



Learning Outcomes

- Grasp the process of designing RPC-based APIs, from initial identification of operations to finalizing details and generating human-readable documentation



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

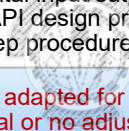


RPC Design Process

The RPC design process utilizes API profiles crafted during API modeling

- Since the API profiles have already outlined operations and fundamental input/output specifications, the RPC API design process becomes a swift three-step procedure

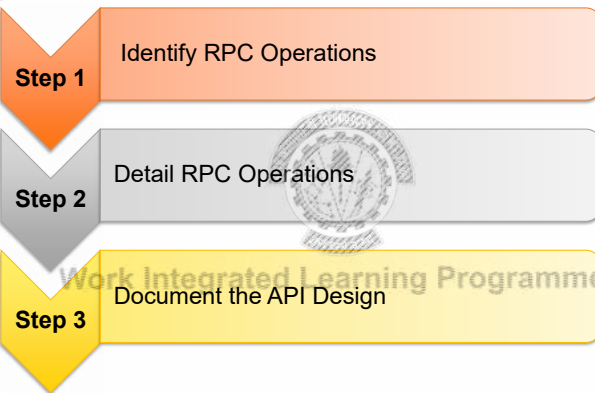
Process can be seamlessly adapted for other RPC-based protocols with minimal or no adjustments



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

RPC Design Process - Steps



BITS WILP | Generated: 12-May-2026 16:47:54

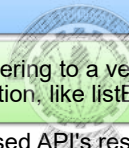


Step 1: Identify RPC Operations

Transition the operations list, along with their descriptions and request/response specifics, into a newly structured tabular format tailored for high-level design

While not mandatory, adhering to a verb-resource operation naming convention, like `listBooks()`,

- Enhances the RPC-based API's resource-centric approach
- Makes it more intuitive for individuals familiar with REST-based APIs



BITS WILP | Generated: 12-May-2026 16:47:54

Step 1: Identify RPC Operations

Operation Name	Description	Request	Response
listBooks()	List books by category or release date		
searchBooks()	Search for books by author, title		
viewBook()	View book details		
viewCart()	View the current cart and total		
clearCart()	Remove all books from the customer's cart		
addItemToCart()	Add a book to the customer's cart		
removeItemFromCart()	Remove a book from the customer's cart		
getAuthorDetails()	Retrieve the details of an author		

A table that captures the initial RPC operations based on the previous API profile examples.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Detail RPC Operations

Elaborate on the request and response specifics for each operation by utilizing the resource definitions and fields documented during API modeling

Most RPC protocols facilitate a parameter list akin to local method invocation

- Outline the input parameters expected in the request and the corresponding values to be returned in the response

Emphasize the importance of standardizing on an error response type to enable consistent handling of server-side errors by clients

Step 2: Detail RPC Operations

Operation Name	Description	Request	Response
listBooks()	List books by category or release date	ListBookRequest -categoryId -releaseDate	ListBookResponse -Book[] or google.rpc.Status + ProblemDetails
searchBooks()	Search for books by author, title	SearchQuery -query	ListBookResponse -Book[] or google.rpc.Status + ProblemDetails
viewBook()	View book details	ViewBookRequest -bookId	Book or google.rpc.Status + ProblemDetails
viewCart()	View the current cart and total	ViewCartRequest -cartId	Cart or google.rpc.Status + ProblemDetails
clearCart()	Remove all books from the customer's cart	ClearCartRequest -cartId	Cart or google.rpc.Status + ProblemDetails
addItemToCart()	Add a book to the customer's cart	AddCartItemRequest -cartId -quantity	Cart or google.rpc.Status + ProblemDetails
removeItemFromCart()	Remove a book from the customer's cart	RemoveCartItemRequest -cartId -cartItemId	Cart or google.rpc.Status + ProblemDetails
getAuthorDetails()	Retrieve the details of an author	GetAuthorRequest: -authorId	BookAuthor or google.rpc.Status + ProblemDetails

The gRPC design complete with request and response basic message details.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 3: Document the API Design

Use the design details from the previous two steps to compose the IDL file for the RPC-based API

- In the case of gRPC, the IDL file is in the Protocol Buffers format

BITS WILP | Generated: 12-May-2026 16:47:54

Step 3: Document the API Design

IDL File for the gRPC Version of the Shopping Cart API

// Shopping-Cart-API.proto3

```
service ShoppingCart {
  rpc ListBooks(ListBooksRequest) returns (ListBooksResponse) {}
  rpc SearchBooks(SearchBooksRequest) returns (SearchBooksResponse) {}
  rpc ViewBook(ViewBookRequest) returns (Book) {}
  rpc ViewCart(ViewCartRequest) returns (Cart) {}
  rpc ClearCart(ClearCartRequest) returns (Cart) {}
  rpc AddItemToCart(AddCartItemRequest) returns (Cart) {}
  rpc RemoveItemFromCart(RemoveCartItemRequest) returns (Cart) {}
  rpc GetAuthorDetails(GetAuthorDetailsRequest) returns (Author) {}
}

message ListBooksRequest {
  string category_id = 1;
  string release_date = 2;
}

message SearchBooksRequest {
  string query = 1;
}

message SearchBooksResponse {
  int32 page_number = 1;
  int32 results_per_page = 2 [default = 10];
  repeated Book books = 3;
}
```

Adapted from Principles of Web API Design

BITS WILP | | Generated: 12-May-2026 16:47:54

```
message ViewBookRequest {
  string book_id = 1;
}

message ViewCartRequest {
  string cart_id = 1;
}

message ClearCartRequest {
  string cart_id = 1;
}

message AddCartItemRequest {
  string cart_id = 1;
  string book_id = 2;
  int32 quantity = 3;
}

message RemoveCartItemRequest {
  string cart_id = 1;
  string cart_item_id = 2;
}

message CartItem {
  string cart_item_id = 1;
  Book book = 2;
  int32 quantity = 3;
}

message Cart {
  string cart_id = 1;
  repeated CartItem cart_items = 2;
}
```

Done!

The high-level design of the RPC-based API is now established, providing a framework for further detailing

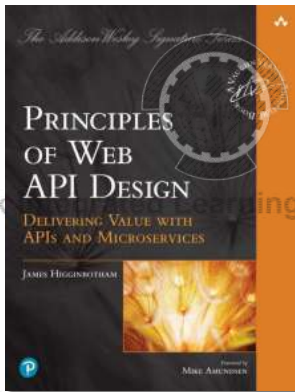
- Subsequent steps involve adding necessary details to finalize the API and utilizing code generators to expedite development and integration tasks

Advisable to generate human-readable documentation using tools like protoc-gen-doc

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- How API modeling serves as a foundational step in the design process, facilitating the translation of customer requirements into RPC-based design outcomes
- RPC API design process : a swift three-step procedure
 - Step 1: Identify RPC Operations
 - Step 2: Detail RPC Operations
 - Step 3: Document the API Design

MCQ1

Which statement accurately describes the RPC design process and its adaptability?

- A) The RPC design process is inherently complex and requires extensive customization for different RPC-based protocols.
- B) API profiles crafted during API modeling streamline the RPC design process into a swift three-step procedure, regardless of the chosen RPC protocol.
- C) The RPC design process is fixed and cannot be adapted for other RPC-based protocols beyond gRPC and Protocol Buffers 3.
- D) The adaptability of the RPC design process is limited and requires significant adjustments when switching between RPC-based protocols.

Correct Answer: B

MCQ2

Which statement accurately reflects the process of transitioning operation descriptions for high-level design?

- A) Adhering to a verb-resource naming convention, such as `listBooks()`, is mandatory for structuring the RPC-based API's tabular format.
- B) Transitioning operation descriptions into a structured tabular format is a complex process, not suited for high-level design.
- C) Following a verb-resource naming convention, like `listBooks()`, is optional but recommended for enhancing the RPC-based API's intuitive resource-centric approach, especially for those accustomed to REST-based APIs.
- D) Utilizing a verb-noun naming convention, such as `listBooks()`, is essential for maintaining consistency across RPC-based APIs, irrespective of the design phase.

Correct Answer: C

MCQ3

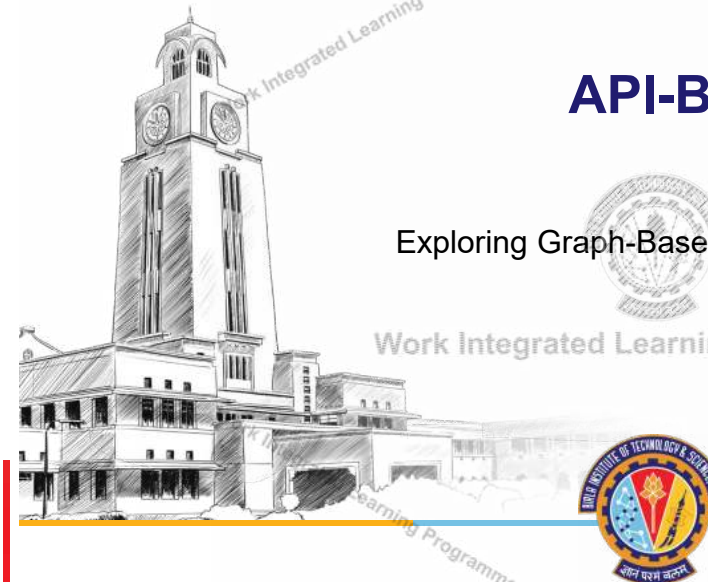
Which statement accurately describes the subsequent steps in finalizing the RPC-based API design?

- A) The high-level design serves as the final blueprint, and no further detailing or documentation is required.
- B) Finalizing the API involves adding unnecessary details to ensure completeness, followed by manual code generation for development.
- C) After establishing the high-level design, the next steps include adding necessary details and utilizing code generators to streamline development and integration tasks, with the advisable addition of generating human-readable documentation using tools like `protoc-gen-doc`.
- D) The API design process concludes with the high-level framework, and human-readable documentation is not considered necessary for further development or integration tasks.

Correct Answer: C

API-Based Products

Exploring Graph-Based APIs | Suresh Gopal



Learning Outcomes

- Define the concept of Graph-Based APIs and differentiate them from traditional API structures
- Understand the fundamental components of graph data structures, including nodes, edges, and properties



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54



Introduction to Graph-Based APIs

Graph-Based APIs facilitate interactions with graph data structures,

- Allowing developers to create, read, update, and delete graph nodes and relationships

Graph data structures represent

- Entities (nodes)
- and connections between them (edges)

Examples:

- GraphQL
- Neo4j
- Amazon Neptune
- Microsoft Graph API



BITS WILP | | Generated: 12-May-2026 16:47:54

Why Graph-Based APIs Matter

Traditional APIs are often based on relational databases or flat data structures

Graph-Based APIs enable modeling of complex relationships,

- Reflects real-world scenarios more accurately
- Empower developers to leverage the power of connected data for better insights and decision-making



Image source: <https://i.yimg.com/vi/cGS6JIYUk1g/maxresdefault.jpg>

BITS WILP | | Generated: 12-May-2026 16:47:54



Graph Data Structures

Nodes: • Represent entities or objects

Edges: • Represent relationships between nodes

Properties: • Additional information associated with nodes and edges

Directed vs. Undirected Graphs: • Directionality of relationships

Graph Query Languages: • Cypher (Neo4j), Gremlin (Apache TinkerPop), SPARQL (RDF databases)

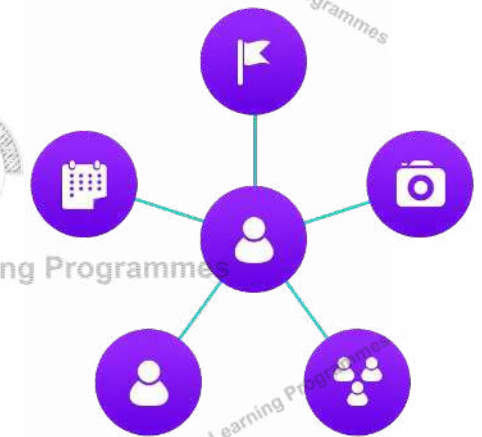


Image source: <https://www.programiz.com/dsa/graph>

BITS WILP | | Generated: 12-May-2026 16:47:54

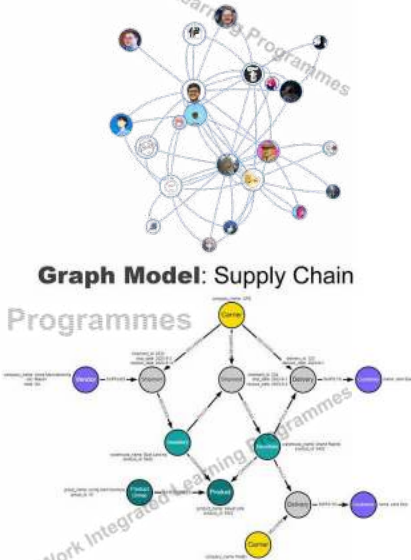


Benefits of Using Graph-Based APIs

Flexible Data Modeling:	Easily represent complex relationships
Efficient Querying:	Traverse connections efficiently for data retrieval
Real-Time Insights:	Analyze interconnected data in real-time
Scalability:	Handle large-scale data with ease.
Interoperability:	Integrate with various platforms and data sources

Real-World Applications

- Social Networks:**
Friend recommendations, network analysis
- Recommendation Systems:**
Personalized recommendations based on user preferences
- Fraud Detection:**
Identifying suspicious patterns in financial transactions
- Knowledge Graphs:**
Connecting structured and unstructured data for semantic search
- Supply Chain Management:**
Tracking products, suppliers, and logistics



Graph Model: Supply Chain

Image source: <https://www.nebula-graph.io/posts/social-networks-with-graph-database-1/>
<https://towardsdatascience.com/why-you-need-a-knowledgegraph-and-how-to-build-it-ac4f35cb79b7>

Considerations

Data Consistency:	Ensuring data integrity across interconnected nodes
Performance Optimization:	Efficient querying for large-scale graphs
Security:	Protecting sensitive data and preventing unauthorized access
Complexity:	Understanding and managing complex relationships and queries
Adoption and Education:	Overcoming learning curves and promoting adoption among developers

Summary

- Graph-Based APIs facilitate interactions with graph data structures, allowing developers to work with nodes and relationships efficiently

MCQ1

What is the primary purpose of Graph-Based APIs?

- A) To facilitate interactions with tree data structures.
- B) To enable developers to manipulate graph nodes only.
- C) To allow developers to interact with graph data structures for creating, reading, updating, and deleting nodes and relationships.
- D) To manage interactions with relational databases.

Correct Answer: C

MCQ2

What components are typically found in graph data structures?

- A) Vertices and edges representing entities and properties, respectively.
- B) Nodes representing entities, properties representing relationships, and edges representing directed or undirected connections.
- C) Vertices representing properties and edges representing relationships, both with associated queries.
- D) Nodes representing entities, edges representing relationships, and properties representing additional information, along with directed or undirected connections.

Correct Answer: D

MCQ3

What are the primary considerations when working with graph-based data structures?

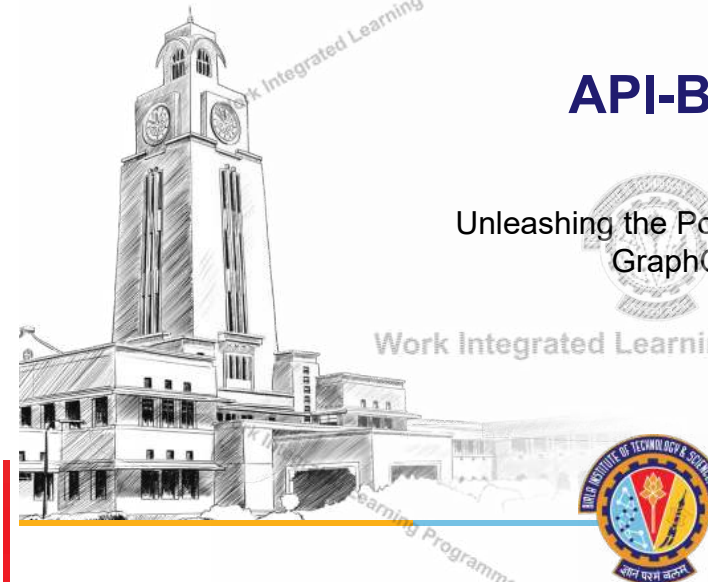
- A) Data integrity, efficient querying, and managing complex relationships are key concerns, with little emphasis on security or adoption challenges.
- B) Security, adoption challenges, and data consistency are the primary considerations, with performance optimization being less significant.
- C) Performance optimization, complexity management, and adoption challenges are crucial, while data consistency and security play secondary roles.
- D) Ensuring data integrity, optimizing performance, addressing security concerns, managing complexity, and overcoming adoption challenges are all important considerations when working with graph-based data structures.

Correct Answer: D

API-Based Products

Unleashing the Power of
GraphQL API

Suresh Gopal



Learning Outcomes

- Understand the fundamental concepts of GraphQL



Work Integrated Learning Programmes

What is GraphQL?

GraphQL is a query language for APIs and a runtime for executing those queries with your existing data

Developed by Facebook in 2012

- Later open-sourced in 2015

Allows clients to request only the data they need, making it efficient and flexible



Why GraphQL?

Advantages:

Single endpoint for data fetching

Client-driven queries

Strongly typed schema

Introspection for documentation

Efficient data loading



GraphQL client



GraphQL client



GraphQL client

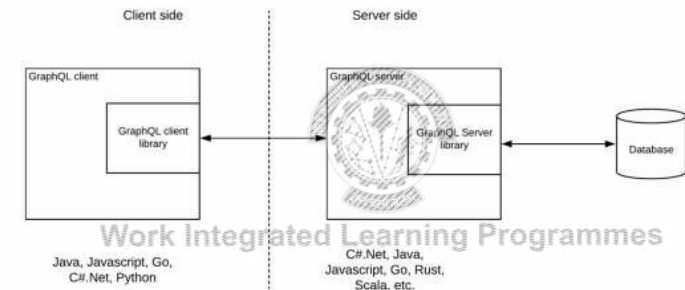


GraphQL server



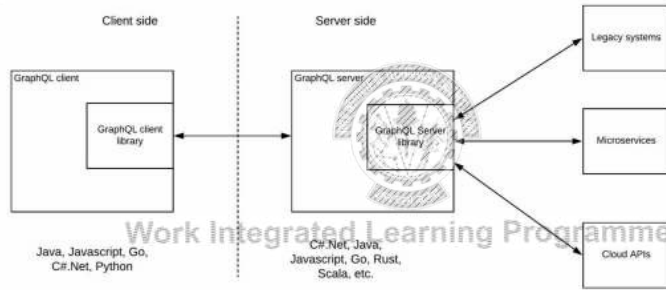
Datasource

GraphQL : Architecture



GraphQL server exposing a database

GraphQL : Architecture



GraphQL server as an integration hub

Source: <https://chanakaudaya.medium.com/graphql-based-solution-architecture-patterns-8905de6ff87e> | Generated: 12-May-2026 16:47:54

GraphQL Component

Components:

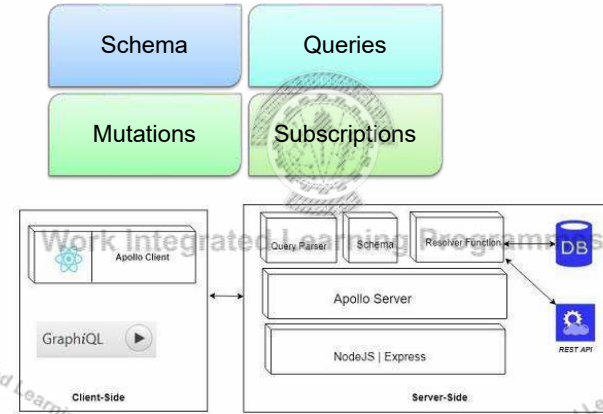


Image source: https://www.tutorialspoint.com/graphql/graphql_application_components.htm | Generated: 12-May-2026 16:47:54

Working with GraphQL API

Client-side: Use GraphQL clients like Apollo Client, Relay, etc.

Server-side: Implement GraphQL in your server using libraries such as Apollo Server, Express GraphQL, etc.

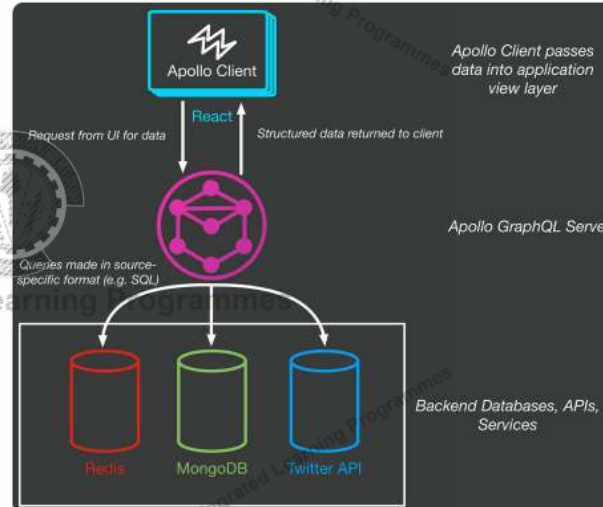


Image source: <https://chathuranga94.medium.com/introduction-to-graphql-3e0142879aba> | Generated: 12-May-2026 16:47:54

Summary

- GraphQL which is a query language for APIs that allows clients to request only the data they need, improving efficiency and flexibility
- Its architecture comprises components like
 - Schema,
 - Queries,
 - Mutations, and
 - Subscriptions enabling efficient data fetching and manipulation

Generated: 12-May-2026 16:47:54

MCQ1

What are the advantages of GraphQL?

- A) Multiple endpoints for data fetching, server-driven queries, loosely typed schema, and lack of introspection for documentation.
- B) Single endpoint for data fetching, server-driven queries, strongly typed schema, and lack of introspection for documentation.
- C) Single endpoint for data fetching, client-driven queries, loosely typed schema, and lack of introspection for documentation.
- D) Single endpoint for data fetching, client-driven queries, strongly typed schema, and introspection for documentation.

Correct Answer: D

MCQ2

How can GraphQL be utilized on both the client-side and server-side?

- A) Client-side usage involves implementing GraphQL in servers, while server-side usage involves employing GraphQL clients.
- B) GraphQL clients like Apollo Client are exclusively used on the server-side, while server implementations are handled through libraries like Apollo Server.
- C) GraphQL clients like Apollo Client are employed on the client-side, while server-side implementations are facilitated through libraries such as Apollo Server.
- D) GraphQL is utilized solely on the server-side, with libraries like Apollo Server and Express GraphQL enabling its implementation.

Correct Answer: C

MCQ3

What are the fundamental components of GraphQL?

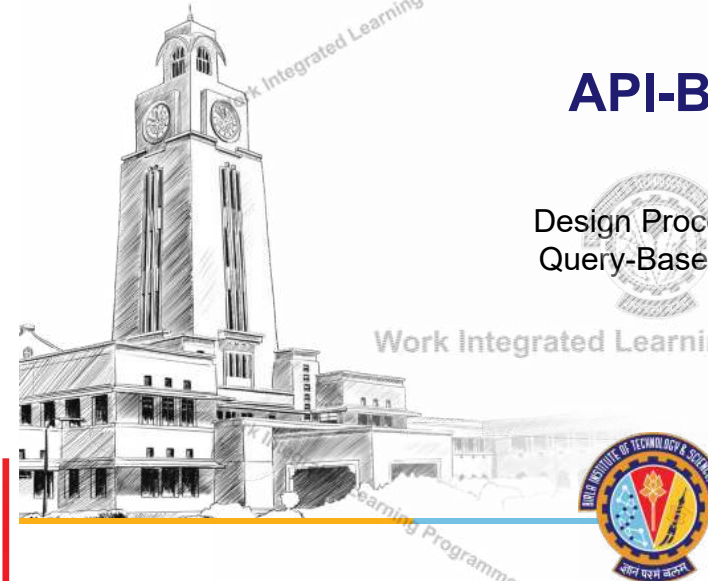
- A) Schema, directives, resolvers, and subscriptions.
- B) Schema, queries, mutations, and subscriptions.
- C) Schema, fragments, resolvers, and subscriptions.
- D) Schema, queries, directives, and mutations.

Correct Answer: B

API-Based Products

Design Process for
Query-Based APIs

Suresh Gopal



Learning Outcomes

- Understand Query-Based API Design Process



Work Integrated Learning Programmes



Query-Based API Design Process

The process used to design a query-based API is similar to that of other API design styles, such as RPC and REST

The primary difference is that the steps require the creation of a resource graph prior to designing the operations

Involves steps

- Step 1: Designing Resource and Graph Structures
- Step 2: Design Query and Mutation Operations
- Step 3: Document the API Design

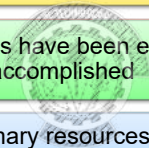


Step 1: Designing Resource and Graph Structures

The initial and pivotal phase in developing query-based APIs entails crafting the graph structure for all resources

If the API modeling tasks have been executed, this phase is likely already accomplished

After pinpointing all primary resources and their associated resources, the subsequent step involves formulating the operations for queries and mutations



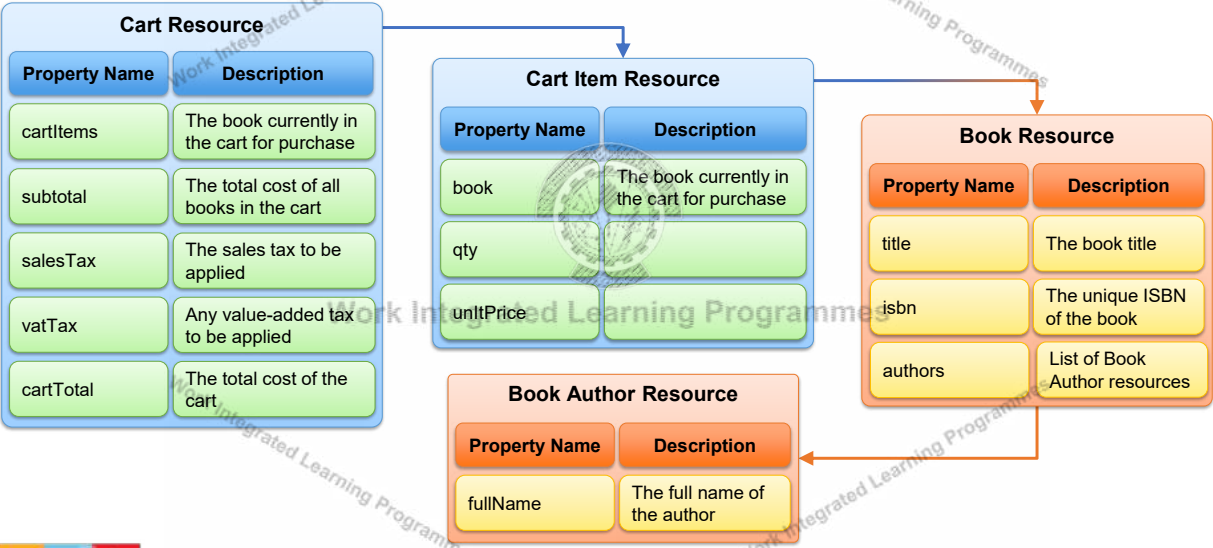
Step 1: Designing Resource and Graph Structures

Book Resource	
Property Name	Description
title	The book title
isbn	The unique ISBN of the book
authors	List of Book Author resources

Book Author Resource	
Property Name	Description
fullName	The full name of the author



Step 1: Designing Resource and Graph Structures



Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Design Query and Mutation Operations

The next step is to migrate all operations captured in the API profile during API modeling

The API profiles capture each operation

- Include a safety classification of safe, idempotent, or unsafe

Classify each operation marked as safe as a query

- Operations marked as idempotent or unsafe will be mutations

GraphQL supports both, so the design can include both query and mutations within the same API definition

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Design Query and Mutation Operations

Operation Type	Operation Name	Description	Request	Response
Query	listBooks()	List books by category or release date		
Query	searchBooks()	Search for books by author, title		
Query	viewBook()	View book details		
Query	viewCart()	View the current cart and total		
Mutation	clearCart()	Remove all books from the customer's cart		
Mutation	addItemToCart()	Add a book to the customer's cart		
Mutation	removeItemFromCart()	Remove a book from the customer's cart		
Query	getAuthorDetails()	Retrieve the details of an author		

The Shopping Cart API profile migrated to a tabular format that helps with query-based API design.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Design Query and Mutation Operations

Once the basic operation details have been captured,

Expand the request and response columns with further details about the input and output values

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Design Query and Mutation Operations

Operation Type	Operation Name	Description	Request	Response
Query	listBooks()	List books by category or release date	<pre>query { Book (categoryId, releaseDate) { ... } }</pre>	Book[]
Query	searchBooks()	Search for books by author, title	<pre>query { Book (searchQuery) { ... } }</pre>	Book[]
Query	viewBook()	View book details	<pre>query { book(bookId) { ... } }</pre>	Book
Query	viewCart()	View the current cart and total	<pre>query { cart(cartId) { ... } }</pre>	Cart

The Shopping Cart GraphQL API design is now expanded with additional details about queries and mutations.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 2: Design Query and Mutation Operations

Operation Type	Operation Name	Description	Request	Response
Mutation	clearCart()	Remove all books from the customer's cart	<pre>mutation clearCart { cartId }</pre>	Cart
Mutation	addItemToCart()	Add a book to the customer's cart	<pre>mutation addItemToCart { cartId bookId quantity }</pre>	Cart
Mutation	removeItemFromCart()	Remove a book from the customer's cart	<pre>mutation removeItemFromCart { cartId cartItemId }</pre>	Cart
Query	getAuthorDetails()	Retrieve the details of an author	<pre>query { BookAuthor (authorId) { ... } }</pre>	BookAuthor

The Shopping Cart GraphQL API design is now expanded with additional details about queries and mutations.

Adapted from Principles of Web API Design

BITS WILP | Generated: 12-May-2026 16:47:54

Step 3: Document the API Design

Conclude the API development process by **documenting the outcomes using the designated format for the selected protocol**

- In the context of GraphQL, a schema serves to outline the available queries and mutations

Advisable to **generate user-friendly documentation** using tools like graphql-docs to enhance accessibility

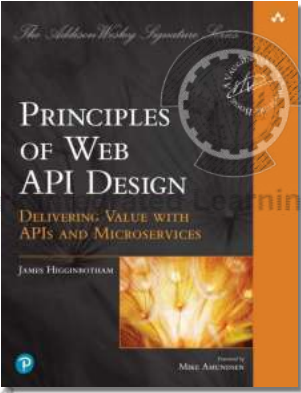
Additionally, **provide an interactive interface through GraphQL Playground**

- To empower developers to formulate requests directly within the browser, facilitating efficient integration code composition

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- Process to design graph-based APIs
 - Step 1: Designing Resource and Graph Structures
 - Step 2: Design Query and Mutation Operations
 - Step 3: Document the API Design



Work Integrated Learning Programmes

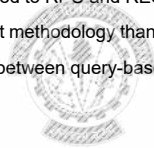


MCQ1

What distinguishes the process of designing a query-based API from other API design styles?

- A) The necessity to create a resource graph before designing operations distinguishes query-based API design from RPC and REST.
- B) Query-based API design involves fewer steps compared to RPC and REST.
- C) Query-based API design follows a completely different methodology than RPC and REST.
- D) The documentation process is the primary difference between query-based API design and other styles like RPC and REST.

Correct Answer: A



Work Integrated Learning Programmes



MCQ2

What is the primary focus during the initial phase of developing query-based APIs?

- A) Crafting query and mutation operations for all resources.
- B) Executing API modeling tasks to accomplish the graph structure.
- C) Documenting the API design for query-based APIs.
- D) Designing the graph structure for all resources.

Correct Answer: D



Work Integrated Learning Programmes

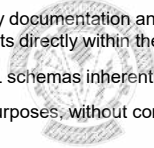


MCQ3

What is the recommended approach for concluding the API development process, specifically in the context of GraphQL?

- A) Document the outcomes using the designated format, focusing solely on schema outlining, without considering user-friendly documentation or interactive interfaces.
- B) Utilize tools like graphql-docs to generate user-friendly documentation and provide an interactive interface through GraphQL Playground, empowering developers to formulate requests directly within the browser.
- C) Skip the documentation phase altogether as GraphQL schemas inherently serve as comprehensive documentation.
- D) Use GraphQL Playground solely for internal testing purposes, without considering its potential benefits for developers during integration code composition.

Correct Answer: B



Work Integrated Learning Programmes



API-Based Products

Why Async APIs? Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

Limitation of Traditional APIs

Traditional discussions regarding Web-based APIs typically revolve around synchronous request/response interactions,

- Typical of REST-based, query-based, and RPC-based APIs
- Straightforward to grasp, making them accessible to developers and non-developers alike, especially those with limited experience in HTTP

However, synchronous APIs have limitations, notably

- Their inability to notify interested parties about changes in resource representation

Learning Outcomes

- Understand the issue with API polling
- Understand why Async APIs are required



Work Integrated Learning Programmes

The Problem with API Polling

Traditional API polling, where clients periodically check with the server for updates, is a common solution for clients needing to be aware of new or modified resources

- While flexible and applicable to various request/response APIs, API polling poses challenges such as complex logic, resource wastage, and potential for poor user experience

API polling involves the client sending GET requests to fetch resource lists, comparing them to previous lists, and determining any changes, often requiring additional coding effort.

Polling



Image source: <https://dzone.com/articles/evaluating-webhooks-vs-polling>

BITS WILP | Generated: 12-May-2026 16:47:54

The Problem with API Polling

The ideal scenario would involve servers notifying interested clients about changes, but this isn't feasible with traditional request/response API styles

Async APIs offer a solution by enabling servers to push asynchronous notifications to interested clients, eliminating the need for constant polling and client-side change detection rules

Async APIs open up new possibilities beyond traditional synchronous Web-based APIs, which rely on HTTP request/response patterns

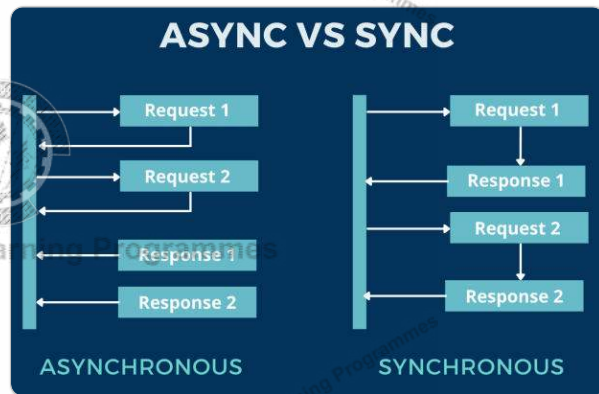


Image source: <https://blog.stackademic.com/asynchronous-api-design-best-practices-server-sent-event-sse-for-real-time-communication-a3a3e20233d2> | Generated: 12-May-2026 16:47:54

Asynchronous APIs

Async APIs, overcome these limitations by enabling server-originated conversations,

- Allowing clients to react to events rather than initiating conversations

Async APIs unlock a digital product or platform's full potential,

- By enabling the development of new capabilities based on event notifications without requiring the API-owning team's direct involvement

Integrating async API design into overall API design efforts empowers teams to innovate solutions based on notifications or data streams



Image source: <https://www.techtarget.com/whatis/definition/synchronous-asynchronous-API> | Generated: 12-May-2026 16:47:54

Open New Possibilities for Digital Business

Async APIs represent digital capabilities, introducing novel opportunities for digital business advancement

Reacting to business events in real-time becomes feasible with async APIs

API efficiency is enhanced with async APIs as constant polling becomes unnecessary for checking state changes

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- The significance of Async APIs



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

What are some limitations of synchronous Web-based APIs?

- A) They lack accessibility to developers and non-developers due to their complexity, particularly for those with limited experience in HTTP.
- B) Synchronous APIs struggle with straightforward interactions, making them less accessible for developers and non-developers alike.
- C) Synchronous APIs cannot notify interested parties about changes in resource representation or the completion of multi-party workflows without client initiation.
- D) Synchronous APIs have no limitations and are universally accessible for both developers and non-developers.

Correct Answer: C

BITS WILP | | Generated: 12-May-2026 16:47:54



MCQ2

What advantage do async APIs offer over synchronous APIs?

- A) Async APIs facilitate straightforward interactions, making them accessible to developers and non-developers alike.
- B) Async APIs struggle with enabling server-originated conversations, limiting clients' ability to react to events.
- C) Async APIs overcome limitations by enabling server-originated conversations, allowing clients to react to events without initiating conversations.
- D) Async APIs require direct involvement from the API-owning team to develop new capabilities based on event notifications.

Correct Answer: C

BITS WILP | | Generated: 12-May-2026 16:47:54



MCQ3

What are the advantages of using async APIs for digital business advancement?

- A) Async APIs introduce complexities that hinder business advancement and require constant polling for state changes.
- B) Async APIs simplify reacting to business events in real-time and enhance API efficiency by eliminating the need for constant polling.
- C) Async APIs introduce inefficiencies and complexities, making real-time event reactions unfeasible.
- D) Async APIs offer no significant advantages over traditional synchronous APIs for digital business advancement.

Correct Answer: B

BITS WILP | | Generated: 12-May-2026 16:47:54



API-Based Products

Messaging Fundamentals - I | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Objectives

- Understand messaging fundamentals



Work Integrated Learning Programmes

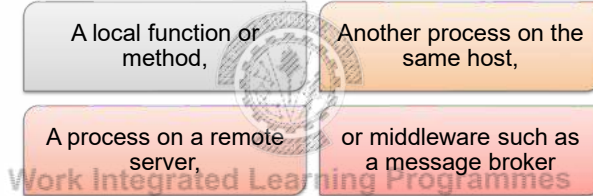
BITS WILP | Generated: 12-May-2026 16:47:54



Messages

Messages contain data that are published by a message producer to a message Receiver

- Receivers may be



BITS WILP | Generated: 12-May-2026 16:47:54

Common Types of Messages

A **command message** requests that work be done immediately or in the future

- Command messages are often imperative
 - CreateOrder, RegisterPayment,
- Sometimes referred to as request messages

A **reply message** provides the result, or outcome, of a command message

- Not all command messages result in a reply message

Event messages tell the receiver about something that happened in the past

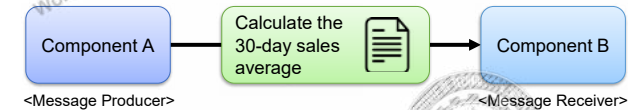
- Typically used when a business event has occurred, a workflow state has changed, or data has been created or modified



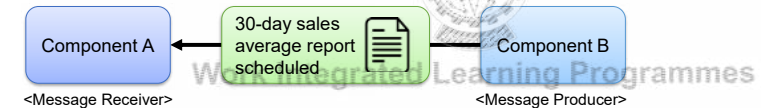
Common Types of Messages

Examples of the three primary types of messages.

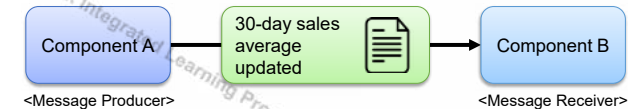
Command Message:



Reply Message:



Event Message:



BITS WILP | Generated: 12-May-2026 16:47:54

BITS WILP | Generated: 12-May-2026 16:47:54

Messaging Styles

Synchronous messaging involves the message producer sending a message and waiting while the receiver processes it and returns a reply

Asynchronous messaging allows the message producer and receiver to operate in their own time rather than waiting upon one another

- The message producer sends the message to the receiver, but the receiver may not be able to process it immediately
- The message producer is free to perform other tasks while waiting for a reply from the message receiver



Image source: <https://mayaberner.medium.com/asynchronous-like-gen-z-bb9c523ac2d1>

BITS WILP | Generated: 12-May-2026 16:47:54

Messaging Localities

Local messaging assumes that messages are sent and received within the same process

- The Smalltalk programming language was built to support sending and receiving messages between objects.

Interprocess messaging exchanges messages between separate processes but on the same host

- Examples include UNIX sockets and dynamic data exchange (DDE)

Distributed messaging involves two or more hosts for messaging

- Messages are transmitted over a network using the desired protocol
- Advanced Message Queuing Protocol (AMQP)
- Message Queuing Telemetry Transport (MQTT),
- SOAPbased Web services, REST-based APIs

BITS WILP | Generated: 12-May-2026 16:47:54

The Elements of a Message

Message body, typically structured in formats like JSON or XML,

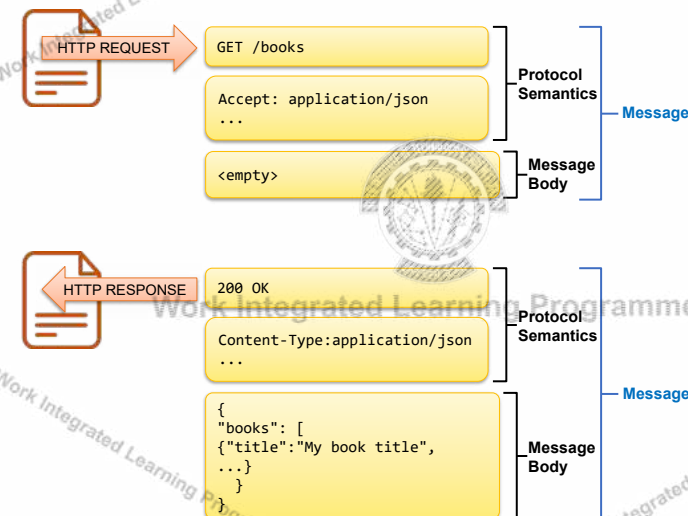
- Although binary or plain text formats are also acceptable

Transport protocol semantics:

- HTTP, MQTT, and AMQP include message headers containing details like
 - Creation timestamps,
 - Time-to-live (TTL),
 - and priority/quality of service

A message is considered complete only when it includes all necessary information for processing over the respective protocol

The Elements of a Message

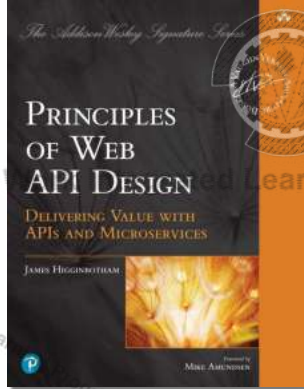


BITS WILP | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

What distinguishes command messages from event messages in message-based communication?

- A) Command messages request immediate or future work and always result in a reply message, while event messages provide the outcome of a command message.
- B) Command messages are imperative and often result in a reply message, whereas event messages inform the receiver about past occurrences, such as business events or workflow changes.
- C) Command messages are typically used for past events, while event messages request immediate actions to be performed.
- D) Command messages are always imperative and result in a reply message, while event messages are only used for future work requests.

Correct Answer: B

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Meaning of message and its types
 - Command, Response and Event
- Messaging styles:
 - Synchronous, Asynchronous
- Messaging Localities:
 - Local, Interprocess and Distributed
- Elements of Message



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

What distinguishes synchronous messaging from asynchronous messaging?

- A) Synchronous messaging involves both the producer and receiver waiting for each other's response, while asynchronous messaging allows them to operate independently.
- B) Synchronous messaging allows the producer to perform other tasks while waiting for a reply, while asynchronous messaging requires constant monitoring of the message queue.
- C) Synchronous messaging enables the receiver to process messages immediately, while asynchronous messaging requires the producer to wait for the receiver to become available.
- D) Synchronous messaging does not involve waiting for a reply, while asynchronous messaging requires both parties to wait for each other's response.

Correct Answer: A

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ3

What are the different types of messaging based on message exchange location?

- A) Local, small-scale, and distributed.
- B) Local, interprocess, and distributed.
- C) Synchronous, asynchronous, and distributed.
- D) Small-scale, interprocess, and distributed.

Correct Answer: B

Learning Outcomes

- Understand fundamentals of
 - Message Brokers
 - Message Streaming

API-Based Products

Messaging Fundamentals - II | Suresh Gopal

Messaging Brokers

Message brokers act as an intermediary between message producers and message receivers

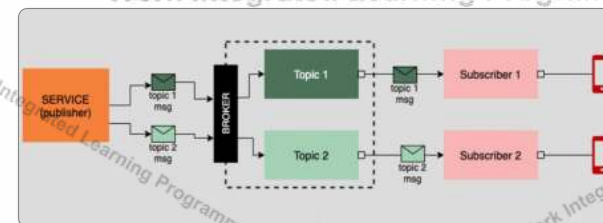
A more loosely coupled design, as producers are only aware of the message broker but not the components ultimately receiving the messages

Examples:

RabbitMQ,

ActiveMQ,

and Jmsmq etc



Messaging Brokers Features

Transactional boundaries

Ensure that messages are published or marked as delivered only if the transaction is committed

Durable subscriptions

Store messages prior to dispatching to message receivers

Client acknowledgement mode

Specifies how a message is considered acknowledged by the client to provide flexibility in balancing performance with failure recovery

Messaging Brokers Features

Message processing failures

are handled by dispatching messages to a different receiver in the event of a failure or outage of the original message receiver

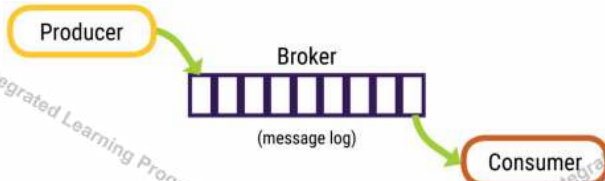
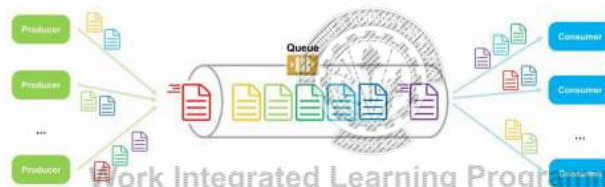
Message priority and TTL

assist the message broker in prioritizing the delivery of messages and removing unprocessed messages if they exceed a specific period of time without being processed

Two Methods of Message Distribution

Point-to-Point Message Distribution (Queues)

Fanout Message Distribution (Topics)



Point-to-Point Message Distribution (Queues)

Facilitates a publisher to send a message to a single subscriber chosen from a pool of registered subscribers

The broker, in this scenario, is tasked with selecting the subscriber to receive the published message for processing,

- Often employing a round-robin or similar selection method

Finds common application in background job processing,

- Where each job should undergo processing just once by a pool of workers



Communicate with messages

Messages put onto queue by sender
Receiver consumes messages from queue

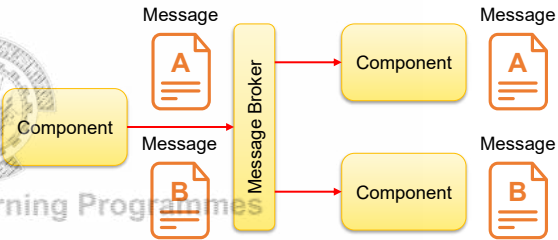
Point-to-Point Message Distribution (Queues)

Only one subscriber will be designated to receive a message from the queue

- If the initially selected subscriber fails to process the message within a specified timeout period, the broker will opt for another subscriber for message processing

Serve a practical purpose in disseminating command messages,

- Ensuring that only one consumer processes a message at a time,
- Maintains consistency, predictability, and preventing duplicate message processing



A point-to-point queue that dispatches each message to a single message receiver subscribed to the queue.

Fanout Message Distribution (Topics)

Topics, ensures that every message published to a topic is disseminated to all currently registered subscribers

- Unlike point-to-point queues, where only one subscriber receives a message, in fanout messaging, each message is processed by every subscriber

The broker overseeing fanout messaging **does not** monitor whether the message was processed by all subscribers or only a subset

- Simply ensures distribution to all

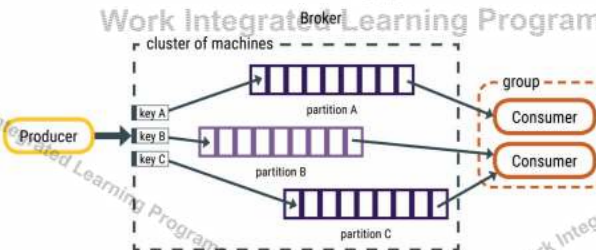
Fanout Message Distribution (Topics)

Every subscriber to the topic receives a copy of each published message

- Enabling independent and parallel processing logic for each event message

Subscribers operate without awareness of one another or the publisher

- Sole acknowledgment is the reception of a new message for processing



Message Streaming

Message brokers **primarily operate in a transactional manner**, aimed at **managing the state of durable subscriptions to facilitate failure recovery of offline receivers**

- Impose limitations on scalability

Message streaming, on the other hand, leverages the extensive expertise garnered from decades of message broker usage but **redistributes certain responsibilities away from the server** while introducing new capabilities to address modern data and messaging requirements

Apache Kafka, Apache Pulsar, and Amazon Kinesis

- Push notifications of new messages to one or more subscribers

Message Streaming

Unlike message brokers, **subscribers within message streaming frameworks have the flexibility to request messages from the topic's available message history at any given point**

- Allows for message replay or seamless continuation of processing from where it was previously halted

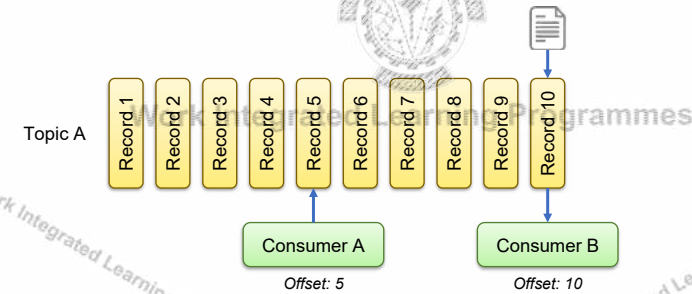
Unlike traditional message brokers, **most streaming servers transfer state management responsibilities from the server to the client**

- Where the client is tasked with tracking the last seen message and handling error recovery

Message Streaming

Facilitated by transitioning from a traditional queue or topic model to **an append-only log structure within streaming servers**

- Logs can either retain all messages indefinitely or limit message history based on specified retention periods

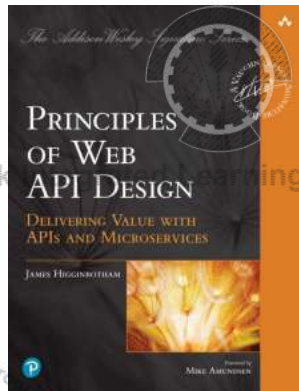


A streaming topic comprising a distributed log of recorded messages, consumed by two separate consumers who have two separate offsets to reflect their current message.

Reference

Principles of Web API Design

- By James Higginbotham



Summary

- Message brokers and its features
- Methods of message distribution
 - Point-to-Point Message Distribution (Queues)
 - Fanout Message Distribution (Topics)
- Message Streaming Fundamentals

MCQ1

What is the primary role of message brokers in message-based communication?

- A) Message brokers directly transmit messages between producers and receivers, ensuring tight coupling between components.
- B) Message brokers serve as intermediaries between producers and receivers, facilitating a more loosely coupled design by decoupling producers from the components ultimately receiving the messages.
- C) Message brokers exclusively handle message reception, while producers and receivers communicate directly with each other.
- D) Message brokers are responsible for generating messages and transmitting them to the appropriate receivers based on predefined rules.

Correct Answer: B

BITS WILP | | Generated: 12-May-2026 16:47:54



MCQ2

What are the key features and functionalities associated with message handling and delivery in message-based systems?

- A) Transactional boundaries, durable subscriptions, client acknowledgment mode, and message priority.
- B) Message processing failures, message priority, TTL, and transactional boundaries.
- C) Durable subscriptions, client acknowledgment mode, message processing failures, and message priority.
- D) Transactional boundaries, durable subscriptions, client acknowledgment mode, and message processing failures.

Correct Answer: A

BITS WILP | | Generated: 12-May-2026 16:47:54



MCQ3

What distinguishes fanout messaging from point-to-point messaging?

- A) Fanout messaging ensures that only one subscriber receives each message, while point-to-point messaging disseminates messages to all registered subscribers.
- B) Fanout messaging disseminates every message to all currently registered subscribers, while point-to-point messaging ensures that only one subscriber receives each message.
- C) Fanout messaging is monitored by the broker to ensure that every subscriber processes each message, while point-to-point messaging does not monitor message processing.
- D) Fanout messaging allows only a subset of subscribers to receive each message, while point-to-point messaging ensures distribution to all registered subscribers.

Correct Answer: B

BITS WILP | | Generated: 12-May-2026 16:47:54



API-Based Products

Async API Styles | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Understand different Async API Styles



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Async API Styles

Async APIs are an API interaction style that allows the server to inform the consumer when something has changed

Variety of API styles:

webhooks

Server-Sent
Events (SSE)

and
WebSocket



BITS WILP | Generated: 12-May-2026 16:47:54

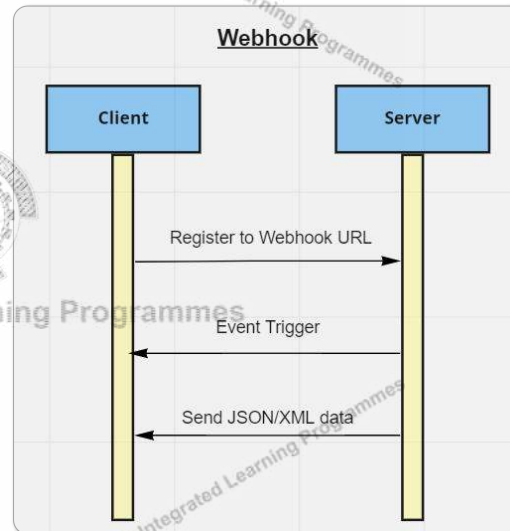
Server Notification Using Webhooks

Webhooks facilitate API servers to notify other servers of events occurring

Unlike traditional callbacks confined within a codebase, webhooks operate via HTTP POST over the Web

Coined by Jeff Lindsay in 2007, webhooks led to the development of REST Hooks patterns for standardized webhook management and security

Suitable for server-to-server communication but not for browser and mobile applications



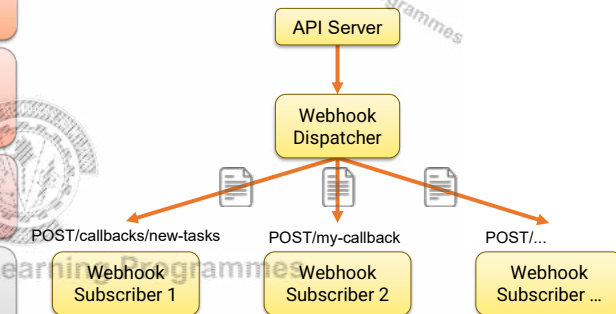
Server Notification Using Webhooks

When an event occurs, the API server sends a POST request to a URL provided by the recipient system

Interested parties register for notifications on specific URLs, such as `https://client_app/callbacks/new-events`

API server dispatches POST requests to subscribers' callback URLs with event details

Webhooks require network accessibility for the API server and the capability to host an API server to receive POST requests



An API server's webhook dispatcher that sends a message to each registered URL that wishes to receive the callback using HTTP POST.



Server Push Using Server-Sent Events

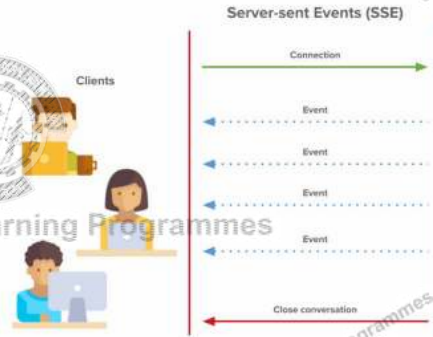
Utilizes the EventSource browser interface standardized under HTML5 by the W3C,

- Enabling servers to push data back to clients over longer-lived HTTP connections

Simplifies server-push notifications without the need for API polling,

- Originally intended for browser use but increasingly employed for both browsers and server-side subscribers

Maintains a standard HTTP connection but prolongs the connection duration to facilitate server-to-client data transmission when available



Server Push Using Server-Sent Events

Subscribers request SSE operations via GET with the media type "text/event-stream", compatible alongside standard request/response interactions like JSON or XML

Upon connection, the server pushes events separated by newlines

- Clients can reconnect to retrieve missed events using the Last-Event-ID HTTP header, aiding failure recovery

Data field formats range from simple data points to JSON payloads, accommodating incremental data processing and complex aggregations

API Consumer
Web/Mobile/Server

GET/tasks/sse-event-stream
Accept: text/event-stream

REST
API Server

task_created:
data: {"id": "12345"}
id: 12345

task_created:
data: {"id": "6789"}
id: 6789

<<Disconnected>>

GET /tasks/sse-event-stream
Accept: text/event-stream
Last-Event-Id: 6789

Using Server-Sent Events (SSE) to allow API servers to push events to the client over a long-lived connection. Connections may be resumed using the Last-Event - Id request header.

Server Push Using Server-Sent Events

SSE supports various use cases, including

- Frontend state change notifications,
- Receiving business events over HTTP
- Incremental data processing

Limitations of SSE include

- API gateway constraints
- Browser compatibility issues
- Lack of bidirectional communication compared to WebSocket protocol

Bidirectional Notification via WebSocket

WebSocket enables bidirectional communication via a full-duplex protocol tunneled through a single TCP connection initiated using HTTP

- Clients can push requests to servers and receive events/responses simultaneously over WebSocket connections

Maintained by the Internet Engineering Task Force as RFC 6455, WebSocket is

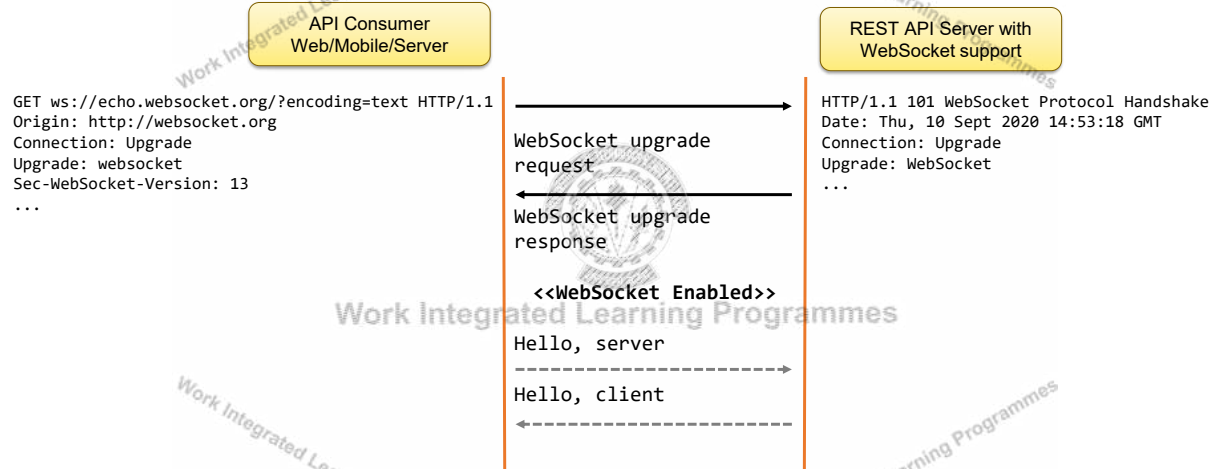
- Widely supported by browsers
- Suitable for various scenarios, including:

Browser-to-server

Server-to-browser

Server-to-server communication

Bidirectional Notification via WebSocket



An example interaction between an API client and server using WebSocket and a plain text subprotocol to create a chat application.

BITS WILP | Generated: 12-May-2026 16:47:54

Bidirectional Notification via WebSocket



While WebSocket initiation uses HTTP, a subprotocol must be selected for communication

- Multiple subprotocols are officially registered with IANA, supporting both text and binary formats

WebSocket is more intricate to implement compared to SSE but supports bidirectional communication

- Allowing both client-to-server and server-to-client data transmission within the same connection

BITS WILP | Generated: 12-May-2026 16:47:54

Selecting an Async API Style



Webhooks:

- Allow server-originated notifications without requiring client initiation
- Suitable for server-to-server communication
- Not applicable for web browsers or mobile apps

Server-Sent Events (SSE):

- Easy to implement on both server and client side
- lack of bidirectional communication
- Ideal for server-push events following RESTful API design.

WebSocket Protocol:

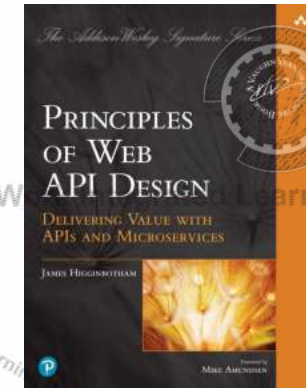
- More complex to implement due to subprotocol support but enables bidirectional communication
- Widely supported across browsers, making it suitable for various scenarios

BITS WILP | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Various Async API styles
 - Webhooks
 - Server-Sent Events (SSE)
 - and WebSocket



Work Integrated Learning Programmes



MCQ1

What is a distinguishing feature of webhooks compared to traditional callbacks?

- A) Webhooks operate within a codebase, while traditional callbacks utilize HTTP POST over the Web.
- B) Webhooks facilitate standardized webhook management and security, unlike traditional callbacks.
- C) Webhooks are suitable for browser and mobile applications, while traditional callbacks are not.
- D) Webhooks enable server-to-server communication via HTTP POST over the Web, while traditional callbacks are confined within a codebase.

Correct Answer: D



Work Integrated Learning Programmes

MCQ2

What technology facilitates server-to-client data transmission over longer-lived HTTP connections without the need for API polling?

- A) Webhooks
- B) REST Hooks
- C) EventSource
- D) Server-Sent Events (SSE)

Correct Answer: D



Work Integrated Learning Programmes

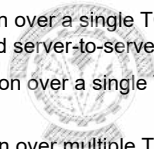


MCQ3

What are the key features and capabilities of WebSocket technology?

- A) WebSocket enables bidirectional communication over multiple TCP connections, suitable only for browser-to-server communication.
- B) WebSocket enables bidirectional communication over a single TCP connection, suitable for various scenarios including browser-to-server, server-to-browser, and server-to-server communication.
- C) WebSocket enables unidirectional communication over a single TCP connection, suitable primarily for server-to-browser communication.
- D) WebSocket enables bidirectional communication over multiple TCP connections, suitable only for server-to-server communication.

Correct Answer: B



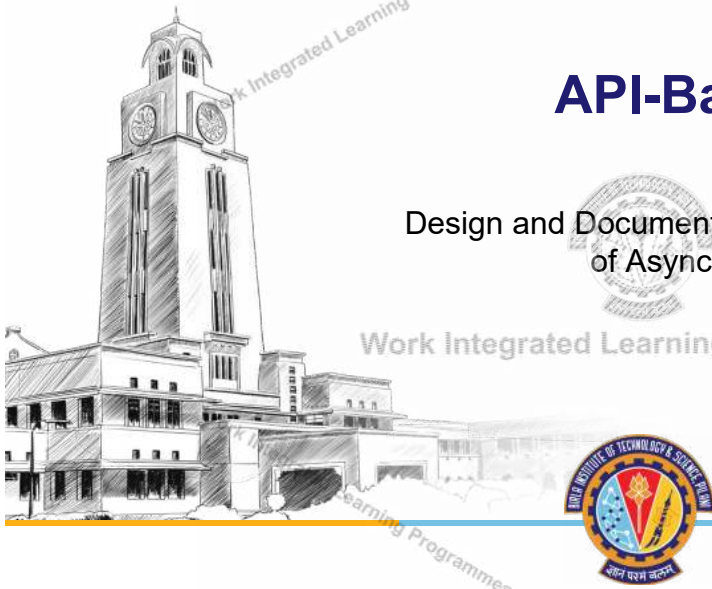
Work Integrated Learning Programmes



API-Based Products

Design and Documentation
of Async APIs

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Designing Async APIs

Similar to the process used to design traditional request/ response APIs using a REST-, RPC-, or query-based style

- Begin with the resources identified during the API modeling step
- Revisit the events identified while capturing the operation details of each API profile
- Then, determine what commands and events would be beneficial for API consumers

Learning Outcomes

- Understand the elements and factors to be considered during design of Async APIs



Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

Factors to be Considered

Command Messages

Event Notifications

Event-Carried State Transfer Events

Event Batching

Event Ordering



Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

Command Messages

Command messages contain all necessary details to request another component to perform a specific task

- Requires careful consideration to include sufficient details for processing the request, including a possible target location for publishing result messages
- Target locations for result messages can
 - URLs for POSTing results,
 - URLs for message broker topics,
 - URLs for shared object stores like AWS S3

Opt for a language-agnostic message format like

- UBER hypermedia format, Apache Avro, Protocol Buffers, JSON, or XML to ensure broader compatibility and interoperability

Event Notifications

Event notifications, also known as thin events, inform subscribers about state changes or business events, providing essential information for subscriber interest assessment

- Subscribers are responsible for fetching the latest resource representation via an API to prevent working with outdated data

Including hypermedia links in thin events facilitates integration of API operations for retrieving the latest resource representation within async APIs

Event Notifications

Thin events are suitable for resources that frequently change,

- Requiring subscribers to obtain the latest representation to avoid stale data usage

While not obligatory, thin events may contain details about specific properties that changed during an update, aiding consumers in determining event relevance

Event-Carried State Transfer Events

Encapsulate all available information at the time of occurrence, eliminating the need to access an API for the entire resource representation

- Provides comprehensive information to subscribers

Reasons for preferring event-carried state transfer over thin events include

- Subscribers seeking a complete snapshot of the resource associated with the event,
- Reducing support for message streaming with complete point-in-time snapshots
- Minimizing API traffic and service coupling in interservice communication scenarios

Event Batching

While many async APIs notify subscribers for each individual message, **certain designs may find value in grouping events into batches**

- Event batching **entails subscribers handling one or more messages within each notification**
- An example of event batching involves **encapsulating notifications within an array**, even if only one event message is present at the time

Batching messages or events allows for grouping based on

- A specific timeframe,
- Number of events per batch,
- or through other grouping factors

BITS WILP | | Generated: 12-May-2026 16:47:54

Event Ordering

Most event-based systems aim to deliver messages in order, but this isn't always guaranteed

- Challenges arise when event receivers go offline and need to recover missing messages while handling new inbound messages
- In complex distributed systems, maintaining message order can be difficult due to multiple brokers and message styles.

Considerations for ensuring message order include **using message sequence numbering or timestamp-based ordering within a single message broker**

BITS WILP | | Generated: 12-May-2026 16:47:54

Documenting Async APIs

The AsyncAPI specification is a standard for defining async messaging channels, supporting various technologies like

- Traditional message brokers,
- SSE,
- Kafka, MQTT, and IoT messaging

Increasingly adopted as a comprehensive solution for defining message schemas and protocol binding specifics across message-driven protocols

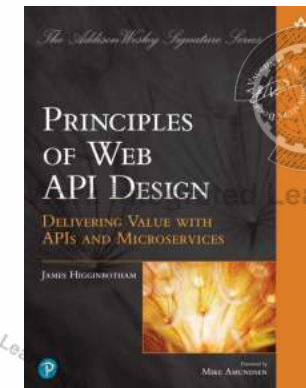
Notably, AsyncAPI is distinct from the OpenAPI Specification (OAS) but draws inspiration from it, aiming to maintain a similar format for ease of adoption

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

Principles of Web API Design

- By James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Elements of Async API design process
- The factors to be considered including
 - Command Messages
 - Event Notifications
 - Event-Carried State Transfer Events
 - Event Batching
 - Event Ordering



Work Integrated Learning Programmes

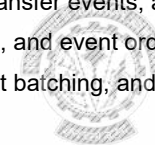


MCQ1

What factors should be considered when designing async APIs?

- A) Command messages, event notifications, and event batching.
- B) Event notifications, event-carried state transfer events, and event ordering.
- C) Command messages, event notifications, and event ordering.
- D) Event-carried state transfer events, event batching, and event ordering.

Correct Answer: B



Work Integrated Learning Programmes

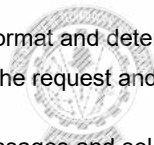


MCQ2

What considerations should be made when designing command messages for async APIs?

- A) Including sufficient details for processing the request and determining target locations for result messages.
- B) Selecting a language-agnostic message format and determining target locations for result messages.
- C) Including sufficient details for processing the request and selecting a language-agnostic message format.
- D) Determining target locations for result messages and selecting a language-agnostic message format.

Correct Answer: A



Work Integrated Learning Programmes

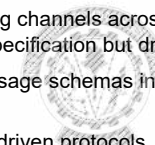


MCQ3

What is the AsyncAPI specification primarily used for, and how does it relate to the OpenAPI Specification (OAS)?

- A) AsyncAPI is a standard for defining message schemas and protocol binding specifics, primarily used for traditional message brokers. It is closely related to the OpenAPI Specification, sharing the same format.
- B) AsyncAPI is used for defining async messaging channels across various technologies like SSE, Kafka, MQTT, and IoT messaging. It is distinct from the OpenAPI Specification but draws inspiration from it for ease of adoption.
- C) AsyncAPI is exclusively used for defining message schemas in IoT messaging protocols. It is unrelated to the OpenAPI Specification.
- D) AsyncAPI is a standard for defining message-driven protocols, primarily used for SSE and Kafka. It shares a similar format with the OpenAPI Specification for ease of adoption.

Correct Answer: B



Work Integrated Learning Programmes



API-Based Products

Essence of API Documentation Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

API Documentation

API documentation serves as human-readable instructions for interacting with an API

Provides detailed information on

Endpoints,

Methods,

Resources,

Authentication protocols,

Parameters,

and headers

Prioritizing API documentation leads to

- Higher API adoption rates,
- Reduced support tickets,
- and increased revenue for public APIs

Learning Outcomes

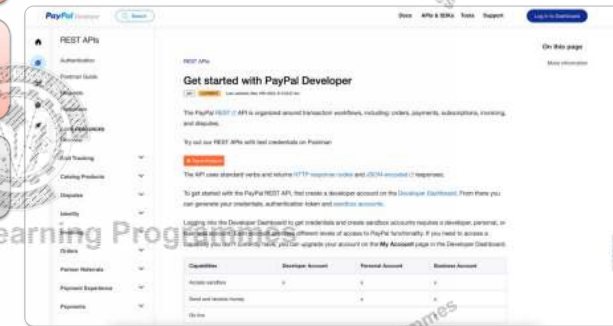
- Explain the importance of effective API documentation in improving
 - Developer experience,
 - Fostering collaboration,
 - Reducing support overhead,
 - and facilitating successful API integration

Why to Document APIs?

Effective API documentation **enhances the developer experience** across private, partner, and public APIs

Public API documentation **helps leaders understand third-party APIs** and their potential **impact on organizational goals**

Private API documentation **promotes cross-team collaboration**



PayPal REST API

Image source: <https://nordicapis.com/5-examples-of-api-documentation-with-great-developer-experience/>

BITS WILP | | Generated: 12-May-2026 16:47:54

API Documentation in an API-First World

API-first development model involves **composing applications using internal or external services delivered through APIs**

- Internal API documentation aids cross-team collaboration, reduces code duplication, and expedites the onboarding process for new employees
- Public API documentation helps potential consumers understand and experiment with an API, leading to increased adoption and revenue

Postman's 2022 State of the API report **highlights documentation as one of the top factors influencing leaders' decisions to integrate with third-party APIs**

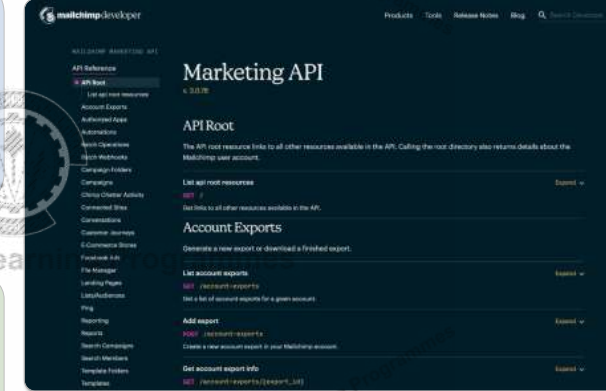
Common Types of API Documentation

Reference documentation:

- Provides detailed information on
 - Every endpoint,
 - Methods,
 - Parameters,
 - and accepted data types,
 - Along with a description of each endpoint's purpose

Tutorials:

- Step-by-step instructions focusing on specific use cases supported by the API, including common workflows like authentication



MailChimp Marketing API

Image source: <https://nordicapis.com/5-examples-of-api-documentation-with-great-developer-experience/>

Common Types of API Documentation

Examples and code samples:

- Demonstrates common API requests and responses in multiple programming languages
- Aiding in understanding the API's functionality

Release notes:

- Updates consumers on important changes to the API, such as **new features, bug fixes, or security patches**, ensuring they stay informed about changes that may affect their integration

Examples of API Documentation

Postman's Public API Network:

- A centralized API catalog allowing producers to share APIs and documentation with a **global community of over 30 million developers**

Exemplary API documentation from leading teams:

- Notable examples include **Stripe, Notion, PayPal, Amplitude, Salesforce, and DoorDash**, among others



<https://www.postman.com/postcon/2025/api-awards/>

Summary

- The importance of effective API documentation
- Common types of API documentation
 - Reference documentation
 - Tutorials
 - Examples and code samples
 - Release notes



Work Integrated Learning Programmes



Reference

Postman API documentation

<https://www.postman.com/api-platform/api-documentation/>

API documentation



Work Integrated Learning Programmes



MCQ1

Which of the following statements best describes the role of API documentation?

- A) API documentation is primarily designed to serve as machine-readable instructions for interacting with an API.
- B) API documentation provides detailed information about an API's available endpoints, methods, resources, and authentication protocols, enhancing the developer experience.
- C) API documentation is only beneficial for public APIs and does not significantly impact the integration process of private or partner APIs.
- D) API documentation is irrelevant for API producers and only serves the needs of API consumers.

Correct answer: B) API documentation provides detailed information about an API's available endpoints, methods, resources, and authentication protocols, enhancing the developer experience.



MCQ2

Which of the following statements accurately reflects the significance of API documentation in an API-first world?

- A) API documentation is irrelevant for organizations adopting an API-first strategy, as the focus is solely on composing applications using microservices.
- B) Internal API documentation primarily benefits consumers by providing a clear understanding of an API's functionalities and usage.
- C) Public API documentation plays a minor role in increasing adoption and revenue, as most consumers prefer to experiment with APIs without referring to documentation.
- D) API documentation is critical for facilitating cross-team collaboration, reducing code duplication, expediting onboarding, and attracting potential consumers to integrate with APIs.

Correct answer: D) API documentation is critical for facilitating cross-team collaboration, reducing code duplication, expediting onboarding, and attracting potential consumers to integrate with APIs.



MCQ3

Which of the following best describes the types of API documentation commonly used to aid consumers in effectively utilizing an API?

- A) Reference documentation provides updates on important changes to an API, such as new features and bug fixes.
- B) Tutorials offer examples of common API requests and responses in multiple programming languages.
- C) Examples and code samples outline step-by-step instructions for using the API, focusing on specific use cases and workflows.
- D) Release notes detail every endpoint of the API, including its methods, parameters, and accepted data types, along with a description of its purpose.

Correct answer: C) Examples and code samples outline step-by-step instructions for using the API, focusing on specific use cases and workflows.

API-Based Products

Elements of API Documentation | Suresh Gopal

Learning Outcomes

- Understand the essential components of API documentation

Model for API Documentation

API documentation serves both humans and machines, providing comprehensive information about API usage

- Should be human-readable, aiding app developers in assessing its suitability for their client apps
- Includes details on licensing policy and usage requirements
- Covers input and output parameters, message format, and error messages
- Enables machine parsing for generating client stubs and server-side skeleton code

Aim is to facilitate seamless integration and development processes

Elements of API Documentation

To make API documentation effective, it should include the following aspects about the API:

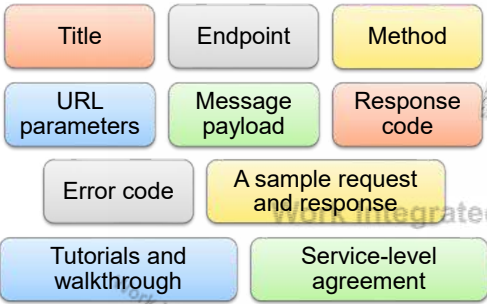


Image source: <https://stoplight.io/api-documentation-guide>

BITS WILP | | Generated: 12-May-2026 16:47:54

Elements - Title, Endpoint, Method, Parameter

```
openapi: 3.0.0
info:
  title: Simple Inventory API
  description: This is a simple API
servers:
  - url: https://virtserver.swaggerhub.com/pravinyash/MySimpleAPI/1.0.0
    description: SwaggerHub API Auto Mocking
paths:
  /inventory:
    get:
      tags:
        - developers
      summary: searches inventory
      description: |
        By passing in the appropriate options, you can search for
        available inventory in the system
      operationId: searchInventory
      parameters:
        - name: searchString
          in: query
          description: pass an optional search string for looking up inventory
          required: false
          style: form
          explode: true
          schema:
            type: string
```



BITS WILP | | Generated: 12-May-2026 16:47:54

Elements - Message Payload, Response & Error Code

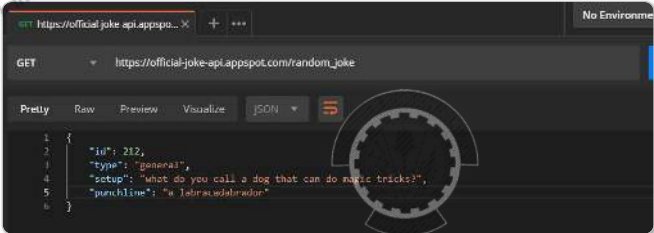
```
InventoryItem:
  required:
    - id
    - manufacturer
    - name
    - releaseDate
  type: object
  properties:
    id:
      type: string
      format: uuid
      example: d290f1ee-6c54-4b01-90ee-d701748f0351
    name:
      type: string
      example: Widget Adapter
    releaseDate:
      type: string
      format: date-time
      example: 2016-08-29T09:12:33.001Z
    manufacturer:
      $ref: '#/components/schemas/Manufacturer'
```

```
responses:
  "200":
    description: search results matching criteria
    content:
      application/json:
        schema:
          type: array
          items:
            $ref: '#/components/schemas/InventoryItem'
  "400":
    description: bad input parameter
```



BITS WILP | | Generated: 12-May-2026 16:47:54

Sample Calls



Source: <https://www.freecodecamp.org/news/rest-api-tutorial-rest-client-rest-service-and-api-calls-explained-with-code-examples/>

BITS WILP | | Generated: 12-May-2026 16:47:54

Tutorials and Walkthrough

Tutorials and walk-throughs enhance understanding through practical examples

- Example code demonstrating API usage aids developers in implementation

Sample SDKs in popular languages like Java, Node.js, C#, PHP, Ruby, and Python expedite adoption.

- Recommended to include SDKs for different platforms such as Android, iOS, and Windows
- **Prioritize tutorials and sample code for the most popular languages and platforms**

Service-Level Agreements (SLAs)

Service-Level Agreements (SLAs) outline non-functional requirements of the API

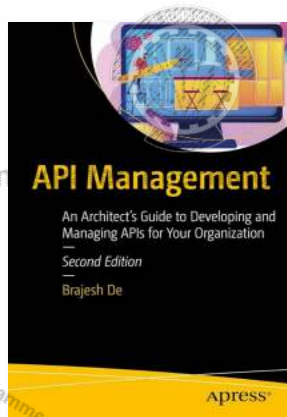
SLA details may include expected throughput, response time, rate limits, maintenance or downtime information, etc.

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



Summary

- Elements of API documentation which encompasses various components such as
 - Title,
 - Endpoint,
 - Method,
 - URL parameters,
 - Message payload,
 - Response and Error codes
 - Sample calls,
 - Tutorials,
 - and service-level agreements (SLAs)

MCQ1

What does the API documentation primarily aim to provide?

- A) Detailed programming instructions
- B) Information for both humans and machines
- C) Licensing policies only
- D) Only message payload structure

Answer: B) Information for both humans and machines

MCQ2

Which of the following is/are elements of API documentation?

- A) Title,
- B) Endpoint,
- C) Method,
- D) URL parameters,

Answer: All

MCQ3

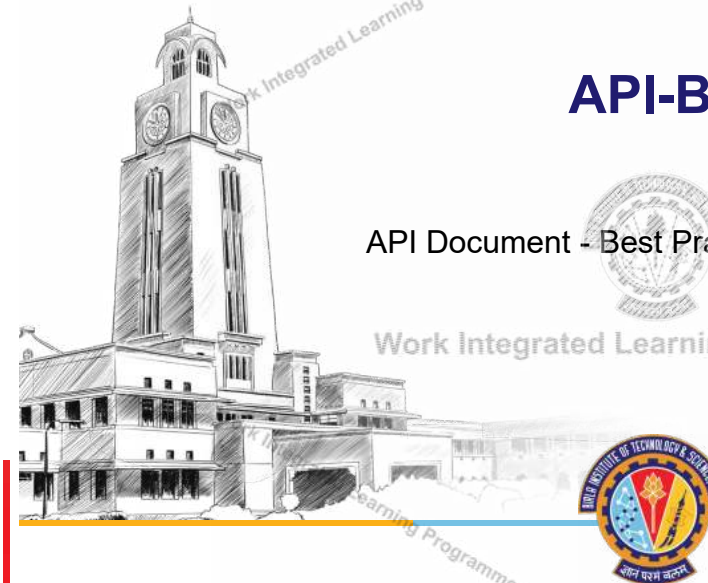
What should the API documentation include regarding error handling?

- A) Only HTTP response codes
- B) Only successful response codes
- C) All possible response codes and error messages
- D) No error codes are necessary

Answer: C) All possible response codes and error messages

API-Based Products

API Document - Best Practices | Suresh Gopal



Learning Outcomes

- How to write API documentation?
- What are some best practices for creating API documentation?



Work Integrated Learning Programmes



How to write API documentation?

Writing API documentation involves several steps and requires familiarity with

- The API's functionality,
- Empathy for its users,
- and a willingness to iterate

To ensure your documentation is effective, one should:

- Understand the API
- Know the audience
- Provide detailed instructions for the most common use cases
- Review, test, and verify the documentation
- Continuously update the documentation



Image source: <https://www.postman.com/api-platform/api-documentation/>

BITS WILP | Generated: 12-May-2026 16:47:54

How to write API documentation?

To ensure effectiveness, one must understand the

- API's purpose,
- Endpoints,
- Methods,
- Parameters,
- Accepted data types,
- and authentication mechanisms

Knowing the audience is crucial,

- As different levels of technical knowledge exist among consumers

Detailed instructions for common use cases should be provided,

- Including code samples and example requests



How to write API documentation?

Thorough review, testing, and verification of documentation are necessary to catch mistakes before publishing

Continuous updating of documentation is essential due to the rapid evolution of APIs

- To avoid confusing consumers and maintain their trust

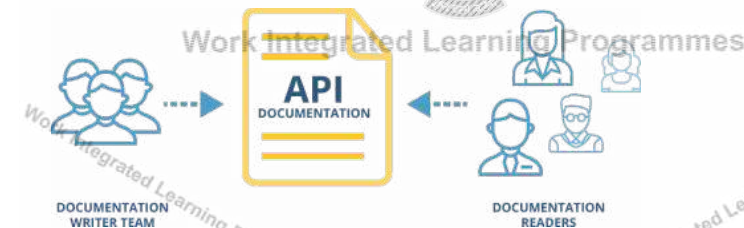


Image source: <https://help.talend.com/en-US/api-designer-user-guide/Cloud/api-documentation>

BITS WILP | Generated: 12-May-2026 16:47:54

Best Practices for Creating API Documentation



API documentation is an **essential deliverable** that has a **significant impact on consumers**

- Its quality can be directly correlated with the overall success of the API

Crucial for teams to adhere to the following best practices when writing API documentation:

- **Tell a compelling story**
- **Keep the documentation up-to-date**
- **Write for a range of audiences**

**API Documentation:
Best Practices**

BITS WILP | | Generated: 12-May-2026 16:47:54

Tell a Compelling Story



API documentation **serves to narrate the unique role an API plays in its software ecosystem**

Should elucidate

- The intended audience,
- Usage guidelines,
- and the benefits it offers

Providing **this broader perspective is crucial for developers to grasp the API's potential and apply technical details effectively**



Image source: https://s3.amazonaws.com/kinlane-productions2/photos/Millais_Boyhood_of_Raleigh.jpg

BITS WILP | | Generated: 12-May-2026 16:47:54

Keep the documentation up-to-date



Many API development teams **ship code changes several times a week, which puts their documentation at risk of falling out of date**

Outdated documentation erodes consumers' trust, especially when updates are not backward compatible

- Essential for teams to systematize the process of **updating their documentation to ensure it always reflects the current state of their API in production**
- Should also **capture updates in a changelog**, which is a dated record of every change to an API's resources and functionality

BITS WILP | | Generated: 12-May-2026 16:47:54

Write for a range of audiences



API documentation is an important resource for a **wide range of software and business professionals**

- Developers will consult an API's documentation to learn how to interact with it
- CTOs may use documentation to help them understand an API's pricing and evaluate whether it will help them meet their business goals

Documentation writers must therefore keep this diverse audience in mind

- For instance, they must thoroughly describe the API's functionality without relying too heavily on technical language or obscuring the larger purpose that the API serves

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

The process of creating API documentation

- Understand the API
- Know your audience
- Provide detailed instructions for the most common use cases
- Review, test, and verify the documentation
- Continuously update the documentation

Reference

Postman API documentation

<https://www.postman.com/api-platform/api-documentation/>

API documentation



MCQ1

What is a crucial aspect of writing effective API documentation?

- a) Memorizing all API endpoints
- b) Understanding the API's purpose and functionality
- c) Having empathy for the developers
- d) Following a rigid documentation template

answer: b and c

MCQ2

What is a key consideration when creating API documentation to ensure it remains useful and relevant over time?

- a) Including complex technical jargon
- b) Providing comprehensive information on endpoints only
- c) Maintaining regular updates and a changelog
- d) Ignoring user feedback

answer: c

MCQ3

How can API documentation cater to a diverse audience with varying levels of technical expertise?

- a) By using only technical language to ensure accuracy
- b) By providing clear context and examples
- c) By excluding use cases and practical examples
- d) By limiting the documentation to basic information

answer: b

API-Based Products

API Languages - Usage

Suresh Gopal

Learning Outcomes

- Learn about use cases for API description languages

Several use cases for API descriptions

API descriptions can support all phases of API design and development

- Use cases are not complete, but they are chosen to convey the central role of API descriptions

Use cases

Communication and Documentation

Design Repository

Contract Negotiation

API Implementation

Client Implementation

Discovery

Simulation

Communication and Documentation

An API serves as a bridge between multiple software systems

- Developers on the **provider side** are occupied with **crafting the API**
- While those on the **consumer side** build applications that **utilize it**

Effective documentation is key to establishing a **shared understanding of the API**,

- Especially considering that developers are often **dispersed across different companies, countries, and time zones**
- Well-documented APIs are **essential for both** the creators of the server-side implementation and the users of the client-side applications

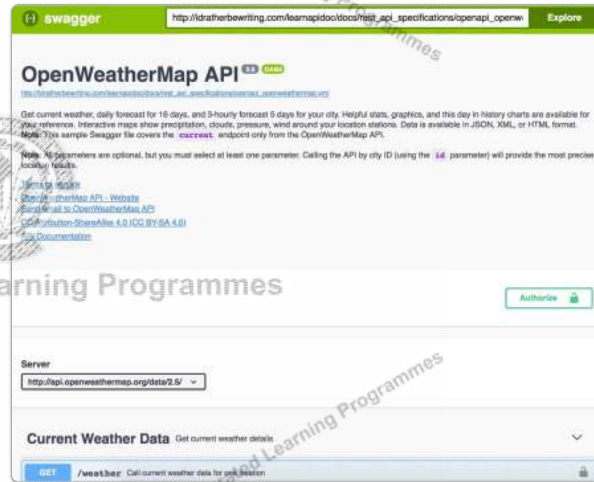


Image source: https://monoh.com/a/6bd84338c365/how-to-turn-off-swagger-ui-in-production#google_vignette
BITS WILP | Generated: 12-May-2026 16:47:54

Design Repository

Serves as a centralized reference for API proxies

- Holds the **authoritative API description**, clarifying doubts about parameters or status codes
- Encompasses **all significant design choices** for each API proxy

Documentation of the entire API portfolio, including API descriptions, is **imperative**

- **API descriptions should be version-controlled**, typically using repositories like GIT or SVN



Swagger



git



SUBVERSION

BITS WILP | Generated: 12-May-2026 16:47:54

Contract Negotiation

Traditionally, app developers would need to wait for the API to be finished

- **Contract first design** allows starting the implementation of the app by the consumer before the provider has finished building the API
- Allows for a very efficient development process with a **much quicker turn around time**
- Allows app developers to bring their apps to market quicker than before

API description can serve as a design contract

- Can be used for agreements **between API designer and API developer** or
- As a contract **between API consumer and API provider**



BITS WILP | Generated: 12-May-2026 16:47:54

Discovery

How does the client know about the capabilities of the API?

- **Client needs to be able to explore or discover the capabilities of the API programmatically**
- With such an automated discovery mechanism, **an app may include new APIs, which have not been known at design time**

To enable such an implementation, **an API description of the API portfolio should be served by a specific endpoint**

- Allows the caller to discover each API within the portfolio by downloading and parsing the API description
- A precondition is that the API provider made the API description available to the consumers



Image source: <https://docs.swagger.io/api-discovery/overview/>

BITS WILP | Generated: 12-May-2026 16:47:54

Simulation

Simulations offer consumers an initial glimpse into the completed API

- During early API design stages, simulations are shared with pilot consumers to gather initial feedback

Creating a simulation requires a model of the real world, which is provided within the API description

- The API description includes interface specifications necessary for building simple simulations or mockups

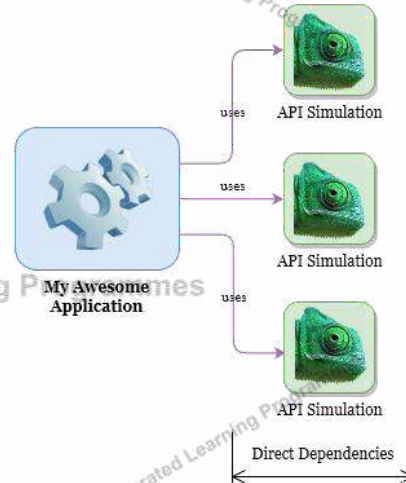


Image source: <https://apisimulator.io/>

BITS WILP | Generated: 12-May-2026 16:47:54

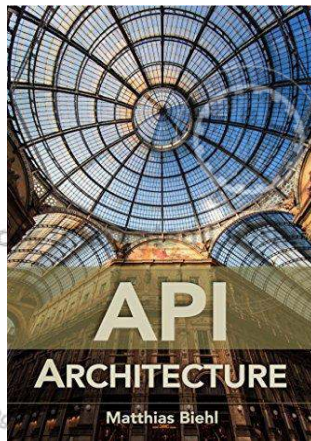
Summary

- How API descriptions play a pivotal role in both contract-first design and simulation processes

Reference

API Architecture

- By **MATTHIAS BIEHL**



MCQ1

Question: What is a key consideration for ensuring effective communication between developers involved in implementing and utilizing an API?

- A) The complexity of the API structure
- B) The geographical location of developers
- C) The documentation quality of the API
- D) The number of software systems connected by the API

Correct Answer: C) The documentation quality of the API

MCQ2

Question: What is the primary purpose of the Design Repository in managing API development?

- A) Storing API proxy source code
- B) Providing a central reference for API descriptions
- C) Tracking API usage statistics
- D) Facilitating communication between developers

Correct Answer: B) Providing a central reference for API descriptions

MCQ3

Question: What advantage does contract-first design offer in comparison to traditional API development methods?

- A) It eliminates the need for API descriptions.
- B) It accelerates the completion of API development.
- C) It allows app developers to start implementation before API completion.
- D) It requires a longer turnaround time for app development.

Correct Answer: C) It allows app developers to start implementation before API completion.

API-Based Products

OpenAPI - Specification | Suresh Gopal

Learning Outcomes

- Understand the structure of OpenAPI specification

OpenAPI - Specification

Can write OpenAPI definitions in YAML or JSON

- All keyword names are case-sensitive

A sample OpenAPI 3.0 definition written in YAML looks like:

```
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description in [CommonMark](http://commonmark.org/help/) or HTML.
  version: 0.1.9
servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main (production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g. Internal staging server for testing
paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        '200':
          # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Metadata

Every API definition must include the version of the OpenAPI Specification that this definition is based on:

- Openapi: 3.0.0

The OpenAPI version defines the overall structure of an API definition –

- What you can document and how you document it

OpenAPI 3.0 uses semantic versioning with a three-part version number.

- Available versions are 3.0.0, 3.0.1, 3.0.2, 3.0.3, 3.0.4, 3.1.0, 3.1.1

```
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description
  version: 0.1.9
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Metadata

The info section contains API information:

- Title, description (optional), version

Title is your API name

Description is extended information about your API

Version is an arbitrary string that specifies the version of your API

Can use semantic versioning like major.minor.patch, or an arbitrary string like 1.0-beta or 2017-07-25

Info also supports other keywords for contact information, license, terms of service, and other details

```
openapi: 3.0.0
info:
  title: Sample API
  description: Optional multiline or single-line description
  version: 0.1.9
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Servers

The servers section specifies the API server and base URL

- Can define one or several servers, such as production and sandbox

```
servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main (production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g. Internal staging server for testing
```

All API paths are relative to the server URL

In the example above, /users means <http://api.example.com/v1/users> or <http://staging-api.example.com/users>

- Depending on the server used

BITS WILP | | Generated: 12-May-2026 16:47:54

Paths

The paths section defines individual endpoints (paths) in API, and the HTTP methods (operations) supported by these endpoints

An operation definition includes

- Parameters,
- Request body (if any),
- Possible response status codes (such as 200 OK or 404 Not Found)
- and response contents

For example, GET /users can be described as attached:

```
paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        '200':
          # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

Parameters

Operations can have parameters passed via

- URL path (/users/{userId}),
- Query string (/users?role=admin),
- Headers (X-CustomHeader: Value)
- or cookies (Cookie: debug=0)

You can define the parameter

- Data types,
- Format,
- Whether they are required or optional,
- and other details

```
1. paths:
2.   /users/{userId}:
3.     get:
4.       summary: Returns a user by ID.
5.       parameters:
6.         - name: userId
7.           in: path
8.           required: true
9.           description: Parameter description in CommonMark or HTML.
10.          schema:
11.            type: integer
12.            format: int64
13.            minimum: 1
14.          responses:
15.            '200':
16.              description: OK
```

Request Body

If an operation sends a request body, use the requestBody keyword to describe the body content and media type.

```
paths:
  /users:
    post:
      summary: Creates a user.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
      responses:
        '201':
          description: Created
```

Responses

For each operation, you can define

- Possible status codes, such as 200 OK or 404 Not Found,
- and the response body schema.
- Schemas can be defined inline or referenced via \$ref

You can also provide example responses for different content types

```
responses:
  '200':
    description: A user object.
    content:
      application/json:
        schema:
          type: object
          properties:
            id:
              type: integer
              format: int64
              example: 4
            name:
              type: string
              example: Jessica Smith
  '400':
    description: The specified user ID is invalid (not a number).
  '404':
    description: A user with the specified ID was not found.
  default:
    description: Unexpected error
```

Authentication

The securitySchemes and security keywords are used to describe the authentication methods used in your API.

Supported authentication methods are:

- HTTP authentication: Basic, Bearer, and so on.
- API key as a header or query parameter or in cookies
- OAuth 2
- OpenID Connect Discovery

```
components:
  securitySchemes:
    BasicAuth:
      type: http
      scheme: basic
  security:
    - BasicAuth: []
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

[OpenAPI documentation](#)



Work Integrated Learning Programmes

Summary

- The elements of OpenAPI specification



Work Integrated Learning Programmes

MCQ1

Which section of the OpenAPI Specification file is primarily used to define the available endpoints and their operations?

- A) Info
- B) Paths
- C) Components
- D) Servers

Answer: B) Paths



Work Integrated Learning Programmes

MCQ2

What is the purpose of the components section in an OpenAPI Specification file?

- A) To define the available operations in the API
- B) To specify the base URLs and server variables of the API's endpoints
- C) To define reusable objects for parameters, request bodies, responses, and schemas
- D) To describe the overall API information such as title, version, and contact details

Answer: C) To define reusable objects for parameters, request bodies, responses, and schemas

MCQ3

Which keyword is used in the OpenAPI Specification file to provide metadata about the API, such as its title, version, and contact information?

- A) Paths
- B) Info
- C) Servers
- D) Components

Answer: B) Info

API-Based Products

Comparison of API
Description Formats

Suresh Gopal

Learning Outcomes

- Compare and contrast various API description formats including
 - OpenAPI,
 - RAML,
 - API Blueprint,
 - and GraphQL SDL

OpenAPI (Formerly Swagger)

OpenAPI is a widely-used specification for describing RESTful APIs

Pros:

- Well-established and widely adopted
- Supports JSON and YAML formats
- Rich ecosystem of tools and integrations

Cons:

- Can be verbose for large APIs
- Complex for beginners



BITS WILP | | Generated: 12-May-2026 16:47:54

RAML (RESTful API Modeling Language)

RAML is a YAML-based language for describing RESTful APIs

Pros:

- Clear and concise syntax
- Built-in support for libraries and data types
- Easy to read and write

Cons:

- Smaller ecosystem compared to OpenAPI
- Limited tooling support



BITS WILP | | Generated: 12-May-2026 16:47:54

API Blueprint

API Blueprint is a Markdown-based language for describing APIs

Pros:

- Simple and intuitive syntax
- Easy to write and maintain
- Supports interactive documentation generation

Cons:

- Less formal than OpenAPI or RAML
- Limited tooling support



BITS WILP | | Generated: 12-May-2026 16:47:54

GraphQL SDL (Schema Definition Language)

GraphQL SDL is a declarative language for describing GraphQL APIs

Pros:

- Strongly-typed schema definition
- Intuitive and expressive
- Built-in support for queries, mutations, and subscriptions

Cons:

- Specific to GraphQL APIs
- Limited to describing GraphQL schemas



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- OpenAPI:
 - Broad adoption, verbose syntax
- RAML:
 - Concise syntax, limited tooling
- API Blueprint:
 - Simple syntax, lacks formality
- GraphQL SDL:
 - Specific to GraphQL, strong typing



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Which of the following API description formats is known for its widespread adoption and rich ecosystem of tools and integrations?

- A) RAML
- B) API Blueprint
- C) GraphQL SDL
- D) OpenAPI (formerly Swagger)

Correct Answer: D) OpenAPI (formerly Swagger)



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Which API description format is characterized by its clear and concise YAML-based syntax and built-in support for libraries and data types?

- A) RAML
- B) GraphQL SDL
- C) API Blueprint
- D) OpenAPI (formerly Swagger)



Correct Answer: A) RAML



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ3

Which API description format utilizes a Markdown-based language and is favored for its simplicity and ease of writing and maintenance?

- A) GraphQL SDL
- B) RAML
- C) OpenAPI (formerly Swagger)
- D) API Blueprint

Correct Answer: D) API Blueprint



BITS WILP | | Generated: 12-May-2026 16:47:54

API-Based Products

API Testing

Suresh Gopal



BITS WILP | Generated: 12-May-2026 16:47:54

What is API Testing?

Process of confirming that an API is working as expected

Developers can

- Run API tests **manually**,
- **Automate** them with an API testing tool

Several types of API tests

- Each one plays a distinct role in ensuring the API remains reliable



Image source: <https://www.postman.com/api-platform/api-testing/> Generated: 12-May-2026 16:47:54

Learning Outcomes

- Learn how API testing can help teams confirm that their API's endpoints, methods, and integrations are functioning as expected



Work Integrated Learning Programmes

“Shifting Left”

Traditionally, API testing has occurred at the **end of the development phase**

- But an increasing number of teams are **running tests earlier in the API lifecycle**

This approach to API testing,

- Known as **“shifting left,”**
- Supports rapid iteration by enabling teams to catch and fix issues as soon as they are introduced

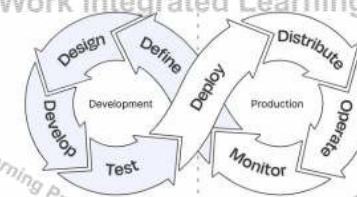


Image source: <https://www.postman.com/api-platform/api-testing/>

Why is API testing important in an API-first world?

Competitive software landscape demands reliability to retain users' trust

- API layer issues can cause user-facing errors, latency, impacting trust, leading to churn
- Pressure on development teams to deliver consistently available, highly performant APIs
- **API-first development model treats APIs as crucial infrastructure components**

API testing ensures continuous verification of quality, health, and performance

- High-quality APIs are essential for delivering a seamless digital experience

API testing and monitoring

Both API testing and monitoring aim to ensure API reliability and performance

API testing is conducted during development to catch issues before production

API monitoring occurs after deployment, focusing on production environment

- Involves gathering and visualizing telemetry data
- Helps in historical analysis and identifying long-term performance trends

Benefits of API testing

Quality assurance:

API testing ensures the API performs as expected, preserving consumer trust and protecting the business's reputation

Early issue detection:

Shift-left approach identifies defects as soon as they are introduced, making development more predictable and helping teams stay on schedule

Resource conservation:

Automation of API testing saves time, allowing team members to focus on innovation

Rapid iteration:

API tests within CI/CD pipelines validate every code change before production, supporting frequent releases and reducing the risk of bugs and regressions

Reference

Postman API testing

<https://www.postman.com/api-platform/api-testing/>

API testing



Summary

- What is API testing
- How it enables “Shifting left” in API-first approach
- Difference between API testing and monitoring
- Benefits of API testing



Work Integrated Learning Programmes



MCQ1

Which of the following statements is true regarding the importance of API testing in a competitive software landscape?

- A) API layer issues do not impact user trust or cause churn.
- B) API testing is unnecessary in an API-first development model.
- C) High-quality APIs are not essential for delivering a seamless digital experience.
- D) Pressure on development teams to deliver consistently available, highly performant APIs underscores the importance of API testing.

Correct answer: D) Pressure on development teams to deliver consistently available, highly performant APIs underscores the importance of API testing.



Work Integrated Learning Programmes



MCQ2

Which statement accurately describes the relationship between API testing and monitoring?

- A) API testing and monitoring are unrelated processes with different goals.
- B) API testing is conducted after deployment, while monitoring occurs during development.
- C) API monitoring focuses solely on catching issues before production.
- D) API testing occurs during development to catch issues before production, while monitoring occurs after deployment to focus on the production environment.

Correct answer: D) API testing occurs during development to catch issues before production, while monitoring occurs after deployment to focus on the production environment.



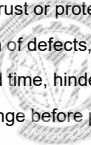
Work Integrated Learning Programmes

MCQ3

Which of the following statements accurately describes the benefits of API testing?

- A) API testing does not play a role in preserving consumer trust or protecting the business's reputation.
- B) The shift-left approach in API testing delays the detection of defects, making development less predictable.
- C) Automation of API testing consumes more resources and time, hindering innovation.
- D) API tests within CI/CD pipelines validate every code change before production, supporting frequent releases and reducing the risk of bugs and regressions.

Correct answer: D) API tests within CI/CD pipelines validate every code change before production, supporting frequent releases and reducing the risk of bugs and regressions.



Work Integrated Learning Programmes



Work Integrated Learning Programmes

API-Based Products

API Testing - Importance and Challenges

Suresh Gopal

Work Integrated Learning Programmes



BITS WILP | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Understand the role and importance of API testing in API life-cycle
- Familiarize with challenges involved in API testing



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

API Testing

API testing focuses on the programmatic interface, testing business logic and communication mechanisms rather than the application's look and feel

Developers build applications reliant on APIs, hence rigorous testing is crucial

Special-purpose software sends requests to the API and reads responses for testing



API Testing

Image source: <https://testgrid.io/blog/api-automation-testing-tools/>

Importance of API testing

Importance of API testing lies in validating adherence to defined standards and contracts

API responses vary based on input data and parameter combinations

- API traffic patterns fluctuate with application usage, necessitating robust handling under different loads

Security testing is a critical aspect of API testing to ensure protection against vulnerabilities

BITS WILP | Generated: 12-May-2026 16:47:54

API Testing Challenges

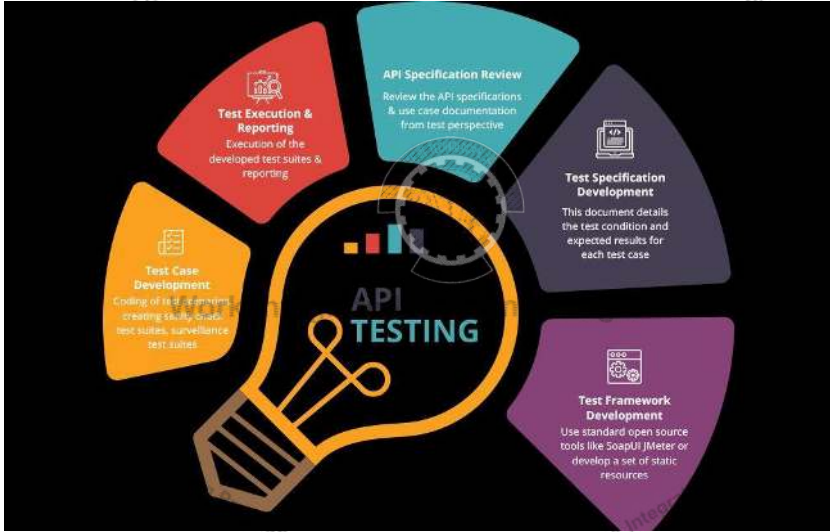


Image source: https://duws858oznvmq.cloudfront.net/hh_5550c951d8.jpg

API Testing Challenges

API testing resembles **white-box testing**, involving **special software** to send messages, receive outputs, and analyze responses

Automating test cases is a significant challenge due to the need to execute multiple APIs in specific sequences, with dynamic parameter values

APIs may be exposed through **management platforms** with **policies** for security, traffic throttling, data transformation, etc., requiring consideration in testing

API Testing Challenges

Challenges in **managing simultaneous connections to backend services and ensuring proper error handling when thresholds are reached**

Test data management is challenging due to the need to handle diverse datasets across multiple test environments

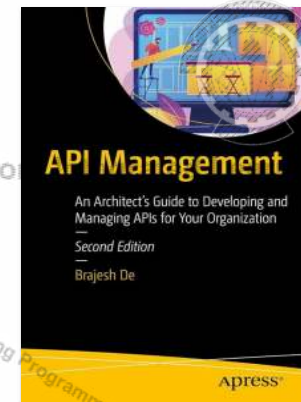
Incorrect test data management can lead to erroneous results, such as using API keys or access tokens from one environment in another

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



Summary

- API testing which centers on verifying programmatic interfaces and communication mechanisms, prioritizing business logic over the application's appearance
- Challenges in API Testing



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ2

Which aspect of API testing resembles white-box testing?

- a) Sending messages to API endpoints
- b) Analyzing API responses
- c) Testing API traffic management
- d) Handling diverse datasets in multiple environments



Work Integrated Learning Programmes

Correct Answer: b) Analyzing API responses

BITS WILP | Generated: 12-May-2026 16:47:54



MCQ1

Which aspect is NOT a focus of API testing?

- a) Testing business logic
- b) Validating the application's look and feel
- c) Validating adherence to defined standards and contracts
- d) Testing communication mechanisms



Correct answer: b) Validating the application's look and feel

BITS WILP | Generated: 12-May-2026 16:47:54



MCQ3

What is a significant challenge in automating test cases for APIs?

- a) Testing API traffic management
- b) Handling diverse datasets across environments
- c) Executing multiple APIs in specific sequences with dynamic parameters
- d) Analyzing API responses



Correct Answer: c) Executing multiple APIs in specific sequences with dynamic parameters

BITS WILP | Generated: 12-May-2026 16:47:54

API-Based Products

API Testing Considerations | Suresh Gopal

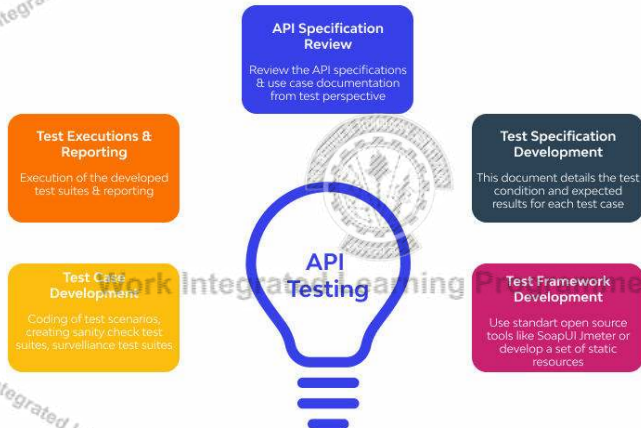


Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

API Testing



Source: <https://www.wallarm.com/what/what-is-api-testing-benefits-types-how-to-start>

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Recognize of the critical aspects of API testing, including
 - Interface specifications,
 - Documentation,
 - and security



Work Integrated Learning Programmes

API Testing Considerations

APIs serve as a bridge for accessing back-end business data and assets

- Generally, the API doesn't encompass the core business logic
- **Mainly facilitates communication**

API testing priorities include:

- Verifying adherence to API interface specifications
- Assessing the comprehensiveness and accuracy of API documentation
- Ensuring robust API security measures are in place

BITS WILP | | Generated: 12-May-2026 16:47:54

API Interface Specification Testing

API Interface defines

- Communication methods with the API, including input parameters and expected responses

Input parameters can be

- Query parameters,
- Form parameters,
- JSON/XML payloads,
- or HTTP headers

Testing focuses on validating API response against documented specifications

BITS WILP | | Generated: 12-May-2026 16:47:54

API Interface Specification Testing

Testing Approach Includes:

Default API behaviour without query parameters

Behaviour when correct query parameter and value are passed

Behaviour with incorrect parameter names or missing values

Handling incorrect parameter values and combinations

BITS WILP | | Generated: 12-May-2026 16:47:54

API Interface Specification Testing

Assessment of:

Default data format for API response

Data format for success and error responses

HTTP response status codes for various conditions

Handling of unexpected HTTP methods, headers, and URLs

BITS WILP | | Generated: 12-May-2026 16:47:54

API Documentation Testing

Need to ensure correctness and currency of API interface documentation

- Vital for effective adoption and satisfaction among developer community

Documentation should be updated with each new API version release

- Testing required to verify documentation reflects latest API updates

Documentation should provide comprehensive information for API interaction

- Interactive documentation requires testing to validate proper responses for each API operation

BITS WILP | | Generated: 12-May-2026 16:47:54

API Security Testing

APIs are access points for business services and data, making them vulnerable to attacks

- Security breaches can damage an enterprise's brand value

Aspects of API security testing

- Authentication and Authorization
- API Fuzzing
- Malformed Payload Injection
- Malicious Content Injection

API Security Testing

Authentication and Authorization:

- Testing access mechanisms and control policies
- Validating API access protected via API Key, Mutual Authentication, or OAuth tokens
- Ensuring correct HTTP error codes for authentication or authorization failures

API Fuzzing:

- Attackers send random input parameters to gain insight into system resources
- APIs should be tested with various input permutations to prevent unauthorized access
- Error responses should only reveal necessary information and not internal system details

API Security Testing

Malformed Payload Injection:

- Protecting APIs against large or complex payloads to prevent system overload
- Testing API's ability to handle unexpected payload sizes or structures

Malicious Content Injection:

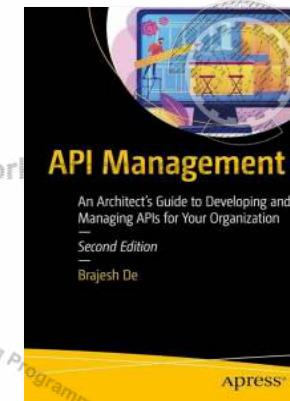
- Injecting SQL, JavaScript, or shell scripts through input parameters
- Testing API behavior when such scripts are injected and ensuring appropriate rejection with error messages
- Testing for various types of injected scripts like SQL, JavaScript, shell script, regular expressions, etc.

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



Summary

- API Interface Specification Testing
- API Documentation Testing
- API Security Testing



Work Integrated Learning Programmes



MCQ1

Which aspect of API Interface Specification Testing focuses on verifying the default behavior of the API when no query parameters are passed?

- A) Validating the HTTP response status codes
- B) Testing the API response format for both success and error conditions
- C) Assessing the behavior when correct query parameters and values are passed
- D) Examining the API response when incorrect parameter names or missing values are provided

Correct Answer: A) Validating the HTTP response status codes



Work Integrated Learning Programmes

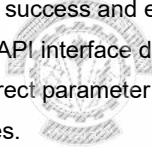


MCQ2

Which aspect is crucial for API Documentation Testing?

- A) Verifying the API response format for both success and error conditions.
- B) Ensuring the correctness and currency of API interface documentation.
- C) Assessing the behavior of API when incorrect parameter values are provided.
- D) Validating the HTTP response status codes.

Correct Answer: B) Ensuring the correctness and currency of API interface documentation



Work Integrated Learning Programmes



MCQ3

Which aspect of API Security Testing focuses on verifying access mechanisms and control policies?

- A) API Fuzzing
- B) Malformed Payload Injection
- C) Authentication and Authorization
- D) Malicious Content Injection

Correct Answer: C) Authentication and Authorization



Work Integrated Learning Programmes



API-Based Products

API Test Automations | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

What is API Test Automation?

API test automation involves using testing tools to execute API tests programmatically.

Allows for **scheduled or frequent testing**,

Often integrated into **CI/CD pipelines**

Designed to **complement manual testing** efforts rather than replacing them entirely

Learning Outcomes

- Learn how API test automation enables teams to continuously validate their work and catch issues before they reach production



Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

What is API Test Automation?

Crucial for agile teams, as it

Ensures **fast-paced development** while systematically verifying API functionality

Helps in identifying and preventing **breaking changes** from reaching production

Enables teams to deliver new features **rapidly and confidently** while optimizing developer resources

API Test Automation and API-First

API test automation is crucial in an API-first development model for **verifying API quality and ensuring it functions as expected**

In an agile development environment, where **small code changes are shipped frequently**, the risk of breaking changes increases

- API test automation **enables teams to iterate quickly without compromising API quality or test coverage**

API Test Automation and API-First

Automated API tests can be

- **Scheduled or integrated into CI/CD pipelines,**
- Allowing for seamless execution whenever new code is pushed

This hands-free approach ensures

- **Issues are detected early**
- **and don't fall through the cracks in the development process**

API test automation plays a **vital role in maintaining the balance between agility and quality in API-first development**

API Test Automation Facilitates “Shift Left”

Historically, software testing has been relegated to the end of development cycles, leading to delays and human errors

Shift-left testing involves **moving testing activities earlier in the development process to mitigate these issues**

API test automation facilitates the shift-left approach by **allowing tests to be run earlier and more frequently in the development process**

Benefits

Early issue detection:

- API test automation allows for the detection of issues as soon as they are introduced, offering immediate feedback to developers
- For instance, CI/CD pipelines can be configured to execute API tests after every code push, catching issues while code is still in progress

Time and resource savings:

- Automated API testing distributes the testing load across the API lifecycle, shortening feedback loops, improving efficiency, and enabling quicker feature releases

Best Practices

Avoid automating flaky tests

- To save time and increase productivity
- Focus on automating simple, straightforward tests and implement retry logic to confirm failures

Reduce the risk of human error

- By standardizing test executions through API test automation, improving confidence and morale within the team

Run tests in parallel

- To ensure fast and efficient execution, especially within CI/CD pipelines, enabling developers to receive feedback quickly without halting their work

Choose a testing tool that seamlessly integrates with the existing workflow

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

Postman API test automation

<https://www.postman.com/api-platform/api-test-automation/>

API test automation



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- What is API test automations and its benefits
- Its role in API-first approach
- Best practices followed in automations



Work Integrated Learning Programmes

MCQ1

What is API test automation?

- a) Manual execution of API tests
- b) Using testing tools to programmatically execute API tests
- c) Running API tests only during development
- d) Ignoring API testing altogether

answer: b



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

What role does API test automation play in an API-first world?

- a) It slows down the development process by introducing additional testing steps.
- b) It ensures API quality by reducing the frequency of code changes.
- c) It helps teams remain agile while maintaining API quality and test coverage.
- d) It replaces manual testing entirely, eliminating the need for human intervention.

Correct answer: c) It helps teams remain agile while maintaining API quality and test coverage.

MCQ3

How does API test automation help teams shift left?

- a) By postponing testing activities until the end of development cycles.
- b) By introducing human errors into the testing process.
- c) By allowing tests to be run earlier and more frequently in the development process.
- d) By increasing delays in the release of new features.

Correct answer: c) By allowing tests to be run earlier and more frequently in the development process.

API-Based Products

Types of API Testing | Suresh Gopal

Learning Outcomes

- Learn about the different types of API testing

API Testing

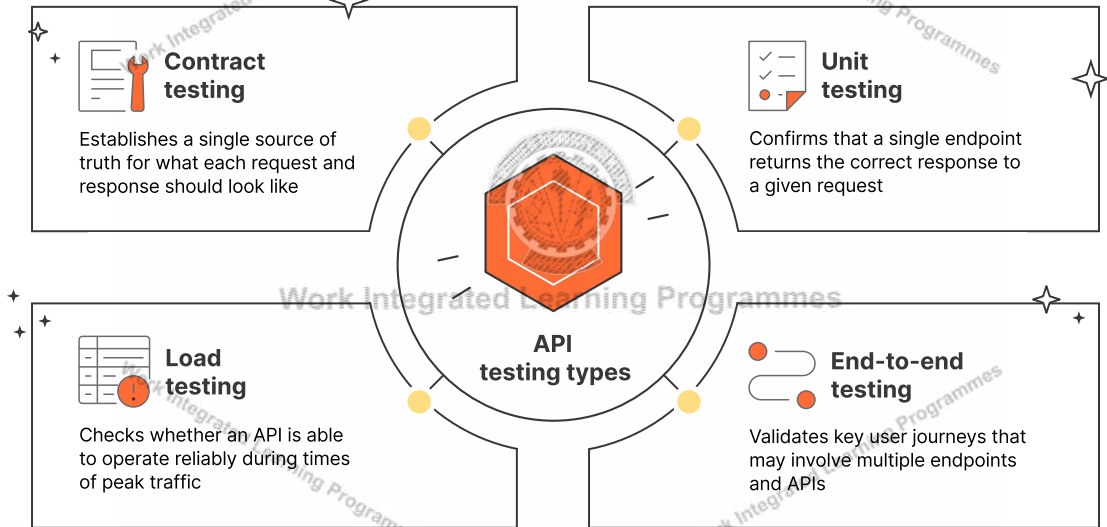
Various methods exist to test APIs, each with distinct purposes

- API testing strategies can be tailored to suit specific project requirements
- Flexibility allows teams to adapt testing methodologies to their unique needs

Four primary approaches are commonly employed,

- With room for customization within each
- **Contract testing**
- **Unit testing**
- **End-to-End testing**
- **Load testing**

Different Types of API Testing



Source: <https://www.postman.com/api-platform/api-testing/>

Contract Testing

An API contract is a human- and machine-readable representation of an API's intended functionality

- Establishes a single source of truth for what each request and response should look like
- Forms the basis of service-level agreements (SLAs) between producers and consumers

API contract testing helps ensure that new releases don't violate the contract by checking the content and format of requests and responses

Unit Testing

API unit testing is the process of confirming that a single endpoint returns the correct response to a given request

Unit tests may validate that an endpoint

- Handles optional parameters correctly,
- or returns the appropriate error message when sent an invalid request

End-To-End Testing

Unit tests help developers ensure that individual endpoints are working as expected

End-to-end tests are used to validate key user journeys that may involve multiple endpoints and APIs

End-to-end API testing involves

- Chaining requests together
- Confirming that each one is working properly
- Which helps teams surface issues in complex workflows before users do

Load Testing

API load testing enables developers to confirm whether their API is able to operate reliably during times of peak traffic

Typically involves using a testing tool to

- Simulate large request volumes
- Measure the resulting response times and error rates

Often performed in anticipation of a significant load increase

- Such as right before a product launch or yearly sale

Reference

Postman API test automation

<https://www.postman.com/api-platform/api-test-automation/>



API test automation

Summary

- Four primary approaches which are commonly employed for API testing
 - Contract testing
 - Unit testing
 - End-to-End testing
 - Load testing

MCQ1

Which of the following best describes an API contract?

- A) A legal agreement between API consumers and producers.
- B) A document outlining the terms of API usage.
- C) A human- and machine-readable representation of an API's intended functionality.
- D) A detailed guide on how to implement an API.

Correct Answer: C) A human- and machine-readable representation of an API's intended functionality.

MCQ2

Which of the following best describes API unit testing?

- A) Testing the integration between different APIs.
- B) Verifying that multiple endpoints work together seamlessly.
- C) Confirming that a single endpoint returns the correct response to a given request.
- D) Ensuring that APIs comply with industry standards.

Correct Answer: C) Confirming that a single endpoint returns the correct response to a given request.

MCQ3

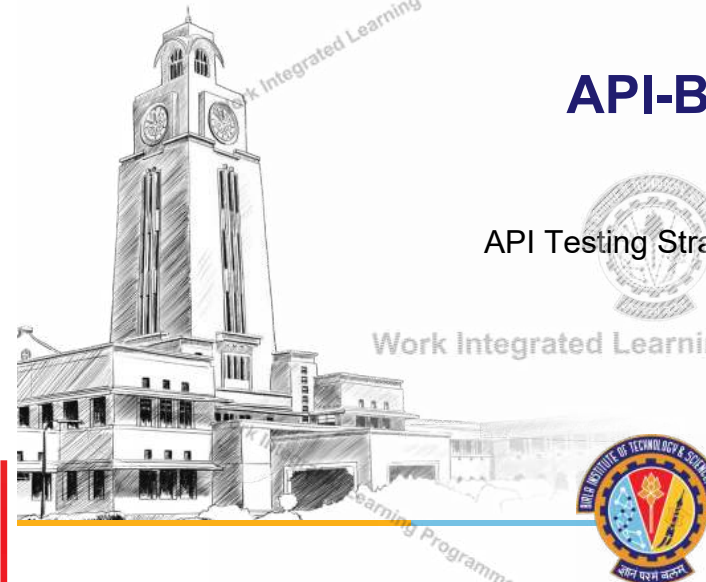
Which of the following best describes API load testing?

- A) Testing the functionality of individual API endpoints.
- B) Ensuring that APIs return the correct response to given requests.
- C) Simulating large request volumes to measure response times and error rates.
- D) Validating key user journeys involving multiple endpoints and APIs.

Correct Answer: C) Simulating large request volumes to measure response times and error rates.

API-Based Products

API Testing Strategies | Suresh Gopal



Learning Outcomes

- Grasp the importance of implementing various API testing strategies to ensure the reliability, security, and functionality of APIs



Work Integrated Learning Programmes

True!

Software defect removal is the most expensive and time-consuming form of work for software.

- Caspers Jones



Work Integrated Learning Programmes

Need for API Testing Strategy

Establishing an API testing strategy is crucial for building an API product or platform

- Choosing the appropriate approach for API testing enhances the supportability of the API program

A well-defined API testing strategy

- Facilitates faster delivery
- Helps mitigate the high cost of defect removal in software development

Automated testing provides insights into the developer experience of an API

Emphasizing its consumer-oriented nature



Acceptance Testing

Acceptance testing, or solution-oriented testing, validates whether an API supports captured job stories

- Ensures that the API solves real problems for customers and produces desired outcomes
- Verifies the collaboration of API operations required for achieving a desired outcome

Tests are composed using only the API interface to assess expected end-to-end functionality

- Writing acceptance tests helps identify poor developer experience and integration issues.

Changes in the API internals during development should not impact the results of acceptance tests



Automated Security Testing

Automated Security Testing is crucial amid **increasing incidents of data breaches**

A continuous process focused on protecting APIs against attacks and data leaks

Key questions include assessing API **vulnerability to attacks and potential data leakage**

Automated Security Testing

Security testing involves

- Design-time review,
- Static and dynamic code analysis
- Runtime monitoring

Design-time and development-time testing include policies and tools to prevent data leaks

- **Authorization policies are crucial to ensuring proper access to API operations**

An API management layer may facilitate runtime monitoring and enforcement of security measures

- **Log analysis helps detect and block malicious attacks.**

Operational Monitoring

APIs serve as the **primary interface for application interactions**, making their availability critical

- **Meeting service-level agreements (SLAs)** is essential to maintain customer satisfaction and avoid financial penalties

Operational monitoring ensures APIs are available and perform as expected while staying within SLA boundaries

- **Addresses the need for infrastructure provisioning to meet performance goals**

Operational Monitoring

Monitoring and analytics solutions play a vital role in API operational monitoring

Analytics verify

- Real-world usage,
- Correctness,
- Performance,
- Utilizing various measurement techniques

Measurement techniques range

- From simple performance logging
- To complex third-party integrations for monitoring and visualization support

User Interface Testing versus API Testing

User Interface Testing and API Testing fulfill distinct roles, with UI tests alone not adequately covering API functionality

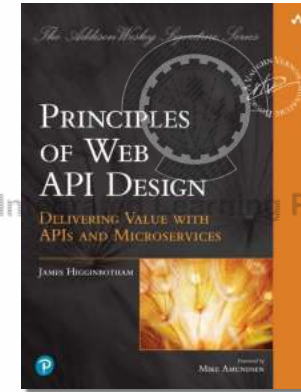
- UI tests exercise the API only as far as the UI interacts with it, potentially overlooking scenarios like client-side validation

API Testing aims to validate the API's ability to handle diverse input scenarios, including erroneous data, independently of UI implementations

- Ensures comprehensive testing before deployment, integrating into the continuous integration and delivery pipeline alongside other automated tests

Reference

Principals of Web API Design



- By James Higginbotham

Summary

- The importance of employing various testing strategies, including
 - Acceptance testing
 - Automated Security Testing
 - Operational monitoring
 - User Interface Testing

MCQ1

Which of the following statements best summarizes the importance of having an API testing strategy?

- A) API testing strategies primarily focus on reducing development costs.
- B) API testing strategies ensure faster delivery but have limited impact on defect removal.
- C) API testing strategies contribute to the success of an API program's supportability and faster delivery while minimizing defect removal costs.
- D) API testing strategies offer insights into the consumer experience but have little impact on supportability or delivery speed.

Correct Answer: C) API testing strategies contribute to the success of an API program's supportability and faster delivery while minimizing defect removal costs.

MCQ2

What is the primary focus of acceptance testing for an API?

- A) Verifying the correctness of individual API operations.
- B) Ensuring the API complies with industry standards.
- C) Validating whether the API meets the needs of customers and produces desired outcomes.
- D) Testing the performance and scalability of the API.

Correct Answer: C) Validating whether the API meets the needs of customers and produces desired outcomes.

MCQ3

What is the primary objective of Automated Security Testing for APIs?

- A) Ensuring the API complies with industry standards.
- B) Detecting and preventing potential data breaches and attacks.
- C) Improving the performance and scalability of the API.
- D) Enhancing the user experience and usability of the API.

Correct Answer: B) Detecting and preventing potential data breaches and attacks.

API-Based Products

API Performance Testing | Suresh Gopal

Learning Outcomes

- Understand the objectives and significance of different type of testing in evaluating API performance under different conditions and loads

Performance of API

APIs are now integral for delivering data and services directly to end users via digital platforms

- With this shift, the performance of APIs under heavy loads becomes crucial, as it directly impacts the overall performance of client applications

Load testing for APIs emerges as a critical aspect to ensure optimal performance and reliability

- Involves comprehensive testing methodologies to assess their performance under various loads and stress conditions

BITS WILP | | Generated: 12-May-2026 16:47:54

Preparing For The Load Test

A well-defined load testing strategy is crucial

- Starting with identifying the APIs to be tested and replicating real-world traffic conditions

Planning involves gathering information on

- Performance requirements such as average and peak throughput for each API
- Throughput distribution,
- Traffic distribution patterns of client apps,
- and expected concurrent users

Test execution may involve

- Generating repetitive loads for API endpoints to establish performance upper bounds,
- Followed by tuning and optimization if necessary

BITS WILP | | Generated: 12-May-2026 16:47:54

Preparing For The Load Test

Simulating realistic traffic patterns involves considering

- Traffic distribution across APIs,
- Traffic growth patterns,
- Handling of success and failure responses,
- and geographically distributed traffic to test for network congestion

Data from production traffic logs can be used to simulate realistic traffic scenarios for load testing,

- Providing valuable insights into API performance under various conditions

BITS WILP | | Generated: 12-May-2026 16:47:54

API Performance Testing

API performance testing should consist of the following,

- With an aim to find the different performance parameters of the APIs and the API platform

Baseline testing

Load testing

Stress testing

Soak testing

BITS WILP | | Generated: 12-May-2026 16:47:54

Baseline Testing

Baseline testing aims to assess system performance under typical expected loads

Results are analyzed to determine average and peak API response times and error rates

CPU and memory utilization are scrutinized to identify and resolve resource bottlenecks

Load Testing

Load testing involves incrementally increasing API traffic volumes to evaluate performance

- Performance metrics like response time and throughput are monitored to assess API performance

The goal is to understand the system's behavior and capacity under expected peak loads, not to identify breaking points

- Server performance metrics such as CPU and heap memory utilization, as well as network port utilization, are analyzed to gauge the platform's capability under high load.

Stress and Soak Testing

Stress testing aims to identify the breaking point of the platform by gradually increasing API traffic load

- Determines the maximum throughput that the system can handle
- The test continues until performance degradation or an increase in errors from API calls is observed

Soak testing assesses system stability over prolonged durations to uncover potential instabilities

- Baseline tests may extend over days or weeks to detect any adverse behaviors during prolonged usage
- The objective is to identify issues with releasing system resources for subsequent execution cycles

API Performance Test Metrics

Look at the following metrics to measure the performance of the individual API and the platform

API response time

API target response time

Throughput

Success and error rate

CPU utilization

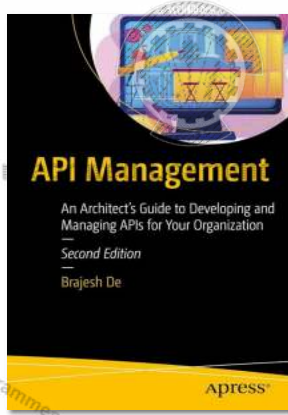
Heap memory utilization

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: What are the key considerations for preparing for API load testing?

- A) Identifying the list of APIs to be tested
- B) Planning for real-world traffic replication
- C) Gathering information on performance requirements
- D) All of the above

Correct Answer: D) All of the above

Summary

- The preparations needs to done for performing API performance testing
- Various methodologies such as
 - Baseline testing,
 - Load testing,
 - Stress testing,
 - and soak testing
- API performance metrics to be observed

MCQ2

Question: Which metrics are essential for measuring API performance during performance testing?

- A) API response time, API target response time, and API throughput
- B) API latency, success and error rates, and CPU utilization
- C) API response time, API latency, and throughput
- D) API target response time, success and error rates, and heap memory utilization

Correct Answer: A) API response time, API target response time, and API throughput

MCQ3

Question: What is the primary objective of load testing for APIs?

- A) To identify the breaking point of the system
- B) To analyze performance metrics like response time and throughput
- C) To determine the minimum expected load for the system
- D) To assess server performance metrics such as CPU utilization and heap memory utilization

Correct Answer: B) To analyze performance metrics like response time and throughput

API-Based Products

API Testing Tools

Suresh Gopal

Learning Outcomes

- Gain a comprehensive understanding of the essential features that API testing tools should possess
- Know about some API testing tools

Key Features for API Testing Tools

Comprehensive Test Coverage:

Ability to test various aspects of APIs including functionality, security, performance, and scalability

Ease of Use:

Intuitive interface and user-friendly design for efficient test creation and execution

Automation Support:

Robust automation capabilities to streamline testing processes and facilitate continuous integration and delivery (CI/CD) pipelines

Key Features for API Testing Tools

Flexibility:

Support for testing both RESTful and SOAP APIs, as well as other common protocols and data formats

Customization Options:

Ability to customize tests and adapt to specific project requirements and environments

Reporting and Analysis:

Detailed reporting features to track test results, identify issues, and analyze performance metrics

Collaboration Tools:

Collaboration features for teams to work together on test creation, execution, and analysis

Nice-to-Have Features

Mocking Capabilities:

Ability to create mock APIs for testing purposes, allowing for isolated and controlled testing environments

Performance Testing:

Built-in tools for performance testing to evaluate API response times, throughput, and scalability

Security Testing:

Integration with security testing tools to assess API security vulnerabilities and ensure data protection

Nice-to-Have Features

Scalability:

Ability to handle large-scale testing scenarios with support for distributed testing and resource optimization

Documentation Generation:

Automatic generation of API documentation to maintain consistency and improve developer experience

Integration Support:

Compatibility with popular development and testing frameworks, as well as third-party tools for seamless integration into existing workflows

Tools Suggestion

Postman:

A popular API testing tool known for its user-friendly interface and robust features

Offers comprehensive test coverage, automation support, and collaboration tools.

Supports both RESTful and SOAP APIs and provides extensive customization options



POSTMAN

Tools Suggestion

Swagger (now known as OpenAPI):

- Not only a specification for defining APIs but also offers tools for testing
- Provides features for API documentation generation, allowing for better understanding and testing of APIs
- Can be integrated with various testing frameworks and CI/CD pipelines



Tools Suggestion

SoapUI:

- A powerful API testing tool specifically designed for testing SOAP and RESTful APIs
- Offers a wide range of features including comprehensive test coverage, automation support, and performance testing capabilities
- Provides advanced features such as mocking capabilities for creating mock APIs and security testing tools



Tools Suggestion

Katalon Studio:

- An all-in-one test automation solution that includes API testing capabilities
- Offers a user-friendly interface with features for test creation, execution, and reporting
- Supports both RESTful and SOAP APIs and integrates seamlessly with other testing frameworks



Comparison of Tools

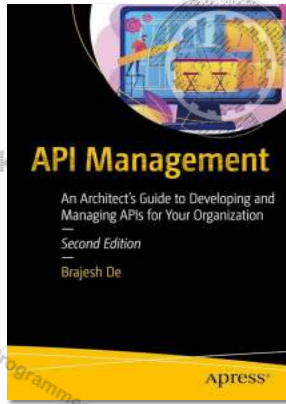
Feature	Postman	SoapUI	Swagger (OpenAPI)	Katalon Studio
Test Coverage	Comprehensive	Comprehensive	Limited	Comprehensive
Ease of Use	Very user-friendly	User-friendly	Moderate	User-friendly
Automation	Yes	Yes	No	Yes
Protocol Support	RESTful and SOAP APIs	RESTful and SOAP APIs	RESTful APIs	RESTful and SOAP APIs
Customization	Extensive customization options	Extensive customization options	Limited customization options	Extensive customization options
Reporting	Yes	Yes	Limited	Yes
Collaboration	Yes	Limited	Limited	Yes
Mocking	No	Yes (SoapUI Pro)	No	No
Performance	No	Yes (SoapU Pro)	No	No
Security	No	Yes (SoapU Pro)	No	No
Scalability	No	Yes (SoapUI Pro)	No	No
Integration	Limited	Limited	Limited	Yes

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by Brajesh De (Author)



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Key Features for API Testing Tools
- Nice-to-Have Features
- API testing Tools suggestion



Work Integrated Learning Programmes

MCQ1

Which of the following is NOT a key feature of API testing tools?

- a) Comprehensive test coverage
- b) Scalability testing only
- c) Ease of use
- d) Automation support

Answer: b) Scalability testing only



Work Integrated Learning Programmes

MCQ2

What is the primary purpose of automation support in API testing tools?

- a) To increase manual testing efforts
- b) To streamline testing processes and integrate with CI/CD pipelines
- c) To replace human testers entirely
- d) To create complex test scenarios only

Answer: b) To streamline testing processes and integrate with CI/CD pipelines



Work Integrated Learning Programmes

MCQ3

What does integration with security testing tools enable in API testing?

- a) Creation of mock APIs
- b) Performance evaluation
- c) Assessment of API security vulnerabilities
- d) Scalability testing

Answer: c) Assessment of API security vulnerabilities

API-based Products

API Gateways - What, where and How?

Suresh Gopal

Learning Outcomes

- Understand the role of an API Gateway as a single entry point in an API-based system, including its core components, key functionalities, and deployment strategies.

What is an API Gateway?

- An API Gateway is a critical management tool positioned at the edge of a system
 - Serves as a single entry point for a set of APIs,
 - Acting as a mediator between consumers such as
 - web or mobile apps,
 - internal systems,
 - or third-party applications
 - and backend services.

What is an API Gateway?

- Consists of two primary components:
 - Control Plane:
 - Manages routes, policies, and telemetry
 - Data Plane:
 - Executes tasks defined in the control plane,
 - Handles network traffic,
 - Enforces policies,
 - and emits telemetry data.

Key Functionalities of an API Gateway

- Acts as a **reverse proxy** to manage all incoming API requests from consumers
- **Orchestrates and aggregates** responses from multiple backend services and external services to fulfill the API requests
- Facilitates **API lifecycle management**, developer onboarding, and governance through features like developer portals and access control.

Key Functionalities of an API Gateway

- Provides crucial functionalities such as:
 - User Authentication
 - Request Rate Limiting
 - Timeouts and Retries
 - Observability:
 - Offers metrics, logs, and trace data.

Deployment of API Gateways

- Typically deployed at the system's edge, but the definition of "edge" varies:
 - For startups and SMBs, often at the edge of the data center or cloud.
 - A single API Gateway can serve as the primary entry point for the entire backend infrastructure.

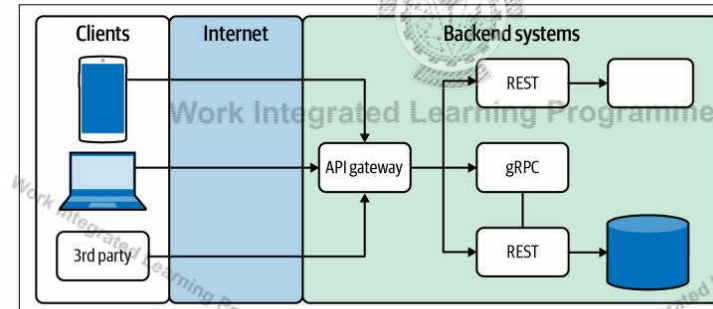


Figure 3-2. A typical startup/SMB API gateway deployment Adapted from "Mastering API Architecture" by Gough, Bryant and Auburn

Deployment of API Gateways

- In large enterprises, API Gateways are often deployed in multiple locations:
 - Initial edge stack at the perimeter of data centers.
 - Additional gateways may be set up for specific products, business lines, or departments, with varying functionalities.

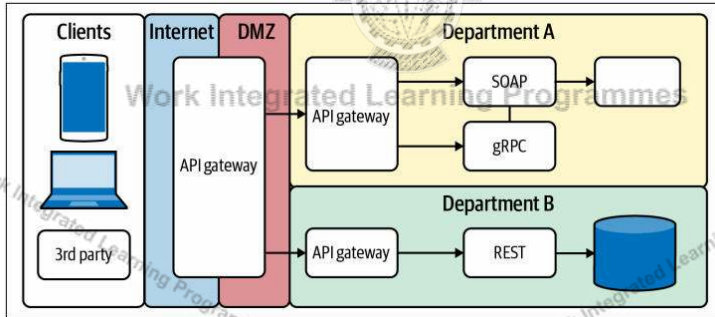
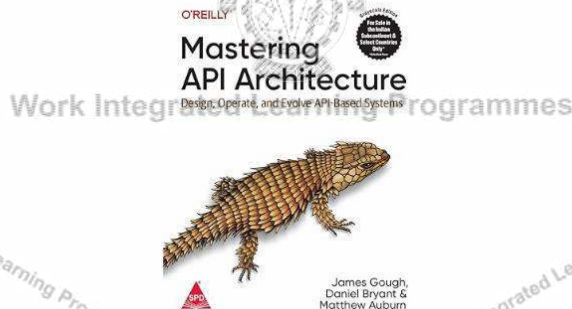


Figure 3-3. A typical large/enterprise API gateway deployment

Adapted from "Mastering API Architecture" by Gough, Bryant and Auburn

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



Summary

- The role and functionality of an API Gateway can differ based on
 - implementation,
 - infrastructure capabilities,
 - and organizational needs.
- It is important to understand the conceptual framework rather than focusing solely on specific implementations.

MCQ1

- What is the primary role of an API Gateway in an API-based system?
 - A) To store data securely
 - B) To act as a single entry point between consumers and backend services
 - C) To replace backend services entirely
 - D) To manage user interfaces

Answer: B) To act as a single entry point between consumers and backend services

MCQ2

- Which two fundamental components make up an API Gateway?

- A) Reverse Proxy and Load Balancer
- B) Data Plane and Control Plane
- C) User Authentication and Rate Limiting
- D) Developer Portal and Observability Tool

Answer: B) Data Plane and Control Plane

MCQ3

- How are API Gateways typically deployed in large organizations?

- A) Only at the cloud edge, without additional components
- B) As a single instance without redundancy
- C) In multiple locations, such as the perimeter of data centers and within individual departments
- D) Directly within user devices to enhance speed

Answer: C) In multiple locations, such as the perimeter of data centers and within individual departments

API-based Products

Why API Gateway? Suresh Gopal

Learning Outcomes

- Understand and articulate the role of an API gateway in modern software architecture, including its benefits

Why Use an API Gateway?

- Modern software architecture requires balancing short-term implementation with long-term maintainability, especially when dealing with APIs and traffic management.
- An API gateway plays a crucial role in addressing many cross-cutting concerns such as
 - maintainability,
 - extensibility,
 - security,
 - observability,
 - product lifecycle management,
 - and monetization

Reduce Coupling

- Loose Coupling and High Cohesion:
 - Software systems designed with loose coupling and high cohesion are easier to understand, maintain, and modify.
 - API gateways promote these principles by acting as a single entry point (facade) or adapter between frontends and backends.
- Simplified Interface:
 - An API gateway can provide a simplified interface to the backend, allowing backend components to change without impacting the frontend
 - Helps maintain system stability and reduces dependencies

Acting as an Adapter/Facade Between Frontends and Backends

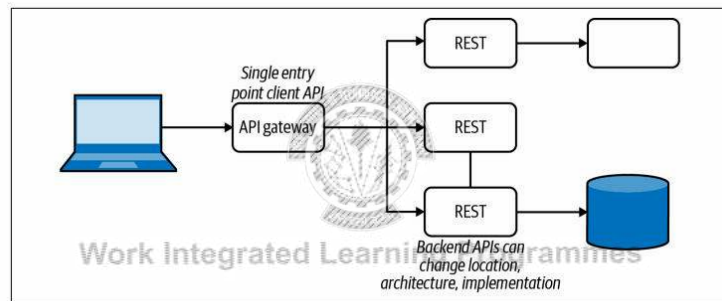


Figure 3-5. An API gateway providing a facade between frontends and backends

Simplify Consumption

- Aggregation and Simplification:
 - API gateways can aggregate multiple backend services into a single API, simplifying the interface for frontend developers.
 - Particularly useful when different backend services have complex or varied APIs
- Protocol Translation:
 - API gateways can translate different protocols (e.g., from SOAP to REST), providing a consistent API interface to consumers

Aggregating and Translating Backend Services

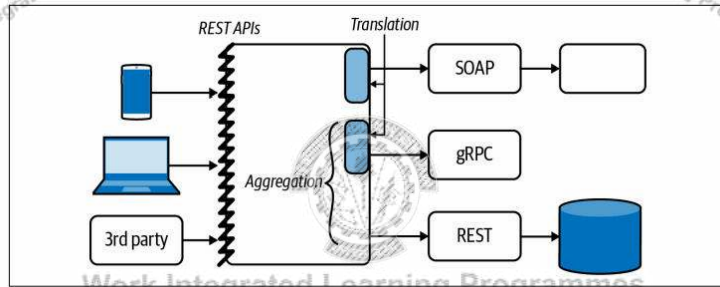


Figure 3-6. An API gateway providing aggregation and translation

- Considerations: Care should be taken to avoid tightly coupling logic within the API gateway with backend service logic, as this could lead to maintenance challenges.

Protect APIs from Overuse and Abuse

- First Line of Defense:
 - An API gateway often acts as the first layer of defense against malicious activities
 - Includes features such as
 - TLS termination, Authentication/authorization,
 - IP allow/deny lists,
 - Rate limiting,
 - Load shedding,
 - and API contract validation
- Observability Strategy:
 - Implementing a comprehensive observability strategy is essential for detecting and mitigating API abuse, whether accidental or deliberate.

Understand How APIs Are Being Consumed

- Observability
 - Performance Metrics:
 - Observability at the API gateway level allows organizations to capture key performance metrics like errors, throughput, and latency, which are critical for meeting business objectives.
 - Correlation Identifiers:
 - By injecting correlation identifiers at the gateway level, organizations can trace requests across various services and systems, enhancing system monitoring and troubleshooting.
 - Dashboards and Alerts:
 - Visual dashboards and alerts are vital for turning observability data into actionable insights.

Manage APIs as Products

- API Lifecycle Management
 - API as a Product:
 - Modern APIs are often managed as products with their own lifecycle, from planning and design to deployment and retirement
 - API gateways support this full lifecycle management (APIM) by providing essential capabilities for API governance and monitoring
 - Strategic Importance:
 - For large organizations, choosing the right API gateway is crucial for supporting their API program strategy.

Monetize APIs

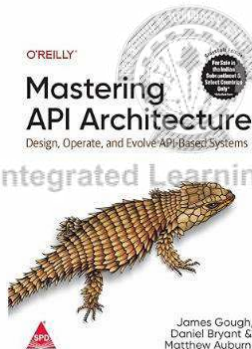
- Account Management, Billing, and Payment
 - Monetization Features:
 - API gateways often include built-in monetization features, such as account management, billing, and integration with payment solutions like PayPal or Stripe
 - Are essential for organizations looking to offer paid APIs to customers
 - Developer Portals:
 - Many API gateways provide developer portals to manage API consumption plans, rate limits, and other options.

Summary

- An API gateway is a powerful tool that addresses numerous architectural challenges, from reducing coupling and simplifying consumption to protecting APIs and managing their lifecycle
- Selecting the right API gateway is a critical decision for modern software architectures, especially when aiming for scalable, secure, and monetizable API solutions.

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



MCQ1

- What is one of the primary roles of an API gateway?
 - a) Reducing coupling between frontends and backends by acting as an adapter or facade.
 - b) Enhancing database performance.
 - c) Providing direct access to the backend databases.
 - d) Replacing the need for a content delivery network (CDN).

Answer: a) Reducing coupling between frontends and backends by acting as an adapter or facade.

MCQ2

- Which feature of an API gateway helps in protecting APIs from overuse and abuse?

- a) Rate limiting and threat detection.
- b) Real-time data analysis.
- c) Direct database access.
- d) Enhanced user interface design.

Answer: a) Rate limiting and threat detection.

MCQ3

- How does an API gateway support API lifecycle management and monetization?
- a) By providing direct server access and increasing bandwidth.
 - b) By managing API versions, supporting billing, and offering developer portals.
 - c) By reducing server load and optimizing query performance.
 - d) By integrating with external content management systems (CMS).
- Answer: b) By managing API versions, supporting billing, and offering developer portals.

API-based Products

History of API Gateways | Suresh Gopal

Learning Outcomes

- **Understand** the historical evolution of API gateways

A Modern History of API Gateways



- With a solid understanding of the "what," "where," and "why" of API gateways, it's insightful to explore their historical evolution
- This reflection on past technologies helps us appreciate current advancements and avoid repeating past mistakes.
- Understanding the historical context of API gateways and traffic management enriches our knowledge and enhances our ability to leverage contemporary technologies effectively.

1990s Onward: The Era of Hardware Load Balancers



The Web's Beginnings

- Origins: Tim Berners-Lee proposed the World Wide Web in the late 1980s, but it gained public awareness during the mid-1990s, leading to the dot-com boom.
- Technological Milestones:
 - Web Browsers: Netscape Navigator (1994)
 - Web Servers: Apache Web Server (1995)
 - Hardware Load Balancers: F5 Networks (1996)

1990s Onward: The Era of Hardware Load Balancers



Functionality and Challenges

- Early Web:
 - Users accessed static HTML pages via HTTP requests.
- Dynamic Content:
 - Implemented using CGI and scripting languages like Perl and C, akin to today's "function as a service (FaaS)."
- Load Balancing:
 - Early hardware load balancers distributed user requests across web servers, providing fault tolerance and improved performance
 - The technology remains in use but has evolved alongside advances in hardware

Early 2000s: The Rise of Software Load Balancers



Evolving Web Needs

- Growth:
 - Increased demand for user-generated content, e-commerce, and system integration.
- Architectural Shifts:
 - Monolithic Applications: Built using Java and .NET.
 - Service-Oriented Architecture (SOA): Introduced Web Service specifications (WS-*).

Early 2000s: The Rise of Software Load Balancers

New Technologies

- Software Load Balancers:
 - HAProxy (2001), NGINX (2002) offered flexibility and cost-efficiency.
- Additional Edge Technologies:
 - Content Delivery Networks (CDNs): Addressed performance bottlenecks.
 - Web Application Firewalls (WAFs): Enhanced security, initially through specialized hardware, later via software (e.g., ModSecurity).

Mid-2000s: Emergence of Application Delivery Controllers (ADCs)

Web 2.0 Era

- Technological Innovations:
 - The launch of the iPhone (2007) and the rise of XMLHttpRequest and Ajax revolutionized web interactions by allowing dynamic content updates without full page reloads.

Mid-2000s: Emergence of Application Delivery Controllers (ADCs)

Advanced Solutions

- Application Delivery Controllers (ADCs):
 - Provided enhanced capabilities like compression, caching, SSL offloading, and traffic shaping.
- Operational Changes:
 - Increased specialization and siloing of edge components led to the rise of DevOps practices to address integration challenges.

Early 2010s: The Birth of First-Generation API Gateways

The API Economy

- Technological Pioneers:
 - Twilio: Simplified telecommunications with API calls.
 - Google Ads API, Stripe: Enabled monetization and payment solutions.
 - Mashape: Founded in 2007, led to the creation of the Kong API Gateway, among others.

Early 2010s: The Birth of First-Generation API Gateways



Features and Focus

- API Management:
 - Focused on managing the software development lifecycle (SDLC) of APIs, with developer portals offering documentation, access controls, and analytics
- Layer 7 Functionality:
 - Enhanced routing based on HTTP metadata, contributing to richer API functionalities.

2015 Onward: The Advent of Second-Generation API Gateways



Microservices and Containerization

- Architectural Shift:
 - The rise of microservices, Docker (2013), and Kubernetes (2015) drove new requirements at the edge.
- Innovative Gateways:
 - Netflix Zuul: Offered dynamic behavior modification and consolidated cross-cutting concerns.
 - Envoy Proxy: Released in 2016, influenced several next-generation gateways, including Ambassador Edge Stack, Contour, and Gloo Edge.

2015 Onward: The Advent of Second-Generation API Gateways



Modern Trends

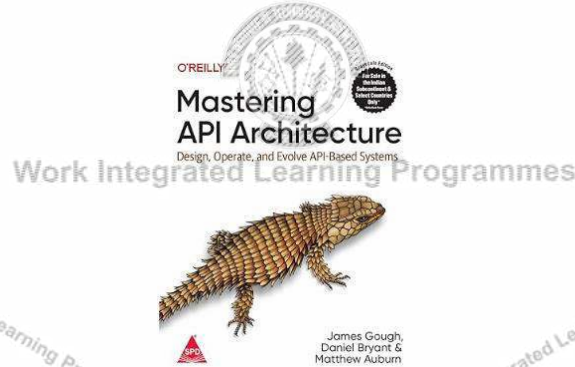
- Developer Focus: Second-generation API gateways emphasized developer self-service and clear separation of concerns.
- Consolidation: Increased focus on integrating both functional and cross-functional requirements within the gateway.

Summary

- The evolution of API gateways mirrors broader technological trends and architectural shifts.
- By understanding the historical context—from hardware load balancers to modern API management solutions—we gain valuable insights into current practices and future directions.
- This perspective enables us to build on solid foundations, appreciate progress, and navigate the complexities of modern technology more effectively.

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

- Which technology was primarily used in the 1990s to distribute user requests across multiple web server instances and provide fault tolerance?
- Software Load Balancers
 - Application Delivery Controllers (ADCs)
 - Hardware Load Balancers
 - Content Delivery Networks (CDNs)

Answer: c) Hardware Load Balancers

MCQ2

- What significant web technology introduced in the mid-2000s allowed web pages to dynamically update content without reloading the entire page?
- HTTP/2
 - AJAX (Asynchronous JavaScript and XML)
 - SOAP (Simple Object Access Protocol)
 - WebSockets

Answer: b) AJAX (Asynchronous JavaScript and XML)

MCQ3

- Which API gateway technology, released in 2013 by Netflix, supported dynamic behavior modification and consolidated various cross-cutting concerns?
- Envoy Proxy
 - Kong API Gateway
 - Zuul
 - Ambassador Edge Stack

Answer: c) Zuul

API-based Products

Other Alternatives | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Routing Traffic to Backend Systems

- **Context:**
 - Many software systems need to route consumer API requests from an external origin to an internal backend application.
 - Traffic can originate from a
 - web browser,
 - mobile app,
 - or external third-party system.
- **Key Requirements:**
 - Reliability
 - Observability
 - Security

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Explore alternatives to API gateways for routing user traffic to backend systems and understand when an API gateway is the most appropriate solution.

Alternatives to API Gateways

- **Question: Is an API Gateway the only solution?**
 - Answer: No, other options exist.
- **Alternatives:**
 - Proxy: Simple solution for routing traffic
 - Load Balancer: Distributes traffic across multiple servers for improved reliability and availability
- **Considerations:**
 - Proxies and load balancers can be suitable for simpler needs.

BITS WILP | | Generated: 12-May-2026 16:47:54

When to Use an API Gateway?

- **Context:**
 - API gateways are commonly used in enterprise contexts
 - They provide more
 - scalability,
 - maintainability,
 - and security
- **When to Consider an API Gateway:**
 - Increasing number of consumers and providers
 - Advanced cross-functional requirements like:
 - Authentication and authorization
 - Rate limiting
 - API management (API keys/tokens, monetization)

Decision Guidelines - Proxy, Load Balancer, or API Gateway?

- **Key Decision Points:**
 - Simple routing vs. advanced requirements
 - Need for API management features
 - Existing organizational mandates or solutions
- **Recommendation:**
 - Use the simplest solution for your current needs, considering immediate future requirements

Guidelines for Choosing Ingress Solutions

- **Guideline Highlights:**
 - For simple routing needs, use a proxy or load balancer
 - For advanced requirements, especially in enterprise settings, use an API gateway with API Management (APIM) features
 - Always perform due diligence to check for existing organizational mandates or solutions

Comparing reverse proxies, load balancers, and API gateways

Table 3-1. Comparing reverse proxies, load balancers, and API gateways

Feature	Reverse proxy	Load balancer	API gateway
Single Backend	*	*	*
TLS/SSL	*		*
Multiple Backends		*	*
Service Discovery			*
API Composition			*
Authorization			*
Retry Logic			*
Rate Limiting			*
Logging and Tracing			*
Circuit Breaking			*

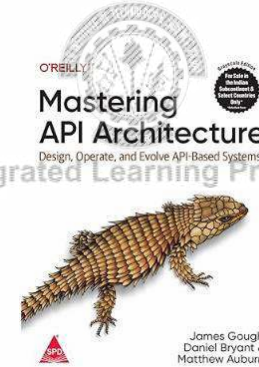
Adapted from "Mastering API Architecture" by Gough, Bryant and Auburn

Summary

- An API Gateway is not the only solution but is often the most comprehensive one for complex scenarios
- The choice between a proxy, load balancer, and API gateway should align with your system's specific requirements and organizational context
- Match your current and foreseeable requirements to the capabilities of each solution, ensuring alignment with organizational policies and scalability needs.

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



MCQ1

- What are the primary functions provided by an ingress solution when routing traffic to a backend system?
- A) Observability, Maintainability, and Flexibility
B) Reliability, Observability, and Security
C) Scalability, Simplicity, and Security
D) Flexibility, Adaptability, and Scalability

Answer: B) Reliability, Observability, and Security

MCQ2

- Which of the following scenarios would most likely require the use of an API gateway over a proxy or load balancer?
- A) Simple routing from a single endpoint to a single backend service
B) Traffic distribution across multiple servers to improve availability
C) Advanced cross-functional requirements such as authentication, authorization, or rate limiting
D) Internal communication between microservices in a small application
- Answer: C) Advanced cross-functional requirements such as authentication, authorization, or rate limiting

MCQ3

- What is the recommended approach when deciding between using a proxy, load balancer, or API gateway?
 - A) Always use an API gateway for all traffic to ensure maximum security.
 - B) Use a proxy for simple routing needs and a load balancer for more complex requirements.
 - C) Always perform due diligence within your organization and use the simplest solution for your requirements.
 - D) Use a load balancer if the organization is an enterprise, regardless of the actual needs.

Answer: C) Always perform due diligence within your organization and use the simplest solution for your requirements.

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Identify and differentiate between the key types of API gateways

BITS WILP | | Generated: 12-May-2026 16:47:54



API-based Products

API Gateway - Taxonomy | Suresh Gopal



Work Integrated Learning Programmes



BITS WILP | | Generated: 12-May-2026 16:47:54

Current API Gateway Taxonomy

- In the software development landscape, the classification of API gateways varies widely.
- Despite a general consensus on their core functionality, different industry segments have distinct needs and perspectives.
- This divergence has led to the emergence of various API gateway subtypes.
- Lets explore the evolving taxonomy of API gateways, examining their specific use cases, strengths, and limitations



BITS WILP | | Generated: 12-May-2026 16:47:54

Traditional Enterprise Gateways



- Purpose:
 - Primarily designed for managing and exposing business-focused APIs
- Integration:
 - Often coupled with a full API lifecycle management solution to handle API release, operation, and monetization at scale



Work Integrated Learning Programmes

Traditional Enterprise Gateways



- Versions:
 - Typically available in open-source editions, but commercial versions are more widely used due to advanced features
- Dependencies:
 - Requires deployment of external services like data stores, which must be maintained with high availability to ensure proper gateway operation
 - Considerations for running costs and disaster recovery plans are crucial



Work Integrated Learning Programmes

Microservices/Micro Gateways



- Purpose:
 - Focused on routing ingress traffic to backend APIs and services
- Features:
 - Generally lack extensive API lifecycle management capabilities.
 - Often available as open-source or lightweight versions of traditional enterprise gateways.



Work Integrated Learning Programmes

Microservices/Micro Gateways



- Deployment:
 - Usually deployed as standalone components, leveraging underlying platforms (e.g., Kubernetes) for managing internal states such as API lifecycle data and rate limiting
- Technology:
 - Built with modern proxy technologies like Envoy, offering strong integration with service meshes that use similar technologies.



Work Integrated Learning Programmes

Service Mesh Gateways

- Purpose:
 - Provide core routing functionality for external traffic entering the service mesh.
- Features:
 - Often lack advanced enterprise features like comprehensive integration with authentication, identity providers, and security solutions (e.g., WAF).
- State Management:
 - Managed using internal implementations or platform-based solutions (e.g., Kubernetes).
- Considerations:
 - Tightly coupled with the service mesh. May not be the best choice if you are not planning to deploy a service mesh.

Comparing API Gateway Types

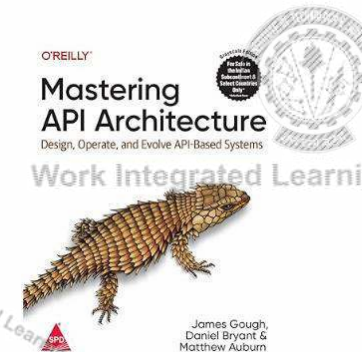
- Traditional Enterprise Gateways:
 - Best suited for large-scale, business-focused API management with comprehensive lifecycle support.
- Microservices/Micro Gateways:
 - Ideal for routing traffic to backend services with minimal lifecycle management, often leveraging modern proxy technologies
- Service Mesh Gateways:
 - Suitable for environments where routing into a service mesh is essential, with a focus on core routing functions rather than extensive enterprise features

Summary

- Understanding the various API gateway types and their unique attributes helps in selecting the right gateway based on specific requirements and use cases.
- Whether focusing on enterprise needs, microservices architecture, or service mesh integration, each type of gateway offers distinct advantages and limitations that should be carefully considered.

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



MCQ1

1. Which type of API gateway is primarily designed for managing and exposing business-focused APIs and often integrates with a full API lifecycle management solution?

- A. Microservices/Micro Gateway
- B. Traditional Enterprise Gateway
- C. Service Mesh Gateway
- D. Lightweight Gateway

Answer: B. Traditional Enterprise Gateway

MCQ2

2. What is a key characteristic of Microservices/Micro Gateways compared to Traditional Enterprise Gateways?

- A. They offer extensive API lifecycle management features.
- B. They are typically available only as commercial versions.
- C. They focus primarily on routing traffic to backend APIs with minimal lifecycle management features.
- D. They are tightly integrated with comprehensive security solutions like WAF.

Answer: C. They focus primarily on routing traffic to backend APIs with minimal lifecycle management features.

MCQ3

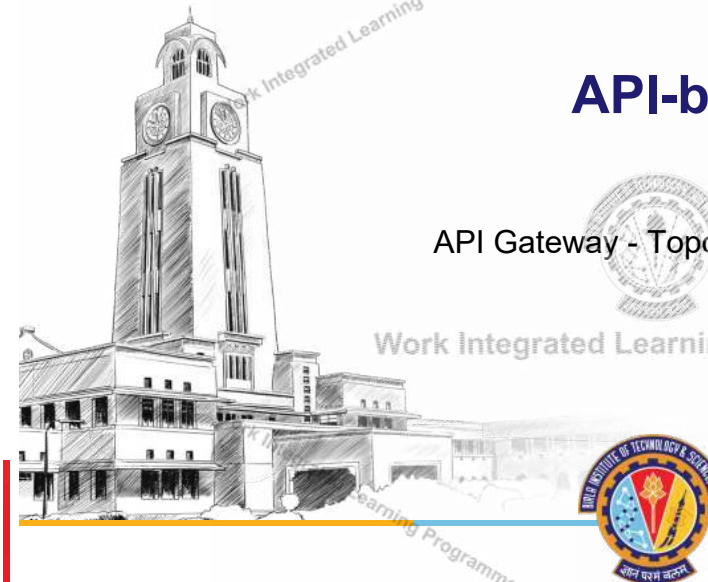
3. Which API gateway type is designed to provide core routing functionality for external traffic entering a service mesh and often lacks advanced enterprise features?

- A. Microservices/Micro Gateway
- B. Lightweight Gateway
- C. Service Mesh Gateway
- D. Traditional Enterprise Gateway

Answer: C. Service Mesh Gateway

API-based Products

API Gateway - Topologies Suresh Gopal



Learning Outcomes

- Understand and compare different API gateway topologies and hosting options, evaluate their benefits and limitations



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

API Gateway Topologies and Management Options

- Organizations require specific API topologies to ensure their API platforms or products are manageable and flexible, addressing functional, nonfunctional, regulatory, and business requirement
- Let's explore
 - common API gateway and API Management (APIM) topologies
 - and hosting options,
 - along with network traffic considerations and common topologies



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

API Gateway Topologies

1. API Gateway Direct to API Server

- This straightforward topology involves routing incoming API requests directly from the API gateway to the backend API servers.
- The API backend typically includes a load balancer and multiple server instances. This approach is suitable for standalone API products and does not require a service mesh.
- Advantages:
 - Simple setup,
 - Direct communication between API gateway and backend,
 - Minimal overhead

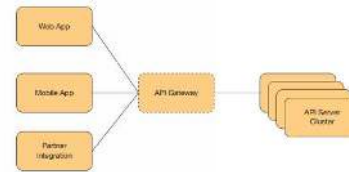


Figure 15.5 API topology 1 showing an API gateway routing in a straight path.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

API Gateway Topologies

2. API Gateway Routing to Services

- This topology involves multiple backend services, with the API gateway determining the appropriate service based on the request path.
- Services might be managed behind a load balancer or within a service mesh, allowing dynamic service communication.
- Advantages:
 - Scalable, modular service management,
 - Can leverage service mesh capabilities.

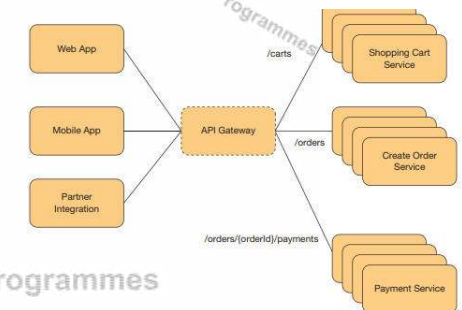


Figure 15.6 API topology 2 showing an API gateway routing to multiple backend services based on the base path of the incoming API request.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

API Gateway Topologies

3. Multiple API Gateway Instances

- Organizations with regulatory requirements or diverse deployments might use multiple API gateway instances
 - can either route to single or multiple backend services or support multitenant SaaS environments
 - enhances reliability and isolates impacts during peak usage.
- Advantages:
 - Improved availability,
 - Isolation of issues,
 - Supports complex regulatory and multi-tenant scenarios.

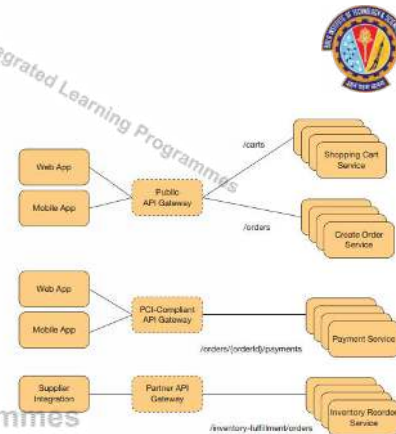


Figure 15.7 API topology 3 showing multiple API gateway instances that support various internal and external API clients, including the isolation of payment processing for PCI compliance and auditing.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

BITS WILP | | Generated: 12-May-2026 16:47:54

API Management Hosting Options

1. Hosted APIM

- Offered as a Software-as-a-Service (SaaS) solution, hosted APIMs are ideal for smaller organizations or those starting their API journey
 - Vendors provide different subscription levels and SLAs.
 - However, costs can increase with high usage, prompting a shift to on-premises solutions.
- Advantages:
 - Easy to start,
 - Low initial setup,
 - Vendor-managed infrastructure.

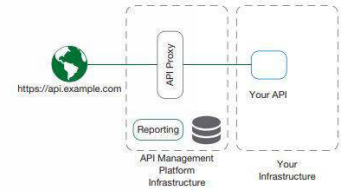


Figure 15.2 The hosted API management option.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

BITS WILP | | Generated: 12-May-2026 16:47:54

API Management Hosting Options

2. On-Premises APIM

- Installed within an organization's data center or cloud infrastructure, on-premises APIMs offer more customization and control
 - can handle internal-facing APIs and meet regulatory requirements but require dedicated operational management.
- Advantages:
 - Greater customization,
 - Control over security and compliance,
 - Supports internal APIs.

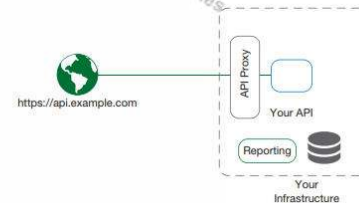


Figure 15.3 The on-premises API management option.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

BITS WILP | | Generated: 12-May-2026 16:47:54

API Management Hosting Options

3. Hybrid APIM

- Combines hosted dashboards and reporting with on-premises API gateway instances
 - model is less common but can reduce the burden of analysis and reporting processes
- Advantages:
 - Balances hosted convenience with on-premises control,
 - Useful when in-house expertise is limited

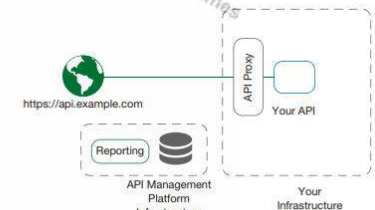


Figure 15.4 The hybrid API management option.

Adapted from "Principles of Web API Design: Delivering Value with APIs and Microservices by James Higginbotham"

BITS WILP | | Generated: 12-May-2026 16:47:54

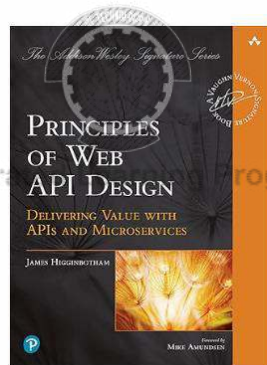
Network Traffic Considerations

- North-South vs. East-West Traffic
- North-South Traffic: Refers to data flow in and out of the data center
 - Requests from external applications are southbound, while responses are northbound.
- East-West Traffic: Involves data flow within the data center, such as between an API and a database.
- Zero Trust Architecture (ZTA):
 - Reduces the distinction between north-south and east-west traffic.
 - All traffic, including public, corporate, and VPN, requires explicit trust and access decisions.

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

- Principles of Web API Design: Delivering Value with APIs and Microservices
 - by James Higginbotham



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Choosing the right API gateway topology and management option depends on organizational needs, regulatory requirements, and the desired level of control and flexibility
- Regularly reassess your API strategy to ensure it aligns with your evolving business and operational goals.

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Which API gateway topology involves routing incoming requests directly to backend API servers without requiring a service mesh?

- A) API Gateway Routing to Services
- B) API Gateway Direct to API Server
- C) Multiple API Gateway Instances
- D) Hybrid API Management

Answer: B) API Gateway Direct to API Server

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

What is a primary advantage of using a hosted API Management (APIM) solution?

- A) Greater customization and control over security
- B) Lower initial setup cost and vendor-managed infrastructure
- C) Ability to manage multiple internal-facing APIs
- D) Full control over the API gateway instances

Answer: B) Lower initial setup cost and vendor-managed infrastructure

MCQ3

In Zero Trust Architecture (ZTA), how is traffic treated compared to traditional network models?

- A) Traffic is automatically trusted based on its source
- B) All traffic, including public, corporate, and VPN, requires explicit trust and access decisions
- C) North-south traffic is prioritized over east-west traffic
- D) East-west traffic is treated with more trust than north-south traffic

Answer: B) All traffic, including public, corporate, and VPN, requires explicit trust and access decisions

API-based Products

Service Mesh - What, where and How

Suresh Gopal

Learning Outcomes

- Understand Service Mesh: Define what a service mesh is and its core functionalities.
- Compare Methods: Recognize the differences between service meshes and traditional communication methods.

Service Mesh

Enhancing Service-to-Service Traffic Management

- We examined how to expose APIs and manage ingress traffic from external systems using an API gateway
- Now, we turn our attention to managing internal traffic—service-to-service communication—using a service mesh
- This shift is crucial as we explore routing, observing, and securing internal APIs with similar goals.

Understanding Service Mesh

- A service mesh is a pattern for managing service-to-service communication within a distributed system.
- Unlike API gateways, which handle external traffic, service meshes focus on internal, east-west traffic within a cluster or data center.
- Components:
 - Control Plane: Manages configuration, routes, and policies.
 - Data Plane: Executes traffic routing, policy enforcement, and telemetry.

Functionality

- Traffic Management:
 - Routes service-to-service communication.
- Resilience:
 - Ensures reliable service interactions.
- Observability:
 - Provides metrics, logs, and traces.
- Security:
 - Enforces policies and encrypts traffic.

Service Mesh vs. Traditional Approaches

- Traditional Approaches:
 - SDKs and Database Drivers: Manage communication via HTTP or TCP/IP but lack advanced features for resilience and observability
- Service Mesh Advantages:
 - Scalability:
 - Handles large numbers of services and interactions.
 - Maintainability:
 - Simplifies management through centralized control.
 - Security:
 - Provides enhanced security features.

Key Functionality Provided

- Proxy Behavior:
 - Acts as a full proxy for inbound and outbound traffic.
- Cross-Cutting Features:
 - User Verification
 - Rate Limiting
 - Timeouts and Retries
 - Metrics and Logging
- Advanced Features:
 - API Lifecycle Management: Service catalogs, developer portals.
 - Governance: Policy auditing and traffic management.

Deployment and Integration

- Deployment Locations:
 - Internal networks or clusters, often with multiple instances across different segments.
- Integration:
 - Works alongside other networking layers in modern stacks, including virtualization and sandboxing.

A typical service mesh topology, deployed across two clusters

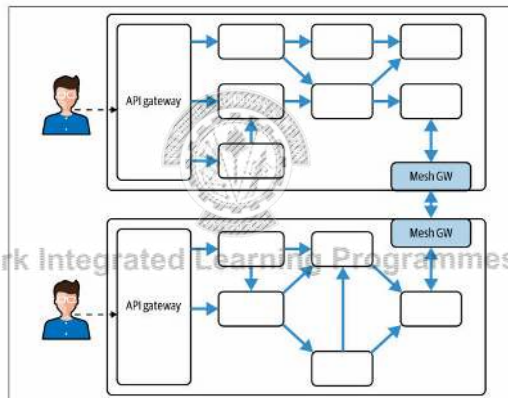


Figure 4-3. A typical service mesh topology, deployed across two clusters (with solid arrows showing service mesh traffic)

Adapted from "Mastering API Architecture" by Gough, Bryant and Auburn

Summary

- A service mesh provides a robust solution for managing internal service-to-service communication with enhanced capabilities for traffic management, resilience, observability, and security.
- While traditional methods have their place, the scalability and advanced features of a service mesh make it a powerful choice for modern, distributed architectures.

Reference

- Mastering API Architecture –
Design, Operate and Evolve Api-Based Systems
By James Gough, Daniel Bryant, Matthew Auburn



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Which component of a service mesh is responsible for defining routes, policies, and telemetry?

- A) Data Plane
- B) Control Plane
- C) Application Layer
- D) Network Interface

Answer: B) Control Plane

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

What is the primary function of a service mesh in a distributed system?

- A) Managing external API traffic
- B) Handling service-to-service communication within a cluster
- C) Routing traffic between user devices and servers
- D) Monitoring hardware performance

Answer: B) Handling service-to-service communication within a cluster

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ3

How does a service mesh differ from a traditional API gateway?

- A) A service mesh manages only external traffic, while an API gateway handles internal traffic.
- B) A service mesh is optimized for internal, service-to-service traffic, while an API gateway handles external, user-to-service traffic.
- C) A service mesh and an API gateway perform the same functions but in different environments.
- D) A service mesh is only used in monolithic architectures, while an API gateway is used in microservices.

Answer: B) A service mesh is optimized for internal, service-to-service traffic, while an API gateway handles external, user-to-service traffic.

BITS WILP | | Generated: 12-May-2026 16:47:54

API-based Products

Potential For API Mischief | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Protecting APIs - An Integrated Approach to Security

- API security is a critical component of API design
- Neglecting API protection can expose organizations to severe threats and potential data breaches
- An integrated API security strategy is essential to safeguard APIs from malicious attacks



Learning Outcomes

- Comprehend, analyze, and apply critical API security principles and practices in real-world scenarios



Importance of API Security

- API design involves more than just defining HTTP methods, paths, resources, and media types
- APIs must be protected to prevent unauthorized access and mitigate risks to both the organization and its customers
- A comprehensive API protection strategy should include the
 - right components,
 - API gateway solutions,
 - and identity and access management systems



Consequences of Poor API Security

Unsecured APIs: APIs with minimal or no security measures are vulnerable to exploitation by attackers.

Real-World Breaches:

- Unauthorized access to user databases, leading to data leaks (e.g., Telegram breach).
- Password reset exploits that bypass security checks and expose sensitive information.
- Exploitation of undocumented APIs meant for internal use, leading to unauthorized access to sensitive data.
- Failure to secure private APIs properly, potentially exposing user locations or sensitive information (e.g., Tinder API issue).

Key API Security Practices

- Authentication (AuthN)
 - Verifies the identity of API callers.
 - Strongly recommended to use OpenID Connect or similar solutions rather than traditional username/password methods.
- Authorization (AuthZ)
 - Controls access to specific API operations based on user roles or scopes.
 - Techniques: API keys, API tokens, OAuth 2.0.

Key API Security Practices

- Claims-Based Access Control
 - Provides fine-grained control over API resources beyond standard authorization methods.
- Rate Limiting (Throttling)
 - Prevents traffic spikes and mitigates denial-of-service (DoS) attacks by limiting the number of API requests.
- Quotas
 - Limits API usage based on subscription levels or agreements, typically on a monthly basis.

Key API Security Practices

- Session Hijack Prevention
 - Uses Cross-Origin Resource Sharing (CORS) and prevents Cross-Site Request Forgery (CSRF) to protect API sessions.
- Cryptography
 - Encrypts data both in transit and at rest. Ensures private keys are safeguarded to prevent data decryption by attackers.
- Mutual TLS (mTLS)
 - Ensures client identity verification and prevents identity forging, especially in inter-service communication.

Key API Security Practices

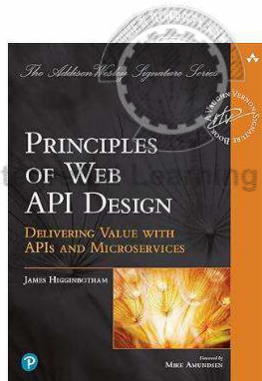
- Protocol Filtering and Protection
 - Filters and blocks invalid requests, enforces HTTPS, and protects against known malicious clients.
- Message Validation
 - Validates incoming data to prevent injection attacks and unauthorized access.
- Botnet and Data Scraping Protection
 - Detects and mitigates malicious botnet activities, data scraping, spam, and DDoS attacks.

Summary

- API security is not a one-size-fits-all solution; it requires a multi-layered approach involving several security components.
- Organizations must adopt best practices and avoid common antipatterns to ensure their APIs are secure.
- Continuous review and adaptation of API security measures are crucial to staying ahead of evolving threats.

Reference

- Principles of Web API Design: Delivering Value with APIs and Microservices
 - by James Higginbotham



MCQ1

- What is the primary reason organizations must adopt a comprehensive API security strategy?
 - A) To increase API response speed
 - B) To ensure APIs are documented
 - C) To protect sensitive data and prevent unauthorized access
 - D) To reduce the cost of API development

Answer: C) To protect sensitive data and prevent unauthorized access

MCQ2

- Which of the following practices is used to verify the identity of API callers before allowing API requests to be processed?

- A) Rate limiting
- B) Claims-based access control
- C) Authentication (AuthN)
- D) Quotas

Answer: C) Authentication (AuthN)

MCQ3

- What is a common misconception that leads some API providers to neglect proper security for their internal APIs?

- A) They believe API security is too expensive to implement.
- B) They think if an API is undocumented, it will remain secure.
- C) They assume APIs are automatically secure when using HTTP.
- D) They rely solely on user passwords for API protection.

Answer: B) They think if an API is undocumented, it will remain secure.

API-Based Products

API Performance Testing | Suresh Gopal

Learning Outcomes

- Understand the objectives and significance of different type of testing in evaluating API performance under different conditions and loads

Performance of API

APIs are now integral for delivering data and services directly to end users via digital platforms

- With this shift, the performance of APIs under heavy loads becomes crucial, as it directly impacts the overall performance of client applications

Load testing for APIs emerges as a critical aspect to ensure optimal performance and reliability

- Involves comprehensive testing methodologies to assess their performance under various loads and stress conditions

BITS WILP | Generated: 12-May-2026 16:47:54

Preparing For The Load Test

A well-defined load testing strategy is crucial

- Starting with identifying the APIs to be tested and replicating real-world traffic conditions

Planning involves gathering information on

- Performance requirements such as average and peak throughput for each API
- Throughput distribution,
- Traffic distribution patterns of client apps,
- and expected concurrent users

Test execution may involve

- Generating repetitive loads for API endpoints to establish performance upper bounds,
- Followed by tuning and optimization if necessary

BITS WILP | Generated: 12-May-2026 16:47:54

Preparing For The Load Test

Simulating realistic traffic patterns involves considering

- Traffic distribution across APIs,
- Traffic growth patterns,
- Handling of success and failure responses,
- and geographically distributed traffic to test for network congestion

Data from production traffic logs can be used to simulate realistic traffic scenarios for load testing,

- Providing valuable insights into API performance under various conditions

BITS WILP | Generated: 12-May-2026 16:47:54

API Performance Testing

API performance testing should consist of the following,

- With an aim to find the different performance parameters of the APIs and the API platform

Baseline testing

Load testing

Stress testing

Soak testing

BITS WILP | Generated: 12-May-2026 16:47:54

Baseline Testing

Baseline testing aims to assess system performance under typical expected loads

Results are analyzed to determine average and peak API response times and error rates

CPU and memory utilization are scrutinized to identify and resolve resource bottlenecks

Load Testing

Load testing involves incrementally increasing API traffic volumes to evaluate performance

- Performance metrics like response time and throughput are monitored to assess API performance

The goal is to understand the system's behavior and capacity under expected peak loads, not to identify breaking points

- Server performance metrics such as CPU and heap memory utilization, as well as network port utilization, are analyzed to gauge the platform's capability under high load.

Stress and Soak Testing

Stress testing aims to identify the breaking point of the platform by gradually increasing API traffic load

- Determines the maximum throughput that the system can handle
- The test continues until performance degradation or an increase in errors from API calls is observed

Soak testing assesses system stability over prolonged durations to uncover potential instabilities

- Baseline tests may extend over days or weeks to detect any adverse behaviors during prolonged usage
- The objective is to identify issues with releasing system resources for subsequent execution cycles

API Performance Test Metrics

Look at the following metrics to measure the performance of the individual API and the platform

API response time

API target response time

Throughput

Success and error rate

CPU utilization

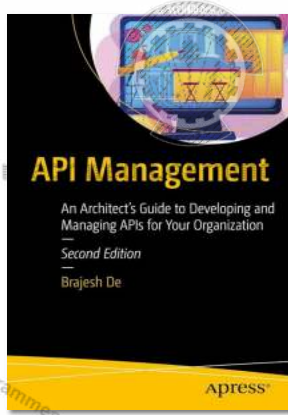
Heap memory utilization

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Question: What are the key considerations for preparing for API load testing?

- A) Identifying the list of APIs to be tested
- B) Planning for real-world traffic replication
- C) Gathering information on performance requirements
- D) All of the above

Correct Answer: D) All of the above

Summary

- The preparations needs to done for performing API performance testing
- Various methodologies such as
 - Baseline testing,
 - Load testing,
 - Stress testing,
 - and soak testing
- API performance metrics to be observed

MCQ2

Question: Which metrics are essential for measuring API performance during performance testing?

- A) API response time, API target response time, and API throughput
- B) API latency, success and error rates, and CPU utilization
- C) API response time, API latency, and throughput
- D) API target response time, success and error rates, and heap memory utilization

Correct Answer: A) API response time, API target response time, and API throughput

MCQ3

Question: What is the primary objective of load testing for APIs?

- A) To identify the breaking point of the system
- B) To analyze performance metrics like response time and throughput
- C) To determine the minimum expected load for the system
- D) To assess server performance metrics such as CPU utilization and heap memory utilization

Correct Answer: B) To analyze performance metrics like response time and throughput

API-Based Products

API Testing Tools

Suresh Gopal

Learning Outcomes

- Gain a comprehensive understanding of the essential features that API testing tools should possess
- Know about some API testing tools

Key Features for API Testing Tools

Comprehensive Test Coverage:

Ability to test various aspects of APIs including functionality, security, performance, and scalability

Ease of Use:

Intuitive interface and user-friendly design for efficient test creation and execution

Automation Support:

Robust automation capabilities to streamline testing processes and facilitate continuous integration and delivery (CI/CD) pipelines

Key Features for API Testing Tools

Flexibility:

Support for testing both RESTful and SOAP APIs, as well as other common protocols and data formats

Customization Options:

Ability to customize tests and adapt to specific project requirements and environments

Reporting and Analysis:

Detailed reporting features to track test results, identify issues, and analyze performance metrics

Collaboration Tools:

Collaboration features for teams to work together on test creation, execution, and analysis

Nice-to-Have Features

Mocking Capabilities:

Ability to create mock APIs for testing purposes, allowing for isolated and controlled testing environments

Performance Testing:

Built-in tools for performance testing to evaluate API response times, throughput, and scalability

Security Testing:

Integration with security testing tools to assess API security vulnerabilities and ensure data protection

Nice-to-Have Features

Scalability:

Ability to handle large-scale testing scenarios with support for distributed testing and resource optimization

Documentation Generation:

Automatic generation of API documentation to maintain consistency and improve developer experience

Integration Support:

Compatibility with popular development and testing frameworks, as well as third-party tools for seamless integration into existing workflows

Tools Suggestion

Postman:

A popular API testing tool known for its user-friendly interface and robust features

Offers comprehensive test coverage, automation support, and collaboration tools.

Supports both RESTful and SOAP APIs and provides extensive customization options



POSTMAN

Tools Suggestion

Swagger (now known as OpenAPI):

- Not only a specification for defining APIs but also offers tools for testing
- Provides features for API documentation generation, allowing for better understanding and testing of APIs
- Can be integrated with various testing frameworks and CI/CD pipelines



Tools Suggestion

SoapUI:

- A powerful API testing tool specifically designed for testing SOAP and RESTful APIs
- Offers a wide range of features including comprehensive test coverage, automation support, and performance testing capabilities
- Provides advanced features such as mocking capabilities for creating mock APIs and security testing tools



Tools Suggestion

Katalon Studio:

- An all-in-one test automation solution that includes API testing capabilities
- Offers a user-friendly interface with features for test creation, execution, and reporting
- Supports both RESTful and SOAP APIs and integrates seamlessly with other testing frameworks



Comparison of Tools

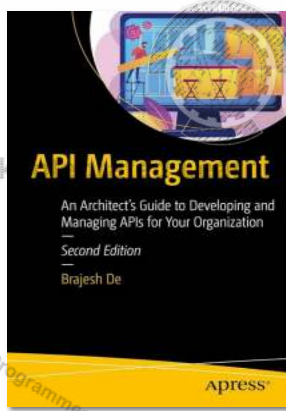
Feature	Postman	SoapUI	Swagger (OpenAPI)	Katalon Studio
Test Coverage	Comprehensive	Comprehensive	Limited	Comprehensive
Ease of Use	Very user-friendly	User-friendly	Moderate	User-friendly
Automation	Yes	Yes	No	Yes
Protocol Support	RESTful and SOAP APIs	RESTful and SOAP APIs	RESTful APIs	RESTful and SOAP APIs
Customization	Extensive customization options	Extensive customization options	Limited customization options	Extensive customization options
Reporting	Yes	Yes	Limited	Yes
Collaboration	Yes	Limited	Limited	Yes
Mocking	No	Yes (SoapUI Pro)	No	No
Performance	No	Yes (SoapU Pro)	No	No
Security	No	Yes (SoapU Pro)	No	No
Scalability	No	Yes (SoapUI Pro)	No	No
Integration	Limited	Limited	Limited	Yes

Reference

API Management:

An Architect's Guide to Developing and Managing APIs for Your Organization

- by **Brajesh De (Author)**



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Key Features for API Testing Tools
- Nice-to-Have Features
- API testing Tools suggestion

MCQ1

Which of the following is NOT a key feature of API testing tools?

- a) Comprehensive test coverage
- b) Scalability testing only
- c) Ease of use
- d) Automation support

Answer: b) Scalability testing only

MCQ2

What is the primary purpose of automation support in API testing tools?

- a) To increase manual testing efforts
- b) To streamline testing processes and integrate with CI/CD pipelines
- c) To replace human testers entirely
- d) To create complex test scenarios only

Answer: b) To streamline testing processes and integrate with CI/CD pipelines

MCQ3

What does integration with security testing tools enable in API testing?

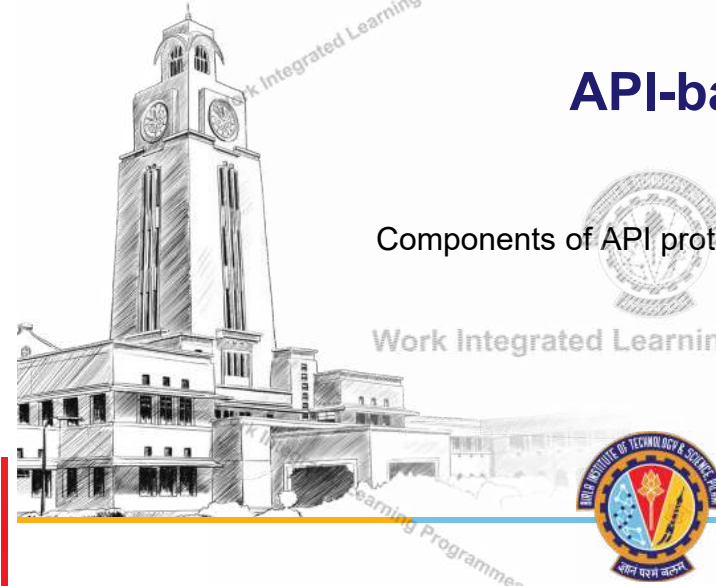
- a) Creation of mock APIs
- b) Performance evaluation
- c) Assessment of API security vulnerabilities
- d) Scalability testing

Answer: c) Assessment of API security vulnerabilities



API-based Products

Components of API protection | Suresh Gopal



Learning Outcomes

- Provide a comprehensive overview of the components involved in API protection



Introduction to API Protection

- Overview:
 - API protection involves various components that collectively form a robust security strategy.
- Importance:
 - Safeguarding APIs is crucial due to their exposure to external and internal threats, including attacks and unauthorized access.



Components of API Protection

- API Gateways
- API Management Layers (APIMs)
- Service Meshes
- Web Application Firewalls (WAFs)
- Content Delivery Networks (CDNs)
- Intelligent API Protection

API Gateways

- An API gateway acts as an intermediary between clients and API servers, managing all incoming and outgoing traffic.
- Key Functions:
 - Acts as a central gatekeeper.
 - May perform protocol transformations.
- Forms:
 - Standalone middleware or part of an API management layer.
- Limitations:
 - Basic security features; advanced API management typically requires an API Management Layer (APIM).

API Management Layers (APIMs)

- APIMs extend the capabilities of API gateways to provide a comprehensive API lifecycle management solution.
- Key Features:
 - Includes publishing, monitoring, protecting, analyzing, and monetizing APIs.
 - Advanced rate limiting, quota management, and subscription-level controls.
- Overlap with WAFs:
 - Some security functions may cross into Web Application Firewalls (WAFs) territory.

Service Meshes

- Service meshes manage network reliability, security, observability, and error handling outside of the individual services.
- Key Components:
 - Sidecar proxies alongside each service for communication handling.
 - Centralized control plane for management and health monitoring.
 - Advantages:
 - Provides resilient, observable network communications.
 - Considerations:
 - May impact performance due to additional network hops.
 - Suitable for large organizations with complex microservices architectures.

Web Application Firewalls (WAFs)

- WAFs protect APIs from network threats like scripting and injection attacks by monitoring OSI layer 3 and 4.
- Key Features:
 - Deeper packet inspection capabilities.
 - Additional protection against Distributed Denial-of-Service (DDoS) attacks.
- Integration:
 - Often merged with APIMs or CDNs, reducing the need for standalone WAFs.

Content Delivery Networks (CDNs)

- CDNs cache content across distributed servers to reduce API server load and enhance performance.
- Key Features:
 - Acts as a reverse proxy for dynamic content.
 - Provides additional DDoS protection.
- Impact on APIs:
 - Helps reduce unwanted traffic, complementing API protection strategies.

Intelligent API Protection

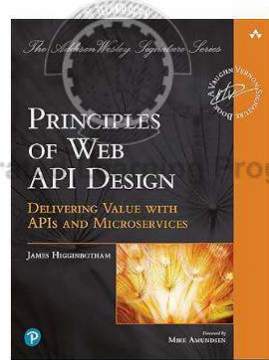
- Advanced protection mechanisms that go beyond traditional API gateways, WAFs, and APIMs.
- Key Challenges:
 - Mitigating automated attacks (e.g., botnets) and data scraping.
- Solutions:
 - Implementing traffic analysis across multiple IP addresses.
 - Advanced detection techniques to identify and block sophisticated attacks.

Summary

- Combining multiple components—API gateways, APIMs, service meshes, WAFs, CDNs, and advanced protection techniques—creates a comprehensive API protection strategy
- Each component has unique strengths and use cases, and organizations should tailor their API security strategy based on their specific needs and architecture.

Reference

- Principles of Web API Design: Delivering Value with APIs and Microservices
 - by James Higginbotham



BITS WILP | Generated: 12-May-2026 16:47:54



MCQ1

- Which of the following components is primarily responsible for managing all incoming and outgoing traffic to an API server and can perform protocol transformations?

- A. Content Delivery Network (CDN)
- B. Web Application Firewall (WAF)
- C. API Gateway
- D. Service Mesh

Answer: C. API Gateway



BITS WILP | Generated: 12-May-2026 16:47:54

MCQ2

- What is a key advantage of using a service mesh in a microservices architecture?
- A. It reduces the need for caching API responses.
 - B. It provides deep packet inspection capabilities.
 - C. It manages network reliability, security, and error handling outside of individual services.
 - D. It replaces the need for an API management layer (APIM).

Answer: C. It manages network reliability, security, and error handling outside of individual services.



BITS WILP | Generated: 12-May-2026 16:47:54

MCQ3

- Which component is known for providing additional protection against DDoS attacks and may be integrated with Content Delivery Networks (CDNs) or API Management Layers (APIMs)?

- A. API Gateway
- B. Web Application Firewall (WAF)
- C. Service Mesh
- D. Intelligent API Protection

Answer: B. Web Application Firewall (WAF)



BITS WILP | Generated: 12-May-2026 16:47:54

API-based Products

Identity and access management | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Identity and Access Management (IAM)

- Identity and Access Management (IAM) provides authentication and authorization services to secure API products or platforms
- Role in API Security:
 - IAM integrates with other middleware components like API gateways to prevent malicious attacks.
- Core Functions of IAM:
 - Integration with third-party vendors using industry standards.
 - Generation of API tokens to replace passwords and manage access controls.

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcome

- provides a comprehensive overview of Identity and access management (IAM), focusing on securing APIs through proper authentication and authorization mechanisms



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

The Role of Passwords and API Keys in API Security

Passwords for API Authentication:

- APIs may use username and password credentials to log in, but this method is discouraged due to several limitations:
- Fragility:
 - Frequent password changes can disrupt API access.
- Lack of Security:
 - Sharing passwords for third-party access compromises security.
- No Support for Multifactor Authentication (MFA).

BITS WILP | | Generated: 12-May-2026 16:47:54

The Role of Passwords and API Keys in API Security



API Keys:

- Simple, long alphanumeric values used as replacements for passwords.
- Characteristics:
 - No expiration date, which increases security risks.
 - Usually found in user profile or settings pages.
 - Resetting API keys may require a manual process.

API Tokens: A Robust Alternative



- What are API Tokens?
 - Represent sessions where a user or third party is authorized to interact with an API.
 - Include an expiration time and support refresh tokens for session renewal.
- Advantages Over API Keys:
 - Expiration Time: Limits the duration of access.
 - Scopes and Controls: API tokens can have multiple access controls (scopes) for different levels of API access (e.g., read-only, read/write).
 - Secure Transmission: API tokens are often passed via HTTP Authorization headers to reduce exposure risks.

API Token Transmission Methods



Common Methods for Passing API Tokens:

- Query Argument in URL:
 - Not recommended as tokens can be logged by servers and accessed by malicious code.
- POST Parameters:
 - More secure, but varies across APIs.
- HTTP Authorization Header:
 - Preferred method due to reduced logging risk and controlled access via CORS headers.

Pass-by-Reference vs. Pass-by-Value API Tokens



- Pass-by-Reference Tokens:
 - Contain only a unique identifier, requiring the server to perform lookups to verify user and permissions.
 - Commonly used for north-south traffic (client-server communication).
- Pass-by-Value Tokens:
 - Contain embedded name/value pairs, minimizing server lookups.
 - Use with caution as they can expose sensitive data.
 - Commonly used for east-west traffic (server-to-server communication).

JSON Web Tokens (JWTs)

- JWTs are a popular standard for pass-by-value API tokens.
- Comprised of three elements:
 - Header,
 - Payload,
 - and Signature,
 - each Base64 encoded and dot-separated.
- Signed with private keys to prevent tampering.
- Usage:
 - Widely used for communicating authorization details in internal traffic (east-west).

BITS WILP | | Generated: 12-May-2026 16:47:54

OpenID Connect

- An identity layer built on top of OAuth 2.0.
- Enables verifying user identities and obtaining profile information via a standardized API.

BITS WILP | | Generated: 12-May-2026 16:47:54

OAuth 2.0

- An industry-standard framework for managing authorization workflows across different types of applications (Web, mobile, desktop)
- Manages complex workflows between data owners, API servers, authorization servers, and third-party applications.

BITS WILP | | Generated: 12-May-2026 16:47:54

Single Sign-On (SSO) and Federated Identity Management

- Importance in Enterprises:
 - SSO simplifies access across multiple applications and third-party services.
 - SAML (Security Assertion Markup Language): A standard used to integrate APIs with existing SSO solutions.
- Provides a seamless experience for enterprise users when accessing APIs through applications.

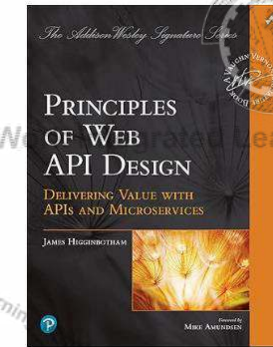
BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Key Takeaways
 - Opt for API tokens over passwords and API keys for robust security.
 - Use standardized transmission methods like HTTP Authorization headers.
 - Leverage OAuth 2.0 and OpenID Connect for secure, scalable authentication and authorization.
 - Implement SSO and federated identity management for cohesive user experiences in enterprise environments.
- Next Steps:
 - Explore resources like JWT.io and OAuth Community websites for deeper learning.

Reference

- Principles of Web API Design: Delivering Value with APIs and Microservices
 - by James Higginbotham



MCQ1

- What is a key advantage of using API tokens over API keys in securing APIs?
 - A. API tokens do not require a refresh process.
 - B. API tokens have no expiration date, providing indefinite access.
 - C. API tokens can have multiple access controls (scopes) for different levels of access.
 - D. API tokens are easier to generate and manage than API keys.

Answer: C. API tokens can have multiple access controls (scopes) for different levels of access.

MCQ2

Which of the following is NOT a recommended method for passing an API token to the server?

- A. As a query argument in the URL
- B. As a POST parameter
- C. Using the HTTP Authorization header
- D. Through a secure WebSocket connection

Answer: A. As a query argument in the URL

MCQ3

What is the primary purpose of OAuth 2.0 in the context of API security?

- A. To provide an alternative to API tokens with no expiration date
- B. To create and manage passwords for API clients
- C. To standardize the authorization workflow for different types of applications
- D. To replace Single Sign-On (SSO) solutions in enterprise environments

Answer: C. To standardize the authorization workflow for different types of applications

API-based Products

OAuth - Overview

Suresh Gopal

Learning Outcomes

- Present a clear, structured overview of OAuth 2.0, focusing on its purpose, how it works, the problems it solves, and the benefits it provides.

Introduction to OAuth 2.0

- OAuth 2.0 is a protocol for authorization that enables applications to access server resources on behalf of users without sharing their credentials.
- Purpose:
 - Provides a secure way for third-party applications to obtain limited access to HTTP services, either with the user's permission or on their behalf.

Real-World Example

- Scenario: A shopping app (e.g., Amazon) posts a review to Facebook on behalf of a you – i.e. user.
- Without OAuth:
 - Users would need to provide their Facebook password to Amazon.
- With OAuth:
 - Users authorize Amazon to use a token to post, without revealing their password.

How OAuth 2.0 Works?

- Authorization Flow:
 - A user (resource owner i.e. You) uses a third-party app (client i.e. Amazon) that needs access to their data on another service (resource server i.e. Facebook)
 - The user is redirected to the service's login page (e.g., Facebook) to grant access.
 - Upon successful login and approval, the user is redirected back to the third-party app, which can now access specific data or perform actions on their behalf.

The Problem OAuth Solves

- Issue with Passwords:
 - Sharing passwords between apps (e.g., Amazon and Facebook) is risky
 - Security risks arise if multiple apps store a user's password.
- Password changes would require updates across all apps, creating a poor user experience.
- OAuth's Solution:
 - OAuth replaces password sharing with tokens that grant limited access, reducing security risks.

Token-Based Access

- How Tokens Work:
 - A token is generated when the user authorizes the app to access their account.
 - Tokens allow specific actions (e.g., posting on Facebook) without revealing the user's password.
- Token Validity:
 - Tokens have a defined expiration period.
 - Access is automatically revoked when the token expires or can be manually revoked earlier.

Benefits of OAuth for Security

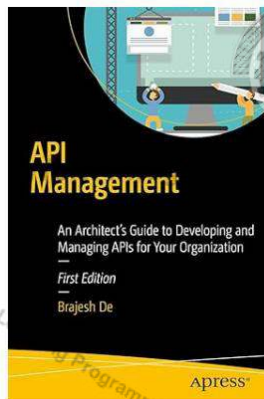
- Improved Security:
 - No need to share passwords across multiple apps.
 - If a password is compromised, users only need to change it in one place.
- API Resilience:
 - OAuth tokens enhance the security of APIs by minimizing dependency on passwords.

Summary

- OAuth 2.0 Advantages:
 - Secure, token-based authorization framework.
 - Protects user credentials and enhances API security.
 - Provides a better user experience by centralizing credential management.

Reference

- API Management: An Architect's Guide to Developing and Managing APIs for Your Organization
 - by Brajesh D



MCQ1

- What is the primary purpose of OAuth 2.0?
 - A. To provide authentication using passwords
 - B. To encrypt data transferred between clients and servers
 - C. To allow third-party applications to access server resources without sharing user credentials
 - D. To perform data backups securely

Answer: C. To allow third-party applications to access server resources without sharing user credentials

MCQ2

- Which of the following is a key advantage of using OAuth 2.0 tokens instead of passwords?

- A. Tokens are easier to remember than passwords
- B. Tokens allow apps to perform actions without the need for storing user passwords
- C. Tokens can never expire
- D. Tokens are only usable by one specific application

Answer: B. Tokens allow apps to perform actions without the need for storing user passwords

MCQ3

- What happens when a user changes their password on a service that uses OAuth 2.0 for authorization?

- A. All tokens are immediately revoked, and users must reauthorize every application
- B. The user must manually update their password in all applications
- C. Tokens continue to work until they expire or are revoked, without needing the new password
- D. The OAuth server automatically updates the password in all connected applications

Answer: C. Tokens continue to work until they expire or are revoked, without needing the new password

API-based Products

Working of OAuth | Suresh Gopal

Learning Outcomes

- Understand the basic concepts of OAuth, including its actors, tokens, scope, and grant types, to effectively manage and implement secure authorization processes.

Introduction to OAuth: Basic Concepts

- Overview OAuth (Open Authorization) is a framework that allows applications to obtain limited access to user accounts on an HTTP service
- Let's cover the fundamental concepts of OAuth, including its
 - actors,
 - tokens,
 - scope,
 - and grant types.

Actors in OAuth

- Resource Owner
 - The end user who owns the data and authorizes access to it.
 - Example: A Facebook user who owns their account's photos and posts.
 - Role: Grants permission to applications to access their resources.
- Client
 - The application requesting access to the resource owner's data.
 - A third-party app trying to access a user's Facebook photos.

Actors in OAuth

- Resource Server
 - The server that hosts the resource owner's protected data.
 - Facebook's server where user data and APIs are hosted.
 - Role: Provides access to the data only after authorization.
- Authorization Server
 - The server that authenticates the user and issues access tokens.
 - Facebook's server that validates user credentials and provides tokens.
 - Often, the resource server and authorization server are combined in practice, such as with API gateways.

Actors in OAuth

Figure 7-3 shows the various actors involved in an OAuth flow.

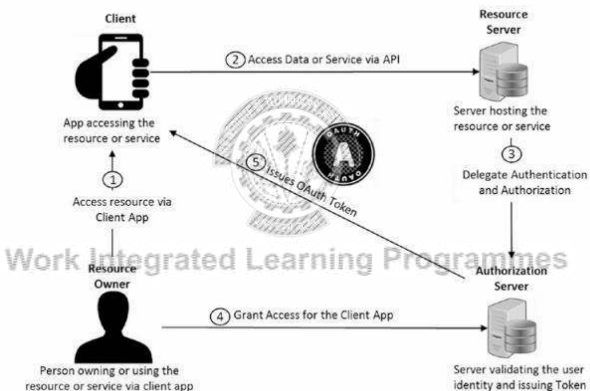


Figure 7-3. The various actors involved in an OAuth flow

Adapted from "API Management by Brajesh De"

Tokens

- Tokens are credentials used to access protected resources.
 - They come in two primary types
- Access Tokens
 - Tokens that grant access to specific resources for a limited time.
- Characteristics:
 - Long string of characters.
 - Typically passed as bearer tokens in HTTP headers.
 - Can have restrictions or scopes defining accessible resources.
 - Expire after a certain period and can be revoked if compromised.

Tokens

- Refresh Tokens
 - Tokens used to obtain new access tokens without re-authenticating.
- Characteristics:
 - Allow the client to request new access tokens without involving the resource owner.

Scope

- Scope specifies what resources and actions the application is authorized to access.
- Characteristics:
 - Defined by the authorization server.
 - Determines the level of access granted.
 - Optional in access token requests.

Grant Types

- OAuth 2.0 defines different methods (grant types) for obtaining access tokens:
 - Authorization Code
 - Client Credentials
 - Resource Owner Password Credentials
 - Implicit

Authorization Code

- Process:
 - User logs in and authorizes the application.
 - Authorization server issues an authorization code.
 - Client exchanges the code for an access token.
- Pros:
 - Secure, involves user authorization.
- Used For:
 - Web applications where user consent is required.

Client Credentials

- Process:
 - Client requests an access token with its credentials.
 - Authorization server validates the client and issues the token.
- Pros:
 - Suitable for machine-to-machine interactions.
- Used For:
 - Server-to-server communications.

Resource Owner Password Credentials

- Process:
 - Client submits user credentials with its request.
 - Authorization server validates credentials and issues a token.
- Pros:
 - Direct but requires user credentials.
- Used For:
 - Trusted applications needing user credentials directly.

Implicit

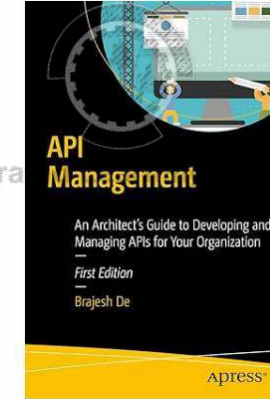
- Process:
 - Access token is directly issued after user authorization.
 - No separate token exchange step.
- Pros:
 - Simplified process.
- Cons:
 - Less secure, no refresh tokens.
- Used For:
 - Mobile and JavaScript apps running in browsers.

Summary

- OAuth is a versatile framework enabling secure and limited access to user resources.
- Understanding its basic concepts, including the roles of different actors, the types of tokens, and various grant types, is essential for implementing effective and secure authorization mechanisms.

Reference

- API Management: An Architect's Guide to Developing and Managing APIs for Your Organization
 - by Brajesh De (Author)



MCQ1

- Which of the following is responsible for issuing access tokens and validating user identity in the OAuth framework?

- A) Resource Owner
- B) Client
- C) Resource Server
- D) Authorization Server

Answer: D) Authorization Server

MCQ2

- What type of token allows a client to request a new access token without re-authenticating the user?

- A) Access Token
- B) Refresh Token
- C) Authorization Code
- D) Identity Token

Answer: B) Refresh Token

MCQ3

- In which OAuth grant type does the client receive the access token directly after the user authorizes the request, without a separate token exchange step?

- A) Authorization Code
- B) Client Credentials
- C) Resource Owner Password Credentials
- D) Implicit

Answer: D) Implicit

BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Outline essential best practices and tips to help future-proof your API

BITS WILP | | Generated: 12-May-2026 16:47:54



API-based Products

Scaling APIs

Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Importance of Availability and Reliability

- When creating an API that others rely on,
 - ensuring **high availability and reliability** is paramount!
- API should consistently
 - perform well
 - load quickly for users
- However, unexpected surges in usage can occur,
 - potentially degrading service quality or disrupting application



BITS WILP | | Generated: 12-May-2026 16:47:54



Key Scaling Strategies

- Scaling Throughput
 - Techniques to enhance the number of API calls handled efficiently
- Evolving API Design
 - Adapting the API to meet changing user needs and trends
- Paginating APIs
 - Implementing strategies to manage large datasets effectively

Key Scaling Strategies

- Rate-Limiting APIs
 - Establishing policies to control the frequency of requests made by developers
- Developer SDKs
 - Providing tools that guide developers in utilizing your API optimally

Optimizing API Performance

- To enhance API's capability to handle increased requests, consider various application-level strategies:
 - Database Query Optimization
 - Sharding Databases
 - Adding Missing Indexes
 - Utilizing Caching
 - Asynchronous Operations
 - Writing Efficient Code
 - Tuning Web Servers

Handling Large Datasets with Pagination

- APIs often need to manage extensive datasets!
- Pagination
 - a proven method for efficiently delivering large data sets to developers

Maintaining Application Health through Rate Limiting

- Despite taking measures to enhance throughput and pagination,
 - **high request rates from developers can still overwhelm your API**
- Rate-limiting is essential for
 - managing these requests
 - maintaining the overall health of your application



Work Integrated Learning Programmes

Empowering Developers with SDKs

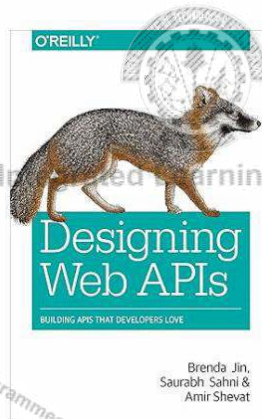
- Finally, scaling APIs is not solely about internal mechanisms
 - Providing Developer SDKs can significantly aid third-party developers in following best practices while integrating with your API



Work Integrated Learning Programmes

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin , Saurabh Sahni , Amir Shevat



Summary

- How by applying these scaling strategies and best practices, you can ensure that your API is well-prepared to handle growth and evolving demands, ultimately leading to sustained success.



Work Integrated Learning Programmes

MCQ1

- Which of the following is NOT a recommended strategy for optimizing API throughput?

- A) Database query optimization
- B) Asynchronous operations
- C) Sharding databases
- D) Hard-coding developer SDKs

Answer:

- D) Hard-coding developer SDKs

MCQ2

- What is the primary purpose of rate-limiting in APIs?
- A) To improve database query performance
- B) To manage the frequency of developer requests
- C) To paginate large datasets
- D) To provide tools for developers to use the API efficiently

Answer:

- B) To manage the frequency of developer requests

MCQ3

- Which of the following is an effective strategy for handling large datasets in APIs?

- A) Using pagination
- B) Optimizing web servers
- C) Sharding databases
- D) Implementing rate limits

Answer:

- A) Using pagination

API-based Products

Scaling Throughput

Suresh Gopal



Learning Outcomes

- Simplify complex API scaling concepts into actionable strategies,
 - helping to effectively communicate the process of identifying bottlenecks, optimizing performance, and applying best practices for scaling.



Work Integrated Learning Programmes

Scaling Throughput

- As the number of API users grows, the throughput (API calls per second) increases
- Optimizing your API to handle this growth is essential for maintaining performance.



Work Integrated Learning Programmes

Architecture of a typical large-scale web application

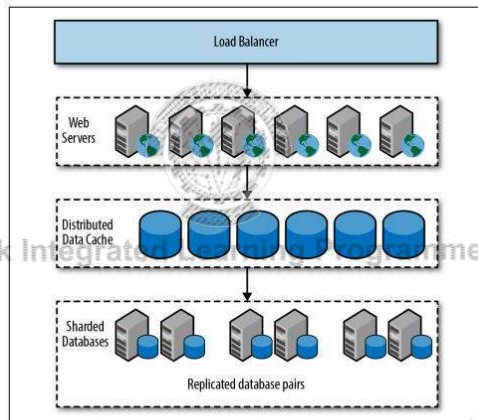


Figure 6-2. Architecture of a typical large-scale web application

Adapted from "Designing Web APIs"

Identifying Bottlenecks

- Importance of Bottleneck Detection:
 - Scaling might require changes to architecture or code
 - Without identifying the bottlenecks, optimizations are guesswork
- Monitoring and Data Collection:
 - Use tools like Amazon CloudWatch, New Relic (Heroku), or Stackdriver (Google) to monitor for bottlenecks.
 - Track disk I/O, network I/O, CPU usage, and memory performance



Common Bottlenecks in APIs

- Disk I/O:
 - Expensive database queries and frequent local disk access.
- Network I/O:
 - External service dependencies and cross-data-center API calls.
- CPU:
 - Inefficient code leading to resource-heavy computations.
- Memory:
 - Insufficient RAM causing slow performance.

Tools for Performance Insights

- Profiling and Monitoring:
 - Profile frequently called API methods to identify slow response times.
 - Performance profiling can detect CPU- or memory-intensive functions.
- Production vs. Development Profiling:
 - Production environments often reveal different issues from development.
 - Enable profiling for a small subset of production traffic to gain deeper insights.

Database Optimization

- Database Profiling:
 - Identify slow database queries using logs like MySQL's slow query log.
- Database Indexes:
 - Indexes help speed up queries by reducing the number of rows scanned.
 - Example: Indexing email addresses in a users table for faster lookups.
- Caution:
 - Over-indexing can lead to performance hits on write operations.

Caching for High Throughput

- Caching Overview:
 - Store frequently used data in memory (e.g., using Memcached) for faster access.
- Database Query Caching:
 - Cache frequent, slow database queries to reduce response times.
 - Check the cache before querying the database and store results for future use.
- Cache Invalidation:
 - Ensure cache is updated or invalidated when data changes to avoid stale results.

Asynchronous Operations

- Handling Expensive Operations:
 - Move time-consuming tasks outside the API request (asynchronously).
- Example:
 - In a file storage system, indexing files after upload can be done asynchronously to speed up response times.
- Task Queues:
 - Use task queue services (e.g., Amazon, Google Cloud, Celery) to manage asynchronous operations efficiently.

BITS WILP | | Generated: 12-May-2026 16:47:54

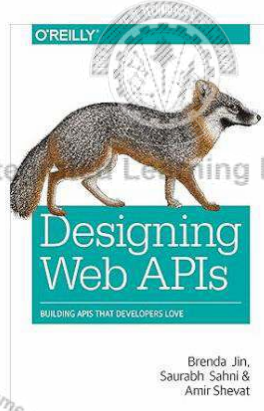
Scaling Throughput with More Resources

- Vertical Scaling:
 - Add more power (CPU, RAM, disk space) to existing servers.
- Horizontal Scaling:
 - Add more server instances and distribute the load via a load balancer.
- Techniques like database sharding and replication can further distribute the load.

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin , Saurabh Sahni , Amir Shevat



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Scaling throughput involves
 - identifying and addressing bottlenecks,
 - optimizing database performance,
 - using caching effectively,
 - and leveraging asynchronous operations.
- Always rely on data-driven insights and best practices to optimize your API's scalability.

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

- Which of the following is not a common bottleneck category when scaling APIs?

- A) Disk I/O
- B) Memory
- C) Data Compression
- D) CPU

Answer:

- C) Data Compression

MCQ2

- What is the primary purpose of database indexing in optimizing API throughput?

- A) To compress database rows for better storage
- B) To store database tables in memory for faster access
- C) To locate rows faster by reducing the number of rows compared in a query
- D) To ensure data is cached closer to the end users

Answer:

- C) To locate rows faster by reducing the number of rows compared in a query

MCQ3

- Which of the following is a recommended best practice for scaling API throughput?

- A) Always prioritize vertical scaling over horizontal scaling
- B) Add caching but skip cache invalidation to keep performance steady
- C) Profile frequently called API methods to identify bottlenecks
- D) Implement expensive operations synchronously to ensure faster responses

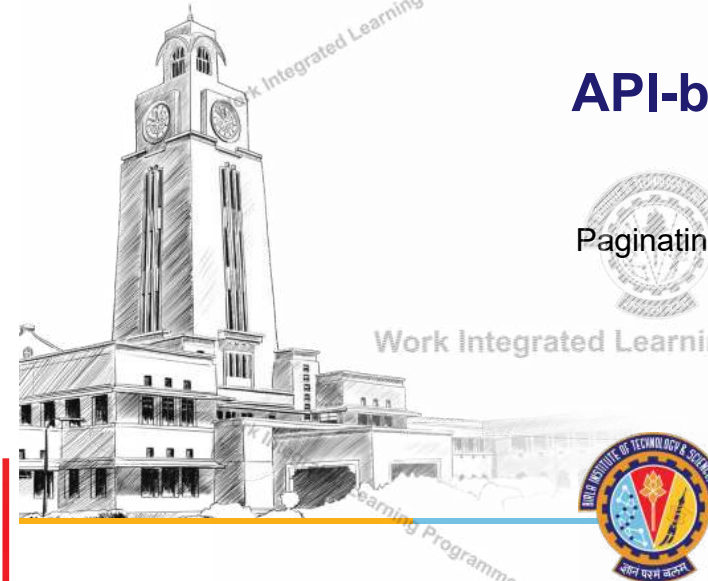
Answer:

- C) Profile frequently called API methods to identify bottlenecks

API-based Products

Paginating APIs

Suresh Gopal



Learning Outcomes

- differentiate between offset-based and cursor-based pagination,
- understand their advantages and disadvantages,
- and apply best practices for implementing efficient API pagination.

Introduction

- In today's digital age, APIs often need to handle large datasets.
 - A single API call can return thousands of items, which may overload the backend or slow down clients unable to process large data volumes.
- To mitigate this, pagination is essential—
 - it breaks large datasets into manageable chunks,
 - improving response times
 - and easing data handling for both clients and servers.

Two primary pagination techniques

- Offset-Based Pagination
- Cursor-Based Pagination

Offset-Based Pagination

- Concept:
 - Offset-based pagination is the most common and simplest technique to implement
- Uses two parameters:
 - Page Size:
 - Defines the maximum number of items to return.
 - Page Number:
 - Specifies the starting point for fetching data.

Offset-Based Pagination (Example)

- APIs like GitHub implement this technique by allowing clients to request data using page and per_page parameters in the URL.
 - `https://api.github.com/user/repos?page=5&per_page=10`

Advantages of Offset-Based Pagination

- Simplicity:
 - Easy to implement for both client and server
- User Experience:
 - Users can directly jump to a specific page without scrolling through all the content.

Disadvantages of Offset-Based Pagination

- Inefficiency with Large Datasets:
 - SQL queries with large offsets can be expensive since the database has to skip many rows before fetching the required data.
- Inconsistency:
 - If data changes frequently (items are added/removed), users may see duplicate or missed items.
- Scalability:
 - It can be challenging in distributed systems, especially for large offsets, where multiple database shards might need to be queried.
- Best Use Case:
 - When the dataset is small or static, and the client can tolerate slight inconsistencies.

Cursor-Based Pagination

- Concept:
 - To overcome the limitations of offset-based pagination, many modern APIs like Twitter, Facebook, and Slack use cursor-based pagination.
- Method doesn't rely on fixed page numbers but instead uses a cursor to mark the last item fetched.
 - Clients request the next set of items using this cursor.

Cursor-Based Pagination(Example)

- Using Twitter API to fetch followers:
 - GET
`https://api.twitter.com/1.1/followers/ids.json?screen_name=saurabhsahni&count=50`
- The response includes a `next_cursor` value to request the next page:
 - GET
`https://api.twitter.com/1.1/followers/ids.json?user_id=12345&count=50&cursor=1374004777531007833`

Advantages of Cursor-Based Pagination

- Performance:
 - Cursor-based pagination is more efficient, especially with large datasets, as queries can be optimized using indexed columns.
- Consistency:
 - Avoids the inconsistencies seen with offset pagination, ensuring each item is retrieved exactly once, even when items are added or removed.

Disadvantages of Cursor-Based Pagination

- Inability to Jump:
 - Clients cannot jump to a specific page directly
 - must traverse through each page sequentially.
- Complexity:
 - Implementing this technique can be more challenging for clients, as they need to store and use cursor values across requests.

Choosing Cursor Types

- Common cursor types include:
 - ID-based Cursors:
 - Uses unique identifiers (e.g., Twitter uses tweet IDs).
 - Timestamp-based Cursors:
 - Suitable for time-based data (e.g., Facebook uses timestamps).
 - Opaque Strings:
 - Encoded values that may include additional information such as IDs or pointers to database shards (used by Slack, GitHub, etc.).

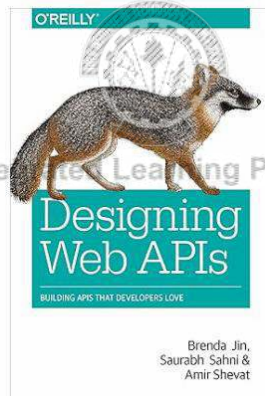
Pagination Best Practices

- Set Default and Maximum Page Sizes:
 - Define sensible defaults and limits to prevent excessive data retrieval.
- Optimize Sorting:
 - Sort data such that newer items appear first, reducing the need to paginate through older data.
- Ensure Backward Compatibility:
 - If introducing pagination later, maintain compatibility with older versions of your API.
- Avoid Sensitive Data in Cursors:
 - Ensure no sensitive information is embedded in cursors, as they can often be decoded by clients.

BITS WILP | Generated: 12-May-2026 16:47:54

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin , Saurabh Sahni , Amir Shevat



BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- Pagination is vital for efficient API performance, especially when dealing with large datasets.
- While offset-based pagination is simple and widely used, it struggles with large datasets and inconsistent results.
- On the other hand, cursor-based pagination is more robust and scalable but comes with added complexity.
- By choosing the appropriate pagination strategy and following best practices, API developers can significantly enhance both server performance and client experience.

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

- Which of the following is a key disadvantage of offset-based pagination?
 - A. It doesn't allow clients to jump to a specific page.
 - B. It is difficult to implement for both clients and servers.
 - C. It becomes inefficient when dealing with large datasets due to expensive SQL queries with large offsets.
 - D. It is not compatible with modern APIs like GitHub and Twitter.

Answer: C

Offset-based pagination becomes inefficient for large datasets because SQL queries need to skip a large number of rows before returning the desired result.

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ2

- What is a primary advantage of cursor-based pagination compared to offset-based pagination?

- A. It allows clients to jump to any arbitrary page.
- B. It offers better performance with large datasets by optimizing queries through indexes.
- C. It is easier to implement and does not require storing cursor values.
- D. It is used mainly for small, static datasets where data rarely changes.

Answer: B

Cursor-based pagination offers better performance for large datasets by efficiently retrieving results through indexed queries.

MCQ3

- Which of the following is a common type of cursor used in cursor-based pagination?

- A. Page Number
- B. Limit Offset
- C. SQL Shard Pointer
- D. Unix Timestamp

Answer: D

A Unix timestamp is a common choice for cursor-based pagination, particularly for time-based data, such as in news feeds or social media APIs.

API-based Products

API Deployment | Suresh Gopal

Learning Outcomes

- covers essential aspects of API deployment, focusing on processes, strategies, and best practices.

API Deployment

- API deployment refers to the process of **transferring an Application Programming Interface (API) from a development environment into a live production environment** where it can be accessed by client applications.
- Importance:
 - APIs serve as the backbone of modern applications,
 - facilitating communication between different software components
 - Effective deployment ensures that APIs are reliable, accessible, and maintainable.

Key Concepts in API Deployment

- Deployment Environment:
- Development:
 - A local or dedicated environment where APIs are created and initially tested.
- Staging:
 - A pre-production environment that closely mimics the production environment for final testing and quality assurance.
- Production:
 - The live environment where APIs serve end-users and must operate with high reliability and uptime.
- Environment Considerations:
 - Configuration, dependencies, and resource allocation often differ between environments.

The API Deployment Process

- Build:
 - Compile the code and create executable files, including dependencies.
- Test:
 - Conduct unit tests (testing individual components), integration tests (testing interactions between components), and regression tests (ensuring existing functionalities work post-change).
- Package:
 - Bundle the API, libraries, and dependencies for deployment. This often involves creating Docker containers or ZIP files.

The API Deployment Process

- Deploy:
 - Move the API to the staging environment for final validation.
- Release:
 - After successful testing in staging, promote the API to the production environment for end-users.

Continuous Integration and Continuous Deployment (CI/CD)

- Continuous Integration (CI):
 - Developers frequently integrate code changes into a shared repository, triggering automated builds and tests. This helps catch issues early.
- Continuous Deployment (CD):
 - Extends CI by automating the deployment of code changes to production. Each change that passes tests is automatically deployed, ensuring rapid delivery of features.
- Popular Tools:
 - Jenkins
 - GitLab CI
 - CircleCI
 - Travis CI

Deployment Best Practices

- Version Control:
 - Use systems like Git to track changes, manage code versions, and collaborate with team members.
- Environment Configuration:
 - Ensure that configurations (like API keys and database URLs) are environment-specific to avoid hardcoding sensitive information.

Deployment Best Practices

- Rollback Mechanism:
 - Prepare a strategy to revert to the previous stable version quickly in case of deployment failures.
- Documentation:
 - Maintain clear documentation on deployment processes, configurations, and any dependencies. This helps in troubleshooting and onboarding new team members.

Challenges in API Deployment

- Downtime:
 - Minimizing downtime is critical for user satisfaction. Consider strategies like blue-green deployment to mitigate this.
- Backward Compatibility:
 - Introducing new features without breaking existing clients is a challenge. Use versioning to manage changes.
- Coordination with Clients:
 - Effectively communicate changes to clients and provide them with necessary documentation and support for smooth transitions.

Summary

- API deployment is crucial for application functionality and user experience.
- Following best practices in deployment can lead to successful API launches.
- Continuous monitoring and feedback are essential for ongoing improvements.
- Next Steps:
 - Explore the importance of learning from each deployment and refining processes continuously.

MCQ1

What is the primary purpose of the staging environment in the API deployment process?

- A) To develop new features
- B) To serve live traffic
- C) To conduct final testing before moving to production
- D) To monitor API performance

Correct Answer: C) To conduct final testing before moving to production

MCQ2

Which deployment strategy minimizes downtime by maintaining two identical environments?

- A) Rolling Deployment
- B) Blue-Green Deployment
- C) Manual Deployment
- D) Automated Deployment

Correct Answer: B) Blue-Green Deployment

MCQ3

What is the main benefit of implementing Continuous Integration (CI) in the API deployment process?

- A) It automates the deployment to production.
- B) It allows for frequent integration of code changes and early detection of issues.
- C) It enables manual deployment processes.
- D) It eliminates the need for testing.

Correct Answer: B) It allows for frequent integration of code changes and early detection of issues.

API-based Products

API Deployment Patterns | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Introduction to API Deployment

- API deployment is the process of making an API available for use, whether internally or externally.
- Importance in modern software development
 - Effective API deployment is crucial for integrating services and enhancing user experiences.
- Overview of strategies
 - Today, we will explore various deployment strategies, their benefits, and best practices.

Learning Outcomes

- Understand various API deployment strategies, including on-premises, cloud, and hybrid approaches, and identify best practices for implementing secure, scalable, and efficient APIs.

Types of API Deployment Strategies

- On-Premises Deployment:
 - Hosted on local servers
 - Ideal for organizations needing complete control over their data and security.
- Cloud Deployment:
 - Hosted on cloud platforms (AWS, Azure, GCP)
 - Provides scalability and accessibility.
- Hybrid Deployment:
 - Combines both on-premises and cloud resources
 - Offering flexibility and redundancy.

Key Considerations for API Deployment

- Scalability:
 - Understand how your API will grow and choose infrastructure that can scale horizontally or vertically.
- Security:
 - Implement robust authentication (OAuth, JWT) and encryption methods to safeguard data.
- Performance:
 - Use techniques like load balancing and caching (Redis, Memcached) to improve response times.
- Monitoring:
 - Use tools like Prometheus or Grafana to monitor API performance and track errors.

BITS WILP | Generated: 12-May-2026 16:47:54

Deployment Automation

- Automation reduces the risk of human error and accelerates the deployment process
- CI/CD practices ensure that changes are automatically tested and deployed, allowing for faster iterations
- Tools like Jenkins or GitHub Actions streamline the build and deployment pipelines.

BITS WILP | Generated: 12-May-2026 16:47:54

Rollback Strategies

- Rollback strategies are essential to minimize downtime and disruption when issues arise.
- Blue-Green Deployment:
 - Maintain two environments (blue and green) to switch between stable and new versions.
- Canary Releases:
 - Gradually roll out the new version to a small percentage of users before full deployment.
- Feature Toggles:
 - Enable or disable features without deploying new code, allowing for safer testing.

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- As we've seen, choosing the right API deployment strategy is critical for success.
- Organizations should prioritize security, scalability, and user feedback to ensure effective deployment.
- Let's aim to implement these best practices in our own projects.

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

What is the primary advantage of using cloud deployment for APIs?

- A) Complete control over hardware
- B) Scalability and accessibility
- C) No dependency on internet connectivity
- D) Simplicity of installation

Answer: B) Scalability and accessibility

MCQ2

Which of the following strategies allows for gradual rollout of a new API version to a small percentage of users?

- A) Blue-Green Deployment
- B) Feature Toggles
- C) Canary Releases
- D) On-Premises Deployment

Answer: C) Canary Releases

MCQ3

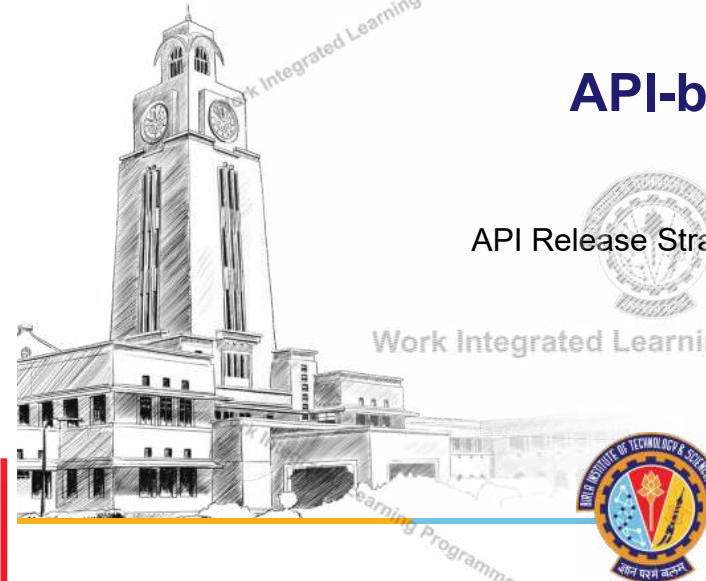
What is a key benefit of implementing Continuous Integration and Continuous Deployment (CI/CD) practices in API deployment?

- A) It eliminates the need for testing.
- B) It reduces the risk of human error during deployment.
- C) It guarantees that the API will never fail.
- D) It simplifies the coding process.

Answer: B) It reduces the risk of human error during deployment.

API-based Products

API Release Strategies | Suresh Gopal



Learning Outcomes

- provide a quick understanding of API release strategies, including versioning, deprecation, and best practices for effective deployment and management



Work Integrated Learning Programmes



Introduction

- What is API Deployment?
 - The process of pushing an API from a development environment to a production environment.
- What is API Release Management?
 - The practices and processes used to manage the release of new API versions, updates, and changes in a controlled and systematic way.
 - Focus on minimizing disruption and maximizing API availability during updates.



Work Integrated Learning Programmes



API Release Strategies

- Importance of Release Strategies
 - Maintain Backward Compatibility:
 - Allows existing clients to continue functioning without changes.
 - Ensure Minimal Disruption:
 - Users can transition at their own pace.
 - Facilitate Smoother Transitions:
 - Well-planned strategies reduce the risk of breaking changes affecting end-users.

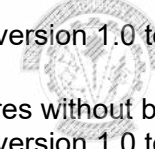


Work Integrated Learning Programmes



Common API Release Types

- Major Releases
 - Releases that introduce significant changes that may break backward compatibility.
 - Example: Transitioning from version 1.0 to 2.0.
- Minor Releases
 - Releases that add new features without breaking existing functionality.
 - Example: Transitioning from version 1.0 to 1.1, introducing new endpoints.
- Patch Releases
 - Releases that include backward-compatible bug fixes.
 - Example: Transitioning from version 1.0 to 1.0.1.



Work Integrated Learning Programmes



Versioning Strategies

- URI Versioning
 - Including the version number directly in the endpoint URL.
 - Example: /api/v1/resource
- Query Parameter Versioning
 - Specifying the version as a query parameter.
 - Example: /api/resource?v=1
- Header Versioning
 - Indicating version in HTTP headers.
 - Example: Accept: application/vnd.yourapi.v1+json
- Content Negotiation
 - Using media types to specify versions.
 - Example: Accepting application/vnd.yourapi.v1+json for version 1.

Semantic Versioning

- A versioning scheme that conveys meaning about the underlying changes.
 - Format: MAJOR.MINOR.PATCH
- Guidelines:
 - MAJOR version: For incompatible API changes.
 - MINOR version: For added functionality in a backwards-compatible manner.
 - PATCH version: For backwards-compatible bug fixes.

Deprecation Strategies

- Why Deprecate?
 - To phase out older features in favor of improved functionality or security.
- Deprecation Process:
 - Announcement: Notify users well in advance through various channels.
 - Migration Paths: Provide clear guidance on how to transition to new versions.
 - Grace Period: Allow a specific timeframe for users to adapt.

API Release Management Best Practices

- Blue-Green Deployment:
 - Keep two production environments (blue and green)
 - Direct traffic to one while updating the other.
- Canary Releases:
 - Gradually release new API versions to a small subset of users before full deployment.
- Automated Rollback:
 - Plan and automate rollback processes in case of deployment failure.

Best Practices for API Releases

- Maintain Backward Compatibility
 - Design new versions to support old clients.
- Automated Testing
 - Ensure that existing functionality remains intact with new releases.
 - Example: Use unit tests and integration tests.
- Documentation
 - Update API documentation with each release to reflect changes.
- Monitoring and Analytics
 - Analyze API usage and performance to identify potential issues after a release.

Summary

- The implementation of structured release strategies is crucial for the long-term success of APIs.
- By adopting best practices in versioning, deprecation, and CI/CD, organizations can enhance the quality, reliability, and user satisfaction of their APIs.
- As you develop and manage your APIs, consider how these strategies can be tailored to fit your specific needs.

MCQ1

What is the primary purpose of implementing a versioning strategy for APIs?

- A) To enhance the aesthetics of API documentation
- B) To ensure backward compatibility and minimize disruption for users
- C) To increase the complexity of API requests
- D) To limit the number of API consumers

Correct Answer: B) To ensure backward compatibility and minimize disruption for users

MCQ2

In Semantic Versioning, what does a change in the MAJOR version number signify?

- A) Backward-compatible new features have been added
- B) Backward-compatible bug fixes have been implemented
- C) Incompatible API changes have been introduced
- D) Minor enhancements or optimizations have been made

Correct Answer: C) Incompatible API changes have been introduced

MCQ3

Which of the following is a benefit of using Continuous Integration/Continuous Deployment (CI/CD) in API release strategies?

- A) It allows developers to work independently without collaboration
- B) It automates the testing and deployment processes, accelerating release cycles
- C) It increases manual testing requirements for every new feature
- D) It eliminates the need for versioning

Correct Answer: B) It automates the testing and deployment processes, accelerating release cycles

API-based Products

Designing API Evolutions | Suresh Gopal

Learning Outcomes

- Analyze the impact of design changes on existing APIs and evaluate strategies to manage those changes effectively to minimize disruption for API consumers.

Introduction

- “APIs are forever.” : Werner Vogels
- APIs are essential in today’s interconnected digital landscape.
- Once an API is deployed, it becomes a long-term commitment to your customers.
- Importance of mindful API design to accommodate future changes without causing disruption.

The Challenge of Managing Change

- Managing change is complex but necessary for API evolution.
- Key Challenges:
 - Developers may find change manageable within a single codebase using refactoring tools and automated tests.
 - Web-based APIs introduce additional challenges, especially with consumer relationships.

The Relationship with API Consumers

- Direct vs. Indirect Relationships:
 - Some teams have direct communication with API consumers, making gradual changes easier.
 - Most API consumers lack personal connections with the API team, complicating change management.

Strategies for Minimizing Impact

- Goal:
 - Minimize impact on customers to prevent churn.
- Considerations for Change:
 - Assess existing API designs.
 - Analyze the impact of proposed changes.
 - Maintain clear communication with consumers.

Understanding the Impact of Change

- Align-Define-Design-Refine (ADDR) Process:
 - Applicable for any organization, from startups to established enterprises.
 - Addresses customer, partner, and workforce needs.
 - Distinction between greenfield (new projects) and brownfield (existing APIs) development.

Conducting an API Design Gap Analysis

- Perform a gap assessment between the current API design and the ideal design.
- Factors to Consider:
 - Terminology differences.
 - Shift from data-centric to resource-centric design.
 - Changes in product vision and direction.

API Design Gap Analysis Steps

- Identify Differences:
 - Compare existing APIs to the ideal design from the ADDR process.
- Assess Impact:
 - Use t-shirt sizing (small, medium, large) to evaluate the value and impact of the changes.
- Determine Best Path:
 - Choose the approach that best serves API consumers.

Making API Design Decisions

- Evaluate the consequences of changes for both the organization and consumers.
- Questions to Consider:
 - Who are the API consumers?
 - What relationships have been established?
 - What value do changes provide to consumers?

Evaluating Consumer Impact

- Understand the implications of API changes:
 - Changes that enhance usability are generally well-received.
 - Changes may unlock new features but could also lead to customer churn.
 - Organizations must prioritize the needs of API consumers to maintain trust.

Strategies for Implementing Change

- Compromise in Design:
 - Continue using existing designs with minor adjustments (e.g., support for old formats).
 - Gradual transition through deprecation strategies, allowing consumers to adapt.

Assessing Design Levels

- Design Levels:
 - Low-level APIs expose database structures directly.
 - High-level APIs focus on outcome-based functionality.
 - Balancing low and high-level APIs is essential to prevent cognitive dissonance among developers.

Options for New Designs

- Choices for Introducing New Designs:
 - Add new design elements to existing APIs.
 - Create a new API as a separate product.
 - Release a new version of the current API.

Resource Considerations

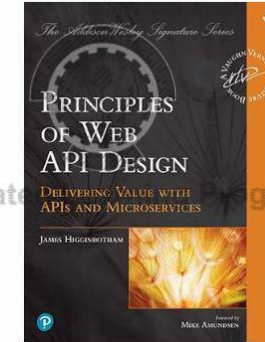
- Consider the resources needed to maintain multiple APIs or versions.
- Effective change management is crucial to ensure smooth transitions.

Building Trust Through Change Management

- Align business and API teams for a unified understanding of design goals.
- Maintain open communication with API consumers to foster trust.
- Principle : Thoughtful API design and an evolutionary approach are key to resilience against change.

Reference

- Principles of Web API Design
 - By James Higginbotham



Summary

- Mindful API design is essential for long-term success.
- Managing change effectively protects relationships with consumers and supports API longevity.
- Final Thought:
 - APIs are forever; careful planning today leads to smoother transitions tomorrow.

MCQ1

What is the primary challenge when managing changes to Web-based APIs?

- A) Lack of documentation
- B) Consumer relationship dynamics
- C) Limited programming languages available
- D) Insufficient server capacity

Correct Answer: B) Consumer relationship dynamics

MCQ2

Which process is recommended for assessing the impact of change on API design?

- A) Create a new API from scratch
- B) Align-Define-Design-Refine (ADDR)
- C) Implement versioning strategies
- D) Perform a market analysis

Correct Answer: B) Align-Define-Design-Refine (ADDR)

MCQ3

What is a key consideration when performing a gap analysis on existing API designs?

- A) The programming languages used
- B) Differing terminology and design styles
- C) The speed of the API response
- D) The number of API consumers

Correct Answer: B) Differing terminology and design styles

API-based Products

API Versioning

Suresh Gopal

Learning Outcome

- Understand the principles and best practices of API versioning

API Versioning

- API versioning is the process of managing changes to an API in a way that allows developers to update and improve the API without breaking existing functionality for users.
- Importance of Versioning:
 - ensures backward compatibility, allowing existing clients to continue functioning as expected.
 - facilitates ongoing development, enabling the addition of new features or improvements.

Why Version an API?

- Reasons for Versioning:
 - Introducing New Features:
 - Adding enhancements or new endpoints.
 - Fixing Bugs or Vulnerabilities:
 - Addressing issues that affect performance or security.
 - Changing Existing Functionality:
 - Modifying features that may not work as intended (breaking changes).
- Impact on Clients:
 - Protects existing users from disruption and provides a stable experience.

Types of API Versioning

- URI Versioning
- Query Parameter Versioning
- Header Versioning
- Media Type Versioning

URI Versioning

- Version is included in the URL.
- Example:
 - GET /api/v1/users vs. GET /api/v2/users
- Pros:
 - Clear and easy to implement.
 - Easy to cache and bookmark for users.
- Cons:
 - Can lead to cluttered URLs if not managed properly.
 - Requires significant changes in clients if the base URL changes.

Query Parameter Versioning

- Version is passed as a query parameter.
- Example:
 - GET /api/users?version=1 and GET /api/users?version=2
- Pros:
 - Allows for multiple versions in a single endpoint.
 - Simple for clients to implement without changing URLs.
- Cons:
 - Less intuitive for users since the version is not visibly part of the URL.
 - May complicate caching strategies as caches might need to differentiate by query parameter.

Header Versioning

- Version is specified in the request headers.
- Example:
 - GET /api/users
 - Accept: application/vnd.yourapi.v1+json
- Pros:
 - Clean URLs without visible version numbers, making it easier for developers.
 - Can handle more complex versioning logic, allowing for fine-grained control.
- Cons:
 - Requires clients to understand and set headers correctly.
 - Less visible to users or stakeholders.

Media Type Versioning

- Version is indicated in the media type.
- Example:
 - GET /api/users
 - Accept: application/vnd.yourapi.v1+json
- Pros:
 - Allows for different representations of the same resource, providing flexibility.
 - Maintains a clean URL structure, avoiding clutter.
- Cons:
 - More complex to implement and understand.
 - Clients must handle media types properly, which may lead to confusion.

Best Practices for API Versioning

- Plan for Versioning Early:
 - Consider potential future changes during the initial API design phase.
- Use Semantic Versioning:
 - Follow the principles of semantic versioning (major.minor.patch) to communicate changes clearly.
- Deprecation Strategy:
 - Have a clear plan for deprecating old versions, including timelines and communications.
- Documentation:
 - Maintain comprehensive and up-to-date documentation for all API versions to aid developers.

Tools for API Versioning

- API Management Platforms:
 - Tools like Postman, Swagger, and Apigee that assist in managing and documenting API versions.
- Versioning Libraries:
 - Libraries and frameworks that facilitate version management (e.g., Spring API versioning).

Summary

- Importance of planning for versioning from the start.
- Understanding the different versioning strategies and their trade-offs.
- Effective communication of changes to clients to ensure a smooth transition.

MCQ1

What is the primary purpose of API versioning?

- A) To increase the size of the API documentation
- B) To ensure backward compatibility while allowing for changes and improvements
- C) To limit the number of users accessing the API
- D) To make APIs more complex and difficult to use

Answer: B) To ensure backward compatibility while allowing for changes and improvements

MCQ2

Which of the following is a common method for API versioning that includes the version in the URL?

- A) Query Parameter Versioning
- B) Media Type Versioning
- C) Header Versioning
- D) URI Versioning

Answer: D) URI Versioning

MCQ3

What is a recommended best practice when implementing API versioning?

- A) Avoid any form of versioning to keep the API simple
- B) Plan for versioning early during the API design phase
- C) Use different APIs for each version without any documentation
- D) Make changes without informing clients about deprecations

Answer: B) Plan for versioning early during the API design phase

API-based Products

API Versioning Strategies | Suresh Gopal

Learning Outcomes

- Understand the principles of API versioning strategies to effectively manage changes while minimizing disruption to existing consumer

Understanding APIs as Contracts

- APIs serve as contracts between providers and consumers.
- Ideal Scenario:
 - In an ideal world, these contracts remain unchanged.
- Reality:
 - Changes may be necessary at times, which could lead to breaking changes that require API consumers to modify their code.

Importance of Nonbreaking Changes

- Objective:
 - When changes are essential, it's crucial to avoid breaking existing integrations.
- Impact:
 - For many consumers, adapting their code to accommodate API changes may not be feasible.
- Understanding Changes:
 - Recognizing what constitutes a breaking change helps establish a versioning policy that supports API evolution without disrupting existing consumers.

Common Nonbreaking Changes

- Nonbreaking changes are typically additive, including:
 - Adding New API Operations:
 - New operations do not impact existing client code.
 - Adding Optional Fields:
 - Clients aren't forced to include new fields.
 - Default Values for Required Fields:
 - Clients using older code will receive default values from the server.
 - Enhancing Response Messages:
 - New fields in responses can be ignored by existing clients unless they use incompatible mapping libraries.
 - Expanding Enumeration Types:
 - New enum values should not disrupt older clients if designed to handle them.

Identifying Incompatible Changes

- Incompatible changes may include:
 - Renaming Fields/Paths:
 - Requires updates in existing client code.
 - Removing Fields:
 - Affects clients relying on those fields.
 - Altering Relationships:
 - Changing from single to multiple relationships necessitates code modifications.
 - Modifying HTTP Methods or Response Codes:
 - Can break existing integrations.

API Versions vs. Revisions

- API Versions:
 - Group API operations and ensure backward compatibility within a version.
 - Each version is treated as a separate product, allowing consumers to migrate at their own pace.
 - Versions can be numeric or date-based (e.g., v1 or 2024-01-01).
- API Revisions:
 - Indicate internal enhancements that do not impact consumers.
 - Revisions are transparent
 - Consumers are unaware of them but can refer to changelogs for updates.

Business Considerations in API Versioning

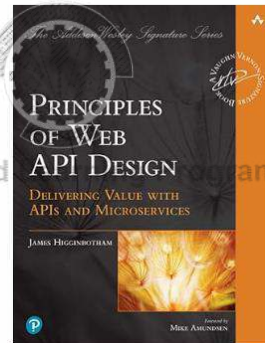
- Migration Decisions:
 - Clients weigh the cost of migration against benefits.
- Impact on Customer Retention:
 - Frequent breaking changes can lead to customer churn.
- Legacy Support:
 - Previous API versions must often be maintained, adding to costs and complexity.
- Infrastructure Implications:
 - Each new API version functions like a new product requiring additional resources.

Summary

- Strategic Importance:
 - Effective API versioning strategies mitigate risks and enhance the longevity of the API.
- Long-term View:
 - Balancing design perfection with customer needs is essential to avoid unnecessary churn.

Reference

- Principles of Web API Design
 - By James Higginbotham



MCQ1

What is the primary purpose of API versioning?

- A) To increase the complexity of the API
- B) To maintain backward compatibility while allowing for API evolution
- C) To make APIs easier to use
- D) To eliminate the need for documentation

Answer: B) To maintain backward compatibility while allowing for API evolution

MCQ2

Which of the following is considered a nonbreaking change?

- A) Renaming fields in the API
- B) Removing an API operation
- C) Adding an optional field to a request message
- D) Changing the HTTP method of an API operation

Answer: C) Adding an optional field to a request message

MCQ3

What does an API revision indicate?

- A) A complete overhaul of the API design
- B) An internal enhancement with no negative impact on consumers of the specific API version
- C) A mandatory update that all consumers must implement
- D) A new version that introduces breaking changes

Answer: B) An internal enhancement with no negative impact on consumers of the specific API version

API-based Products

Rate-Limiting APIs | Suresh Gopal

Learning Outcomes

- Understand the importance of API rate-limiting, its implementation strategies, and best practices for maintaining application reliability and preventing abuse.

Introduction to Rate-Limiting in APIs

- Why is Rate-Limiting Important?
 - Protects Infrastructure:
 - Safeguards your application from overload, ensuring reliability and availability.
 - Prevents Abuse:
 - Helps prevent misuse, such as denial-of-service (DoS) attacks or mass user registrations.
 - Maintains Developer Trust:
 - Ensures that your product is available and functional for all users and developers.

What is Rate-Limiting?

- Definition:
 - Rate-limiting controls the number of API requests a client can make within a specific time period.
- Example:
 - GitHub's API allows up to 5,000 requests per hour
- Purpose:
 - To prevent excessive traffic and protect your API from crashing due to a surge in requests.

Developing a Rate-Limiting Policy

- Key Considerations:
- Global vs. Granular Rate Limits:
 - Global: A single rate limit for all API endpoints.
 - Granular: Specific rate limits for individual API endpoints based on resource consumption.
- Rate-Limit per User, App, or IP:
 - Based on the API's authentication method (per-user, per-application, or per-IP).

Developing a Rate-Limiting Policy

- Handling Occasional Traffic Bursts:
 - Support bursts beyond the sustained limit for specific use cases.
- Allowing Exceptions:
 - Provide higher rate limits to trusted developers or partners upon valid requests.

Rate-Limiting Algorithms

- Token Bucket:
 - Allows traffic bursts while maintaining a steady rate.
 - Example: Slack, Stripe.
- Fixed-Window Counter:
 - Limits requests to a fixed number within a set time interval.
 - Example: Twitter.
- Sliding-Window Counter:
 - Tracks traffic within a moving window of time to handle bursts.
 - Example: Instagram.

Token Bucket Algorithm: How it Works?

- Concept:
 - A “bucket” of tokens refills at a fixed rate.
 - Each request removes tokens, and when the bucket is empty, requests are denied.
- Benefits:
 - Ideal for APIs that need to handle occasional traffic surges.
- Example:
 - Rate-limit of 20 requests/minute, with bursts up to 50 requests allowed.

Fixed-Window Counter Algorithm

- How it Works:
 - Limits requests in fixed intervals (e.g., 20 requests per minute).
- Limitations:
 - Can allow bursts at the boundary of two intervals, potentially exceeding the rate limit.
- Use Case:
 - Simple to implement, used by APIs like Twitter.

Sliding-Window Counter Algorithm

- How it Works:
 - Divides a time window into smaller buckets, tracking requests in each.
- Advantage:
 - Prevents large traffic bursts by summing recent buckets.
- Example:
 - Used by Instagram for steady traffic control.

Making Developers' Lives Easier

- Best Practices for Developer-Friendly Rate-Limiting:
- HTTP 429 Status Code:
 - Inform developers when rate limits are exceeded.
- Custom Response Headers:
 - X-RateLimit-Limit: Max allowed requests.
 - X-RateLimit-Remaining: Requests left in the current interval.
 - X-RateLimit-Reset: Time when the limit resets.
 - Rate-Limit Status API: Provide an API to check rate-limit status.

Documenting Rate Limits

- Why Documentation Matters:
 - Helps developers understand rate-limit policies upfront.
 - Provides guidance on handling rate limits in applications.
- Examples:
 - GitHub, Twitter, and Facebook offer detailed documentation on rate limits.

Rate-Limiting Best Practices

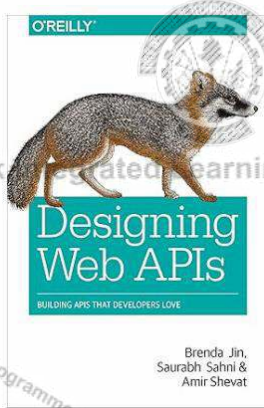
- Key Tips:
 - Choose the algorithm that suits your traffic patterns.
 - Ensure common use cases aren't heavily rate-limited.
 - Offer exceptions for trusted partners, but assess the impact on infrastructure.
 - Start with lower thresholds and adjust as necessary.
 - Implement exponential back-off for smoother API retries after hitting limits.

Summary

- Rate-limiting protects your API's infrastructure and product from misuse.
- Choosing the right algorithm and policies ensures both application reliability and developer satisfaction.
- Clear documentation and developer-friendly features help prevent confusion and unnecessary friction.

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin , Saurabh Sahni , Amir Shevat



BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

What is the primary purpose of implementing rate-limiting in APIs?

- a) To improve the API's response time
- b) To control the number of users accessing the API
- c) To protect infrastructure and prevent abuse
- d) To reduce API development costs

Answer: c) To protect infrastructure and prevent abuse

MCQ2

Which of the following algorithms allows occasional traffic bursts while maintaining a steady limit on the API's traffic?

- a) Sliding-window counter
- b) Token bucket
- c) Fixed-window counter
- d) Linear bucket

Answer: b) Token bucket

MCQ3

Which HTTP status code should be returned when a developer exceeds the API rate limit?

- a) 200 OK
- b) 403 Forbidden
- c) 404 Not Found
- d) 429 Too Many Requests

Answer: d) 429 Too Many Requests

API-based Products

Deprecating APIs | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Overview of API Deprecation

- API deprecation refers to the process of phasing out an API operation or product, signaling to consumers that it will no longer be supported in the future.
- Impact of Sudden API Shutdowns
 - Sudden shutdowns can lead to:
 - Increased downtime for applications relying on the API.
 - Significant time and resource loss as teams scramble to find alternatives.
 - Frustration and loss of trust among developers and consumers.



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Equipped with best practices for establishing effective deprecation policies and communication strategies that maintain consumer trust during API transitions.



Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

The Importance of a Deprecation Policy

- Building Consumer Trust
- Trust as a Foundation:
 - A well-defined deprecation policy establishes a foundation of trust between API providers and consumers.
 - Consumers are more likely to continue using an API if they feel informed and supported during changes.



BITS WILP | | Generated: 12-May-2026 16:47:54

Key Benefits of a Deprecation Policy

- **Transparency:**
 - Clear guidelines help consumers understand the timeline and reasons for deprecation.
 - Reduces uncertainty and enhances consumer confidence in the API provider.
- **Smooth Transition:**
 - A structured policy allows consumers to plan and implement necessary changes.
- **Enhanced Collaboration:**
 - Maintaining open lines of communication fosters collaboration and feedback between providers and consumers.

Key Components of a Deprecation Policy

- **Documentation Standards**
 - Document the deprecation policy within API program standards.
- **Policy Elements**
 - When deprecations are permitted.
 - Steps to follow during the deprecation process.
 - Minimum deprecation duration before retirement.
 - Establishing a migration path for consumers.
 - Clear definitions in the API's terms of service.

Benefits of a Deprecation Policy

- **Enhanced Preparedness**
 - Organizations with a solid deprecation policy are better positioned to manage API changes.
- **Maintaining Trust**
 - Open communication and clear guidelines ensure continued trust from API consumers.

Announcing a Deprecation

- **Significance of Communication**
 - Effective communication is critical for maintaining trust during deprecation.
- **Communication Methods**
 - Well-crafted emails detailing the changes.
 - Notification banners on web dashboards.
 - Alerts in API documentation.
 - Blog posts or dedicated landing pages addressing FAQs.
 - Frequent social media updates linking to official announcements.

Ensuring Effective Communication

- Multi-Channel Strategy
 - Utilize various communication methods to reach all consumers.
 - Recognize that employee turnover may impact email communications.



Work Integrated Learning Programmes

Technical Implementation

- Using OpenAPI Descriptions
 - Implement the deprecated: true indicator for HTML documentation warnings.
- GraphQL and gRPC Considerations
 - Similar provisions exist for their schema and IDL formats.



Work Integrated Learning Programmes

Sunset Header RFC

- Programmatic Notifications
 - Use the Sunset Header RFC to inform consumers about retirement timelines.
 - Include guides in API documentation for using the Sunset Header effectively.



Work Integrated Learning Programmes

Logging and Alerts

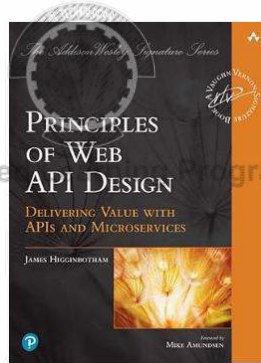
- Implementing Warnings in Helper Libraries
 - Add deprecation warnings in logs or console messages.
 - Ensure backend integrations alert monitoring dashboards about deprecated APIs.



Work Integrated Learning Programmes

Reference

- Principles of Web API Design
 - By James Higginbotham



Summary

- Establishing a clear deprecation policy and effective communication strategy is vital.
- Prioritizing consumer trust and transparency during the deprecation process leads to better outcomes.

MCQ1

What is the primary purpose of establishing a deprecation policy for APIs?

- A) To increase the complexity of the API
- B) To maintain consumer trust and facilitate smooth transitions
- C) To eliminate the need for API documentation
- D) To ensure APIs are always available

Answer: B) To maintain consumer trust and facilitate smooth transitions

MCQ2

Which of the following is NOT a recommended method for announcing an API deprecation?

- A) Sending well-written emails
- B) Posting a blog about the deprecation
- C) Ignoring consumer feedback
- D) Adding notification banners on web dashboards

Answer: C) Ignoring consumer feedback

MCQ3

What technical implementation can be used to inform consumers about deprecated APIs in OpenAPI descriptions?

- A) Adding a new API endpoint
- B) Using the deprecated: true indicator
- C) Changing the API version
- D) Removing documentation altogether

Answer: B) Using the deprecated: true indicator

API-based Products

API Management capabilities | Suresh Gopal

Learning Outcomes

- introduce the fundamental concepts of API management platforms, detailing their key services, benefits, and strategic importance for modern enterprises.

Meeting Modern Customer Expectations

- Current Landscape:
 - Customers demand access to enterprise data and services through multiple digital devices and channels.
- Enterprise Challenge:
 - To meet these expectations, enterprises need to open their assets in an agile, flexible, secure, and scalable manner.
- Key Solution:
 - APIs serve as a window into an enterprise's data and services, allowing seamless communication between applications.

Managing APIs Effectively

- API Management Need:
 - Once an API is created, it requires effective management to ensure it serves internal, partner, and external developers efficiently.
- Core Functions of API Management Platforms:
 - Developer Engagement:
 - Engage developers to use APIs effectively.
 - Business Insights:
 - Gain insights through usage analytics.
 - Security & Protection:
 - Ensure secure and reliable data access.
 - Scalability:
 - Support growing demand across digital channels.

BITS WILP | Generated: 12-May-2026 16:47:54

Core Capabilities of an API Management Platform

- An API management platform enables you to:
 - Developer Enablement:
 - Provide tools and resources to help developers easily use APIs.
 - Secure, Reliable, and Flexible Communication:
 - Ensure robust data transfer while maintaining security.
 - Lifecycle Management:
 - Oversee APIs from creation to retirement.
 - API Auditing & Analytics:
 - Track usage, monitor performance, and ensure compliance.

BITS WILP | Generated: 12-May-2026 16:47:54

Benefits of an API Management Platform

- Accelerated Digital Outreach:
 - Expand the reach across multiple digital platforms and devices.
- Partner Adoption:
 - Foster partnerships by offering accessible APIs.
- Monetization of Digital Assets:
 - Generate revenue by exposing valuable services via APIs.
- Analytics for Optimization:
 - Use insights to refine and enhance digital transformation efforts.

BITS WILP | Generated: 12-May-2026 16:47:54

Core Capabilities of an API Management Platform

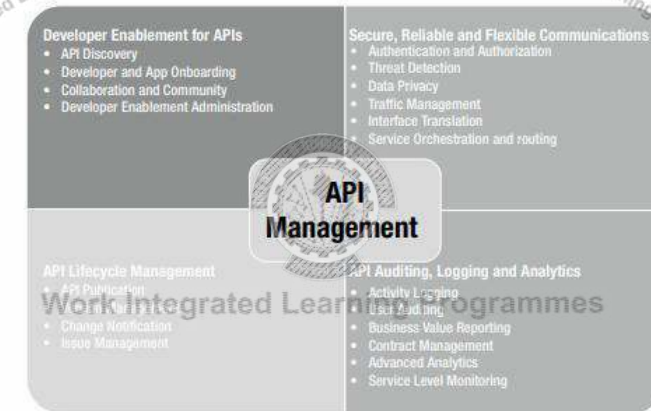


Figure 2-2. API management capabilities

Adapted from "API Management – An Architect's Guide to Developing and Managing APIs for your Organization"

BITS WILP | Generated: 12-May-2026 16:47:54

Delivery Models of API Management Platforms

- Cloud-Based Service:
 - Delivered by vendors in a public cloud, offering scalability and ease of access.
- On-Premise Deployment:
 - Deployed in private clouds for greater security and control.
- Hybrid Approach:
 - Combines hosted services and on-premise solutions for flexibility and security.

Three Key Services of an API Management Platform

- API Gateway Services:
 - Manage APIs created from existing services.
 - Add security, traffic management, and routing features.
- Analytics Services:
 - Monitor API traffic, app performance, and developer engagement.
 - Provide operational insights and business metrics.
- Developer Portals:
 - Facilitate developer onboarding, API documentation, and community management.
 - Enable API monetization through easy access for third-party developers.

Three Key Services of an API Management Platform

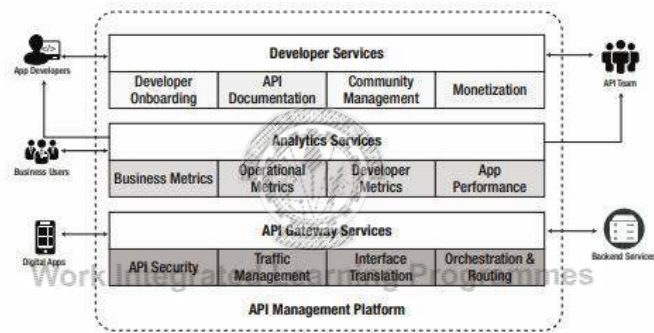
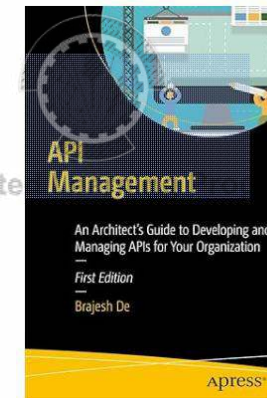


Figure 2-3. API management platform services

Reference

- API Management – An Architect's Guide to Developing and Managing APIs for your Organization
 - by Brijesh De



Summary

- The Role of API Management in Digital Transformation
 - Digital Enablement:
 - API management platforms are crucial for enabling businesses to offer seamless digital services across multiple channels.
 - Strategic Importance:
 - APIs, when managed effectively, unlock unique enterprise assets and support continuous growth in the digital economy.

Next Steps: Understanding API Management Services

- Explore how API gateway, analytics, and developer portals work together to enhance digital services.
- Learn about the lifecycle of APIs and best practices for API security, scalability, and performance optimization.

MCQ1

What is the primary role of an API in modern digital services?

- A) To store enterprise data
- B) To enable communication between applications and backend systems
- C) To secure enterprise networks
- D) To monitor digital channels

Answer:

- B) To enable communication between applications and backend systems

MCQ2

Which of the following is NOT a core capability of an API management platform?

- A) Developer Enablement
- B) Traffic Management
- C) API Lifecycle Management
- D) Product Inventory Control

Answer:

- D) Product Inventory Control

MCQ3

Which service in an API management platform helps in monitoring API traffic and providing business insights?

- A) Developer Portals
- B) API Gateway Services
- C) Analytics Services
- D) Security Services

Answer:

- C) Analytics Services

API-based Products

API Auditing, Logging and Analytics | Suresh Gopal

Learning Outcomes

- Understand the key components and processes of API lifecycle management, including API creation, publication, version management, change notification, and issue management, to ensure effective API development and deployment

API Lifecycle Management

- Definition:
 - API lifecycle management encompasses the processes involved in controlling how an API is developed, deployed, and consumed.
- Purpose:
 - Enables developers to interact with APIs, allowing them to build applications, report issues, and request new features.

Key Capabilities of API Lifecycle Management

- API Creation
- API Publication
- Version Management
- Change Notification
- Issue Management



Work Integrated Learning Programmes



API Creation

- API as a Facade:
 - APIs serve as an interface between users and backend services.
- API Proxy:
 - Acts as a secure facade for exposing backend services to users.
- Policies:
 - API proxies should support policies for:
 - Security
 - Traffic Management
 - Message Translation
 - Encryption & Filtering
 - Caching & Orchestration
 - Routing
- Testing & Deployment:
 - After design and development, APIs must be tested using built-in consoles to streamline the process before publishing.



Work Integrated Learning Programmes

API Publication

- Publishing for Discovery:
 - After creation, APIs must be deployed to environments where they can be consumed.
- Developer Portal:
 - API specifications and details are shared here for developers to discover and use in applications.
- Rollbacks:
 - If there are issues with deployment, rolling back to previous versions should be easily managed.



Work Integrated Learning Programmes



Version Management

- Evolution of APIs:
 - APIs need to adapt to evolving business needs, requiring multiple versions to exist simultaneously.
- Versioning Features:
 - Support for older versions for existing users
 - Ability to deprecate and retire older versions smoothly.
 - Deprecated APIs continue to serve existing traffic but prevent new sign-ups.
- Retirement:
 - Deprecated APIs should be retired with ample notice to avoid maintenance overheads.



Work Integrated Learning Programmes



Change Notification

- Impact of Changes:
 - API changes may affect existing consumers, so notifications are essential.
- Mechanisms:
 - Notifications can be sent via:
 - Email
 - SMS
 - Social media
- Notification Types:
 - Release updates (new features or releases)
 - Planned or unplanned downtime
 - Deprecation alerts



BITS WILP | | Generated: 12-May-2026 16:47:54

Issue Management

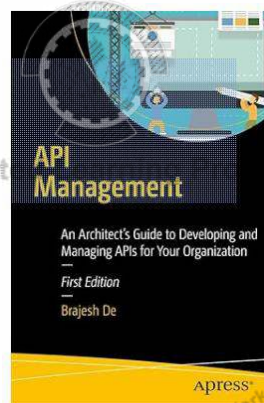
- Logging Issues:
 - Consumers should have a platform to log issues or report bugs related to the API.
- Support Tickets:
 - Developers should be able to raise tickets for assistance.
- Integration:
 - The API management platform should integrate with enterprise systems to support seamless issue management.



BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

- API Management – An Architect's Guide to Developing and Managing APIs for your Organization
 - by Brijesh De



BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Key Takeaways:
 - API lifecycle management ensures smooth creation, deployment, and version control.
 - Proper change notifications and issue management foster better collaboration between API providers and consumers.
 - An efficient API management platform boosts development speed, security, and consumer satisfaction.



BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

What is the primary purpose of an API proxy in API lifecycle management?

- A) To create a database for API data storage
- B) To serve as a secure facade for exposing backend services to consumers
- C) To manage user authentication and authorization
- D) To monitor API performance

Answer: B) To serve as a secure facade for exposing backend services to consumers

MCQ2

Which of the following is NOT a feature supported by policies attached to an API proxy?

- A) Security
- B) Traffic Management
- C) User Interface Design
- D) Caching

Answer: C) User Interface Design

MCQ3

When an API version is marked as deprecated, what should happen to new consumers?

- A) They can continue to use the deprecated API without restrictions
- B) They should be informed and prevented from signing up for the deprecated API
- C) They will receive automatic updates to the deprecated API
- D) They must use the deprecated API for all applications

Answer: B) They should be informed and prevented from signing up for the deprecated API

API-based Products

Developer Enablement for APIs | Suresh Gopal



Learning Outcomes

- Understand the importance of API auditing, logging, and analytics in optimizing API performance and business value.



Work Integrated Learning Programmes

API Insights

- Businesses need insights into their API programs to drive the right investments.
- Understanding API usage, who uses it, and its value is key to decision-making.
- This information helps enhance API performance and optimize business value.



Work Integrated Learning Programmes

The Role of API Gateways

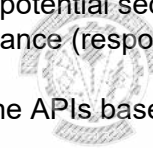
- API Gateways:
 - Serve as a central hub for API traffic.
 - Provide tools to measure, monitor, and report API usage.
 - Enable dashboards for API analytics and monetization reports to assess business value.



Work Integrated Learning Programmes

Importance of API Analytics

- API Analytics:
 - Offers insight to make informed decisions about API development.
 - Helps identify the success or potential security risks (e.g., malicious traffic).
 - Tracks API adoption, performance (response time, throughput), and global usage.
 - Enables API providers to refine APIs based on usage metrics.



Work Integrated Learning Programmes

Key API Analytics Questions

- Key Metrics:
 - Is API traffic increasing, and why?
 - How well is the developer community adopting the API?
 - What is the API's performance (response time, errors)?
 - What devices and regions are accessing the API?
 - How are APIs being used in apps?

Activity Logging

- Activity Logging:
 - Logs detailed information for each API request.
- Tracks:
 - Who is using the API.
 - What apps/devices are calling the API.
 - Geographical location of API calls.
- Captures metadata like IP address, HTTP method, URI, and response times.
- Provides insights into API performance metrics and error rates.

User Auditing

- User Auditing:
 - Helps review historical data on API usage.
- Tracks:
 - Who accessed the API.
 - When and how the API was used.
 - Frequency of API calls by different users.
- Helps identify usage trends and potential issues.

Business Value Reports

- Monetization Reports:
 - Measures revenue generated by APIs.
 - Supports both direct and indirect monetization models.
 - Tracks engagement metrics like:
 - Number of unique users.
 - Number of registered developers.
 - Number of active apps using the API.
 - Assesses the overall business value of API programs.

Advanced Analytics

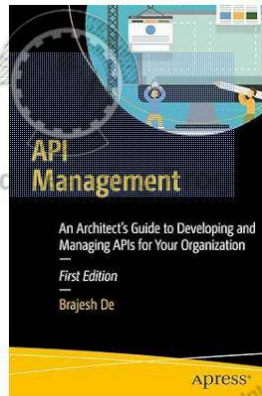
- Custom Analytics:
 - The platform can log custom variables for advanced reporting.
 - Enables API administrators to create custom reports.
 - Helps identify opportunities for API improvement and expansion.

Service-Level Monitoring

- Performance Monitoring:
 - Tracks latency across both the platform and backend systems.
 - Provides insights into performance bottlenecks.
 - Offers detailed reports on API errors, classified by:
 - Type.
 - Frequency.
 - Severity.
 - Helps troubleshoot and resolve issues effectively.

Reference

- API Management – An Architect's Guide to Developing and Managing APIs for your Organization
 - by Brijesh De



Summary

- API Auditing and Analytics:
 - Crucial for optimizing API performance, security, and business value.
 - Provides the necessary tools for informed decision-making.
 - Empowers businesses to continuously improve their API programs based on real-time data.

MCQ1

What is the primary purpose of API analytics?

- A) To monitor network performance
- B) To gain insights into API usage and performance for informed decision-making
- C) To restrict unauthorized access to the API
- D) To automate API deployment processes

Answer: B) To gain insights into API usage and performance for informed decision-making

MCQ2

Which of the following is typically tracked by activity logging in an API management platform?

- A) The color scheme of the API dashboard
- B) The IP address and device type of the client accessing the API
- C) The salary of developers working on the API
- D) The number of pages accessed by users on the website

Answer: B) The IP address and device type of the client accessing the API

MCQ3

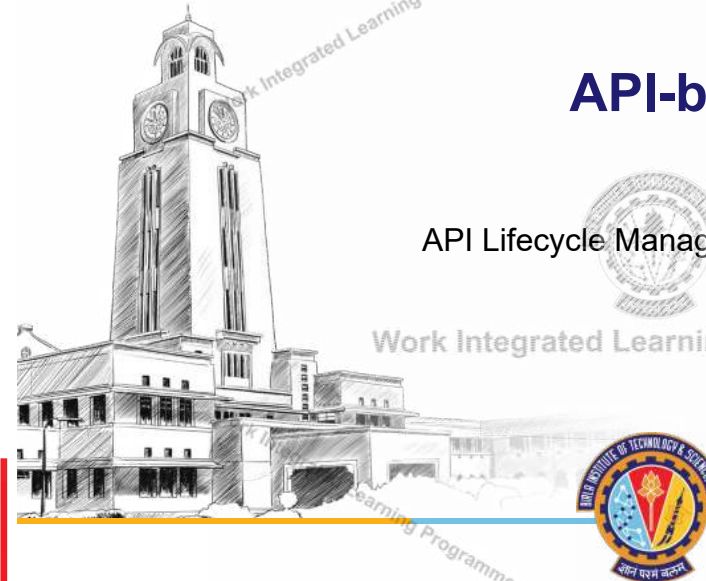
What do business value reports in an API management platform primarily measure?

- A) The aesthetic appeal of API documentation
- B) The monetary and engagement value generated by the API program
- C) The API's storage and memory usage
- D) The number of errors occurring in the API

Answer: B) The monetary and engagement value generated by the API program

API-based Products

API Lifecycle Management | Suresh Gopal



Learning Outcomes

- Understand the key components of developer enablement for APIs, including the role of a developer portal, onboarding processes, documentation, and community management to drive API program success.



Work Integrated Learning Programmes



Developer Enablement for APIs

- Key Idea:
 - The success of an API program relies heavily on the engagement and support of the developer community.
- Overview:
 - Developers use APIs to create mobile apps or integrate multiple applications.
 - To succeed, they need access to the right tools, documentation, and a supportive environment.



Work Integrated Learning Programmes



Importance of Developer Involvement

- Application Developers:
 - Build custom integrations.
 - Create apps using APIs.
- Key Requirements:
 - Awareness of available APIs.
 - Understanding of functionalities.
 - Access to a "playground" for testing APIs.



Work Integrated Learning Programmes



Developer Enablement Platform Essentials

- API Management Platform:
 - Should provide essential services that enable developers to effectively build applications.
- Developer Portal:
 - A crucial component that offers documentation, support, and a community space for developers to collaborate.



Work Integrated Learning Programmes



What is a Developer Portal?

- Definition:
 - A developer portal is a customized website that acts as a gateway between the API provider and the developer community.
- Features:
 - Comprehensive API documentation.
 - How-to guides and terms of use.
 - Community interaction through blogs and forums.
 - API monetization tools (billing, reports).

API Catalog and Documentation

- API Catalog:
 - Publicizes the available APIs, often referred to as an API registry.
 - Developers can search by metadata, tags, or functionality.
- Documentation Includes:
 - API functionality, interface, and usage guides.
 - Terms of use and version control.

Developer Support Resources

- Key Elements of Support:
 - Well-designed REST APIs are intuitive, but documentation is key for further developer success.
- Support Features:
 - API documentation that includes how-to guides.
 - Test consoles for hands-on interaction with APIs.
 - Sample code and SDKs to speed up the development process.

Developer Onboarding Process

- Simplified Access:
 - Developers must register to access APIs, either independently or as part of a company.
 - Self-registration and API selection should be straightforward.
- API Key Generation:
 - After successful registration and approval (manual or automatic), API keys are issued.
 - Key management for companies, allowing modification or revocation as needed.

Community Management and Collaboration

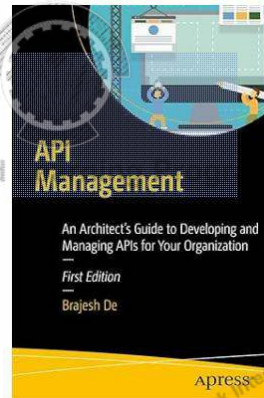
- Developer Collaboration:
 - Developers often seek advice and shared experiences from their peers.
 - Blogs and forums are vital tools for community interaction.
- Community Features:
 - Blogs can feature API usage experiences or updates from the provider.
 - Discussion forums for troubleshooting and sharing best practices.
 - Support ticketing system for reporting API issues.

Building a Dynamic Developer Community

- Community Building:
 - A robust developer portal fosters a dynamic community through active engagement.
 - Collaboration and shared learning enhance API adoption and usage.
- Key Strategies:
 - Moderated blogs and forums to ensure quality content.
 - Regular updates from the API provider to keep the community informed.

Reference

- API Management – An Architect's Guide to Developing and Managing APIs for your Organization
 - by Brijesh De



Summary

- Developer enablement is crucial to API success.
- A well-designed developer portal offers documentation, support, onboarding, and community engagement.
- Encouraging collaboration and providing resources fosters a strong developer community, driving the API program forward.

MCQ1

What is the primary purpose of a developer portal in an API program?

- A) To monitor API performance.
- B) To document APIs, provide support, and foster community engagement.
- C) To control API security.
- D) To generate revenue from APIs.

Answer: B) To document APIs, provide support, and foster community engagement.

MCQ2

Which of the following is NOT a key feature of a developer portal?

- A) API catalog and documentation.
- B) Embedded test consoles for API interaction.
- C) A platform to modify API infrastructure.
- D) Blogs and forums for community collaboration.

Answer: C) A platform to modify API infrastructure.

MCQ3

In the developer onboarding process, what is typically generated after successful registration and approval?

- A) API documentation.
- B) A developer account.
- C) An API key or client ID.
- D) A community support ticket.

Answer: C) An API key or client ID.

API-based Products

Types of API Users | Suresh Gopal



Learning Outcomes

- Understand the **diverse roles and responsibilities of API users**



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



Various types of API users

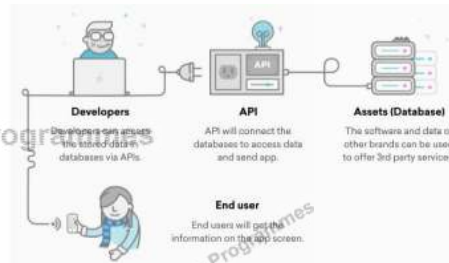
- API users can vary widely based on their roles, needs, and permissions
- Some common types of API users:
 - Developers
 - Administrators
 - End Users
 - Data Analysts
 - Business Executives
 - Integration Partners
 - IoT Devices
 - Bots and Automation Scripts
 - Testing and Quality Assurance (QA) Team



<https://techdocs.broadcom.com/us/en/ca-enterprise-software/layer7-api-management/api-developer-portal/5-0/user-types-roles-and-permissions.html>

Developers

- Primary Role:
 - Developers are the **core users of APIs**, responsible for leveraging APIs to **build applications, integrate systems, or access data programmatically**
- Responsibilities:
 - write code to interact with APIs
 - handle authentication,
 - process responses,
 - integrate API functionality into applications
- Skills:
 - Proficiency in programming languages**
 - Understanding of API documentation,
 - Knowledge of authentication methods are essential skills for developers



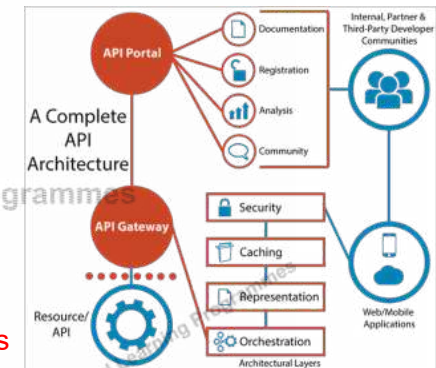
<https://www.peerbits.com/blog/complete-guide-to-api-development.html>

BITS WILP | Generated: 12-May-2026 16:47:54



Administrators

- Primary Role:
 - manage the API infrastructure and its associated resources**
- Responsibilities:
 - configure access controls,
 - manage API keys,
 - monitor usage and performance metrics
 - ensure security compliance,
 - maintain API documentation
- Skills:
 - Strong knowledge of API management platforms**
 - security best practices,
 - system administration skills are required



<https://www.softwaretestinghelp.com/api-management-tools/>

BITS WILP | Generated: 12-May-2026 16:47:54



End Users

- Primary Role:
 - consume services or applications that are powered by APIs
- Responsibilities:
 - interact with the user interface of applications without directly interacting with the underlying APIs
- Skills:
 - may require minimal technical skills
 - depending on the complexity of the application they're using



<https://www.google.com/url?sa=i&url=https%3A%2F%2Fintellisoft.io%2Ftoday-s-world-s-most-popular-tools-for-testing-rest-api%2F&psig=AOvVaw0YSo4Ix6mTIJG6OEN2h447&ust=1715340904026000&source=images&cd=vf&opi=89978449&ved=0CBIQjRxqFwoTCNjdjcm8gIYDFQAAAAAdAAAAABAR>

BITS WILP | Generated: 12-May-2026 16:47:54

Data Analysts

- Primary Role:
 - use APIs to access and analyze data from various sources
- Responsibilities:
 - extract data through APIs,
 - perform analysis,
 - create reports,
 - visualize insights to support decision-making processes
- Skills:
 - Proficiency in data analysis tools,
 - understanding of API data formats,
 - knowledge of querying languages (e.g., SQL) are essential



<https://usa.bootcampcdn.com/wp-content/uploads/sites/108/2020/04/data-analyst-skills-2020.jpg>

BITS WILP | Generated: 12-May-2026 16:47:54

Business Executives

- Primary Role:
 - utilize APIs to access high-level analytics or integrate with third-party services
- Responsibilities:
 - leverage APIs to make strategic decisions,
 - improve business processes,
 - enhance operational efficiency
- Skills:
 - Understanding of business objectives,
 - Data interpretation skills,
 - Knowledge of API capabilities relevant to business needs are important

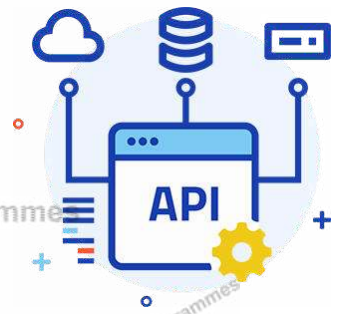


<https://www.uplead.com/wp-content/uploads/2021/11/business-executives-1.png>

BITS WILP | Generated: 12-May-2026 16:47:54

Integration Partners

- Primary Role:
 - exchange data with system through APIs
- Responsibilities:
 - integrate their systems with yours to enable seamless data exchange,
 - automate processes, or enhance functionality
- Skills:
 - Familiarity with API documentation
 - Data mapping,
 - Integration protocols (e.g., REST, SOAP) is necessary



https://sp-ao.shortpixel.ai/client/q_lossless,ret_img/https://www.intelebiz.com/wp-content/uploads/2019/01/api_integrations.png

BITS WILP | Generated: 12-May-2026 16:47:54

IoT Devices

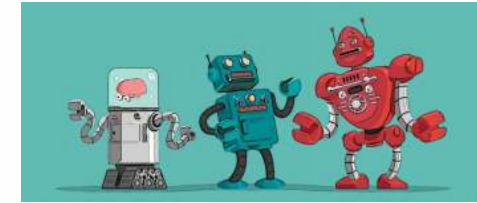
- Primary Role:
 - IoT devices communicate with servers or cloud platforms via APIs to send data or receive commands
- Responsibilities:
 - collect and transmit data to API endpoints,
 - trigger actions based on received commands,
 - handle communication protocols
- Skills:
 - Embedded systems development, networking knowledge,
 - understanding of API communication protocols are required



<https://bytebeam.io/blog/top-industrial-iot-device-examples/>

Bots and Automation Scripts

- Primary Role:
 - automate tasks by interacting with APIs
- Responsibilities:
 - perform repetitive actions such as data retrieval, posting updates,
 - executing commands on behalf of users or systems
- Skills:
 - Programming skills,
 - Familiarity with API endpoints,
 - Scripting languages are necessary



<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.freecodecamp.org%2Fnews%2Fbuilding-bots%2F&psig=AOvVaw0TUJfUi1LCA2B697LM0XnO&ust=1715341175775000&source=images&cd=vfe&opi=89978449&ved=0CBIQjRxqFwoTCMjqqs69gIYDFQAAAAAdAAAAABAR>

Testing and Quality Assurance (QA) Teams

- Primary Role:
 - test APIs for functionality, performance, and reliability
- Responsibilities:
 - design and execute test cases,
 - analyze API responses,
 - identify bugs or performance issues
 - ensure API compliance with specifications
- Skills:
 - Knowledge of testing methodologies,
 - API testing tools,
 - Scripting languages for test automation are important



https://res.cloudinary.com/upwork-cloud/image/upload/c_scale,w_1000/v1687774964/catalog/1673272999419645952/kkbzknfrhsrbpkhvxpe.jpg

Summary

- the diverse roles and responsibilities of API users which is crucial for designing, implementing, and managing APIs effectively to meet the needs of various stakeholders

MCQ1

Who is responsible for managing API infrastructure, configuring access controls, and monitoring usage?

- A) Developers
- B) End Users
- C) Administrators
- D) Integration Partners

Answer: C) Administrators

MCQ2

Which API user type is primarily responsible for extracting and analyzing data from various sources through APIs to support decision-making?

- A) Data Analysts
- B) Business Executives
- C) Integration Partners
- D) Security Auditors

Answer: A) Data Analysts

MCQ3

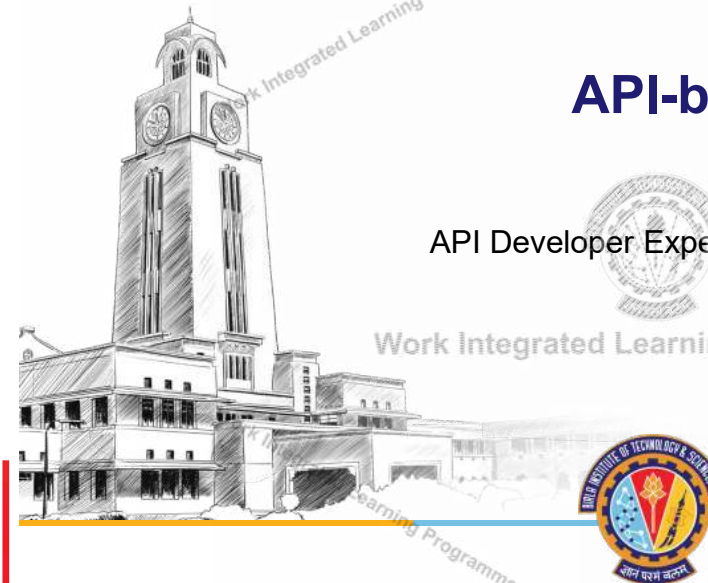
Which API user type communicates with servers or cloud platforms via APIs to send data or receive commands, such as triggering actions based on received commands?

- A) Bots and Automation Scripts
- B) Testing and Quality Assurance (QA) Teams
- C) IoT Devices
- D) Developers

Answer: C) IoT Devices

API-based Products

API Developer Experience | Suresh Gopal



Learning Outcomes

- explore the importance of DX in driving API adoption and success



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



What is Developer Experience (DX)?

- Developer Experience refers to the overall experience developers have when using an API or any development tool
- encompasses
 - usability,
 - documentation,
 - support,
 - community,
 - and the ease of integration
- A positive DX empowers developers to efficiently utilize APIs, leading to increased adoption and satisfaction



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54



<https://waydev.co/wp-content/uploads/2024/04/What-Is-DX--1024x576.jpg>

Importance of API Developer Experience

- **Enhanced DX leads to faster integration:**
 - Developers can quickly understand and implement APIs, reducing time-to-market for applications
- **Improved usability boosts productivity:**
 - Intuitive APIs with clear documentation save developers time and effort, increasing productivity
- **Increased adoption and retention:**
 - Developers are more likely to continue using APIs with positive DX, leading to higher adoption rates and lower churn
- **Positive reputation and word-of-mouth:**
 - Developers share their experiences with APIs, influencing others' decisions and contributing to the API's reputation



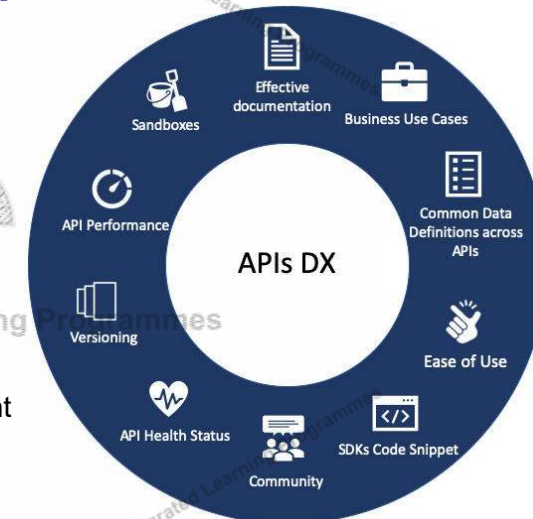
https://media.lipdn.com/dms/image/D4D22AQEyofu4_21CMw/feedshare-shrink_800/0/1689522644596?e=2147483647&v=beta&t=_Dg6CUZvBdYMi7ZVn2PshPw-IJ0zDqccyr8IUrv20e8

BITS WILP | Generated: 12-May-2026 16:47:54



Components of API Developer Experience

- **Documentation:**
 - Clear, comprehensive documentation is essential for understanding API functionality, endpoints, parameters, and authentication methods
 - Include code examples, use cases, and tutorials to facilitate integration
- **Usability:**
 - APIs should be intuitive and easy to use, with well-designed endpoints and consistent naming conventions
 - Provide SDKs, libraries, and code snippets in popular programming languages to streamline development



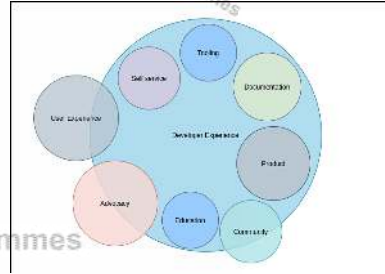
<https://neosalph.com/wp-content/uploads/2023/02/APIs-DX.png>

BITS WILP | Generated: 12-May-2026 16:47:54



Components of API Developer Experience(2)

- **Support:**
 - Offer responsive support channels, such as email, forums, and chat, to assist developers with inquiries and issues
 - Create a knowledge base and FAQ section to address common questions and troubleshooting steps
- **Developer Portals:**
 - Developer portals serve as a centralized hub for API resources, including documentation, SDKs, forums, and community contributions
 - Customizable portals allow developers to access relevant information tailored to their needs.



https://miro.medium.com/v2/resize:fit:1362/0*WCG8PpIO69y9rFeO

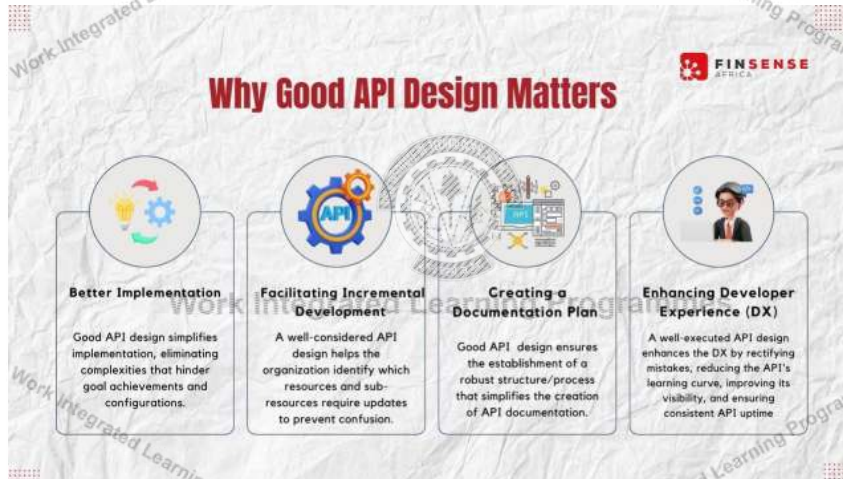
BITS WILP | Generated: 12-May-2026 16:47:54

Strategies to Enhance API Developer Experience

- **Design with developers in mind:**
 - Conduct user research to understand developers' needs and preferences
 - Iterate on API design based on feedback to improve usability and functionality
- **Prioritize documentation:**
 - Invest in creating clear, concise, and up-to-date documentation
 - Include interactive examples and sandbox environments for hands-on learning
- **Foster developer engagement:**
 - Organize hackathons, webinars, and developer meetups to foster a sense of community
 - Recognize and reward developers who contribute valuable feedback or code to the API ecosystem.

BITS WILP | Generated: 12-May-2026 16:47:54

How Good API design improves DX?



https://media.licdn.com/dms/image/D4D12AQEA72n9CNtFbw/article-cover_image-shrink_720_1280/0/1698322059210?e=2147483647&v=beta&t=nUhw5nEKmzNuhe6E3Pox5luQNphxNpNUT3ez2_8VAUK

BITS WILP | Generated: 12-May-2026 16:47:54

Summary

- what DX entails,
- its impact on developers,
- and strategies to enhance it

BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

What is Developer Experience (DX)?

- A) The degree to which an API is secure
- B) The overall experience developers have when using an API or development tool
- C) The process of designing APIs for end-users
- D) The speed at which developers can integrate APIs into their applications

Correct Answer: B) The overall experience developers have when using an API or development tool

MCQ2

Which of the following is NOT a component of API Developer Experience?

- A) Documentation
- B) Usability
- C) Pricing model
- D) Support

Correct Answer: C) Pricing model

MCQ3

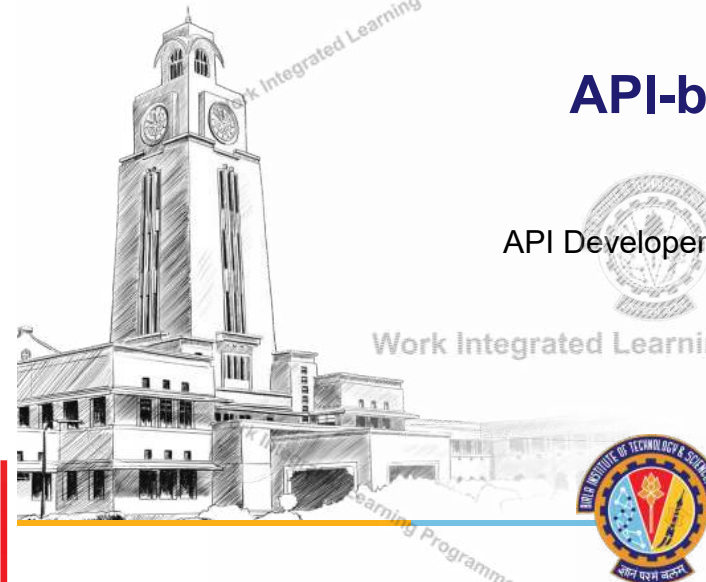
What is one strategy mentioned to enhance API Developer Experience?

- A) Ignoring developer feedback
- B) Providing outdated documentation
- C) Offering multiple support channels with timely responses
- D) Keeping API design static without iteration

Correct Answer: C) Offering multiple support channels with timely responses

API-based Products

API Developer Portal | Suresh Gopal



Learning Outcomes

- Understand the **role and significance of API Developer Portals** in modern software development

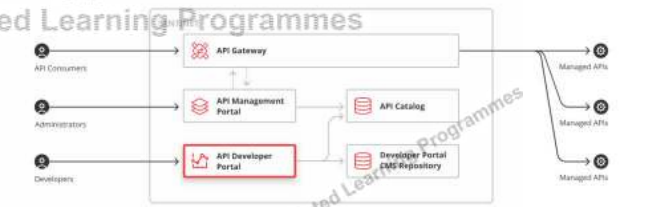


Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Transition to API Developer Portals

- While APIs are essential, **Accessibility and usability of APIs are equally crucial for their successful adoption and utilization**
- This transition brings us to the concept of **API Developer Portals**,
 - which serve as centralized platforms designed to **streamline**
 - the API discovery,**
 - integration,**
 - and management processes**

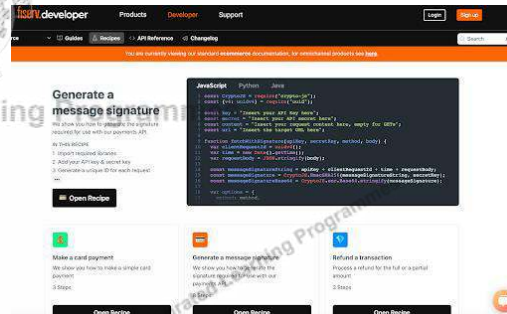


<https://nevatech.com/how-it-works/api-developer-portal>

BITS WILP | Generated: 12-May-2026 16:47:54

API developer portal

- A centralized platform that **provides developers with access to APIs**
 - along with **the necessary documentation, tools, resources, and support** to effectively use those APIs in their applications
- Serve as a gateway for developers to**
 - discover,**
 - explore,**
 - understand,**
 - and integrate APIs into their projects**

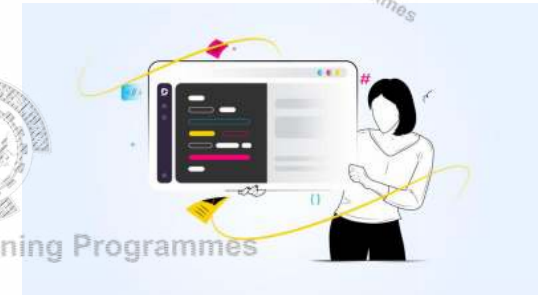


<https://document360.com/blog/api-developer-portal-examples/>

BITS WILP | Generated: 12-May-2026 16:47:54

Key Features

- Documentation
- API Explorer
- Developer Onboarding
- Code Samples and SDKs
- Developer Support

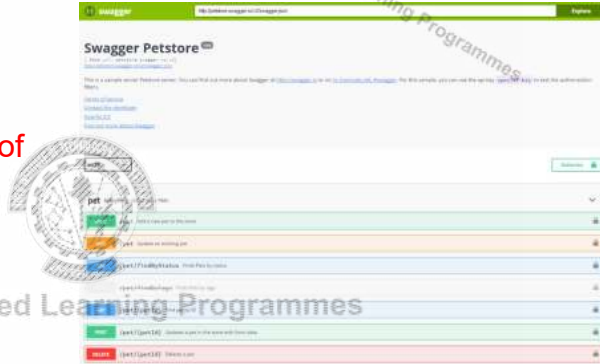


<https://document360.com/blog/api-developer-portal-examples/>

BITS WILP | Generated: 12-May-2026 16:47:54

Documentation

- The most critical feature!
- Includes detailed explanations of
 - API endpoints,
 - parameters,
 - request/response formats,
 - authentication methods,
 - and usage examples
- Well-structured documentation empowers developers to understand and utilize APIs effectively

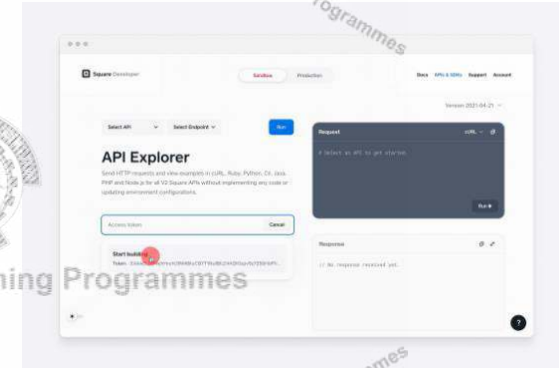


<https://static1.smartbear.co/swagger/media/blog/wp/swaggeruiexample-1024x701.png>

BITS WILP | Generated: 12-May-2026 16:47:54

API Explorer

- Allows developers to **interact** with APIs directly from the portal
- Developers can
 - make test requests,
 - view responses in real-time,
 - experiment with different parameters and endpoints without writing any code
- facilitates rapid prototyping and testing of API functionality



<https://developer.squareup.com/docs/devtools/api-explorer>

BITS WILP | Generated: 12-May-2026 16:47:54

Developer Onboarding Resources

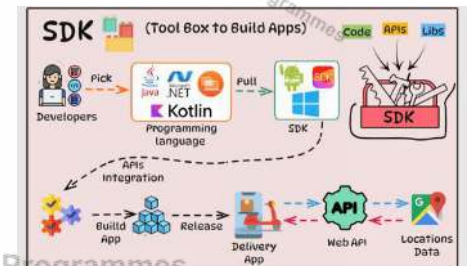
- Offer resources and guides to help developers get started with using the APIs
- Resources may include
 - tutorials,
 - getting started guides,
 - step-by-step instructions for obtaining API keys,
 - setting up authentication,
 - and making initial API calls
- Accelerate the learning curve and ensure a smooth integration process



BITS WILP | Generated: 12-May-2026 16:47:54

Code Samples and SDKs

- To simplify integration, it provides
 - pre-built code samples,
 - sample applications,
 - and software development kits (SDKs) in various programming languages
- Resources offer templates and boilerplate code that developers can adapt to their specific use cases
 - speeding up the development process and reducing errors



https://www.linkedin.com/posts/alexxybyte_systemdesign-coding-interviewtips-activity-7123696152117436416-BADK/

BITS WILP | Generated: 12-May-2026 16:47:54

Developer Support Channels

- API Developer Portals offer **channels for developers** to
 - seek assistance,
 - ask questions,
 - report issues,
 - and engage with the API provider's support team or community forums
- Support channels may **include**
 - documentation feedback mechanisms,
 - email support,
 - live chat,
 - forums,
 - and knowledge bases



<https://nytjahuomenna.files.wordpress.com/2014/01/some-channels-and-purposes.jpg?w=745>

Summary

- API Developer Portals which plays a crucial role in modern software development
 - **by providing developers with the tools and resources they need to leverage APIs effectively**
- Key features of API developer portal
 - **Documentation**
 - **API Explorer**
 - **Developer Onboarding**
 - **Code Samples and SDKs**
 - **Developer Support**

MCQ1

What is the primary purpose of an API Developer Portal?

- A) To restrict access to APIs
- B) To facilitate communication between APIs
- C) To provide developers with tools and resources for API integration
- D) To replace APIs with alternative technologies

Correct Answer: C) To provide developers with tools and resources for API integration

MCQ2

Which of the following is NOT a key feature of an effective API Developer Portal?

- A) Comprehensive documentation
- B) Intuitive user interface
- C) Limited developer support
- D) Developer onboarding resources

Correct Answer: C) Limited developer support

MCQ3

Why are API Developer Portals essential in modern software development?

- A) They restrict access to APIs for security purposes
- B) They facilitate communication between different software applications
- C) They provide developers with tools and resources to utilize APIs effectively
- D) They replace the need for APIs with alternative technologies

Correct

Answer: C) They provide developers with tools and resources to utilize APIs effectively

API-based Products

API Catalog

Suresh Gopal

Learning Outcomes

- Learn how **private and public API catalogs** can improve API
 - discoverability,
 - collaboration,
 - and governance

API sprawl

Uncontrolled proliferation of APIs within an organization

- Challenges:
 - Redundancy,
 - Inconsistency,
 - Governance,
 - and maintenance difficulties
- Causes:
 - **Independent development and deployment by various teams**
- Consequences:
 - Inefficiencies,
 - Complexity,
 - Security vulnerabilities,
 - higher costs

<https://www.akamai.com/glossary/what-is-api-sprawl>

-
- The diagram illustrates the Backend Architecture. On the left, three external services are listed: API Consumers, API Providers, and Developers. Arrows from these services point to a central 'Backend' box. Inside the 'Backend' box, there are three main components: API Gateway, API Management Portal, and API Developer Portal. The API Gateway and API Management Portal are connected to the API Developer Portal. The API Developer Portal is connected to the API Catalog and the Developer Portal CMS Repository. The API Catalog and the Developer Portal CMS Repository are connected to three managed services on the right: Managed API, Managed API, and Managed API.

BITS WILP | | Generated: 12-May-2026 16:47:54

- API Catalog
 - Internal APIs
 - Process Order APIs
 - Ship Products APIs
 - Public APIs
 - Take Orders APIs
 - Track Orders APIs

BITS WILP | | Generated: 12-May-2026 16:47:54

-



BITS WILP | | Generated: 12-May-2026 16:47:54

-
- ZIMLAB**
- Home API Catalog
- ## API Catalog
- List of APIs available to be consumed. These APIs were obtained from the configured API Catalog.
- How to call them: How to log in: [Show all API Types](#)
- | | | | |
|---|---|---|--|
| <p>Amazon S3 API
 import
 definition</p> | <p>Facebook API
 import and handle
 definition</p> | <p>Flickr API
 import, edit
 definition</p> | <p>Google Maps API
 importing, view
 definition</p> |
| <p>Instagram API
 import, photos
 definition</p> | <p>Last.fm API
 import
 definition</p> | <p>LinkedIn API
 importing, user
 definition</p> | <p>Twitter API
 importing, tweet
 definition</p> |

BITS WILP | | Generated: 12-May-2026 16:47:54

Reference

- [Postman API catalog](#)



Work Integrated Learning Programmes

Summary

- API catalogs which acts as **centralized repositories**,
 - simplifying the discovery and utilization of APIs within organizations



Work Integrated Learning Programmes

MCQ1

What is the primary purpose of an API catalog within an organization?

- a) To increase the complexity of development processes
- b) To streamline the discovery and utilization of APIs
- c) To restrict access to APIs for certain teams
- d) To promote redundancy and inconsistency in API development

Correct answer: b) To streamline the discovery and utilization of APIs

MCQ2

Which of the following is a consequence of API sprawl?

- a) Improved efficiency in development processes
- b) Reduction of security vulnerabilities
- c) Increased complexity and operational costs
- d) Streamlined governance and maintenance

Correct answer: c) Increased complexity and operational costs

MCQ3

How can organizations mitigate the challenges posed by API sprawl?

- a) By encouraging independent development of APIs by various teams
- b) Through effective API cataloging, standardization, and governance frameworks
- c) By limiting access to APIs to only a select few developers
- d) By avoiding the use of APIs altogether

Correct answer: b) Through effective API cataloging, standardization, and governance frameworks

API-based Products

Developer Resources | Suresh Gopal

Learning Outcomes

- Develop understanding about the common API developer resources

Why Developer Resources?

- Simply having a great API isn't sufficient if developers don't know how to utilize it effectively
 - crucial to offer learning materials alongside API design to support developers
- Different types of developer resources impact different stages of the development cycle
- Developer resources encompass various assets aimed at enhancing API usage

Developer Resources

- Developer resources encompass various assets aimed at enhancing API usage
 - API Documentation
 - Samples and Snippets
 - Rich Media
 - Community Contribution
 - Software Development Kits and Frameworks
 - Development Tools

Code Samples

- Providing code samples and snippets is an effective way to enhance developers' utilization and productivity with your API
- Well-designed samples
 - incorporate best practices, reducing errors and misuse of the API
 - should be provided in common programming languages, with high readability and comprehensive comments
- Even a single consistent code sample can be valuable to developers of various backgrounds

Code Samples ...

- Code samples typically
 - address specific API use cases
 - or demonstrate integrations between different aspects of the API
- Reference apps
 - like Google I/O app or Twitch's chat app,
 - illustrate business use cases rather than specific API functionalities
- The challenge
 - lies in balancing readability, usability, and relevance to the use case to facilitate developers' learning and implementation

Snippets

- Concise, contextual code samples accompanying tutorials, reference docs, or FAQ answers
 - consist of a few lines of code,
 - briefly documented but highly readable
- Unlike code samples, snippets are part of complete code and don't require variable declarations or imports
- Advisable to provide snippets in multiple programming languages for ease of implementation

```
function createResource(properties) {
  var resource = {};
  var normalizedProps = properties;
  for (var p in properties) {
    var value = properties[p];
    if (p && p.substr(-2, 2) == '[]') {
      var adjustedName = p.replace('[]', '');
      if (value) {
        normalizedProps[adjustedName] = value.split(',');
      }
      delete normalizedProps[p];
    }
  }
}
```

https://developers.google.com/youtube/v3/code_samples/code_snippet_instructions#javascript

Rich Media

- Various learning preferences exist among developers, including
 - reading,
 - listening,
 - and watching
- While written tutorials and documents remain popular, there's a rising trend in using rich media such as
 - short videos,
 - webinars,
 - and live Q&A sessionsto engage and educate developers.

BITS WILP | Generated: 12-May-2026 16:47:54



Videos

- Serve as effective tools for introducing new technologies, sharing general best practices, and delving into specific topics
 - Creating videos has become more accessible with the availability of high-definition recording devices
- However, producing high-quality videos still requires professional equipment, practice, and editing expertise, which can be costly and challenging
- Live sessions offer a practical starting point for recording videos, leveraging presentations from events to share with a wider developer audience.

BITS WILP | Generated: 12-May-2026 16:47:54



Office Hours

- Dedicated time slots where developers can receive assistance and have their questions addressed regarding an API
 - provide an opportunity for developers to interact directly with the API team and receive guidance
- The Slack Developer Platform team initiated weekly online office hours due to resource constraints,
 - enabling developers to join via a public video call link and ask questions
 - facilitated learning from shared inquiries among multiple participants,
 - enhancing the overall experience and knowledge exchange



BITS WILP | Generated: 12-May-2026 16:47:54



Webinars and Online Training

- Serve as effective online education platforms for developers,
 - offering a somewhat collaborative learning experience
- Through webinars,
 - speakers present topics using online tools like Zoho and invite developers to join the training sessions
- Presentations are broadcast to the audience,
 - often supplemented with screen sharing or videos to enhance understanding

BITS WILP | Generated: 12-May-2026 16:47:54



Community Contribution

- Building comprehensive resources to educate and engage developers with your API requires significant time and investment
 - However, leveraging community contributions can alleviate some of these challenges and enrich the developer experience
- A thriving developer community often generates valuable content such as
 - tutorials,
 - videos,
 - code samples,
 - and answers to questions
- Examples include
 - Google Developers Experts collaborating on various resources,
 - Slack managing open-source SDKs

Reference

- Designing Web APIs
 - By Brenda Jin, Saurabh Sahni, Amir Shevat

Summary

- the importance of providing comprehensive resources for developers, such as
 - Code samples,
 - Snippets,
 - Videos
 - Office hours
 - Webinars,
 - Community contributions

MCQ1

What is a key characteristic of webinars mentioned in the discussion?

- a) Webinars are exclusively presented by API developers.
- b) Webinars involve interactive sessions where developers can ask questions.
- c) Webinars are primarily focused on providing written documentation.
- d) Webinars are only accessible to developers who attend events in person.

Answer: b) Webinars involve interactive sessions where developers can ask questions.

MCQ2

Which statement accurately reflects the role of community contribution in API development?

- a) Community contributions replace the need for basic documentation provided by API developers.
- b) Community contributions focus solely on financial support for API maintenance.
- c) Community contributions enrich the developer experience by generating content like tutorials and code samples.
- d) Community contributions are discouraged as they often lead to inconsistencies in API documentation.

Answer: c) Community contributions enrich the developer experience by generating content like tutorials and code samples.

MCQ3

Why are code samples and snippets considered valuable resources for developers?

- a) They eliminate the need for developers to engage with API documentation.
- b) They provide comprehensive explanations of API functionalities.
- c) They enhance developer productivity by offering reference examples of API usage.
- d) They require developers to have advanced programming skills to understand.

Answer: c) They enhance developer productivity by offering reference examples of API usage.

API-based Products

Developer Resources | Suresh Gopal

Learning Outcomes

- Develop understanding about the common API developer tools and frameworks

Developer Resources

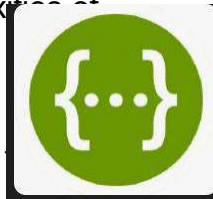
- Developer resources encompass various assets aimed at enhancing API usage
 - API Documentation
 - Samples and Snippets
 - Rich Media
 - Community Contribution
 - Software Development Kits and Frameworks
 - Development Tools

Software Development Kits and Frameworks

- Developers **possess diverse proficiency levels**, impacting their comfort with accessing web APIs directly
- Developing SDKs and frameworks facilitates easier access to APIs for developers**
- Offer **the advantage of integrating best practices and security measures**,
 - reducing the need for developers to implement these elements themselves

SDKs

- SDKs **simplify API interaction by offering a code library instead of direct API calls**, preferred by developers for its ease of use
- SDKs **encapsulate APIs**, shielding developers from the complexities of direct API handling
 - SDK interfaces **require readability and documentation**
- Providing SDKs in **developers' preferred languages** is essential and effectiveness
 - Regular SDK **updates** are crucial to align with API enhancements, ensuring access to new features



Frameworks

- Frameworks simplify API usage by providing functionality aligned with developers' use cases, alleviating API complexity**
 - Botkit exemplifies how frameworks can streamline the development of conversational interfaces and bots by encapsulating intricate functionalities
- Well-maintained frameworks and SDKs ease API upgrades and**
 - minimizing disruption and enabling seamless integration of new features
- Hosting SDKs and frameworks on platforms like GitHub
 - enhances accessibility and encourages community collaboration, fostering innovation and improvement.



Botkit

Development Tools

- API might appear daunting and opaque to developers without adequate support tools
- Providing tailored tools can empower developers to troubleshoot and address challenges independently
 - Identifying and addressing common pain points is essential, but understanding developers' unique needs is paramount
 - Analyzing developers' requirements enables the definition of specialized tools tailored to your service

Debugging and Troubleshooting

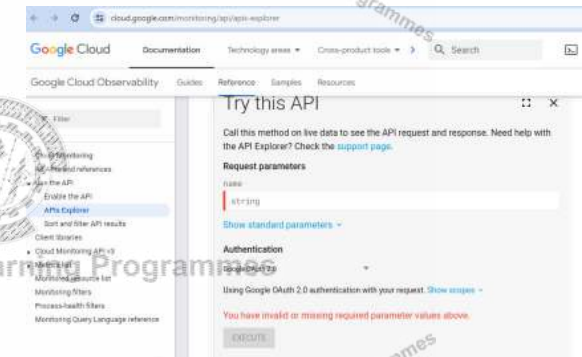
- Despite clear error messages, pinpointing the exact cause of failure can still be challenging for developers
- Providing developers with tools for analyzing, debugging, and troubleshooting API calls can significantly enhance their troubleshooting capabilities
 - range from basic web pages displaying associated logs to sophisticated step-by-step debuggers integrated into development environments
 - In most cases, simpler debugging tools suffice for resolving the majority of issues

Sandboxes

- Sandboxes and API testers facilitate rapid testing and validation of API usage correctness for developers
- Sandboxes offer a secure and isolated environment, such as
 - simulated image lists,
 - enabling developers to experiment with API functionalities without impacting actual product data

API Testers

- API testers, typically included in API documentation, allow developers to execute API calls, sometimes on live data
 - Google's APIs Explorer
- Developers can utilize API testers like APIs Explorer without requiring coding skills; valid input parameters are sufficient for making API requests



<https://cloud.google.com/monitoring/api/apis-explorer>

Summary

- How the developer experience can be enhanced in utilizing APIs through various tools and strategies including
 - Software Development Kits and Frameworks
 - SDKs
 - Frameworks
 - Development Tools
 - Debugging and Troubleshooting
 - Sandboxes and API Testers

Reference

- Designing Web APIs
 - By Brenda Jin, Saurabh Sahni, Amir Shevat

MCQ1

Which of the following best describes the purpose of Software Development Kits (SDKs)?

- a) They serve as abstraction layers over APIs, simplifying API interaction for developers.
 - b) They are tools for analyzing, debugging, and troubleshooting API calls.
 - c) They provide secure and isolated environments for testing API usage.
 - d) They facilitate communication between different programming languages.
- Correct answer: a) They serve as abstraction layers over APIs, simplifying API interaction for developers.

MCQ2

What is the primary function of frameworks in API development?

- a) To provide a safe and isolated environment for API testing.
 - b) To simplify API usage by offering functionality tailored to developers' use cases.
 - c) To generate automatic SDKs across multiple programming languages.
 - d) To abstract API changes and facilitate seamless integration of new features.
- Correct answer: b) To simplify API usage by offering functionality tailored to developers' use cases.

MCQ3

Which tool enables developers to analyze, debug, and troubleshoot API calls without the need for coding?

- a) APIs Explorer
- b) API documentation
- c) SDKs
- d) Sandboxes

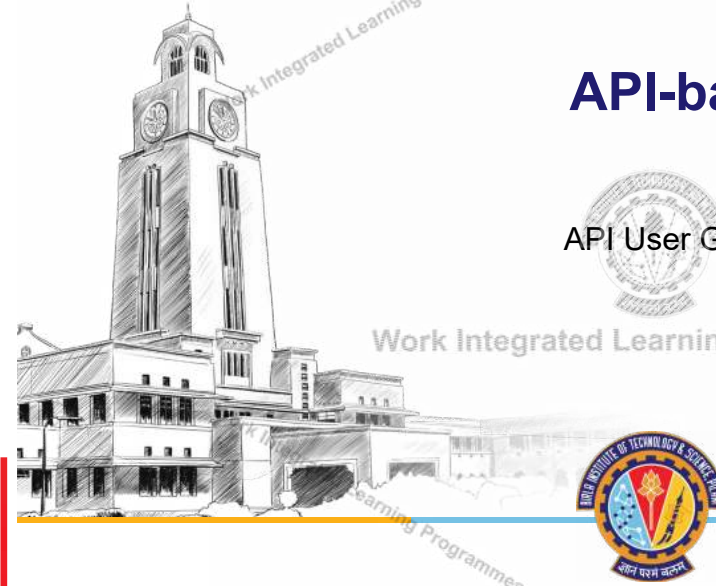
Correct answer: a) APIs Explorer



API-based Products

API User Guides

Suresh Gopal



Learning Outcomes

- Develop a comprehensive understanding of API user guides



API user guide

- A comprehensive document or set of instructions designed to help developers effectively utilize an Application Programming Interface (API)
- The API user guide typically includes:
 - Introduction
 - Getting Started
 - Endpoint Documentation
 - Authentication and Authorization
 - Rate Limiting
 - Error Handling
 - Sample Code
 - Best Practices
 - Terms of Use
 - Version History
 - Glossary



Ride-Sharing App API User Guide

Ride-Sharing App API User Guide

Introduction

Welcome to the Ride-Sharing App API, providing access to ride booking and management functionalities. This user guide will help you integrate ride-sharing features into your applications.

Getting Started

To use the Ride-Sharing App API, you need to sign up for an API key on our developer portal. Once registered, you can start making requests to our endpoints.

Authentication

API requests require authentication using your API key. Include your API key in the request headers as follows:

```
Authorization: Bearer YOUR_API_KEY
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Ride-Sharing App API User Guide(2)

Endpoints

The Ride-Sharing App API provides the following endpoints:

1. Request a Ride

Endpoint:

```
POST /rides/request
```

Description:

Request a ride for a specific location.

Parameters:

- `"pickup_latitude"` (required): Latitude of the pickup location.
- `"pickup_longitude"` (required): Longitude of the pickup location.
- `"dropoff_latitude"` (required): Latitude of the drop-off location.
- `"dropoff_longitude"` (required): Longitude of the drop-off location.
- `"passenger_count"` (optional): Number of passengers (default: 1).

Example Request:

```
json
POST /rides/request
{
  "pickup_latitude": 37.7749,
  "pickup_longitude": -122.4194,
  "dropoff_latitude": 37.7879,
  "dropoff_longitude": -122.4075,
  "passenger_count": 2
}
```

Example Response:

```
json
{
  "ride_id": "123456",
  "status": "pending",
  "driver_eta": 5
}
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Ride-Sharing App API User Guide(3)

2. Get Ride Details

Endpoint:

```
bash
GET /rides/:ride_id
```

Description:

Get details of a specific ride by its ID.

Parameters:

- `"ride_id"` (required): ID of the ride.

Example Request:

```
bash
GET /rides/123456
```

Example Response:

```
json
{
  "ride_id": "123456",
  "status": "accepted",
  "driver_name": "John Doe",
  "driver_phone": "+1234567890",
  "car_model": "Toyota Prius",
  "car_license_plate": "ABC123",
  "eta": 5,
  "pickup_location": {
    "latitude": 37.7749,
    "longitude": -122.4194
  },
  "dropoff_location": {
    "latitude": 37.7879,
    "longitude": -122.4075
  }
}
```

BITS WILP | | Generated: 12-May-2026 16:47:54

Ride-Sharing App API User Guide(4)

Rate Limiting

The Ride-Sharing App API has rate limits in place to prevent abuse. Each API key is limited to 1000 requests per day.

Security

All API requests and responses are encrypted using HTTPS. Ensure that you keep your API key secure and do not expose it publicly.

Examples and Tutorials

For code examples and tutorials on how to use the Ride-Sharing App API in various programming languages, please visit our [developer documentation](#).

BITS WILP | | Generated: 12-May-2026 16:47:54

Ride-Sharing App API User Guide(5)

FAQs

Q: How can I track the location of my ride?

A: Use the "GET /rides/:ride_id" endpoint to get real-time updates on the ride status and location.

Q: Can I schedule a ride for a later time?

A: Currently, scheduling rides for a later time is not supported.

Contact

For support or inquiries, please contact us at support@ridesharingapp.com.

Summary

- Purpose and Structure of API User Guides

MCQ1

What is the purpose of an API user guide?

- A) To market the API to potential users
- B) To provide instructions for developers on how to use the API effectively
- C) To showcase the API's features and capabilities
- D) To implement security measures for the API

Answer: B) To provide instructions for developers on how to use the API effectively

MCQ2

Which section of an API user guide typically includes details such as endpoint URLs, HTTP methods, parameters, and response formats?

- A) Getting Started
- B) Authentication and Authorization
- C) Endpoint Documentation
- D) Sample Code

Answer: C) Endpoint Documentation

MCQ3

What does the "Rate Limiting" section of an API user guide explain?

- A) How to obtain API credentials
- B) How errors are handled by the API
- C) Describes any rate limiting policies enforced by the API
- D) Recommendations and best practices for using the API efficiently

Answer: C) Describes any rate limiting policies enforced by the API

API-based Products

Mock API Implementation

Suresh Gopal

Learning Outcomes

- gain the ability to effectively simulate API behavior across different stages of development

Mock API implementations

- Mock API implementations **serve as substitutes for real APIs during development, testing, or when integrating with third-party services**
 - Simulate the behavior of real APIs, allowing developers to work independently of the actual backend services
- Common approaches to implementing mock APIs:
 - Manual Mocking
 - Static Mocking with JSON Files
 - Dynamic Mocking with Data Generators
 - Proxy-based Mocking
 - Generated Mocks from API Specifications
 - Client-side Mocking Libraries

Manual Mocking

- **Developers manually create mock endpoints**
 - using frameworks like Flask (Python), Express.js (Node.js), or Spring Boot (Java)
- Responses are hardcoded or generated programmatically based on expected API behavior
- **Suitable for simple APIs or scenarios where full control over responses is needed**
- Time-consuming to set up and maintain, limited scalability for complex APIs

Static Mocking with JSON Files

- **Mock responses are stored in JSON files and served by a simple HTTP server**
- Developers create a directory structure representing API endpoints, with each JSON file containing a mock response
- Tools like json-server (JavaScript) or WireMock (Java) can be used to serve these static mock responses
- Offers a quick **way to create mock APIs for testing or frontend development**
- **Limited flexibility for dynamic responses, scalability issues for large datasets**

Dynamic Mocking with Data Generators

- **Mock responses are dynamically generated using data generators or factories**
- Developers define templates or schemas for responses and populate them with random or predefined data
- Libraries like Faker (multiple languages) or Mockaroo (web-based) generate realistic data for mock responses
- **Useful for simulating large datasets or generating varied responses for testing purposes**
- **Requires additional libraries or tools, setup overhead for data generation**

Proxy-based Mocking

- **A proxy intercepts requests made to the real API and returns mock responses instead**
 - Developers configure the proxy to route requests to mock endpoints based on predefined rules or conditions
- Tools like Charles Proxy (desktop), MockServer (Java), or Service Virtualization solutions provide proxy-based mocking capabilities
- **Ideal for mimicking the behavior of existing APIs without modifying client-side code**
- **Requires setup of proxy server, may not be suitable for offline development or isolated testing.**

Generated Mocks from API Specifications

- **Mock APIs are automatically generated from API specifications**
 - e.g., OpenAPI/Swagger
- Tools like Swagger Codegen, OpenAPI Generator, or Postman can generate mock servers based on API documentation
- **Generated mocks closely mimic the structure and behavior of the real API, facilitating API-first development workflows**
- **Dependency on API documentation, may not cover all edge cases or business logic**

BITS WILP | | Generated: 12-May-2026 16:47:54

Summary

- Understanding of various mock API implementation techniques which enables developers to efficiently simulate API behavior for development, testing, and integration purposes

BITS WILP | | Generated: 12-May-2026 16:47:54

Client-side Mocking Libraries

- Libraries intercept HTTP requests made by the client application and return mock responses
- Examples include MirageJS (JavaScript), Retrofit Mock (Android), or Nock (Node.js)
- **Useful for frontend development, enabling developers to work offline or in isolation from the backend**
- **Limited to client-side testing, may not cover backend API changes or errors.**

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ1

Which mock API implementation technique offers full control over responses but is time-consuming to set up and maintain?

- A) Dynamic Mocking with Data Generators
- B) Static Mocking with JSON Files
- C) Manual Mocking
- D) Proxy-based Mocking

Answer: C) Manual Mocking

BITS WILP | | Generated: 12-May-2026 16:47:54

MCQ2

Which mock API implementation technique is characterized by storing mock responses in JSON files and serving them via a simple HTTP server?

- A) Dynamic Mocking with Data Generators
- B) Static Mocking with JSON Files
- C) Client-side Mocking Libraries
- D) Generated Mocks from API Specifications

Answer: B) Static Mocking with JSON Files

MCQ3

Which mock API implementation technique involves intercepting requests made to the real API and returning mock responses instead, suitable for testing against existing APIs?

- A) Generated Mocks from API Specifications
- B) Proxy-based Mocking
- C) Client-side Mocking Libraries
- D) Dynamic Mocking with Data Generators

Answer: B) Proxy-based Mocking

API-based Products

Understanding Developers
Ecosystem

Suresh Gopal

Learning Outcomes

- Develop an understanding of different types of developer-users for your APIs

Introduction to Developer Ecosystems

- Building a scalable, well-designed API is essential, but it is only the first step in fostering an ecosystem.
- Misconception:
 - Many companies believe that releasing an API guarantees developer interest and usage.
- Reality:
 - Without a strategy to engage developers, many APIs go unused, leading to wasted resources.

Defining a Developer Ecosystem

- A developer ecosystem comprises a community of developers and partners interacting with a technology or platform.
- Analogy to Natural Ecosystems:
 - Just as ecosystems in nature involve various organisms interacting, a developer ecosystem features collaboration and competition among developers.
- Examples of Successful Ecosystems:
 - Google and Android: A vast network of developers creating apps.
 - iOS Developer Community: Active engagement through meetups and forums.
 - Microsoft: A diverse ecosystem supporting various platforms and services.

The Role of Developers

- Developers are crucial for:
 - Extending Product Features:
 - Example: Slack apps enhance the Slack experience.
 - Creating New Use Cases:
 - Example: Google Cloud Platform allows developers to innovate.
 - Driving Adoption:
 - iOS's popularity is significantly attributed to its extensive app ecosystem.

Understanding API Developers

- Different types of developers engage with your API, each with unique motivations and use cases:
 - Hobbyists:
 - Experiment for fun and feedback.
 - Hackers:
 - Use APIs for innovative solutions, often for profit.
 - Business-Focused Users:
 - Solve specific business problems with minimal technical knowledge.
 - Professional Developers:
 - Assess API based on stability and fit for projects.

The Hobbyist

- Profile:
 - Early adopters who enjoy experimentation.
 - Provide valuable feedback and insights into API functionality.
- Benefits:
 - Quick to adopt new features and vocalize opinions.
- Challenges:
 - Use cases may diverge from intended applications (e.g., using a facial recognition API on pets).

The Hacker

- Profile:
 - Professional developers or entrepreneurs using APIs for practical applications.
 - Focused on innovation and efficiency.
- Benefits:
 - Early adopters critical for product feedback and growth.
 - Often have high technical proficiency and can handle steep learning curves.

The Business-Focused, Tech-Savvy User

- Profile:
 - Individuals who may not identify as developers but seek solutions to specific problems.
 - Examples include financial analysts or IT personnel automating workflows.
- Importance:
 - This audience is typically larger than the traditional developer base.
 - Requires different educational resources and support.

The Professional Developer

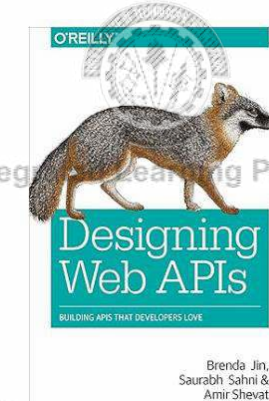
- Profile:
 - Evaluates APIs based on product maturity, stability, and fit for specific projects.
 - Sensitive to breaking changes that could disrupt their work.
- Willingness to Pay:
 - More likely to invest in APIs that solve real business problems.
 - Appreciates tools and services that streamline integration and development.

Summary

- Key Takeaway:
 - Creating a thriving developer ecosystem requires more than just launching an API.
- Active Engagement:
 - Consistently communicate with and support your developer community.
- Long-Term Commitment:
 - Building relationships with developers is essential for sustained adoption and success.

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin, Saurabh Sahni , Amir Shevat



MCQ1

Which type of API developer is characterized as an early adopter who experiments with the API for enjoyment and provides valuable feedback?

- A) Professional Developer
- B) Business-Focused User
- C) Hacker
- D) Hobbyist

Answer: D) Hobbyist

MCQ2

What distinguishes a "Hacker" in the context of API development?

- A) They focus solely on business use cases without technical expertise.
- B) They are motivated by innovation and often seek to create new applications with the API.
- C) They avoid using the latest features and prefer stable versions of the API.
- D) They primarily use the API for personal projects without a commercial goal.

Answer: B) They are motivated by innovation and often seek to create new applications with the API.

MCQ3

Which type of API developer is typically sensitive to breaking changes and evaluates the API based on its stability and product fit?

- A) Hobbyist
- B) Hacker
- C) Professional Developer
- D) Business-Focused User

Answer: C) Professional Developer

API-based Products

Building a Developer Strategy | Suresh Gopal

Learning Outcomes

- Understand the key stages in developing a comprehensive developer strategy to enhance API engagement and developer success.

Importance of a Robust Developer Strategy

- Importance of a Robust Developer Strategy
 - Ensures effective engagement and support for developers.
 - Drives the adoption and success of your API/platform.
- Overview of the Stages:
 - Developer Segmentation
 - Distilling the Value Proposition
 - Developer Funnel
 - Ecosystem Mapping
 - Tactical Implementation

Developer Segmentation

- Purpose:
 - Understand who your audience is.
- Key Attributes:
 - Identity:
 - Differentiate between Frontend, Backend, Full Stack, Mobile, and Enterprise developers.
 - Developer Proficiency:
 - Identify the skill levels needed to effectively use your API.
 - Platform of Choice:
 - Understand the differences between mobile and web developers.

Identity and Proficiency

- Identity:
 - Recognize granularity in developer identity (e.g., iOS vs. Android).
 - Encourage community engagement through meetups and events.
- Developer Proficiency:
 - Tailor your API complexity based on the proficiency levels.
 - Example: APIs for Augmented Reality may require more expertise compared to simpler APIs like Google Apps Script.

Platform of Choice

- Understanding Developer Preferences:
 - Identify preferred platforms for engagement.
- Examples:
 - Mobile Development: iOS vs. Android
 - Console Development: Xbox vs. PlayStation
- Strategy:
 - Tailor engagement strategies based on these platforms to meet specific needs.

Preferred Tools and Use Cases

- Development Language and Tools:
 - Identify the tools your audience uses regularly (e.g., IDEs, frameworks).
- Common Use Cases:
 - Gather information on the primary tasks developers aim to accomplish.
- Communication Importance:
 - Regularly ask developers about their challenges and feedback.

Communication Preferences

- Determining Communication Channels:
 - Identify how developers prefer to receive updates and information:
 - Social Media (Twitter, LinkedIn)
 - Email Notifications
 - Events and Industry News
- Strategy:
 - Establish multiple channels for effective outreach and engagement.

Market Size and Geographical Distribution

- Assessing Developer Base:
 - Determine the current number of developers and the addressable market.
- Global Mapping:
 - Identify key developer hubs around the world to understand localization needs.
- Importance:
 - Continuous market analysis helps refine your strategy.

Distilling the Value Proposition

- Defining Your Value Proposition:
 - Clearly articulate why developers should use your API/platform.
- Core Questions:
 - What unique benefits does your API offer?
 - What is your competitive advantage over others?
- Examples of Value Propositions:
 - Stripe API: Simplifies online payment processing.
 - YouTube API: Enables embedding video players seamlessly.

Validating the Value Proposition

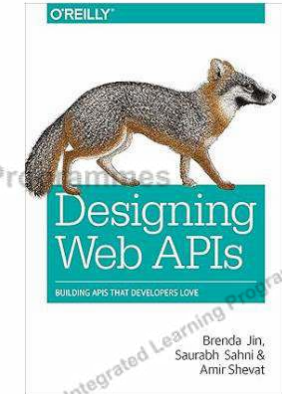
- Importance of Validation:
 - Test and validate the significance of your value proposition to developers.
- Example:
 - Performance metrics comparison (10 ms vs. 30 ms) should be relevant to common developer use cases.
- Understanding Pain Points:
 - Regularly gather feedback to ensure your value proposition addresses actual developer needs.

Summary

- Understanding and engaging developers is crucial for API success.
 - Focus on segmentation, value proposition, and tactical implementation.
- Call to Action:
 - Encourage further discussion and questions on enhancing developer strategies.

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin, Saurabh Sahni , Amir Shevat



MCQ1

What is the primary purpose of developer segmentation in building a developer strategy?

- A) To increase API security
- B) To understand who your audience is and their attributes
- C) To enhance marketing strategies
- D) To improve product pricing

Correct Answer: B) To understand who your audience is and their attributes

MCQ2

Which of the following best describes the value proposition of an API?

- A) A statement on the company's mission
- B) The unique benefits and advantages that appeal to developers
- C) A list of technical specifications for the API
- D) A marketing campaign for the API

Correct Answer: B) The unique benefits and advantages that appeal to developers

MCQ3

What is a critical factor in determining the preferred means of communication for developers?

- A) The latest technology trends
- B) The programming languages used
- C) How developers stay informed about updates and news
- D) The geographical distribution of developers

Correct Answer: C) How developers stay informed about updates and news

API-based Products

API Developer Programme | Suresh Gopal

Learning Outcomes

- Understand the key components and strategies for developing effective developer programs that foster ongoing engagement and adoption of APIs across diverse developer audiences

Overview of API Development

- APIs serve as a bridge between applications, enabling seamless interaction and data exchange.
- Importance of Initial Resources for Developers
 - Providing comprehensive documentation, SDKs, and tutorials is critical for easing the onboarding process for new developers.
 - Continuous Engagement is Essential for Success

Overview of API Development

- Developer engagement shouldn't end at launch
 - ongoing support is vital to foster adoption and innovation.
- Role of Developer Relations Teams
 - These teams act as liaisons between developers and the company, ensuring feedback loops and addressing developer needs.

Understanding Developer Programs

- Initiatives designed to support developers in integrating and building solutions using your API.
- Activities to Drive Developers of All Sizes
 - Programs may include workshops, forums, and resources aimed at fostering API usage across diverse developer demographics.
- Involvement of Developer Relations and Marketing Teams
 - Collaboration ensures that developers receive tailored content and marketing outreach that meets their needs.

Breadth and Depth Analysis

- Two Axes of Developer Programs
 - Depth Axis:
 - Focus on a smaller group of high-impact developers.
 - Breadth Axis:
 - Engage a wider audience, including small and midsize companies and hobbyists.

Deep Developer Programs

- Goal:
 - Engage a Small Group of High-Impact Developers
 - Focused strategies to leverage the expertise of top developers.
- Types of Programs:
 - Top Partner Program:
 - Identify and engage top users based on significant use cases.
 - Beta Program:
 - Offer early access to new features to gather valuable feedback.
 - Design Sprints:
 - Collaborative sessions aimed at solving specific challenges together.

Top Partner Program

- Identification Process:
 - Analyze use cases to identify top partners that will yield high impact.
- Engagement Strategy:
 - Build strong relationships with these partners, providing tailored support (white-glove service) to maximize collaboration.

Beta Program

- Steps for Running a Beta Program:
 - Ideation:
 - Identify key developers and desired use cases.
 - Recruitment:
 - Invite selected partners to participate.
 - Onboarding:
 - Share specifications and gather initial feedback.
 - Joint Building:
 - Maintain regular check-ins for support and collaboration.
 - Launch Prep:
 - Collaborate on marketing strategies and materials.
 - Launch Day:
 - Ensure coordination with partners for a smooth rollout.

Design Sprints

- Process Overview:
 - Understand:
 - Present API capabilities and gather developer needs.
 - Define:
 - Clarify the problem to address.
 - Diverge:
 - Brainstorm multiple design solutions.
 - Decide:
 - Select the most promising idea for implementation.
 - Prototype:
 - Develop a working mockup or prototype.
 - Validate:
 - Test the prototype with end customers for feedback.

Broad Developer Programs

- Focus on Scale
 - Aim to engage as many developers as possible with low-touch, scalable initiatives.
- Key Activities:
 - Host community events and meetups.
 - Organize hackathons to spur innovation.
 - Create online content like videos and tutorials to assist developers.

Community Events and Meetups

- Goal:
 - Build Self-Sustaining Developer Communities
 - Foster ongoing relationships and knowledge sharing among developers.
- Example:
 - Google Developer Groups (GDGs)
- Host regular meetups, hackathons, and training sessions to enhance community engagement and support.

Hackathons

- Structure and Outcomes:
 - Encourage developers to brainstorm and prototype solutions during a focused time frame.
- Clear Guidelines are Essential for Success
 - Provide structure to ensure productivity and alignment with goals.
- Contributions to Awareness and Feedback Collection
 - Use hackathons as a platform for gathering insights and driving engagement.

Speaking at Events

- Importance of Advocacy:
 - Employ full-time advocates to represent your API at global events, enhancing visibility and credibility.
- Recordings of Sessions Provide Long-Term Value
 - Archive and share sessions to extend reach and impact over time.

Online Content and Support

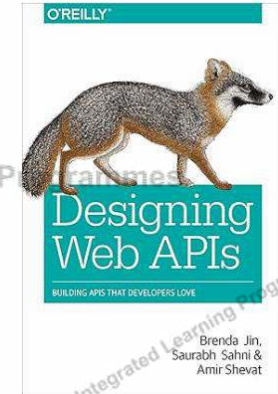
- Building a Video Program:
 - Develop ongoing content like tutorials and live streams to engage with developers continuously.
- Engage with Developers on Platforms like YouTube and Twitch
 - Utilize popular platforms to reach a wider audience and maintain developer interest.

Summary

- Highlight the importance of developer engagement after API launch to foster a strong adoption rate.
 - Tailoring Programs to Different Developer Needs
- Differentiate strategies for deep vs. broad engagement to address various developer audiences.
- Emphasize that a well-structured approach to developer engagement is vital for long-term success.

Reference

- Designing Web APIs: Building APIs That Developers Love
 - by Brenda Jin, Saurabh Sahni , Amir Shevat



MCQ1

What is the primary goal of deep developer programs?

- A) To engage a large audience of casual developers
- B) To provide general resources for all developers
- C) To engage a small group of high-impact developers
- D) To run community events and hackathons

Answer: C) To engage a small group of high-impact developers

MCQ2

Which of the following activities is essential for the ongoing engagement of developers post-API launch?

- A) Creating a marketing brochure
- B) Offering unlimited API access
- C) Running community events and meetups
- D) Publishing a blog post

Answer: C) Running community events and meetups

MCQ3

What is a key benefit of conducting beta programs for API developers?

- A) It allows for unlimited developer access to the API
- B) It provides early access to new features for feedback
- C) It eliminates the need for documentation
- D) It focuses solely on marketing strategies

Answer: B) It provides early access to new features for feedback

API-based Products

Building API Product Strategy | Suresh Gopal

Learning Outcomes

- Develop a comprehensive understanding of API product strategies to drive business innovation and growth

Why APIs Matter for Business Success?

- APIs
 - enhance product ecosystems,
 - power platform-based business models,
 - and support seamless integration with third-party services
- Crucial for
 - driving digital transformation
 - opening new revenue channels through API monetization

The Strategic Importance of an API Product

- API as a Product:
 - Treating APIs as products ensures they are designed, deployed, and maintained with the same rigor as customer-facing applications.
- Digital Transformation through APIs:
 - Allow organizations to
 - be agile,
 - extend services,
 - and accelerate innovation
- **An effective API strategy is key to remaining competitive in today's digital economy.**

Key Elements of an API Product Strategy

- Three elements
 - Understanding Your API Users
 - Use Cases Aligned with Business Goals
 - Monetization Models

Understanding Your API Users

- Internal users (developers within your company):
 - APIs streamline internal processes and enable faster product development.
- Partners:
 - Partner integrations are crucial for extending the reach of your services
- External developers:
 - Enabling third-party developers to build on your platform expands your ecosystem.

Use Cases Aligned with Business Goals

- Identify the target audience for your APIs and clearly define their needs.
- Make sure your API strategy complements your core business goals, such as
 - improving customer experience
 - or increasing revenue streams.

Monetization Models

- Freemium:
 - Offer basic functionality for free, with paid premium features
- Pay-as-you-go:
 - Charge based on API consumption, ideal for businesses with varying API usage
- Subscription:
 - Provide access based on monthly or annual payment tiers, depending on usage levels.

Designing a Winning API

- API Usability:
 - Easy-to-understand documentation is crucial for ensuring smooth adoption by developers.
 - Simplify onboarding with clear guides, SDKs, and sandbox environments for testing.
- Security and Compliance:
 - Ensure robust authentication mechanisms (e.g., OAuth) and compliance with data protection regulations (e.g., GDPR, HIPAA).
 - Protect APIs against attacks like DDoS and ensure encrypted data transmission.

Designing a Winning API

- Scalability and Reliability:
 - Design APIs that can handle increased traffic and usage spikes. Leverage cloud infrastructure for flexibility.
 - Establish monitoring systems to ensure uptime, detect issues early, and maintain performance consistency.

API Ecosystem Management

- API Lifecycle Management
 - Start with well-planned development, focusing on use cases and ease of integration.
 - Regularly update APIs to support new features, fix bugs, and maintain performance.
 - Plan for deprecation phases to ensure smooth transitions for users.

API Ecosystem Management

- Building a Developer Community
 - Foster engagement by providing strong developer support through forums, detailed documentation, SDKs, and APIs testing tools like Postman.
 - Organize hackathons and webinars to build awareness and excitement around your API offerings.

API Ecosystem Management

- Developer Feedback and Continuous Improvement
 - Create open communication channels with developers, enabling them to provide feedback and suggest improvements.
 - Keep evolving the API to better meet developer and business needs.

Challenges and Best Practices

- Challenges:
 - Ensuring security and data privacy, managing version control, and handling legacy APIs.
- Best Practices:
 - Prioritize user-friendly documentation.
 - Adopt agile practices for continuous delivery and improvement of APIs.
 - Plan for scalability and robustness to ensure the API can handle future growth.

Reference



Summary

- Key Takeaways
 - APIs are essential for driving business innovation, enhancing product ecosystems, and creating new revenue streams.
 - A winning API strategy revolves around understanding user needs, building robust and secure APIs, and creating an ecosystem of engaged developers.
- Next Steps:
 - Review your organization's current API strategy and identify areas for improvement.
 - Consider how APIs can play a role in your future business models and partnerships.

MCQ1

Which of the following is NOT a key element of a successful API product strategy?

- A) Understanding API users
- B) Aligning use cases with business goals
- C) Ignoring developer feedback
- D) Defining a clear monetization model

Answer: C) Ignoring developer feedback

MCQ2

In the API ecosystem management, which of the following is essential for engaging external developers?

- A) Regularly deprecating APIs
- B) Restricting access to only premium users
- C) Building a strong developer community with support and tools
- D) Offering no support or feedback channels

Answer: C) Building a strong developer community with support and tools

MCQ3

Which of the following monetization models is characterized by charging users based on the number of API calls they make?

- A) Subscription model
- B) Freemium model
- C) Pay-as-you-go model
- D) Open-source model

Answer: C) Pay-as-you-go model

API-based Products

API Users Adaption Strategy | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Understanding User Needs

- Key Areas of Focus:
 - Ease of Use:
 - Developers value APIs that are intuitive and easy to implement.
 - Flexibility:
 - Offering multiple ways to integrate (REST, GraphQL, etc.) ensures broader adoption.
 - Reliability:
 - High uptime, robust error handling, and clear versioning support user confidence.



BITS WILP | | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Provide insights on identifying and addressing the unique needs of different API users to ensure successful API adoption and long-term engagement

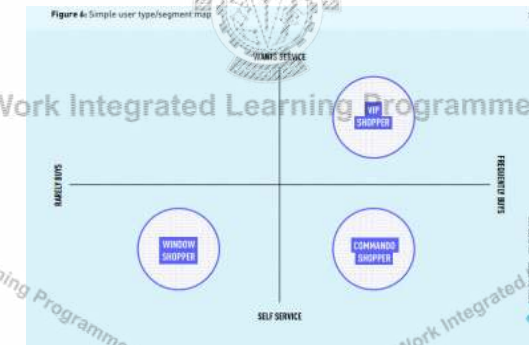


Work Integrated Learning Programmes

BITS WILP | | Generated: 12-May-2026 16:47:54

Introduction to API Users

- API users are central to the success of API products.
 - Identifying and understanding your user base ensures the API is tailored to meet their needs.
 - Importance of user experience in API adoption.



BITS WILP | | Generated: 12-May-2026 16:47:54

Types of API Users

- Key API User Personas:
 - Internal Developers:
 - Developers within the organization using APIs to build new services or enhance internal tools.
 - Partners:
 - External entities collaborating with the company, using APIs for data exchange or integrations.
 - Third-party Developers:
 - Independent developers or companies integrating APIs into their own products.
 - End Users:
 - Non-technical users who indirectly benefit from API functionality (e.g., through apps).

Internal Developers

- Purpose:
 - Build internal services, automate tasks, or create integrations with other enterprise systems.
- Needs:
 - Clear documentation, ease of use, and strong support systems.
- Challenges:
 - Ensuring consistency across various internal systems and APIs.

Partner API Users

- Purpose:
 - Access data or services for integration into their own systems.
- Needs:
 - Stable, secure APIs with strong SLA agreements.
- Challenges:
 - Ensuring security and reliable performance, particularly with sensitive data.

Third-Party Developers

- Purpose:
 - Use the API to enhance or create new products for their own users.
- Needs:
 - Comprehensive documentation, easy onboarding, and scalability.
- Challenges:
 - Creating a developer-friendly ecosystem, building trust, and fostering long-term adoption.

End Users

- Purpose:
 - Indirect users who benefit from apps and services powered by APIs.
- Needs:
 - Seamless performance without glitches.
- Challenges:
 - Ensuring the API performs reliably at scale, as issues will affect the end user's experience.

Developer Experience (DevEx)

- Importance:
 - A strong developer experience drives API adoption.
 - Tools like SDKs, detailed documentation, and active developer forums play a significant role.
 - Investing in DevEx ensures a smoother user journey and reduces friction.

Building a Developer Community

- Fostering a community where developers can
 - share insights,
 - report issues,
 - and find solutions.
- The value of developer evangelism:
 - nurturing a sense of belonging and long-term engagement.

Customizing for Different Users

- Key Strategies:
 - Provide different onboarding experiences tailored to the needs of various user types (e.g., quick-start guides for internal users, more robust documentation for third-party developers).
 - Offer customizable access levels and feature sets based on user type.
 - Engaging users through direct feedback loops to improve the API.

Tools and Resources for API Users

- Key Resources:
 - API Documentation:
 - User-friendly, detailed, and regularly updated.
 - Sandboxes & Testing Environments:
 - Allow users to try the API without real-world consequences.
 - Developer Portals:
 - Provide a central hub for all user needs, from onboarding to troubleshooting.
 - Support Channels:
 - Offering help via chat, forums, and dedicated customer service.

Summary

- Aligning API Development with User Needs
 - Success hinges on thoroughly understanding and serving your API users.
 - Engaging with users early and often helps ensure the API evolves with their needs.
 - A strong developer experience is key to long-term success and adoption.

Reference



MCQ1

Which group of API users primarily focuses on building internal services or automating tasks within an organization?

- a) Third-party developers
- b) Partners
- c) Internal developers
- d) End users

Correct Answer: c) Internal developers

MCQ2

What is a key challenge when developing APIs for third-party developers?

- a) Ensuring high uptime and security
- b) Managing sensitive data for partners
- c) Providing consistent developer experience across internal systems
- d) Fostering a developer-friendly ecosystem and building trust

Correct Answer: d) Fostering a developer-friendly ecosystem and building trust

MCQ3

Which resource is essential for improving developer experience (DevEx) and ensuring smooth API adoption?

- a) Detailed documentation and active developer forums
- b) SLA agreements for partners
- c) Real-time API usage monitoring
- d) Complex onboarding processes

Correct Answer: a) Detailed documentation and active developer forums

API-based Products

Role of API Users Personas in API Strategy

Suresh Gopal

Learning Outcomes

- help convey the importance of understanding API users through personas, supporting a strategy that resonates with different user needs and promotes API success.

Introduction to API Users

- API users are diverse, and understanding them is critical to API success.
- APIs serve different user groups (e.g., developers, business users).
 - Success hinges on knowing who will use the API, their needs, and pain points.
 - This is where User Personas become crucial.

What is a User Persona?

- Definition:
 - A user persona is a semi-fictional representation of your ideal API users based on real data and insights.
 - Represents the goals, behaviors, and pain points of a specific group of users.
 - Helps API teams tailor the design, functionality, and documentation to specific user needs.

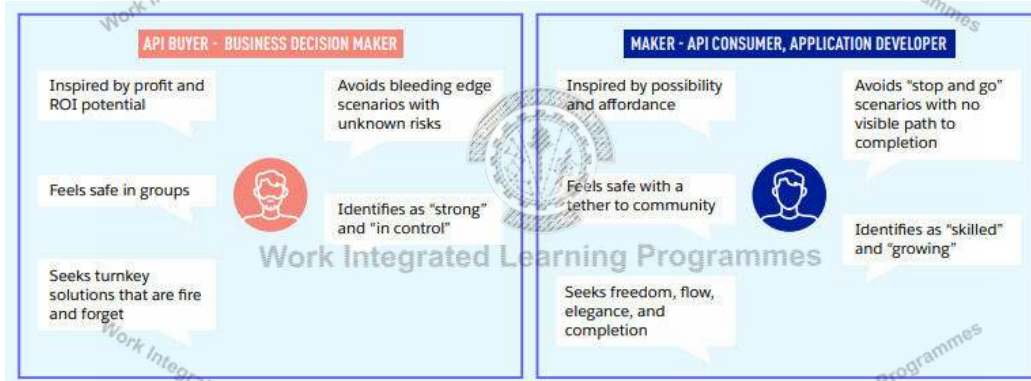
The Role of User Personas in API Strategy

- Developing user personas ensures the API aligns with user expectations and solves real problems.
 - Prioritize API features based on user personas.
 - Create more targeted and user-friendly API documentation.
 - Enhance the API's usability and adoption across different user groups.

Building User Personas: Key Components

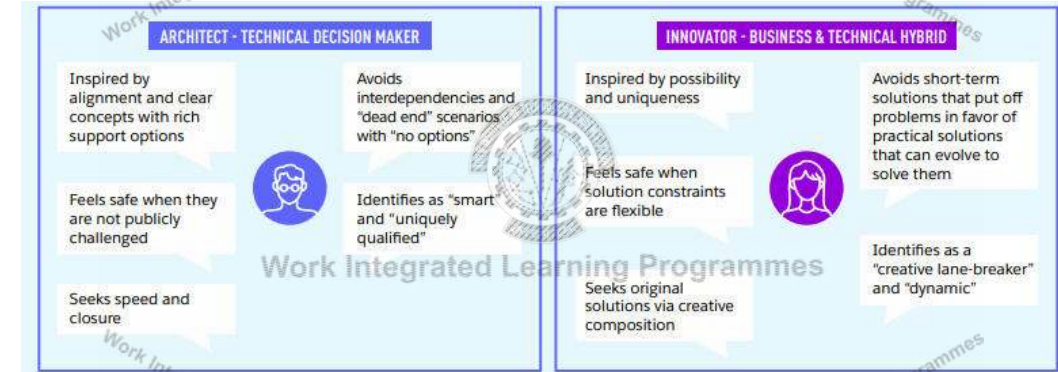
- A user persona is built from a blend of research and real user data.
- Components to Include:
 - Demographics:
 - Role, experience level, industry, etc.
 - Technical Skills:
 - Proficiency with technology, coding knowledge, etc.
 - Goals:
 - What are they trying to achieve with the API?
 - Pain Points:
 - What challenges do they face when working with APIs?
 - Use Cases:
 - How they will use the API in their workflows.

User Persona Example



BITS WILP | Generated: 12-May-2026 16:47:54

User Persona Example



BITS WILP | Generated: 12-May-2026 16:47:54

User Persona Example



BITS WILP | Generated: 12-May-2026 16:47:54

Research Methods for Developing Personas



- Use a combination of qualitative and quantitative methods to develop user personas.
 - Interviews & Surveys:
 - Direct conversations with developers and users.
 - Analytics:
 - Analyzing API usage data to identify user behavior and patterns.
 - User Feedback:
 - Reviewing support tickets, forums, and user reviews to understand pain points.
 - Competitor Analysis:
 - How users engage with similar APIs.

BITS WILP | Generated: 12-May-2026 16:47:54

Mapping API Features to User Personas

- User personas should directly inform the API design and features.
 - Developer Needs:
 - Focus on ease of integration, code samples, and detailed documentation.
 - Business Needs:
 - Include analytics, reporting features, and non-technical guides.
 - Administrator Needs:
 - Emphasize security, scalability, and management tools.

User Persona-Driven API Documentation

- Tailor API documentation to meet the needs of each user persona.
 - For Developers:
 - Provide interactive documentation, code samples, and SDKs.
 - For Business Users:
 - Offer use case examples, clear terminology, and performance metrics.
 - For Admins:
 - Include guides on security, API monitoring, and troubleshooting.

Summary

- Why User Personas Matter for API Success
 - A deep understanding of API users via user personas leads to better design, higher adoption, and long-term success.
 - Ensure API design, documentation, and support cater to real user needs.
 - Use personas to guide ongoing product strategy and development.

Reference



MCQ1

What is a user persona in the context of API strategy?

- A) A fictional character used in marketing campaigns
- B) A representation of the ideal API users based on data and insights
- C) A technical specification document for API development
- D) A type of API documentation aimed at developers

Correct Answer: B) A representation of the ideal API users based on data and insights

MCQ2

Which of the following is NOT a key component of a user persona?

- A) Demographics
- B) Technical Skills
- C) API Version History
- D) Goals

Correct Answer: C) API Version History

MCQ3

Why is it important to regularly update user personas?

- A) To reduce the number of personas used
- B) To ensure they reflect current user needs and behaviors
- C) To comply with industry regulations
- D) To create a standard for API documentation

Correct Answer: B) To ensure they reflect current user needs and behaviors

API-based Products

API Go-To-Market strategy – I **Suresh Gopal**



Learning Outcomes

- Understand the key components and strategies involved in developing an effective Go-To-Market strategy for APIs to drive successful market entry and adoption.



Work Integrated Learning Programmes

Understanding the Market Landscape

- Market trends in API usage
 - (e.g., growth of microservices, cloud computing)
- Key players in the API ecosystem
 - (e.g., API management platforms)



Work Integrated Learning Programmes

Market Trends in API Usage

- Growth of Microservices:
 - Increasing adoption of microservices architecture has led to a surge in the demand for APIs, enabling developers to build and deploy applications as a collection of loosely coupled services.
 - Microservices facilitate independent deployment and scaling of services, resulting in more agile and efficient development processes.



Work Integrated Learning Programmes

Market Trends in API Usage

- Rise of Cloud Computing:
 - The shift to cloud-based services has accelerated the need for APIs to integrate various cloud applications and services, allowing for seamless data flow and functionality across platforms.
 - APIs are essential for leveraging cloud capabilities, such as data storage, computing power, and application hosting, making them critical in cloud ecosystems.



Work Integrated Learning Programmes

Key Players in the API Ecosystem

- API Management Platforms:
 - Companies like Apigee, AWS API Gateway, and MuleSoft provide tools for designing, deploying, and managing APIs, ensuring security, scalability, and performance.
 - These platforms offer features such as analytics, monitoring, and developer portals to enhance the API experience.

Key Players in the API Ecosystem

- Integration Services:
 - Services like Zapier and Postman help facilitate API integrations and testing, making it easier for developers to connect various applications and services.
- Marketplaces:
 - Online platforms such as RapidAPI and AWS Marketplace allow API providers to list their APIs, making them accessible to a wider audience and encouraging adoption.

Identifying Target Markets

- Segmentation
 - (e.g., developers, enterprises, startups)
- Understanding buyer personas
 - (e.g., CTOs, product managers, developers)

Segmentation

- Developers:
 - The primary users of APIs who require robust documentation and support to integrate APIs into their applications efficiently.
- Enterprises:
 - Large organizations that seek to leverage APIs for internal applications, third-party integrations, and enhancing their service offerings.
- Startups:
 - Emerging companies that rely on APIs to build innovative solutions quickly and cost-effectively, often using third-party services to enhance their products.

Understanding Buyer Personas

- CTOs (Chief Technology Officers):
 - Decision-makers focused on technology strategy and infrastructure, interested in APIs that can enhance operational efficiency and scalability.
- Product Managers:
 - Professionals responsible for defining product features and user experiences, looking for APIs that provide unique functionalities or enhance existing products.
- Developers:
 - Technical users who evaluate APIs based on ease of use, documentation quality, and support, often influencing purchasing decisions through their experiences.

Value Proposition

- Defining Your API's Value Proposition
 - What makes your API unique? (e.g., features, performance, ease of use)
- Customer pain points and how your API addresses them
- Case Studies
 - Examples of successful API value propositions in the market

Defining Your API's Value Proposition

- What Makes Your API Unique?
- Features:
 - Highlight standout features of your API, such as:
 - Comprehensive documentation for easy integration
- Performance:
- Emphasize reliability, speed, and scalability, ensuring that your API can handle high volumes of requests without performance degradation.
- Ease of Use:
 - Focus on user experience, including:
 - Intuitive design and structure of the API
 - User-friendly onboarding processes and extensive tutorials

Customer Pain Points and How Your API Addresses Them

- Integration Complexity:
 - Many developers face challenges integrating APIs due to poor documentation or lack of support.
 - Solution: Provide comprehensive guides, sample code, and an interactive sandbox for testing.
- Performance Issues:
 - Slow response times and reliability can hinder user experiences and application performance.
 - Solution: Highlight the speed and reliability of your API, perhaps backed by performance metrics.

Customer Pain Points and How Your API Addresses Them



- Lack of Customization:
 - Developers often need tailored solutions for specific use cases.
 - Solution: Offer flexible and customizable endpoints that allow users to adjust parameters to meet their needs.



Work Integrated Learning Programmes

Examples of Successful API Value Propositions in the Market



- Stripe:
 - Value Proposition:
 - Simplifies payment processing for developers by providing an easy-to-integrate API with excellent documentation and developer support.
 - Outcome:
 - Has become the go-to solution for payment processing for businesses of all sizes, significantly reducing the time to integrate payment functionality.



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Examples of Successful API Value Propositions in the Market



- Twilio:
 - Value Proposition:
 - Offers communication APIs that allow developers to embed messaging, voice, and video capabilities into applications with minimal setup.
 - Outcome:
 - Widely adopted by businesses for real-time communication, enabling faster customer interactions and improved user engagement.



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Summary

- A comprehensive GTM strategy provides a structured approach to entering the market, helping to effectively reach target audiences and differentiate from competitors.
- By clearly defining goals, target markets, and messaging, organizations can allocate resources efficiently, ensuring maximum impact from marketing efforts.
- A strategic approach allows for the identification of unique value propositions, ensuring that the API stands out in a crowded marketplace.



Work Integrated Learning Programmes



BITS WILP | Generated: 12-May-2026 16:47:54

MCQ1

What is the primary purpose of a Go-To-Market (GTM) strategy for APIs?

- A) To develop API documentation
- B) To define pricing models
- C) To outline the approach for entering the market and driving adoption
- D) To conduct competitor analysis

Answer: C) To outline the approach for entering the market and driving adoption

MCQ2

What is a key benefit of having a well-defined Go-To-Market (GTM) strategy for APIs?

- A) It eliminates the need for user feedback.
- B) It ensures maximum impact from marketing efforts by optimizing resource allocation.
- C) It guarantees immediate success in the market.
- D) It focuses solely on technical documentation of the API.

Answer: B) It ensures maximum impact from marketing efforts by optimizing resource allocation.

MCQ3

What is an essential element of a successful API Go-To-Market strategy?

- A) Focusing solely on technical features
- B) Engaging with developer communities and providing robust support
- C) Limiting market outreach to a single platform
- D) Avoiding feedback from customers

Answer: B) Engaging with developer communities and providing robust support

API-based Products

Go-To-Market strategy - II | Suresh Gopal



Learning Outcomes

- Analyze effective distribution, marketing strategies for APIs to enhance market entry and drive user adoption.



Work Integrated Learning Programmes

Distribution Channels

- Choosing the Right Distribution Channels
 - Direct sales vs. indirect sales (e.g., resellers, partners)
 - Online marketplaces for APIs (e.g., RapidAPI, AWS Marketplace)
- Building a Developer Portal
 - Importance of a user-friendly portal for API documentation and support



Work Integrated Learning Programmes

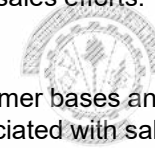
Direct Sales vs. Indirect Sales

- Direct Sales:
 - Involves selling the API directly to end-users (developers, enterprises) through a dedicated sales team or online platform.
- Advantages:
 - Greater control over the sales process and customer relationships.
 - Direct feedback from customers can inform product improvements.
- Considerations:
 - Requires investment in sales personnel and resources for outreach and engagement.



Direct Sales vs. Indirect Sales

- Indirect Sales (e.g., Resellers, Partners):
 - Involves leveraging third-party partners or resellers to distribute the API, expanding market reach without direct sales efforts.
- Advantages:
 - Access to established customer bases and networks of partners.
 - Lower overhead costs associated with sales personnel.
- Considerations:
 - Potential loss of control over customer interactions and brand messaging.
 - Need for effective partner management and training.



Work Integrated Learning Programmes

Online Marketplaces for APIs:

- Examples
 - RapidAPI: A platform that aggregates multiple APIs, making it easier for developers to discover, connect, and manage APIs.
 - AWS Marketplace: Offers APIs and services for AWS users, simplifying integration within the Amazon ecosystem.
- Advantages:
 - Increased visibility and discoverability among a broader audience of developers.
 - Simplified payment and billing processes through the marketplace platform.
- Considerations:
 - Marketplace fees may impact overall revenue.
 - Competition within the marketplace can be intense, necessitating a strong value proposition.

Building a Developer Portal

- Importance of a User-Friendly Portal for API Documentation and Support:
 - Comprehensive Documentation:
 - Interactive Tools:
 - Features such as API explorers, code samples, and interactive sandboxes enhance the developer experience and promote hands-on engagement with the API.
 - Support and Community:
 - Providing forums, FAQs, and support channels fosters a sense of community and assists developers in resolving issues quickly.
 - Tracking and Analytics:
 - Implementing analytics within the portal helps monitor usage patterns, gather feedback, and identify areas for improvement.

Marketing Strategy

- Creating Awareness and Generating Leads
 - Content marketing strategies (e.g., blogs, whitepapers, case studies)
 - Social media and community engagement (e.g., GitHub, Stack Overflow)
 - Webinars and workshops to educate potential users
 - Engaging with Developer Communities and Forums

Content Marketing Strategies

- Blogs:
 - Publish informative and relevant articles that address common challenges, industry trends, and use cases for your API.
- Whitepapers:
 - Develop in-depth resources that provide valuable insights, research findings, or case studies relevant to your API's use case.
- Case Studies:
 - Showcase success stories of customers who have effectively implemented your API to solve problems or enhance their applications.

Social Media and Community Engagement



- Platforms:
 - Utilize platforms like GitHub and Stack Overflow to engage directly with developers, answering questions and providing support.
 - Share updates, announcements, and technical resources on social media channels like Twitter and LinkedIn to foster community engagement.
- Community Building:
 - Actively participate in developer forums, discussion groups, and meetups to create connections and establish your API as a trusted resource.
 - Encourage user-generated content and feedback to create a sense of ownership and community around your API.

Webinars and workshops to educate potential users



- Educational Initiatives:
 - Host webinars to educate potential users about your API's capabilities, best practices, and integration techniques.
 - Offer hands-on workshops that guide participants through real-world use cases and implementation strategies.
- Lead Generation:
 - Collect attendee information to build your email list and follow up with additional resources or promotional offers.

Engaging with Developer Communities and Forums



- Actively participate in relevant online communities (e.g., Reddit, Dev.to) by answering questions and providing insights, positioning yourself as an industry expert.
- Foster relationships with influential developers and community leaders to amplify your reach and credibility.

Summary



- Together, effective distribution channels and a comprehensive marketing strategy are integral to the successful Go-To-Market approach for APIs.
- By combining direct and indirect sales methods, leveraging online marketplaces, and implementing targeted marketing initiatives, organizations can drive awareness, enhance user adoption, and ensure long-term success in the API landscape.

MCQ1

Which of the following is a key advantage of using indirect sales channels for API distribution?

- A) Greater control over customer relationships
- B) Access to established customer bases through partners
- C) Higher revenue per sale
- D) Simplified API documentation

Answer: B) Access to established customer bases through partners

MCQ2

What is the primary benefit of building a user-friendly developer portal for an API?

- A) To increase the number of sales representatives
- B) To enhance the developer experience by providing comprehensive documentation and support
- C) To limit access to the API for only selected users
- D) To create a direct sales channel

Answer: B) To enhance the developer experience by providing comprehensive documentation and support

MCQ3

Which marketing strategy focuses on educating potential users about an API's capabilities?

- A) Content marketing
- B) Direct sales
- C) Social media engagement
- D) Webinars and workshops

Answer: D) Webinars and workshops

API-based Products

Go-To-Market strategy - III Suresh Gopal



Learning Outcomes

- Understand how to provide effective customer support, gather feedback for continuous improvement, and measure API success using key performance indicators to refine the Go-To-Market strategy.

Customer Success and Support

- Importance of Customer Support
 - Providing robust documentation and resources (e.g., FAQs, tutorials)
 - Establishing a support team for onboarding and troubleshooting
- Gathering Feedback
 - Importance of customer feedback for continuous improvement
 - Utilizing feedback to enhance API features and performance

Providing robust documentation and resources

- Providing Robust Documentation and Resources:
 - Comprehensive Documentation
 - FAQs and Tutorials
- Interactive Resources: Offering interactive tools, such as API explorers or sandboxes, allows users to experiment with the API and test features without setting up a full integration, enhancing their learning experience.

Establishing a support team

- Onboarding Assistance:
 - A dedicated support team can provide personalized onboarding sessions, guiding new users through the integration process and helping them understand key features.
- Troubleshooting and Technical Support:
 - Ongoing technical support is vital for resolving issues users encounter during implementation or usage. Quick response times and effective communication can significantly improve the user experience and foster long-term relationships.

Establishing a support team

- Customer Education:
 - Regular webinars, workshops, or one-on-one support sessions can be organized to educate customers about advanced features or best practices for using the API effectively.



Work Integrated Learning Programmes

Gathering Feedback

- Importance of customer feedback for continuous improvement
- Utilizing feedback to enhance API features and performance



Work Integrated Learning Programmes

Importance of customer feedback

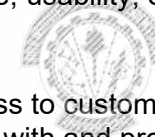
- Feedback Channels:
 - Setting up multiple channels for collecting feedback, such as surveys, feedback forms, and forums, helps gather diverse insights from different users.
- Active Listening:
 - Actively listening to customers' experiences allows you to identify pain points, discover unmet needs, and gather suggestions for improving the API.



Work Integrated Learning Programmes

Utilizing feedback to enhance API features and performance

- Iterative Improvements:
 - Use feedback to inform your product roadmap, making iterative updates to the API that improve performance, usability, or feature set.
- Building Trust:
 - Demonstrating responsiveness to customer feedback fosters trust and loyalty. Users are more likely to stick with and promote an API that evolves based on their input.



Work Integrated Learning Programmes

Measuring Success

- Key Performance Indicators (KPIs)
 - Metrics to track API success (e.g., number of active users, API calls, revenue)
 - Tools for monitoring API performance and user engagement

Metrics to track API success

- Number of Active Users:
 - Track the number of unique users actively utilizing the API over a given time period. This provides insight into user retention and adoption rates.
- API Calls (Usage Volume):
 - Monitor the number of API calls made by users. A high number of API calls indicates strong engagement and reliance on your API.

Metrics to track API success

- Revenue:
 - Measure the revenue generated through paid models (e.g., subscription fees, pay-per-call) to assess financial performance.
- Customer Satisfaction (CSAT) and Net Promoter Score (NPS):
 - Collect feedback through surveys to gauge customer satisfaction and the likelihood of them recommending your API to others.
- Error Rates and Uptime:
 - Track error rates and uptime as technical performance metrics. A low error rate and high uptime are critical for maintaining user trust.

Tools for Monitoring API Performance and User Engagement

- API Analytics Platforms:
 - Google Analytics for APIs or Mixpanel: These tools track user interactions and provide insights into usage patterns, popular endpoints, and user engagement.
- API Management Platforms (e.g., Apigee, AWS API Gateway):
 - Provide built-in tools for monitoring performance metrics, security, and traffic management.

Tools for Monitoring API Performance and User Engagement



- Monitoring Tools (e.g., Datadog, New Relic):
 - Offer real-time tracking of API performance, alerting teams to latency issues, downtime, and other technical problems.
- Customer Feedback Tools:
 - Tools like SurveyMonkey or Qualtrics can help collect direct feedback on user satisfaction, helping to align product enhancements with user needs.

Summary

- Customer Success and Support:
 - Providing robust documentation, proactive onboarding, and gathering continuous feedback are critical to ensuring customer satisfaction and driving the iterative improvement of your API.
- Measuring Success:
 - Monitoring key performance indicators (KPIs) like user engagement and API usage, along with utilizing data-driven insights, helps refine the Go-To-Market strategy and ensure long-term API success.

MCQ1

What is the primary benefit of gathering customer feedback for an API?

- A) To increase pricing tiers
- B) To identify customer pain points and improve API features
- C) To reduce the need for customer support
- D) To decrease the number of API calls

Answer:

- B) To identify customer pain points and improve API features

MCQ2

Which of the following is an essential metric to track API success?

- A) Number of customer support tickets
- B) Number of API documentation pages
- C) Number of API calls made by users
- D) Number of marketing emails sent

Answer:

- C) Number of API calls made by users

MCQ3

Why is it important to regularly review key performance indicators (KPIs) for an API?

- A) To increase user activity by sending more marketing emails
- B) To make data-driven decisions and refine the Go-To-Market strategy
- C) To limit the API's features to certain customers
- D) To track competitors' API pricing strategies

Answer:

- B) To make data-driven decisions and refine the Go-To-Market strategy

BITS WILP | Generated: 12-May-2026 16:47:54

Learning Outcomes

- Understand the various pricing models for APIs to effectively determine the best approach for maximizing revenue and meeting customer needs.

BITS WILP | Generated: 12-May-2026 16:47:54



API-based Products

Go-To-Market strategy - IV | Suresh Gopal

BITS WILP | Generated: 12-May-2026 16:47:54

Pricing Strategy

- Pricing Models for APIs
 - Freemium vs. paid models
 - Usage-based pricing (e.g., pay-per-call, tiered pricing)
 - Subscription models (e.g., monthly or annual fees)
- Competitor Analysis
 - Overview of pricing strategies of competitors in the market

BITS WILP | Generated: 12-May-2026 16:47:54



Pricing Models for APIs

- Pricing-Based Models:
 - Freemium Models
 - Paid Models
- Usage-Based Pricing:
 - Pay-per-Call
 - Tiered Pricing
- Subscription Models

Freemium vs. Paid Models

- Freemium Model:
 - Provides a basic version of the API for free with limited features.
 - Encourages users to upgrade to a paid version for access to advanced features.
 - Effective for attracting users and allowing them to test the API before committing to a paid plan.

Freemium vs. Paid Models

- Paid Model:
 - Users pay from the start for full access to the API's features.
 - Payment can be structured as a one-time fee or recurring payments (e.g., subscriptions).
 - Suitable for APIs offering specialized services where immediate access to full features is critical.

Usage-Based Pricing

- Pay-per-Call:
 - Charges users based on the number of API calls they make.
 - Cost is directly proportional to usage, allowing flexibility for scaling usage up or down.
 - Ideal for customers who want to control their costs based on usage patterns.

Usage-Based Pricing

- Tiered Pricing:
 - Offers multiple pricing tiers based on usage limits or access to additional features.
 - Allows users to choose plans that fit their specific needs and upgrade as they scale.
 - Attracts both small users and enterprise customers by offering flexibility.

Subscription Models

- Monthly or Annual Fees:
 - Users subscribe to the API for a recurring fee, providing predictable revenue streams.
 - Subscription terms are typically monthly or annual, with discounts often provided for longer commitments.
 - Suitable for APIs with steady usage patterns, where customers prefer fixed costs for predictable budgeting.

Overview of Pricing Strategies of Competitors

- Competitive Landscape:
 - Freemium Models:
 - Many competitors offer free access to basic API features with premium upgrades, making this a common strategy to attract developers and small businesses.
 - Pay-per-Call Systems:
 - Some competitors charge users based on the number of API calls, allowing customers to scale their usage and costs according to their needs.
 - Subscription-Based Models:
 - Competitors may offer monthly or annual subscription plans, providing consistent access to the API with fixed fees, often tiered based on usage limits or advanced features.

Overview of Pricing Strategies of Competitors

- Market Positioning:
 - Value and Cost Comparison:
 - Assess how your API's features, performance, and pricing stack up against competitors. Are you offering more for less, or do you provide premium features at a higher cost?
 - Differentiation Opportunities:
 - Identify ways to stand out by offering unique features, better support, flexible pricing, or improved performance.
 - Highlight gaps in competitors' offerings that your API can fill, whether through pricing models or product capabilities.

Summary

- Selecting the right pricing model is essential for maximizing revenue while meeting customer expectations and fostering long-term API adoption
- Selecting the right pricing model depends on the API's target audience, value proposition, and customer usage patterns. Balancing affordability with value is critical for long-term success.

MCQ1

What is a key characteristic of the freemium model for APIs?

- A) Users pay a one-time fee for full access.
- B) Users have free access to basic features, with an option to upgrade to premium features.
- C) Users are charged based on the number of API calls they make.
- D) Users subscribe on a monthly or annual basis.

Answer:

- B) Users have free access to basic features, with an option to upgrade to premium features.

MCQ2

In usage-based pricing, what does "pay-per-call" refer to?

- A) Charging users a fixed monthly fee regardless of usage.
- B) Users are charged based on the total number of API calls they make.
- C) Offering a basic version of the API for free.
- D) Providing access to multiple pricing tiers.

Answer:

- B) Users are charged based on the total number of API calls they make.

MCQ3

What advantage do subscription models provide for API providers?

- A) They allow customers to pay only when they use the API.
- B) They create predictable revenue streams through recurring fees.
- C) They limit user access to basic features only.
- D) They do not require a competitive analysis.

Answer:

- B) They create predictable revenue streams through recurring fees.

API-based Products

Introducing API Analytics Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

Introduction to API Analytics

- What is API Analytics?
 - API analytics is the process of collecting, monitoring, and analyzing data generated by API usage to evaluate its performance, reliability, and overall effectiveness.
- Key to ensuring that APIs meet business objectives and provide value to users.
- Core Purpose:
 - Assess how well the API meets its intended objectives
 - Provide insights into API usage patterns, helping stakeholders make data-driven decisions.

Learning Outcomes

- Understand the role of API analytics in monitoring performance, optimizing usage, addressing key challenges, and exploring emerging trends for future improvements in API security and real-time analysis.

Why is API Analytics Important?

- Why is API Analytics Important?
 - Ensures API Efficiency:
 - Helps identify and resolve performance issues like latency, slow response times, and errors.
 - Monitors API Usage:
 - Tracks user engagement, which endpoints are most popular, and how users interact with the API.
 - Optimizes API Performance:
 - Provides data to improve functionality, scalability, and overall user experience.
 - Aligns API with Business Goals:
 - Ensures the API delivers value to the business by meeting key performance indicators (KPIs) and user expectations.

Why API Analytics Matters?

- Performance Monitoring
- User Insights
- Error Detection
- Optimization
- Scalability



Work Integrated Learning Programmes



Why API Analytics Matters?

- Performance Monitoring:
 - Continuously tracks how well your API is performing.
 - Helps identify latency issues, downtime, and slow responses, ensuring optimal API performance.
- User Insights:
 - Provides visibility into how and by whom the API is being used.
 - Offers data on user demographics, usage patterns, and most-used endpoints, enabling personalized user experience and better decision-making.

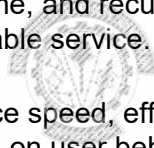


Work Integrated Learning Programmes



Why API Analytics Matters?

- Error Detection:
 - Detects and logs errors, such as failed requests or authorization issues.
 - Pinpoints bottlenecks, downtime, and recurring problems, allowing for faster troubleshooting and more reliable service.
- Optimization:
 - Leverages analytics to enhance speed, efficiency, and usability.
 - Helps refine API design based on user behavior and performance data, leading to smoother user experiences.
- Scalability:
 - Analyzes traffic patterns to ensure the API can handle increased load.

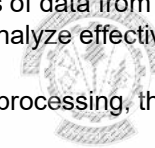


Work Integrated Learning Programmes



Challenges in API Analytics

- Data Overload:
 - Issue:
 - APIs generate vast amounts of data from multiple endpoints and users, which can be difficult to manage and analyze effectively.
 - Impact:
 - Without proper filtering and processing, the sheer volume of data can obscure critical insights.



Work Integrated Learning Programmes



Challenges in API Analytics

- Real-Time Monitoring:
 - Issue:
 - Ensuring timely alerts for performance degradation or failures can be challenging, especially for APIs with high traffic.
 - Impact:
 - Delays in identifying issues can lead to extended downtime, lost revenue, and poor user experiences.

Challenges in API Analytics

- Complex User Behavior:
 - Issue:
 - APIs are often used by diverse clients with different usage patterns, which can be hard to interpret and optimize for.
 - Impact:
 - Mixed and unpredictable behaviors make it difficult to identify trends and prioritize improvements, leading to inefficient API optimization efforts.

Future of API Analytics

- AI-Powered Analytics:
 - What it is:
 - Leveraging machine learning algorithms to predict API performance trends and detect anomalies before they affect users.
 - Impact:
 - Predictive insights enable proactive maintenance and optimization, improving API reliability and user satisfaction.

Future of API Analytics

- Increased Focus on API Security Metrics:
 - What it is:
 - Enhanced monitoring tools focused on detecting API security threats, such as unauthorized access attempts, data breaches, and rate-limiting violations.
 - Impact:
 - Proactive security analytics help safeguard APIs from growing cyber threats, ensuring data integrity and user privacy.

Summary

- API analytics involves collecting, monitoring, and analyzing data to assess API performance and usage, ensuring alignment with business goals.
- API analytics is essential for performance monitoring, gaining user insights, error detection, optimization, and ensuring scalability.
- Common challenges include managing data overload, real-time monitoring, and interpreting complex user behaviors from diverse clients.

MCQ1

What is the primary purpose of API analytics?

- A) To increase the number of API endpoints
- B) To collect, monitor, and analyze API data for performance and usage evaluation
- C) To automatically fix errors in APIs
- D) To reduce the cost of API development

Correct Answer: B

MCQ2

Which of the following is a common challenge in API analytics?

- A) Easy integration with all monitoring tools
- B) Predicting API performance accurately
- C) Managing large volumes of API request data
- D) Eliminating all API downtime

Correct Answer: C

MCQ3

Which emerging trend in API analytics focuses on enhancing security monitoring?

- A) AI-powered analytics
- B) Real-time dashboards
- C) Increased focus on API security metrics
- D) Latency optimization

Correct Answer: C

API-based Products

Defining Key API Metrics | Suresh Gopal



BITS WILP | | Generated: 12-May-2026 16:47:54

What are API Metrics?

- API metrics provide quantifiable measures of the performance, usage, and effectiveness of an API.
 - help in monitoring, troubleshooting, and improving API performance.
 - Essential for ensuring reliability, user satisfaction, and business growth.



Work Integrated Learning Programmes

Learning Outcomes

- Understand and articulate the key metrics for measuring API performance, including performance, usage, error, business, and reliability metrics, to effectively monitor and optimize API effectiveness.



Work Integrated Learning Programmes

Why API Metrics Matter?

- Importance of API Metrics:
 - Ensure smooth functionality and performance.
 - Provide insights for optimization.
 - Measure scalability and capacity.
 - Track user experience and engagement.
 - Enable proactive issue detection and resolution.



Work Integrated Learning Programmes

Types of Key API Metrics

- Performance Metrics
- Usage Metrics
- Error Metrics
- Business Metrics
- Reliability & Availability Metrics



Work Integrated Learning Programmes



Performance Metrics

- Response Time (Latency):
 - Measures the time taken to respond to an API request.
 - Impact: Slow responses lead to poor user experience.
- Throughput:
 - Number of API requests processed per unit of time.
 - Impact: Indicates capacity under load.
- Request Size:
 - Monitors the size of requests (payloads).
 - Impact: Larger sizes can increase latency.



Work Integrated Learning Programmes



Usage Metrics

- Request Volume:
 - Number of API requests over a period of time.
 - Impact: Helps understand API traffic patterns.
- User Demographics:
 - Tracks who is using the API and how often.
 - Impact: Provides insights into customer engagement.
- API Endpoints Usage:
 - Tracks which API endpoints are used most frequently.
 - Impact: Helps prioritize endpoint optimizations.

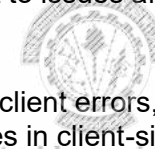


Work Integrated Learning Programmes



Error Metrics

- Error Rate:
 - Percentage of failed API requests over total requests.
 - Impact: High error rates point to issues affecting performance.
- Types of Errors:
 - Categorizes errors (e.g., 4xx client errors, 5xx server errors).
 - Impact: Helps diagnose issues in client-side usage vs. server issues.
- Latency of Failed Requests:
 - Tracks response time of failed requests.
 - Impact: Provides clues about system stress before failure.



Work Integrated Learning Programmes



Business Metrics

- Revenue per API Call:
 - Measures revenue generated per API request.
 - Impact: Direct business value from API usage.
- User Retention:
 - Tracks how often users return to use the API.
 - Impact: Indicates API's ongoing value to customers.
- Adoption Rate:
 - Number of new users or clients integrating with the API.
 - Impact: Shows API market penetration and potential growth.

Reliability & Availability Metrics

- Uptime:
 - Measures the amount of time the API is available.
 - Impact: High uptime indicates reliability.
- Mean Time Between Failures (MTBF):
 - Average time between API failures.
 - Impact: Tracks the reliability and durability of the API.
- Mean Time to Recovery (MTTR):
 - Average time it takes to recover from failure.
 - Impact: Measures how quickly issues can be resolved.

Best Practices for Tracking API Metrics

- Use Monitoring Tools:
 - Tools like Postman, Prometheus, or Datadog to track API metrics in real-time.
- Set Thresholds and Alerts:
 - Define acceptable ranges for metrics and set alerts when they are exceeded.
- Implement Logging:
 - Store API activity logs to assist with detailed analysis and troubleshooting.
- Regular Audits:
 - Perform routine checks to ensure metrics are in line with performance goals.

Summary

- Monitoring the right API metrics is critical to ensure optimal performance and business success.
- A combination of performance, usage, error, and business metrics should be tracked to gain a complete picture.
- Continuous monitoring and proactive adjustments are essential for maintaining a high-quality API service.

MCQ1

Which of the following metrics measures the percentage of failed API requests?

- A) Response Time
- B) Throughput
- C) Error Rate
- D) Uptime

Answer: C) Error Rate

MCQ2

What does the 'Mean Time to Recovery (MTTR)' metric indicate?

- A) The average time taken for an API to respond
- B) The average time between API failures
- C) The average time taken to recover from an API failure
- D) The number of successful API requests per time period

Answer: C) The average time taken to recover from an API failure

MCQ3

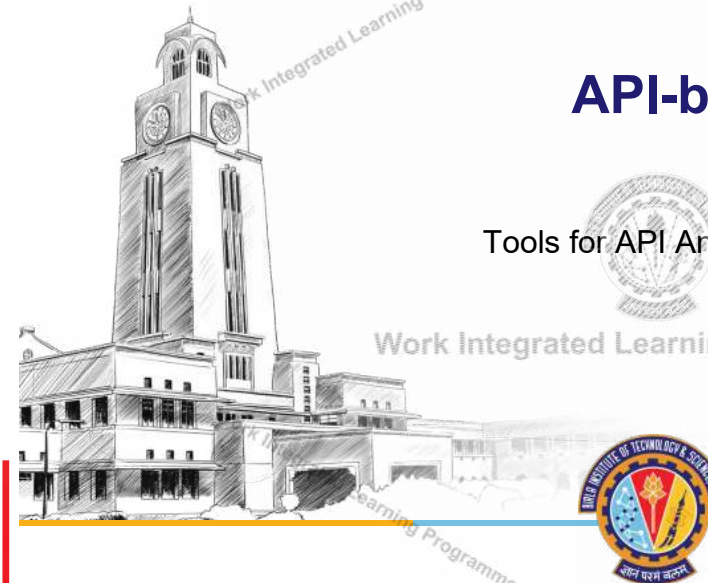
Which type of metric would track the number of requests processed by an API over a specific period?

- A) Business Metrics
- B) Usage Metrics
- C) Performance Metrics
- D) Reliability Metrics

Answer: B) Usage Metrics

API-based Products

Tools for API Analytics | Suresh Gopal



Learning Outcomes

- Understand the importance and benefits of API analytics tools for monitoring and optimizing API performance



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Why is API Analytics Important?

- API Analytics involves tracking, collecting, and analyzing data generated by API usage.
 - helps developers and businesses understand how their APIs are performing, who is using them, and what issues might be arising.
- Why is it Important?
 - Optimize API Performance
 - Understand Usage Patterns
 - Enhance User Experience

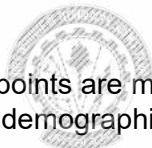


Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Key Benefits of API Analytic Tools

- Performance Monitoring:
 - Real-time insights into how well the API is performing, helping identify latency or downtime quickly.
- Usage Insights:
 - Analytics can show which endpoints are most popular, what kind of traffic patterns exist, and which user demographics interact most frequently.
- Error Tracking:
 - Identify patterns in errors and exceptions that can then be addressed to improve reliability and stability.
- Business Impact:
 - API analytics can help measure how APIs contribute to business outcomes (e.g., revenue through API integrations or increased customer retention via third-party applications).

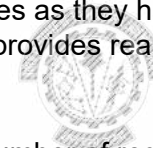


Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Common Features of API Analytics Tool

- Real-time Monitoring:
 - These tools provide live dashboards that track key API metrics such as uptime, response times, and error rates as they happen.
 - Example: AWS CloudWatch provides real-time metrics and dashboards to monitor AWS APIs.
- Usage Reporting:
 - Historical reports about the number of requests, data transferred, and overall API load over time.
 - Example: Google Analytics offers limited support for API tracking, reporting on custom API events.



Work Integrated Learning Programmes

BITS WILP | Generated: 12-May-2026 16:47:54

Common Features of API Analytics Tool

- Alerts and Notifications:
 - Automatically notify developers or operators when something goes wrong (e.g., slow response times or increased error rates).
 - Example: New Relic allows you to set custom alerts based on API performance thresholds.
- Data Visualization:
 - Tools often provide charts and graphs to make it easier to interpret large volumes of data.
 - Example: Postman provides analytics in the form of visualized test results to monitor request success rates and performance.

Popular API Analytics Tools - Overview

- Google Analytics:
 - Primarily used for tracking website interactions but can be configured for basic API tracking using event-based triggers.
- Key Features:
 - Tracks basic API performance metrics.
 - Integration with other Google products (Google Cloud, Google Tag Manager).
 - Limited real-time monitoring but good for high-level reporting.

Popular API Analytics Tools - Overview

- Postman:
 - Widely known as an API testing tool but offers monitoring and analytics features for API performance.
- Key Features:
 - In-built tools for API testing and debugging.
 - Monitors the number of requests, response times, and success/error rates.
 - Provides logs and visualizations for better understanding of API performance.

More API Analytics Tools

- New Relic:
 - A leading platform for monitoring API performance and user interactions.
- Key Features:
 - Real-time performance metrics, including response time and throughput.
 - Comprehensive data for identifying API performance bottlenecks.
 - Integrates with a variety of services, making it ideal for large, distributed systems.

More API Analytics Tools

- AWS CloudWatch:
 - Monitors AWS-based APIs and provides performance metrics for AWS services.
- Key Features:
 - Tracks CPU usage, request count, error rates, and response times.
 - Integration with other AWS services for broader cloud monitoring.
 - Provides alarms and notifications for anomalies.

Advanced API Analytics Tools

- Kong:
 - A powerful API gateway that offers built-in analytics capabilities.
- Key Features:
 - Monitors traffic, response times, and error rates for APIs.
 - Provides advanced analytics on API usage patterns.
 - Includes plugins for added functionality like logging, rate-limiting, and authentication.

Summary

- API analytics tools are essential for ensuring the performance, stability, and reliability of modern APIs.
- From basic monitoring to advanced real-time metrics, the right toolset can help you optimize your API strategy and enhance the user experience.
- Final Thought:
 - As APIs become central to business operations, investing in proper API analytics tools and practices is essential for long-term success.

MCQ1

What is the primary purpose of API analytics?

- A) To design new APIs
- B) To track, collect, and analyze data generated by API usage
- C) To improve user interface design
- D) To develop API documentation

Correct Answer: B) To track, collect, and analyze data generated by API usage

MCQ2

Which of the following is a key metric to track in API analytics?

- A) Number of developers
- B) API response time
- C) API documentation updates
- D) Number of APIs created

Correct Answer: B) API response time



MCQ3

Which tool is known for both API testing and performance monitoring?

- A) Google Analytics
- B) New Relic
- C) Postman
- D) AWS CloudWatch

Correct Answer: C) Postman

